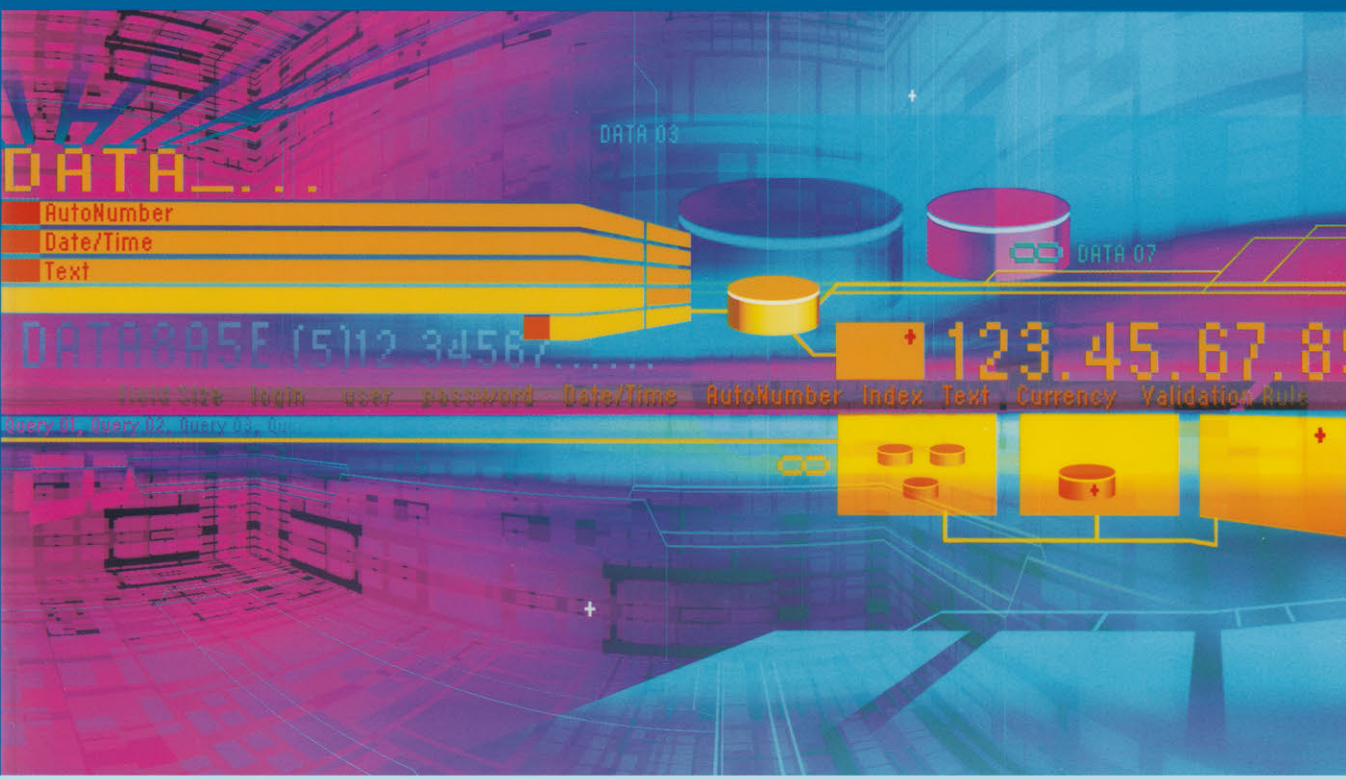
Tools of the Trade

Practical Web Database Design

Chris Auld, Allan Kent, Rudy Limeback, Nigel Stanger, Thearon Willis



Important database design steps

Structured Query Language

Database and platform neutral

glasshaus

labor-saving devices for web professionals

Free technical support for this book at www.glasshaus.com

Summary of Contents

Introduction		**1**
Chapter 1:	Database Introduction	**5**
Chapter 2:	Core Database Concepts	**23**
Chapter 3:	Creating and Using Relational Databases with SQL	**53**
Chapter 4:	Fundamentals of Database Design	**101**
Chapter 5:	Planning the Database	**139**
Chapter 6:	Implementing the Database	**155**
Chapter 7:	Database Security	**177**
Chapter 8:	Advanced Database Feature	**197**
Chapter 9:	Database Maintenance	**225**
Appendix:	Available Database Platforms	**257**
Index		**281**

Practical Database Design for the Web

Chris Auld

Allan Kent

Rudy Limeback

Nigel Stanger

Thearon Willis

© 2002 Apress
Originally published by glasshaus in 2002

ISBN 978-1-59059-194-9 ISBN 978-1-4302-5377-8 (eBook)
DOI 10.1007/978-1-4302-5377-8

Practical Database Design for the Web

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical articles or reviews.

The authors and publisher have made every effort in the preparation of this book to ensure the accuracy of the information. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, glasshaus nor its dealers or distributors will be held liable for any damages caused or alleged to be caused either directly or indirectly by this book.

Cover Image

The Cover Image was provided by Raphael J. Wiszowaty

Raphael J. Wiszowaty was born in Poland in 1978 and immigrated to the United States in 1994 together with his family. His interest in art and design began when he was still in elementary school. He developed a serious vision of becoming an artist and designer when he joined Gallery 37 in Chicago in 1996. His five year training involved working with different media: painting, sculpture, graphic design, animation, video, and sound.

Currently he is a student at Columbia College Chicago, with already completed concentration in Graphic Design. He will graduate with honors and BFA in Interactive Multimedia in June of 2003. If you would like to find out more about Raphael's work, please feel free to visit his site at: *www.multimus.com*

labor-saving devices for web professionals

© 2002 glasshaus

Trademark Acknowledgements

glasshaus has endeavored to provide trademark information about all the companies and products mentioned in this book by the appropriate use of capitals. However, glasshaus cannot guarantee the accuracy of this information.

Credits

Authors
Chris Auld
Allan Kent
Rudy Limeback
Nigel Stanger
Thearon Willis

Techincal Reviewers
Robin Dewson
Damien Foggon
Dan Maharry
Dan O'Reilly
David Schultz
Alex Shiell
Rick Stones

Proof Reader
Agnes Wiggers

Technical Editors
Mark Waterhouse
Chris Mills

Commissioning Editor
Chris Mills

Publisher
Viv Emery

Managing Editor
Liz Toy

Project Manager
Sophie Edwards

Graphic Editors
Rachel Taylor
Pip Wonson

Cover
Dawn Chellingworth

Indexer
Bill Johncocks

About the Authors

Chris Auld

Chris Auld is Chief Architect at New Zealand based software firm eMedia. Chris divides his time between architecting large systems, building mobile applications, and running around like a headless chicken evangelising all that is "new" and "only just released". Chris is a Microsoft platform specialist and a self-professed XML geek – he even has the car number plate to prove it! Chris is a graduate of Otago University in Law and Information Science.

In his "spare" time, he collects University Degrees and DJs Dark Progressive House music, and partakes in silly ritualistic adventure sport activities such as kitesurfing, whitewater kayaking and mountain biking.

I would like to thank: my Mum and Dad for getting me to where I am and for supporting me in where I am going; my darling girlfriend for enduring my late nights in front of the computer screen. I also need to thank the glasshaus crew for their encouragement and support, despite my being on the other side of the world with 12 hours time difference.

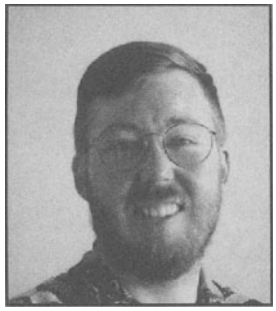
Allan Kent

Allan has been programming seriously for the last nine years and other than a single blemish when he achieved a diploma in Cobol programming, is entirely self-taught. He runs his own company where they try to make a living out of making a lot of noise and playing Quake. When that doesn't work they make a lot of noise while doing development and design for an ad agency. Allan lives in Cape Town, South Africa with his girlfriend and four cats.

Rudy Limeback

Rudy Limeback is a web and SQL consultant in Toronto Canada. Rudy started as a Cobol programmer in 1971, turning to database design with IDMS soon after. Subsequent positions included database software sales, data administration and management, data warehousing, and data modeling. Rudy began using SQL with DB2 in 1987, and has also worked with SQL Server, Access, Oracle, and MySQL. His interest in the Internet started in 1996 with a personal web site. Rudy was one of the founders of the *http://evolt.org/* Web developer community and remains active on its discussion list, helping web developers use SQL for dynamic sites. Rudy is also a regular contributor on several database forums. Visit *http://rudy.ca/* for more information or just to say hello.

Nigel Stanger

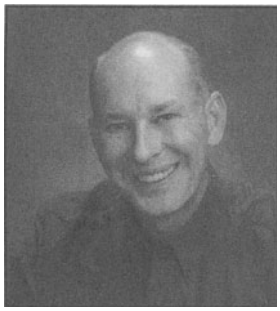


Nigel is a lecturer in the Department of Information Science at the University of Otago School of Business in Dunedin, New Zealand. He has almost fifteen years experience with database systems, mainly relational, but also object and other types. He teaches both introductory and advanced database courses, covering everything from database theory and SQL to database tuning and data warehouses.

He has research interests in database system performance (especially in distributed and web-based environments), wireless mobile databases and XML-based systems. His PhD thesis was on translating between data modeling representations. He's also recently been looking at spam filtering using neural networks.

He's a keen mountain biker and goes on at least one major cycle tour every year. He also enjoy car trialing and recently helped set part of the course for an event. He has been a Mac user since 1985, defiantly cheerful in a sea of Windows machines.

Thearon Willis



Thearon is a Senior Systems Engineer with over 20 years programming experience. He currently develops Intranet applications, XML Web services and server-side and client-side utilities using Visual Basic .NET, ASP, DHTML, XML, JavaScript, VBScript, VB COM Components and SQL Server. He lives with his wife Margie and daughter Stephanie in the Raleigh, North Carolina area.

As always, I want to thank my wife Margie and my daughter Stephanie for the patience they have shown while I write another book. Without their love and support, none of this would be possible.

DATA_...
AutoNumber
Date/Time
ext
123.45.67.89
AutoNumber Index Text Currency

Table of Contents

Introduction 1

Support and Feedback 3

1: Database Introduction 5

What is a Database? 5

A Short History Lesson 8

Database Models 10

DBMS 17

Databases on the Web 18

Summary 20

2: Core Database Concepts 23

Introduction 23

But Is All This Theory Actually Useful? 24

What is a Data Model? 25

The Relational Model of Data 27
The Basics: Tables, Rows, and Columns 28
Domains and Data Types 31
Identifying Rows: Primary Keys 34
Linking Related Tables: Foreign Keys 36
Normalization 37
A Quick Recap 42
Data Integrity 42
Metadata and the Data Dictionary 44
Physical Data Access Methods 45
Sequential Access 46
Indexed Access 47
Direct Access 48
Fitting It All Together 49
Summary 50

3: Creating and Using Relational Databases with SQL 53

Introduction 53
A Note on the Examples 54
Structured Query Language 54
A Little History 55
Dialects 55
Syntax Components 56
Data Definition 57
CREATE TABLE 57
DROP TABLE 63
ALTER TABLE 64
CREATE INDEX, DROP INDEX 66

Data Manipulation....67
Creating Data: INSERT....67
Updating Data: UPDATE....69
Deleting Data: DELETE....70
The WHERE Clause....71
Reading Data from the Database....72
Querying in SQL: SELECT....73
Joins....77
Useful Reporting Features....84
Other Miscellaneous Useful Stuff....90
Techniques for Building SELECT Queries....94
A Detailed Example....95
Summary....99

4: Fundamentals of Database Design 101

Data Modeling....101
Why Bother Modeling?....102
Modeling Entities, Attributes, and Relationships....103
Entities....104
Attributes....104
Relationships....105
Common Structures....105
One-to-Many....106
Many-to-Many....107
One-to-Many Recursive....109
Many-to-Many Recursive....112
One-to-One....113
E-Commerce Example Database Diagram....116

Integrity Constraints 116
Entity Integrity 117
Attribute or Domain Integrity 121
Referential Integrity 128
Denormalizing for Performance 131
Using CASE Tools for Modeling 132
CASE Demonstration – Visio 2002 132
Importance of Sample Data 136
Summary 137

5: Planning the Database **139**

Gather Information 140
Store Owner 141
Customer 144
Payment Processing 145
Identify Entities and Attributes 146
Identify Entity Relationships 147
Product Relationships 147
Category Relationships 150
Customer Relationships 151
Prepare Conceptual Data Model 151
Summary 152

6: Implementing the Database **155**

Target Database System 156
Naming Conventions 156
Table Names 157
Column Names 157

Associative Entities for Many-to-Many Relationships 159
Data Types and Nullability 161
Character Data Types 161
Numeric Data Types 161
Date Data Types 162
Nullability 162
Natural and Surrogate Primary Keys 163
Foreign Keys and Relational Integrity 164
Table and Column Definitions 165
Product 166
Category 167
ProductCategory 168
Customer 168
Orders 169
OrderItem 170
The Overall Database 170
Database Indexes 171
Sample DDL 172
Sample SQL 173
Search for Products 173
Add a Product to a Customer's Order ("add to cart") 174
Summary 175

7: Database Security **177**

Designing a Security Model 178
Identifying Your Target Audience 178
Choosing an Authentication Method 179
Identifying the Type of Data Access Needed 180

Defining Users and Groups 181
Using Strongly Typed Passwords 182
Encrypt Your Passwords 182
How to Secure Your Database 183
Table Permissions 183
Stored Procedure Permissions 184
View Permissions 184
Setting Permissions 185
Inline SQL vs. Stored Procedures 186
Using Views to Restrict Data Access 191
Network Security for Your RDBMS 192
Summary 193

8: Advanced Database Features 197

Advanced Queries 197
Sub-Queries 198
Stored Procedures and Triggers 205
Stored Procedures 206
Triggers 207
Physical Database Tuning 209
Query Plans, or, What's That RDBMS Thinking? 210
Indexes 213
Types of Index 215
Managing Concurrency 216
Locking 216
Transactions 217
Summary 222

9: Database Maintenance **225**

Why a Developer Should Know This 226

Backup and Restore 226

Backing Up Your Database 227

Restoring Your Database 231

Modifying Tables 234

Changing Columns 235

Adding Columns 238

Dropping Columns 239

Adding Indexes 243

Reorganizing Data and Indexes 246

Dropping an Index....247

Drop Existing....248

Building Scripts for Production Implementation 249

Scripts to Populate Tables....249

When to Perform Database Maintenance 254

Summary 254

Appendix: Available Database Platforms **257**

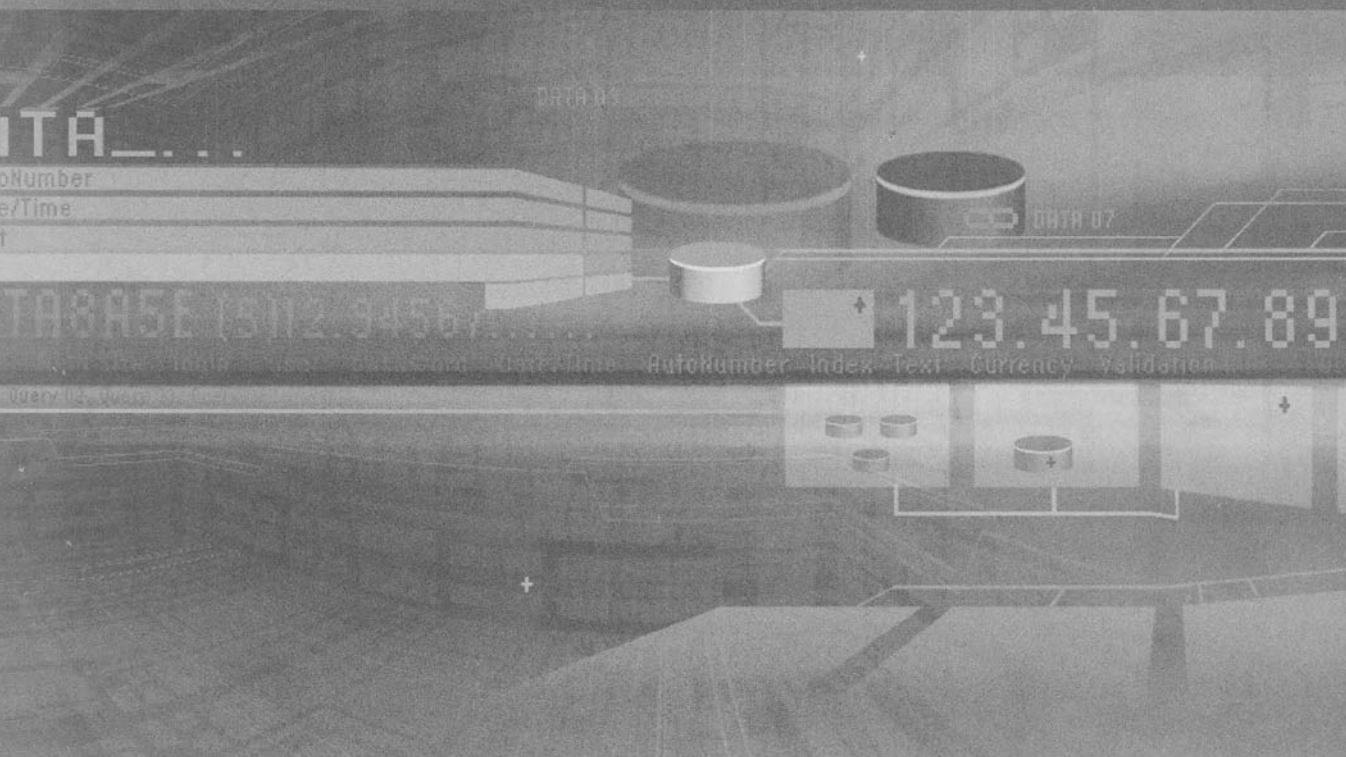
MySQL 257

Oracle 265

Microsoft SQL Server 271

Summary 278

Index **281**


DATA 05
DATA 07
123.45.67.89
AutoNumber
Index
Text
Currency
Validation

Introduction

"What is a web database?" you may be asking.

Answer: It's a database that's connected to the Web in some way.

"What's the difference between a web database and any other database?"

Another good question. We will be answering this, and many others, throughout this book as we look at how to design a database for use with web-based applications and web sites.

What's It All About?

Let's cut straight to the chase – this book will take you from knowing what a database is, to knowing how to design one effectively so that it makes good use of storage space, and provides fast and efficient data access.

The main consideration when designing a database for use with the Web is the length of time people are prepared to wait for their information. Visitors to an online shopping site will be more impatient than visitors to a shop – after all, they can see what's in the shop as soon as they walk in, but they have to wait until your site has downloaded before they can see what's there. Whether you're trying to sell things to people and using the database as a catalog, or you're offering an information service and using the database to provide the content, if it's going over the Web you'll need it to be available quickly.

Of course, there are ways you can do this with existing databases, but your site will work much better if you design the database from the beginning with the Web in mind. *"So how do we do that? Where do we start?"*

Read on, as we give you a solid beginning, from which well-designed databases will spring. We'll provide an introduction to databases for those of you who aren't familiar with them, then move swiftly on to look at the core aspects of a database, as well as the basic building blocks of your design. We take you through an example of planning a database, followed by designing and implementing it, before we move on to cover the all-important topic of security. There are sections on advanced techniques and database management to help you get the most out of your new database.

One thing to bear in mind is that, throughout this book, we have used standard code wherever possible. For databases, the standard is **SQL**, the **Structured Query Language** (pronounced "sequel"). As we'll see in *Chapter 2*, SQL has progressed through a number of stages, each adding more functionality. Unfortunately, database vendors don't always follow the standard as closely as other standards are followed, and the standard doesn't cover all aspects of databases.

Where there is departure from the standard, we'll warn you and pick a database to use for an example. In these cases, if the relational database management system you wish to use differs from the one in the example, then you may have to do a spot of research to work out how to alter the syntax so you can use it in your own projects. This has the advantage of making the book as useful as possible to as large a number of people as possible – even if the syntax in the examples is not identical to what you require, the principles will still be valid.

Who's This Book for?

This book is for you if you want to design (and build) databases for use with the Web. This book won't present platform-specific database implementations, but we will take you through the steps you need to follow to ensure that your database does the job you want it to do, and does it well.

This book is for you, whether you've never really used databases before, but want to start incorporating them into your web sites now or in the future, or you have built database-driven web sites before, but want to learn how to make them work better.

What Do I Need to Begin?

To get the most out of this book, you'll need access to a database. For the most part, it won't matter which one you use as we've chosen to present things in a standard manner. The *Appendix* provides details of three of the most frequently used databases: MySQL, Oracle, and SQL Server.

What's Inside?

As we've said, we're going to take you from perhaps knowing next to nothing about databases to being able to design efficient and effective databases for use with the Web.

Chapter 1 gives us a thorough introduction to what databases are, how they have evolved, and what forms they take. Fortunately, modern relational databases work better on hard disks than they would do on a huge tape reel.

In *Chapter 2*, we provide a brief on the key elements of database theory – no, don't run away so soon; we'll cover only those elements you'll need if you're going to do a good job.

Chapter 3 moves on from the theory and walks us through using SQL to get the most out of a relational database. Actually, you'll need SQL to get anything out of a relational database, as well as to put data into it, and even build it in the first place.

1

- Types of database
- Uses of databases on the Web
- Introducing the DBMS

Author: Allan Kent

In *Chapter 4* we take the basics of relational databases and tell you about the fundamentals of database design. This is where we can begin to apply the theory to a practical model, and see how we implement what seem like simple relationships.

In *Chapter 5*, we take an even more practical approach as we guide you through the preparatory steps we need to take before we even begin designing the database. After all, there's no point designing the perfect database for managing site content if you actually want a database to provide details of your e-commerce venture.

Chapter 6 moves us on from gathering information and makes a start on designing the database. Now we know what's going to be stored in it, we can make sure that we have all the right things in the right places.

In *Chapter 7*, we examine the critical issue of database security. Under the microscope are means of authentication, deciding who gets to do what with your database, ways to stop people running unwanted SQL in your database, and using error handling to stop people finding out how your database is structured.

In *Chapter 8*, we run through some advanced techniques, which will make your database a smoother and more efficient place. Why bother typing queries in by hand every time you want the information, when you can type them in once and save them in the database to be run later.

Chapter 9 looks at database maintenance, and what you can do to keep your web database in top condition. Up for discussion are how and when to back up your data, how to make changes to your data structure, and what to do with indexes to make them more efficient.

Finally, the *Appendix* talks about three of the main databases you're likely to come across: MySQL, Oracle, and SQL Server. For each, we look at departures from standard SQL, and what options they give us.

Support and Feedback

Although we aim for perfection, the sad fact of book publication is that a few errors will slip through. We would like to apologize for any that have reached this book despite our efforts. If you spot an error, please let us know about it using the e-mail address *support@glasshaus.com*. If it's something that will help other readers then we'll put it up on the errata page at *http://www.glasshaus.com*.

This e-mail address can also be used to access our support network. If you have trouble running any of the code in this book, or have a related question that you feel that the book didn't answer, please mail your problem to the above address quoting the title of the book (*Practical Web Database Design*), the last 4 digits of its ISBN (**1205**), and the relevant chapter and page number.

Web Support

You'll want to go and visit our web site, at *http://www.glasshaus.com*. It features a freely downloadable compressed version of the full code for this book, in both `.zip` and `.sit` formats. You can also find details of all our other published books, author interviews, and more.

Database Introduction

Anyone who has used a computer for a significant period of time will have heard of the term database. If you've spent some time on the Internet, you've interacted with one in some way, whether you were aware of it or not. If you've ever placed an order online, or participated in a discussion forum, or even just used a search engine, then you've experienced the power of a database.

In this, the first chapter of Practical Web Databases, we will be providing you with an introduction to the world of databases. We'll start off by taking a quick look at what a database is. We'll then spend some time giving you some background information on databases – when they started and where they come from.

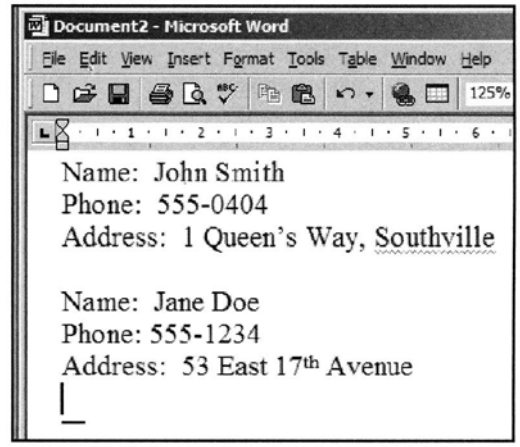
Once we've done with the history lesson, we'll look at the various types of databases that are out there, and what they are used for. We'll then look at what benefits you'll get by using a DBMS (database management system), and end the chapter by looking at how databases are used on the Web.

What Is a Database?

The simple answer is, "a database is anything that stores data", although we'll want to be more specific. If you've ever had to work on a computer in an office environment, you'll probably have spent some time typing information into a spreadsheet program like Microsoft Excel, Lotus 1-2-3, or if you are older, Borland Quattro-Pro. This method of storing information in a tabular format is one of the most basic ways of storing information. Sure, you could store your information by typing it all into a Microsoft Word document, but it wouldn't be useful. Consider the differences:

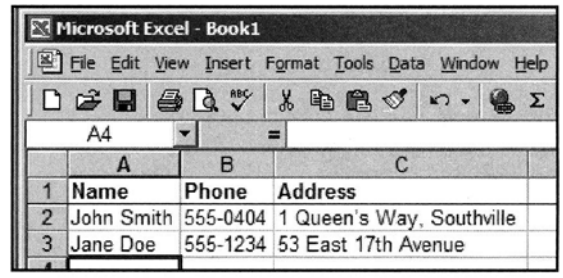
Microsoft Word:

By having your data sitting in paragraphs in your Word document, you could use Word's *Find* function to search for a specific word or phrase, and then look at the information that appears in the vicinity of that word, but there would be no fixed structure to your information, other than the structure you impose.



Data stored by paragraphs in Word

Microsoft Excel is a better option than Word and is closer to what we consider a database. The data is structured and laid out in such a way that we can sort the data easily by any column, and by scanning down the column we can quickly see where we have missing information or where we have similar records:



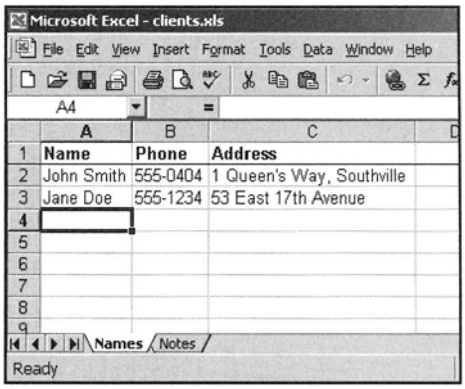
The same data stored in Excel – more structured

Look familiar? A lot of smaller companies get by with a system just like this. And it's fine if you don't have a lot of information, or you're the only person working with the information. The moment your information starts growing, or a co-worker needs to use the data at the same time, things start getting out of hand.

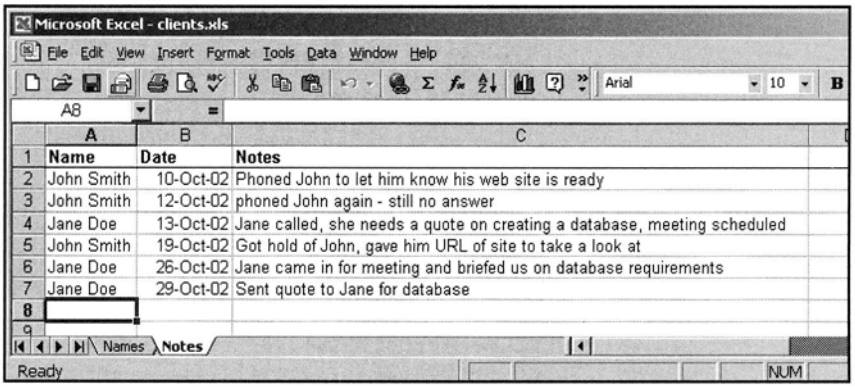
Suddenly you're struggling to find records, you're forgetting to update certain sections of your document or spreadsheet, your data is getting out of date, you've got lots of long entries that are mostly the same information, you're running out of dimensions for your data, and in your rush you're not entering the data in any consistent way.

Luckily there is a solution – store all that data in a database. This is where we need to be more precise about what a database is, so let's try, "A database is a program that manages multi-dimensional data for multiple simultaneous users". What does this mean, though? A database program is designed for the task of storing and managing data in an efficient manner; we tell it how we want the data structured, give it the data, and it will look after it. When we want to search the data, the database will provide us with an easy means to get what we want quickly. We can also easily add extra dimensions to our database. In our Excel spreadsheet our 2 dimensions were our rows and columns – columns were the fields for our information and the rows were our records.

If we wanted to add an additional dimension, for example including a list of notes for that person, as well as the date that that note was entered, we start running into a problem. Where do we store the notes? We don't know how many notes we will have over time, so we can't practically add them all to a single cell at the end of our row of information. So what we would do is create a new sheet and enter them in there:

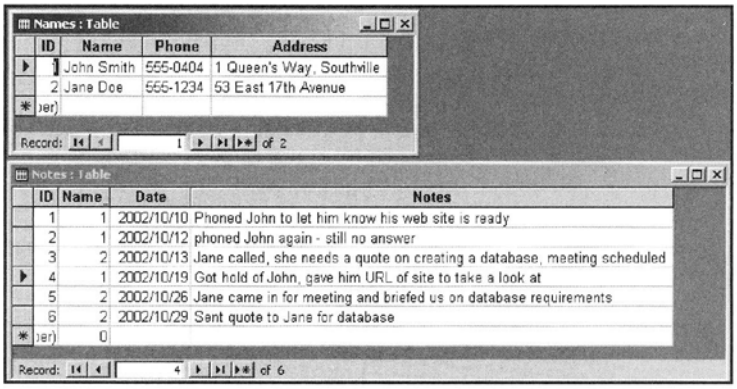


The notes are on a second sheet



The Notes sheet, complete with notes about our customers

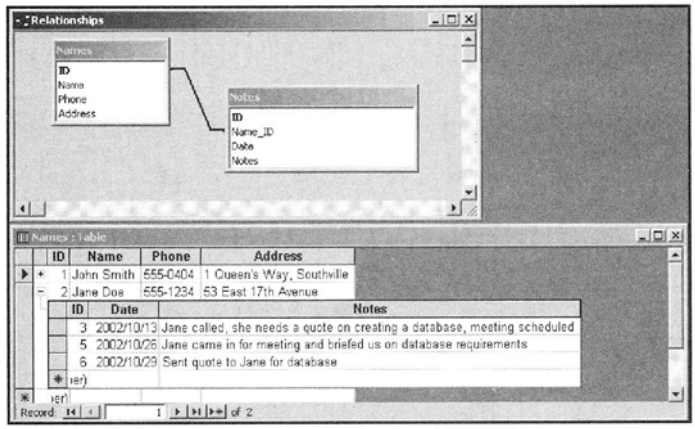
Our new **Notes** sheet now contains notes that pertain to that client. Excel has no practical way for us to relate the records in the **Notes** sheet with the client records in the **Names** sheet, so we are still going to have to sort the data and manually search through it to find notes for a specific client. Also notice how we have to duplicate the client name in each of the **Notes** records, taking up unnecessary space. By using a database we would simply relate the **Notes** to the client **Name** and the database would take care of it for us:



The first step – moving the data into a database (Access in this case)

All we have done here is create tables in Microsoft Access with the same information as we had in Excel. The only difference is that we have replaced the client name in the **Notes** table with the client ID from the **Names** table. So instead of Jane Doe, we use the number 2.

The next step is to relate the two tables together:



Relating notes to names is easy with a database

We've related the ID of the **Names** table to the `Name_ID` in the **Notes** table. If we now re-open the **Names** table, Access allows us to see all of the related notes for that client.

Using a database gives you the following advantages:

1. Your data will be structured and ordered
2. Many people will be able to work on the data at the same time
3. You will be able to search for related information more easily
4. You will be able to report on that information more easily
5. You will be able to build relationships between data in your database

The benefits far outweigh the effort required in learning how to set up and use a database. The rest of this book will help you to understand enough of the theory behind databases to be able to design efficient and effective databases for use with your web sites and applications.

A Short History Lesson

History was the first subject I elected to drop at school, so we'll keep this as short and as interesting as possible. We'll look at the different types of database model after seeing what became available and when.

In the 1960s we saw a development in the amount of data that a computer could store. In order to address the fact that a computer now had a large amount of data on it, people started developing new and more efficient ways to store this data. The data was stored in such a way that it could be processed easily, rather than concentrating on how it was structured overall. The systems were primitive in that there was no standard system or API for accessing the data, meaning that no two databases were the same. A user of the system would have to know how the database was structured on disk in order to input or retrieve any information. The two database models that we got from this era of computing were the Hierarchical and Network models. While these were efficient and practical in their time, they are seldom used nowadays.

The 1970s saw what is probably the most important jump in database theory. In 1970 a researcher from IBM, E.F. Codd, published a paper (*A Relational Model of Data for Large Shared Data Banks*), in which he proposed a relational model for databases. He outlined how, using calculus and algebra, a system could be created that would allow the storage and retrieval of large amounts of data. His paper spoke about an English-like language that people could use to interact with the database and introduced the storage of information in tables.

Besides the fact that he had defined the relational database model that is in popular use today, he started people thinking about the organization of the data within the database as separate from the way it is physically stored on disk.

This sparked two prototypes of relational database systems. The one, Ingres, using QUEL as a query language, led in time to the development of further relational database management systems from Ingres, Wang, and Sybase. The other, System R with the SEQUEL language, led to IBM developing DB2, and systems from Oracle and HP. At about the same time, Honeywell Information Systems were also developing a relational database system based on these principles, and in 1976 released the first commercial relational database system.

Also in 1976 Peter Chen proposed something called the Entity-Relationship model. This model looks at the database structure from the perspective of how the data is going to be used, rather than how the tables are structured. This model is still in use today, and you will take a look at this in *Chapter 2*.

Something that came out of the IBM group was a database language called **SQL**, the **Structured Query Language**. In the 1980s we start seeing commercial database systems built on the SQL standard. By the mid-1980s SQL was the de facto standard and database systems built on the Hierarchical and Network model were being used less and less. Big-name database systems from this period include DB2, Paradox, Dbase, and Watcom SQL.

During the 1990s all of the major players of the database world that we know today are getting in on the act – Sybase, IBM, Oracle, Informix, and later Microsoft, after buying the technology from Sybase. We also see the introduction of tools for rapid client application development such as Powerbuilder and Visual Basic, and the world was introduced to the client-server model. At about the same time we see the beginnings of Object Database Management Systems.

By the middle of the 90s we have the World Wide Web and the use of databases in this area is realized. Towards the end of the 90s we have all sorts of technologies for accessing databases from the Web – ADO (Active Data Object), ODBC (Object Database Connectivity), and JDBC (Java database connectivity). Through these technologies and their own native database functions, Perl, Java, Active Server Pages, Cold Fusion, and PHP emerge as web programming languages. With the continuing boom in Internet activity, companies begin to adopt online order processing, and databases are used for recording and analyzing buying trends. Also by this time we start seeing the emergence of open source alternatives such as PostgresSQL and MySQL. Now for the first time a relational database management system (RDBMS) is available to anyone who wants one, regardless of their purchasing power.

Currently we're seeing a huge growth in open source alternatives. PostgreSQL and MySQL are developing into true RDBMSs to challenge the big names. Not only is the software free, but it also runs on a free operating system (as well as most other operating systems).

Databases are being used for bigger and bigger projects. In the scientific field, Geographical Information Systems and exercises like the Human Genome project are exploiting the power of databases when analyzing huge sets of data. System R was initially tested with 8MB of data, which was a lot for the early 1970s. Developers at CERN are aiming to have a distributed database system up and running by 2005 that can store an Exabyte of data. An Exabyte is 1024 Petabytes. Still in the dark? A Petabyte is 1024 Terabytes, and a Terabyte is 1024 Gigabytes. It's 1 x 10^{18} Bytes. Which is a lot.

Database Models

Now that we've had a quick introduction to where databases have come from, let's take a look at the different database models that are out there. A number of these were mentioned in the history, and this is where we will explain how they work.

Hierarchical

The Hierarchical database model is the oldest database model and in the 1970s it served its purpose. Mainframes used tapes to store data and in this environment the hierarchical model was efficient. The logical model matched the physical model.

In the model, information is stored in a hierarchy of tables. All the relationships between the tables are parent/child relationships. In this relationship a parent can have multiple children, but a child can only have one parent.

Let's take a look at a hypothetical example. Suppose glasshaus, the publishers of this book, wanted to store their information in a hierarchical database. Since they have books on more than one topic or area of interest, we would need to categorize the books into divisions. They would also need to record what books were being worked on, and who was working on what book. Also very important would be to record some details about the authors, such as mailing address or banking details.

Let's see how that structure would look:

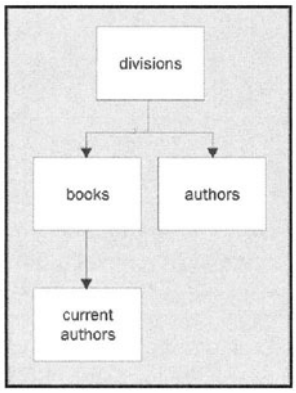


Table	Purpose
Divisions	The different divisions at glasshaus
Books	The books that are being written at the moment
Current Authors	The authors working on a specific book
Authors	All the authors available

If they wanted to get the details of all the authors working on this book, they would need to determine which division the book fell under. Then they would have to query the database to get the book details. Step down again with another query to get the identification or social security numbers of the authors working on this book. Once they have that they can then go back to the **Divisions** table, but this time query down to the **Authors** table to get the specific details.

All queries in a hierarchical database model must start at the top-level table in the hierarchy and work their way down. Not only is this tedious but anybody using the database would have to know exactly how the database has been structured. You will also have a large duplication of data in a database like this. If one author is working on more than one book, you have details of that single author attached to each of the books he or she is working on, scattered all over your database.

A good analogy of the way that a hierarchical database works is *Windows Explorer* – you start browsing your hard drive at *My Computer* and you have to browse to each folder before you can browse into the next one. A commercial example of a hierarchical database is IBM's IMS.

> *A good analogy of the way that a hierarchical database works is Windows Explorer*

As the need to store more complex data structures arose, more advanced database models followed. The next step was the Network Database model.

Network

As we saw, in a hierarchical database model you had to start at the top of the structure, and each child table could only have one parent. With the network model you can start querying the database at any table you wish, and move either up or down the hierarchy of tables. Not only that, but also by replacing the parent/child concept with one of association, a table could be linked to more than one 'parent'.

The name Network Model comes from this – the relationships between the data in the tables form a network of links.

Data in the Network Model is structured using Records, Record Types, and Set Types.

- A Record is a single row of data – the Record stores the attributes and the values of the data in this specific record.
- A Record Type is a collection of Records of the same type.
- A Set Type is a relationship between two Record Types. The nature of the Set Type makes it very efficient when running certain queries, but it is not very flexible.

Commercial products that used this database model were IBM's DL/1 and Computer Associates' IDMS.

Using the same example as we used for the hierarchical model, a Record Type would be one of the tables such as **Divisions** or **Authors** – these would be made up of individual Records. A Set Type would be a pre-defined query that we build into the database that relates Record Types. We could create a Set Type that relates books to divisions, and another Set Type that relates authors to books. Because we can build these Set Types between any Record Types in the database, it is far more flexible than the hierarchical model. We are however, still limited to using Set Types that have been created in the database.

Neither the Hierarchical nor the Network model is used today for database systems. You will still find them in use, but only as a way of accessing legacy data.

Relational

As mentioned in the history lesson, the Relational model is based on Edgar Codd's paper from 1970. In this model, the structure of the data and the way in which it is stored on disk, are independent of one another.

Data in a relational database is stored in the following way:

Database	**The top level of the structure. Made up of a collection of tables.**
Table	A table is made up of at least one record. Each of the records in a table will contain the same kind of information, for example author details.
Record	A record is made up of at least one field, and represents a single piece of information about something.
Field	A field contains a single value that represents a part of a record.

By way of an example, if we wanted to store information about authors, we would want to know their name, e-mail address, and billing information. Each of these things would be a field. By collecting all the fields together and entering some values in them, we would have a record of information that was specific to an author. We would then keep all of the records in a single collection, and that would be the **Authors** table. The **Authors** table would be one of the tables within the database.

As you would expect in a database model called Relational, the database maintains relationships between the tables. The strength of the model is in the types of relationships that it offers:

- One-to-one
- One-to-many
- Many-to-many

Since the Relational model is the model that you are most likely to encounter in the real world, let's take a moment to look at relationships. We'll come back to the relational model throughout the book, but this introduction will suffice for understanding the differences from the other models.

One-to-One Relationship

In a one-to-one relationship between table **X** and table **Y**, each record in table **X** can only have one matching record in table **Y**, and for each record in table **Y**, there can only be one matching record in table **X**.

This is not a common relationship type, as it would be far easier to just store the information from both tables in one single table, such is the nature of the relationship. If you had a table with a large number of fields, it may be more manageable to split it into two separate tables and link them with a one-to-one relationship. For example where only some records have additional material, it would make sense to split the table into a core information table, and an additional information table.

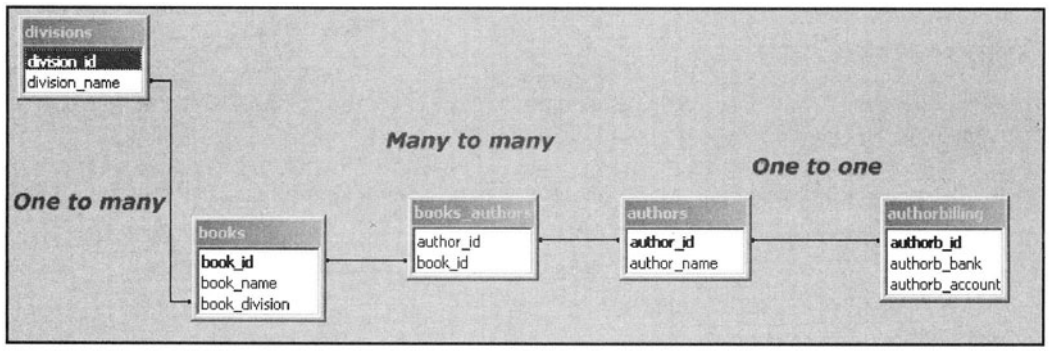
One-to-Many

A one-to-many relationship type is the most common that you will find. In this relationship, a record in table **P** can have multiple related records in table **S**. In our previous example we would link the divisions and books table with a one-to-many relationship. One division can have many books written for it, but a book would only fall under one division.

Many-to-Many

In a many-to-many relationship, any record in table **A** can have many related records in table **B**, and any record in table **B** can have many related records in table **A**. This form of relationship is made by creating a third table that links the two tables in the many-to-many relationship. Each of the two linked tables in this relationship is linked to the third new table by way of one-to-many relationships.

Let's take a look at these graphically:



- A single division can have many books – the divisions table is linked to books with a one-to-many relationship: **division_id** linked to **book_division**.

- An author is linked to his or her billing information in the authorbilling table by way of a one-to-one relationship (although remember these tables could be combined into one). **author_id** links to **authorb_id**.
- A single book will have multiple authors, and any one author may work on more than one book, therefore the many-to-many relationship between books and authors is created through the use of the **books_authors** table. The books table is linked to **books_authors** in a one-to-many relationship, and the authors table is linked to **authors_books** in a one-to-many relationship.

Object

At around the same time as object-oriented programming languages (like C++) were starting to appear, an interest grew in developing an object-oriented database system. The main reason for this was that certain organizations and companies needed to store large collections of data that did not fit well in a relational model. By integrating object-oriented programming features with database capabilities you get a database system where the database objects appear as programming language objects.

The concept of an object-oriented system is that the object itself is a collection of data and functions or methods that can be run on the data.

Because of this feature of the model, object-oriented database systems do not have a standard language like SQL that can be used to manipulate the data. The fact that the information is stored as an object results in a better storage method, and because the designer of the database defines the ways in which the object can be manipulated, there is very little opportunity for a user of the database to corrupt the information. Although it requires more effort and someone with specialist skills to set up an Object Database, the skills that are required are simply an ability to develop the object to begin with and being able to program in one of the object-oriented languages.

The next time you watch Formula One, look for the Computer Associates advertising on the McLaren cars. They use CA's Jasmine ODBMS to track the car's performance.

Object Relational

An Object Relational database model is simply an amalgamation of the Relational and Object database models. The model came about primarily for two reasons.

- There was a growing need for an effective way of storing large amounts of multimedia data in a database.
- Because the ways in which programmers structure their data and the way in which a database stores that data, are often not the same.

Programmers found themselves writing code that converted data between different formats every time they read from or wrote to the database. The Object model circumvents this problem by storing the data as an object itself, so when the data is read off the disk, it is already in the format that Java or C++ uses. This means that it is not a simple matter to move across to an Object model if you haven't planned to use it from the start. By creating an Object Relational model, people could carry on using a Relational model, but store the information in its native object form.

Multimedia Database

A multimedia database is one that, as the name implies, stores multimedia data (images, video, or audio) and is optimized for this kind of data. The media data is stored in the database along with metadata that describes the media. The users of the database query this metadata, and the media that the metadata describes is returned.

The problem of the sheer size of multimedia data still exists. Although MP3 and JPEG formats are compressed to some extent, most multimedia applications will want to provide media of a high quality. Because of this most multimedia databases have to be accessed via a high-speed LAN connection.

Distributed

All computers have a limit, and it is for that reason that the Distributed database system came about. While not strictly a database model, it is a database configuration that is used and therefore worth a look at.

The Distributed system is a way of adding extra computing power to your database without upgrading the physical machine. A number of smaller machines share the workload of a single database system and appear to the user of the database as a single unit. Each individual computer in the system is referred to as a node and is connected to the others by way of a network.

Distributed systems come in two flavors – homogeneous and heterogeneous.

A homogeneous distributed database system is one in which all of the nodes that make up the system are running the same database system and will have a common database layout. Some database management systems such as Oracle will support a structure such as this.

A common scenario for a setup such as this is where a high-traffic web site sets up a number of mirror servers that all contain the same information and application scripts. The request for information is routed to a non-stressed server for serving. There is an issue of replication – data integrity has to be maintained across all nodes in the system. Today's DBMSs that support homogeneous distributed database systems do have built-in fault tolerance and replication features.

A heterogeneous system is typically set up to support a legacy system already in place. Each of the nodes can run different database management systems and data is not normally duplicated across servers. Nodes are linked together to provide access to information that a given node may not have.

File Storage

In the strictest sense of the word, storing information in a file on the disk classifies it as a database. As long as the information is structured in some consistent way, you can call it a database.

This kind of 'database' is a lot more common than you would think. Flat-file format databases exist all over the web and drive many applications. Programming languages like Perl provide excellent text-handling functions, and people who do not have access to a database in the traditional sense can easily develop a system using a collection of flat format files.

One of the ways of storing information in a flat-file is to use a format known as comma-separated values (CSV). A single line, or record, in the text file is broken up into individual fields with commas.

```
Country,Currency Code,Capital City,Continent
United States,USD,Washington D.C.,North America
South Africa,ZAR,Pretoria,Africa
United Kingdom,GBP,London,Europe
```

The web application then has to read the file line by line, parse the line into fields, and only then can it decide what to do what that record of information.

XML

A flat-file format that warrants a special mention is XML (Extensible Markup Language). XML is a bit of a buzzword on the Internet at the moment and while it was designed to be much more than just a cheap alternative to getting a real database, you will often find it used in this way.

As a database format XML has the disadvantages that you would find with any flat-file formats – access to the data in an XML file is slow because the files have to be read off the disk and there is no compression or optimization in the way they are saved. This being said, there are some advantages to using XML over a plain text file. The nature of XML is such that it is portable across systems. Unlike plain text files, however, XML can describe the structure and type of data that is being stored. Because of this, XML is more human readable than CSV files.

Getting back to our original example, if we exported our client names as a CSV file, it would look like:

```
Name,Phone,Address
John Smith,555-0404,"1 Queen's Way, Southville"
Jane Doe,555-1234,53 East 17th Avenue
```

Compare this to the same data, but stored as XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<Names>
  <client name="John Smith">
    <telephone>555-0404</telephone>
    <address>1 Queen's Way, Southville
</address>
  </client>
  <client name="Jane Doe">
    <telephone>555-1234</telephone>
    <address>53 East 17th Avenue </address>
  </client>
</Names>
```

> *XML can describe the structure and type of data*

Due to its popularity, most programming languages have developed support for XML. Programming methods exist to parse XML documents, pinpoint specific information, add new XML records, and alter existing information. Through the use of XML and XSLT (Extensible Stylesheet Language Transformations) we can apply styles to the XML data and display it neatly in XHTML format.

This brings us rather neatly to another database model, the XML database.

For more information on XML, take a look at Practical XML for the Web, from glasshaus, ISBN 1904151086.

XML Database

An XML database, or native XML database, is a database that has been designed specifically to store XML documents.

An XML database allows you to perform queries using an XML query language such as XPath, so if your XML document was stored in an XML database, you could easily get it to return all documents that contain an italicized word in the opening paragraph.

XPath and XQuery
XPath is an XML query language that allows you to address a specific part of an XML document. Unfortunately the XPath query is not powerful enough to group or sort the query results. XQuery on the other hand is far more powerful but at the moment is read-only, so inserting and modifying information in the XML document is not possible. With time this functionality should be included. You can read about XPath and XQuery in more detail at http://www.w3c.org/.

There are many databases that can handle XML. Most database systems today are XML-enabled, in that they can import and export XML, and may have additional features for handling XML documents. None of the big-name database companies has brought out a native XML database yet. There are however a number of smaller companies that have both open source and commercial offerings. XDB from Zvon, Ozone, and Xindice from the Apache group are some of the open source options. For a commercial database there are XIS, Infonyte, and TeraText to name a few.

A native XML database has certain advantages over using a simple XML file in that it contains features of a Database Management System, and allows you to create XML document collections and use XML query languages such as XPath or XQuery. In the next section we will look at the DBMS in more detail.

DBMS

It's all well and good having a nice big database with lots of tables and relationships that's chock-full of data. But how do you manage it all? More interestingly, how do you manage more than one person working with the data at once? This is where a DBMS or Database Management System comes into play.

A Database Management System is a software application that is designed to store and manage databases. To the DBMS, a database is a specific collection of related tables that we have said makes up one database, which is likely to be one collection among many. Any database created using a DBMS is practically useless without it.

A DBMS provides functionality to both the end users and the administrators of the database. For the end user, the DBMS acts as a single point of entry or interface into any number of separate databases. The person on the other end of the browser or application doesn't care that the titles for the books they are ordering are coming from a database in Birmingham, UK, while the actual order is being sent somewhere in the US.

For the administrator of the database, the DBMS provides a lot more functionality. It provides security features to make sure that the right people can access the relevant sections of the database and the wrong people don't get to access anything. It also provides a front-end for the administrator to allow adding, editing, and deleting information in the database, and changing security settings. Most importantly it does most of its work in the background – logging access to the database, performing backups, and recovering from any faults.

By using a DBMS to manage your database you will also be optimizing your use of resources. Because a DBMS brings databases together, you can minimize data redundancy by having the same data stored only once within the system. Without a DBMS different departments within an organization would have had to maintain their own data. A vast majority of the data that would be stored would be common to each of the systems run by the individual departments. By pulling all of the databases into a central system, the duplicate data can be eliminated.

An obvious advantage of this is data integrity. When each department had its own database, they would have each been responsible for its upkeep and maintenance. If a client's details changed, someone would have to make sure that the details were changed in each of the systems. By storing common information in one place only, when one department makes an update, it is immediately updated for all the other departments.

The database management systems that we will be dealing with for the rest of the book are Relational Database Management Systems. There are vast numbers of RDBMSs available for you to choose from, Oracle 9, Sybase ASE, and Microsoft SQL Server 2000 being amongst the best known.

Databases on the Web

Now that we have dealt with some background information on databases, we can take a look at the why and where of using them on the Web.

First of all, in what situation would you want to use a database?

- To provide data that has already been stored in a database
- As the back-end, or storage component to a web application
- To store and manage constantly changing material

Existing Database Content

If you were in a situation where you had a large database of information that you wanted to make available on the Internet, you could just export all of the information to static HTML pages and publish the information in that way. By keeping it in its native database format, however, you would keep the functionality that is provided by a DBMS and you could make that functionality available to the users of your web site. It would be far more effective if users could search for information stored in your database, rather than having to browse around to find the exact information they need.

As we said earlier, if you had a library of documents stored in XML in a native XML database, users could make fairly advanced searches across your data by using the XML features inherent in that database. You could also reuse the existing data in many applications and, since it's all in one place, any updates need only be performed once.

> *...users could search for information stored in your database*

Web Applications

I am sure that most people have interacted with a web application at some point. Every e-commerce web site is driven from some database back-end. All the items that are available in the catalogs are simply records in a database somewhere. When you are searching for that CD or book, the e-commerce application is querying a database and returning only the results relevant to you. Compare that to some of the older web sites that have their catalog published as static HTML – you get to browse through categories (if you are lucky) and once you find the correct category, you have to browse through every item in that category until you find the item you want. E-commerce sites also store your customer information in the same database so that the next time you log in you won't have to fill it all in again.

Another popular use of databases on the Web is for collaboration systems. These can be as basic as a discussion forum or as advanced as a fully-fledged document management and approval system. Every time you post a message on a discussion forum, that message is being stored in a database somewhere. When you log into the forum with your username and password, the login script will check that username and password against a list of usernames and passwords in the database that is driving the forum.

Some web sites will use a database for very basic things. Preferences for what you want to see on the web site can be stored in a database and while it is a fairly simple thing to implement, it can make a huge difference to your appreciation of the site.

Many companies and organizations are using web-based systems for the day-to-day management of the company. Project management, cost estimates, invoicing, and billing can all be migrated into a web environment. These intranet systems run off a central database, thereby minimizing data redundancy. All of the company data is stored in one central place and cross-department reports can now be created without having to pull data from a number of different systems.

Dynamic, Changing Material

There are also web sites that do not require the special features of a database management system, but because their content is changing on a daily or hourly basis, it makes sense to drive the site from a database. Sites like CNN that provide news are a good example of this. In order to remain current and maintain a loyal subscriber base, these sites need to keep up to date with breaking news stories. By storing the content in a database the team of web editors can quickly add content to the site without having to mess around in an HTML editor.

An area that is seeing a lot of development is the field of Content Management Systems, or CMS. These are database-driven web sites where the client controls the maintenance of the information on the site through the use of an integrated authoring environment. Many of the features and requirements of a CMS, such as separating content and presentation, multiple users authoring for the web site at the same time, versioning control, and reporting, would not be possible without using a database in the back-end.

Take a look at *Content Management Systems*, from glasshaus, ISBN 190415106X for more information on CMS.

Databases can provide invaluable features and functionality to a web site, but when would you not want to use a database? Just because we can store the content in a database, doesn't mean that we should. While reading this book you will learn what is involved in designing a database for use with the Web. Once you have these facts in hand you will be able to weigh up the advantages that a database will give you against the amount of work that would be required in setting up the database and the scripts to access it.

Some tasks that can be implemented with a database can be achieved just as easily and with fewer overheads on your server by using an alternative. Using a database to store header and footer information for your web site is unnecessary as you could better achieve this with a server-side include.

Summary

During the course of this chapter we have taken a brief look at the background to databases and database theory. While background information and history is interesting, it is not essential in your quest to create web databases. The important facts to take away from this chapter are:

- Relational databases, the database model that is most prevalent, which we will explore in more detail through the rest of the book. Three different types of relationships connect tables: one-to-one, one-to-many, and many-to-many.
- XML, which was not strictly designed to function as a 'database'. That being said, for small tasks it does make a fairly useful alternative to implementing a fully-fledged database system.
- DBMS, a database management system, which provides a single unified front-end to our database system. The DBMS application not only brings our data together in one place, but also provides tools for the database administrator to manage the running of the database, the structure of the data, and the actual data itself.
- We use a database on the Web for content delivery, either from a static database or, in the case of a CMS, a dynamic system with frequent updates. We can also use a database as the back-end to any one of the multitudes of web applications that you will encounter on the Internet.

2

- How databases work
- Tables, columns, keys
- Data types, relationships, normalization

Author: Nigel Stanger

Core Database Concepts

Introduction

In *Chapter 1* we learned about the history of databases and database management systems (DBMS), where and when databases are useful (and just as important, *not* useful), and how databases are used on the Web. In this chapter we're going to look at some of the theory underlying modern database systems. Much of this chapter will look at the most common data model used by database systems today: the relational model. We'll introduce you to the core concepts of the relational model and how they interact, and show you examples of what relationally structured data look like. We'll also introduce some other important concepts that are applicable to any database, relational or otherwise. Here's a brief outline of what we'll be looking at:

- Data models: what are they and why are they needed?
- The relational model of data
  - Basic structures: tables, rows, columns
  - Domains, data types, and missing data
  - Identifying rows: primary keys
  - Linking related tables: foreign keys
  - Removing data redundancy: normalization
- Data integrity: maintaining the validity of your data
- Metadata: data about data
- Physical data access methods (indexes and the like)

But Is All This Theory Actually Useful?

> Those who are enamored of practice without theory are like a pilot who goes into a ship without rudder or compass and never has any certainty where he is going. Practice should always be based upon a sound knowledge of theory.
> *Leonardo da Vinci*

Unfortunately "theory" has become a bit of a dirty word with some database practitioners. It's not that unusual to hear comments like "Yes, yes, this theory's all very well for the academics in their ivory towers, but I have to build real systems, and, well, it's not very *practical*, is it?" Well, nothing could be further from the truth. How do you think you would react if the structural engineer who was designing a new building for you said something like this?

> *A well-designed database will be easier to maintain, expand, and extend, and will store data more efficiently*

Just like you can't build sound bridges and buildings without a solid understanding of engineering theory, you can't build sound databases without a solid understanding of database theory. Sure, you can design and build databases without that theoretical understanding, and they'll probably work most of the time, but look out if you push them too far. If you understand how the database is supposed to work then you stand a fighting chance of understanding the causes when it isn't.

Well-designed databases tend to scale up a lot better than poorly designed ones, meaning that as they get larger through holding more data they retain a good level of performance, something that's critical in the web environment. Take a growing online retailer, for example. Storing your catalog in a badly designed database isn't going to make too much difference to your clients when there are only a small number of items. When the catalog grows to ten times its original size your client is going to have to wait a lot longer to get the information they want from a search. A well-designed database will be easier to maintain, expand, and extend, and will store data more efficiently. Unfortunately a lot of web developers (and other database developers for that matter) don't have a good database theory background, so the databases they build don't perform as well as they could.

The other reaction that often occurs when someone hears the word "theory" is that their eyes glaze over and they think to themselves "Oh no, *theory*, it's going to be complicated and hard to understand". Again, the reality is often quite different. Many powerful mathematical and physical theories are quite simple in concept and relatively easy to understand. The concepts underlying the relational model in particular are remarkably simple, yet the model is extremely powerful and flexible.

This shouldn't be taken to imply that designing and building relational databases is easy! It can be very easy to design and build small, single-purpose, single-user databases (you only need to look at the various database construction "wizards" in Microsoft Access to realize this). Designing and building large, complex databases that can scale to large numbers of users and high transaction rates, on the other hand, is difficult. It all depends on the complexity of the environment that the database has to deal with. We'll look at some principles of effective database design in *Chapter 4*.

What Is a Data Model?

In the *Introduction* to this chapter, we said that the relational model is "the most common data model used by databases today". What do we mean by a "data model"? Unfortunately, there are two slightly different but widely accepted meanings for the term "data model":

1. "A data model is a formalized method of structuring and manipulating data."
2. "A data model is a structured description of some real-world phenomenon."

Data models of the first type are effectively the basis for building database management systems. They specify how a DBMS should represent data internally, and how it should present data to users and application programs. The relational model is an example of this kind of data model. This meaning tends to be used mostly by database theoreticians and DBMS implementers.

Data models of the second type are the basis for building an individual database or information system. They define the database structures that are needed to solve a particular problem, and the internal relationships between these structures. An entity-relationship diagram describing a particular library database is an example of this kind of data model. This meaning tends to be used more by database designers and systems analysts. You may occasionally see data models of this type referred to as "**schemas**".

In other words, the first type of data model defines the generic infrastructure that we can use to build *any* database; the second type of data model defines the structure of a single *specific* database. A real-world analogy is the difference between building regulations and the plans for a particular building. The regulations set up the framework within which buildings are constructed. They define the generic rules for constructing any kind of building, and are analogous to the first type of data model described above. The detailed floor plans of a specific building are analogous to the second type of data model.

Obviously we need to be very clear what we actually mean when we say "data model". This chapter and *Chapter 3* use the first definition (a formalized method of structuring and manipulating data), but we may use the second definition in later chapters when talking about a specific database.

So, to return to our original question, what is a data model? The most widely accepted definition of a data model amongst database theoreticians is the following:

*A **data model** is a formal framework for structuring and manipulating data. Every data model should have the following three aspects*

- **Structure**: Some way of imposing structure on data.
- **Integrity**: Some way to ensure the consistency and validity of data.
- **Operations**: Some way to manipulate data.

If you look at the three aspects listed above you'll see that they each address a different aspect of data storage. **Structure** tells a database management system (DBMS) how data is to be represented; **integrity** tells a DBMS how to prevent invalid data from being entered into a database (or existing data being made invalid); and **operations** provide ways to add, delete, modify, and retrieve data.

A particular advantage of "modern" data models is that they reduce our need to know the physical representation of data. Back in the 1960s there were no database management systems as such, so if you wanted a database, you had to write all the data management code yourself. To do this you needed intimate knowledge of how the data were physically stored: where the files were, what they were called, and their internal structure. Flexibility wasn't on the menu.

It's not that these early databases had no data models at all; they did, but the data models they used were very low-level and closely tied users and application programmers to the physical location of the data. The solution was to layer a kind of "virtual" or *logical* data model on top of the physical representation. Logical data models hide all of the physical details from users and applications, leaving those up to the DBMS to deal with. This is called physical data independence, which is a fundamental concept underlying all database management systems.

Physical data independence means the developer only has to know how to access and control a DBMS. In theory it's possible to totally change the physical data storage representation without having to rewrite any of your application programs. This is because they are written to the logical data model, which hasn't changed. The DBMS vendor will have to define a new mapping from the physical representation to the logical data model, but that's their problem, not the programmers'.

Another nice side-effect of physical data independence is that it suddenly becomes much easier to create a general query language, because you don't need to know where, or how, the data are stored physically, just where they are in the logical structure. Query languages fall into the operations aspect of data models, and we'll talk about them in *Chapter 3*. In particular, we'll look at a language called SQL (Structured Query Language), which is used by all modern relational DBMS products.

The Web was originally intended to have physical data independence of a sort. URLs (Uniform Resource Locators) were never really intended as something that end users would ever see, because a URL is effectively a physical path to the location of a web page (unless it's dynamically generated). If we move a web page, its URL changes. Instead of URLs, we were supposed to use something called a Uniform Resource Identifier (URI), which was a logical identifier that never changed, and independent of the physical location. Unfortunately the Web took off so rapidly that URIs got lost in the dust. They are starting to make something of a comeback with XML, however.

> *Most current DBMS products are now based on the relational model*

Three major logical data models were invented during the late 1960s and early 1970s: the hierarchical model, the network model, and the relational model. As we saw in *Chapter 1*, the hierarchical and network models tended to be inflexible and difficult to use, and were superseded by the relational model in the early 1980s. Most current DBMS products are now based on the relational model (giving us the term "relational DBMS" or "RDBMS"), so this is what we'll be spending much of this chapter looking at.

The Relational Model of Data

Like a lot of things in the computing industry, the relational model is not new. In fact, it celebrated its 30th birthday in 2000. The fact that the relational model has lasted this long says a lot about its underlying strengths.

A mathematician called Edgar Codd invented the relational model. While working at IBM in the late 1960s, Codd saw at first hand the problems caused by the lack of physical data independence. Some logical data models had already been developed by this stage, notably early forms of the hierarchical and network models, but they were all ad hoc solutions without any real formal basis. This tended to make these models inflexible and difficult to use. Mathematicians are used to precise formal models, so the ad hoc nature of these other models bothered Codd considerably. He therefore set out to develop a logical data model with a formal mathematical foundation. He chose set theory and predicate logic as his infrastructure and worked from there, eventually introducing his model to the rest of the world in his paper "A relational model of data for large shared data banks", published in the July 1970 issue of *Communications of the ACM*. Check it out if you're interested – it's surprisingly easy to read and understand. A partial version is available online at *http://www.acm.org/classics/nov95/*.

Things sort of snowballed from there. Other researchers began extending and improving the original model, and IBM began to show interest in building commercial products based on it. This interest led to the development of the System R prototype during the mid-1970s, and eventually to IBM's SQL/DS product in 1981. This eventually evolved into IBM's flagship relational product, DB2. IBM weren't the first company to come out with a relational DBMS product though; a small company called Relational Software, Inc. beat them to it in 1979. You may not recognize the name, but you've almost certainly heard of them. Their product was called Oracle.

There was an explosion in relational products during the 1980s (DB2, Oracle, Sybase, Informix, and many more). They developed an early reputation as being huge resource hogs, but these problems were rapidly solved by improvements in both software and hardware, and now most commercial databases are managed by a relational DBMS of some sort, whether it's MySQL serving a small business web site on a Linux box or Oracle running a giant corporate data warehouse on an IBM mainframe. Here's a short list of some well-known modern relational DBMS products and their vendors:

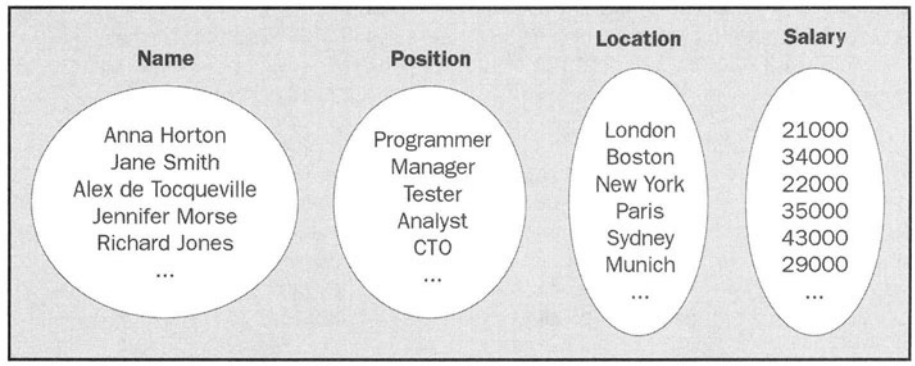
- Oracle9i, Rdb (Oracle)
- DB2, Informix (IBM)
- SQL Server, Access (Microsoft)
- Sybase, SQLAnywhere (Sybase)
- Interbase (Borland/open source)
- MySQL, PostgreSQL (open source)
- Teradata (NCR)
- ...and many more

We'll spend the rest of this section looking at the structure of the relational model. We'll look at integrity after that, and operations will be covered in *Chapter 3*.

The Basics: Tables, Rows, and Columns

You may remember us saying just before that the relational model is mathematically based. Don't panic! You don't need a math degree to understand and use the relational model, any more than you need an engineering degree to drive a car. The structural aspect of the relational model has only a few major concepts. There are formal mathematical definitions of all these concepts, but you don't need these to understand what's going on.

The relational model is, not surprisingly, based around a mathematical concept known as a **relation**. Suppose you have a bunch of sets, just like the ones you learned about in math class. Sets by definition can only contain values of a particular kind. Let's say you have a set of employee names, a set of employee positions, a set of employee locations, and a set of employee salaries, for example:



These sets are all obviously related to each other, in that each set contains values that represent a particular property of an employee, and each individual employee will have one value from each of these sets. For example, our first employee might have the name Jane Smith, the position Manager, a location of London and a salary of 35000. The second employee might be Richard Jones, Analyst, New York, 34000, and so on.

If we continue in this way we end up with a table-like structure, where each column corresponds to one of the sets of values mentioned above, and each row corresponds to an individual employee. That is:

EMPLOYEE			
Name	**Position**	**Location**	**Salary**
Jane Smith	Manager	London	35000
Richard Jones	Analyst	New York	34000
Anna Horton	Programmer	London	22000
Alex de Tocqueville	Programmer	Paris	21000

This entire structure is a relation, but in general usage we usually just refer to it as a "table" (for obvious reasons). There are two important things to note at this point:

1 When visualized as a table, relations look very similar to spreadsheets, in that they have a collection of rows and columns and data values are stored in the "cells" of the table, but this similarity is only superficial. Relations are fundamentally different from spreadsheets and the two must not be confused.

2 If you've had any previous exposure to database or systems analysis concepts, you may have heard of something called a "relationship". This is not the same thing as a relation, and the two should never be confused (yet another example of very similar terms with different meanings). Relationships can be used to link associated relations, and we'll tell you how to do that in a little while. To avoid any further confusion, we'll use the term "table" from now on.

Let's look at the previous table in a little more detail. Here are some interesting details to note about this table, and about tables in general:

- The table has a name: EMPLOYEE.
- It has four columns called Name, Position, Location, and Salary, each containing a piece of data on a particular employee. Each value in a column is drawn from a set of values associated with that column. This is called the domain of the column; we'll come back to this in the next section.
- Each "cell" in the table contains only a single value. For example, there are no employees that have multiple positions. This is actually a requirement of the relational model: columns are only allowed to contain single values. (This is actually the definition of something called first normal form, which we'll come back to when we talk about normalization later in this chapter.)
- The table can have as many rows as you need. The table above happens to have just four rows, but it could just as easily have four billion or none at all.
- The table can also have as many columns as you need (within the limits of the RDBMS product you're using). A large number of columns is unusual though, because columns represent properties or attributes of the things we're modeling (see below).
- No two rows are identical; that is, they all have at least one value different (unlike a spreadsheet that may contain duplicates). This is because each row represents a different instance of the things that we're storing data about. This is an important feature of the relational model, and we'll come back to it later when we talk about primary keys.
- There's no requirement for the rows to be in any particular order. We could shuffle the rows around, but it would still be the same table because the data values haven't changed (unlike a spreadsheet, where moving rows about can destroy the data). As long as we keep all the values in a row together, everything is fine because we're not changing the values for any one instance of the things that we're storing data about.
- Similarly, there's no requirement for the columns to be in any particular order. We could swap the positions of the Name and Salary columns, but it would still be the same table because once again, the data hasn't changed. Note that we're swapping the positions of entire columns, not just individual data values. If we did that the data would very likely end up making no sense (an employee might end up with a salary of "Paris" or a name of "43000").

If we want to be formal, we should really refer to tables as **relations**, rows as **tuples**, and columns as **attributes**, but in this book we will stick to the common names. Database theoreticians tend to get all bent out of shape when people use these terms incorrectly, but for all practical purposes they're interchangeable. Note however that the terms "file", "record", and "field" should **not** be used as equivalents to "table", "row", and "column". That's because files, records, and fields are really physical data structures, whereas tables, rows, and columns are logical data structures (there's that physical vs. logical thing again). Most modern RDBMS products don't guarantee a one-to-one correspondence between logical and physical data structures; for example, a logical table might be split across several physical files to improve performance. Because of this, "table", "row", and "column" are used regularly in the industry, whose lead we will follow.

> *The columns of a table correspond to properties that we want to store in the database*

Each table in a database usually corresponds to something in the real world that interests us. These might be physical things like employees, customers, products, or buildings; or they might be more abstract things like orders, sales, invoices, or contracts. The columns of a table correspond to properties that we want to store in the database, like names, salaries, prices, addresses, invoice numbers, etc. That's one reason why most tables usually don't have a huge number of columns, but we'll see a more important reason in the section on *Normalization*. Each row in a table corresponds to a unique occurrence of the thing we're storing data about, like "employee Jane Smith" or "contract number KN-116-7252".

Here are some more fairly typical examples of tables:

PRODUCT				
ProductNo	**Description**	**ReorderLevel**	**Price**	**QtyInStock**
AQX88916	Flush widget	1000	23.95	4937
AAD62726	Left-handed doodad	0	4.95	10673
FGE91822	Grunge nut	5000	0.50	155016
AHL46785	Flange bracket	25000	1.35	51745
DHU69863	Self-righting socket	5	2.37	52

CUSTOMER				
CustomerNo	**First**	**Last**	**Address**	**CreditLimit***
4649-4673	Richard	Johnston	14 West Avenue	1000
1166-3461	Amelia	Waverley	995 Forth Street	*<null>*
7671-3496	Antonio	Gonzales	55B Granary Lane	*<null>*
6794-1674	Diane	Adams	364 East Road	1500
1113-7741	Wayne	Jones	42 York Street	*<null>*

SALE						
SaleNo	**SaleDate**	**CustomerNo***	**ProductNo**	**Qty**	**Amount**	**Salesrep**
12345	Aug 12 2002	4649-4673	AQX88916	1	23.95	Dave Williams
12346	Aug 12 2002	1113-7741	AQX88916	7	167.65	Sara Thompson
12347	Aug 13 2002	1166-3461	AHL46785	3705	5001.75	Li Qing
12348	Aug 13 2002	*<null>*	DHU69863	50	118.50	Sara Thompson
12349	Aug 14 2002	1166-3461	DHU69863	940	2227.80	Sara Thompson
12350	Aug 14 2002	7671-3496	DHU69863	42	99.54	Sara Thompson
12351	Aug 14 2002	6794-1674	AQX88916	55	1317.25	Dave Williams

(* We'll explain what the *<null>* entries mean in the next section.)

It can be a bit cumbersome to draw an actual table when we want to say what, for example, the **CUSTOMER** table looks like. One common notation is the following:

```
EMPLOYEE(Name, Position, Location, Salary)
PRODUCT(ProductNo, Description, ReorderLevel, Price, QtyInStock)
CUSTOMER(CustomerNo, First, Last, Address, CreditLimit)
SALE(SaleNo, SaleDate, CustomerNo, ProductNo, Qty, Amount, Salesrep)
```

OK, that covers the basic relational structures. Don't worry about the `CustomerNo` column appearing in both tables above; we'll come back to this point after we've had a closer look at the actual data values inside the tables.

Domains and Data Types

When we store data in a database, everything ultimately gets stored as bits on disk, but this isn't a particularly helpful representation for general use. Real-world data comes in many different forms, but the three most common types of data we come across are numbers, dates, and text. All DBMS products support a collection of **data types** that correspond to these three basic types of data, plus usually a few extra data types for things like True/False values, images, video, audio, and so on. Data types tell the DBMS how it should store and (just as important) return the data, while at the same time hiding the physical representation from users and applications, just like most programming languages.

What data types would you expect to find in a RDBMS? For numeric data, you can reasonably expect at least integer and decimal types. There should be a date data type of some sort, which often incorporates the time as well, but you're just as likely to find a separate data type for date and time values. For text, there should be some sort of character string data type (either fixed or variable length, or both). Other data types you may find include:

- Time intervals, such as "three days, six hours and forty-five minutes" or "four score and seven years ago".
- Raw binary data, often known as binary large objects or BLOBs, which are used for storing such things as word processor documents in their native format, or more commonly images.
- Large amounts of text data, usually called character large objects or CLOBs, and intended for storing text that might be several megabytes in size, or, more recently, structured text like XML documents.
- Booleans or True/False values.

Every RDBMS supports a language called Structured Query Language (SQL), which has a well-defined set of standard data types. We'll look at some of these in a little more detail in *Chapter 3*.

Technically, data types are not part of the relational model at all. Instead of saying that each value is of a particular data type, the relational model says that all the values in a column are drawn from a well-defined collection of values. Think back to the previous section where we built a table by pulling values from different sets. Each of these sets specifies the allowable values in a column and is called a **domain**. A domain can be anything from "any alphanumeric character string" through "any integer greater than 4000" to "any one of the values [1, 42, 'table', ?, 'July 12 1937']". The last example, while allowed by the relational model, is unusual in practice because it contains values of many different data types. It's more normal for the domain of a column to contain values of only a single data type.

It turns out that not many RDBMS products directly support domains, including big names like Oracle. This isn't because domains aren't useful; it's probably more because domains weren't included in the products when they were first created, and you can achieve most of the effect of domains with constraints (which we'll cover in *Chapter 3*) or other data entry mechanisms. As a result, there's no strong incentive to add them now. Those RDBMS products that do support domains only support restricted single data type domains like "salary is an integer greater than 20,000".

Missing Data: Nulls

At some point while entering data into a database, you're going to hit a situation where you don't have a value to put in a column. Two reasons for this are:

1. You don't to know what the value is; for example, the customer didn't provide their e-mail address, either because they forgot, or they didn't want to. This is generally referred to as "value unknown", "value missing", or "value not supplied".
2. The value never existed in the first place; for example, the customer isn't on the Internet and therefore doesn't have an e-mail address. This is generally referred to as "value inapplicable". Other examples include maiden names for males, middle initials for people with no middle name, and commission rates for employees who aren't on commission.

What do you do when you have missing or unknown data? If you were filling out a paper form, you would just leave the field blank. What's the equivalent of this in a database?

Of course, an empty, blank, or zero-length string ("") doesn't mean there is no value – it means that the value is a blank string, a fact which is more important in programming than it is in the real world. Since databases have to deal with programming languages they need a way to represent this difference.

If we're dealing with text data like e-mail addresses, then we could just insert an empty or blank string. That might work fine for text, but what about numbers? There's no such thing as an "empty" or "blank" number. In the past it was common practice to use numbers outside the valid range to represent missing values (such as -1 for monetary values like prices). This is fine until you want to do something like work out averages or sums, and it's no use at all for columns that can contain any number. Dates are just as bad as numbers: there's no such thing as an "empty" or "blank" date, and the trick of using an invalid date falls apart if we need to subtract one date from another to calculate the time interval between them..

What we need is something that can be used with any data type to indicate that no value has been stored; that the value is unknown or missing. We can't use blank or zero, so we have to invent something else. Fortunately, Codd thought of this when he was developing the relational model, and he introduced the concept of **null**. You'll usually see this referred to as "null value", but that's technically incorrect, because null isn't actually a value. Null is a special flag that you put in the database in place of an actual value. With null we can make anything "unknown" when we need to. For example, returning to our **SALE** table from before:

SALE						
SaleNo	**SaleDate**	**CustomerNo**	**ProductNo**	**Qty**	**Amount**	**Salesrep**
12345	Aug 12 2002	4649-4673	AQX88916	1	23.95	Dave Williams
12346	Aug 12 2002	1113-7741	AQX88916	7	167.65	Sara Thompson
12347	Aug 13 2002	1166-3461	AHL46785	3705	5001.75	Li Qing
12348	Aug 13 2002	*<null>*	DHU69863	50	118.50	Sara Thompson
12349	Aug 14 2002	1166-3461	DHU69863	940	2227.80	Sara Thompson
12350	Aug 14 2002	7671-3496	DHU69863	42	99.54	Sara Thompson
12351	Aug 14 2002	6794-1674	AQX88916	55	1317.25	Dave Williams

The `CustomerNo` for sale 12348 is null: it has no value. Perhaps it was a cash sale, for which there wouldn't be a customer number (the value is inapplicable). Similarly in the **CUSTOMER** table, not all customers are allowed credit:

CUSTOMER				
CustomerNo	**First**	**Last**	**Address**	**CreditLimit**
4649-4673	Richard	Johnston	14 West Avenue	1000
1166-3461	Amelia	Waverley	995 Forth Street	*<null>*
7671-3496	Antonio	Gonzales	55B Granary Lane	*<null>*
6794-1674	Diane	Adams	364 East Road	1500
1113-7741	Wayne	Jones	42 York Street	*<null>*

dit limit. We can't give them a zero credit limit because this implies that they actually *are* allowed credit (this may seem somewhat picky, but it's an important distinction from a business perspective). As in the previous example, the `CreditLimit` value is inapplicable for some customers, so we use nulls instead of actual values.

Be aware that some methods for displaying data tables will display a blank entry for nulls, while others, as here, display '*<null>*' or something similar. It's important to know how your RDBMS deals with nulls and setting up tables; you will most likely have to specify whether a column is allowed to contain nulls. Trying to insert nulls into a column that doesn't allow them will cause problems.

> *It's important to know how your RDBMS deals with nulls*

A side-effect of the "not a value" idea is that a null can't be manipulated in any meaningful way with "real" values. For example, the question "is null > 2000?" is meaningless; we might just as well ask "is fish > 2000?". We'd get the same result if we asked "what is null + 2000?". The usual answer to such questions is "unknown", or sometimes "null". Following this through to its logical conclusion, we find that the question "is null = null?" is just as meaningless as any other question involving nulls. This sounds a little weird, but makes sense once you stop thinking of null as an actual value.

You should be careful when using nulls in your database, because they can have weird side-effects on any sort of expression, especially logical ones that return a True/False value. Also, standard built-in functions like `AVERAGE` (or `AVG`) and `SUM` will generally ignore nulls (as the SQL standard says they should). As a result, if you call the `AVERAGE` function expecting it to include all rows, you might not get the result you expect. Nulls are also awkward for application programs to deal with, as they must first test whether a column is null before attempting to read from it.

To complicate the issue further, some RDBMS products may not support nulls at all. One solution with these products is to define a separate "valid" column for every column that can contain nulls. These "valid" columns indicate whether or not the associated column contains a valid value. Effectively you're simulating null by using an extra column in the table. For example, we could add an `AllowedCredit` column to the **CUSTOMER** table that contains the value `0` or `N` if the customer is not allowed credit, and `1` or `Y` if the customer is allowed credit. Applications checking the table could find out whether or not to ignore the value of `CreditLimit` by first checking the value of `AllowedCredit`.

What this all boils down to is that if you need to use nulls, you should always check the documentation for your RDBMS carefully to find out how it handles them (if at all).

An alternative to nulls is to use **default values**. That is, if we don't supply a value, the RDBMS fills it in with some default that we've specified earlier. Returning to the sale example above, we could create a dummy customer row for "Cash Sale", then use its customer number as the default value for `CustomerNo` in **SALE** (this may or may not be a good solution depending on how the business works). As to which you should use, null or default value, it really depends on what you're storing. If you can come up with a sensible default value, then it probably makes more sense to use that and avoid the extra complications that nulls introduce. If there's no obvious default value or a default value just doesn't make sense, use null. Of course, if there's no possibility of values being missing, then you don't have a problem.

Identifying Rows: Primary Keys

We said earlier that an important property of a table was that no two rows were identical. This is actually a fundamental property of any table. Think about it: if you had two identical rows, each representing a different real-world thing, how could you tell them apart? You couldn't. That's why the relational model requires every row to be different, even if by only one value.

The relational model takes this concept one step further, by saying that every table **must** have a column or collection of columns that can be used to uniquely identify each row. This unique identifier is called the **primary key** of the table. The primary key can be a single column (like social security number), or it can be the concatenation of several columns (such as order number and product number). Single-column primary keys are generally preferable, but there are situations where concatenated or **composite** primary keys occur naturally – see *Chapter 4* for more on composite primary keys

Although the relational model says that all tables must have a primary key, many RDBMSs do not require this.

Let's go back to the **PRODUCT** table that we saw earlier:

PRODUCT				
ProductNo	**Description**	**ReorderLevel**	**Price**	**QtyInStock**
AQX88916	Flush widget	1000	23.95	4937
AAD62726	Left-handed doodad	0	4.95	10673
FGE91822	Grunge nut	5000	0.50	155016
AHL46785	Flange bracket	25000	1.35	51745
DHU69863	Self-righting socket	5	2.37	52

Looking at this table, `ProductNo` is almost certainly a unique product code, so we could just use that as the primary key. We could maybe use `Description`, but there's no guarantee that it will stay unique (this is true of most names or descriptions), or even remain unchanged; primary key values shouldn't change.

Let's look at another example:

EMPLOYEE			
Name	**Position**	**Location**	**Salary**
Jane Smith	Manager	London	35000
Richard Jones	Analyst	New York	35000
Anna Horton	Programmer	London	22000
Alex de Tocqueville	Programmer	Paris	21000

This one is a bit trickier. `Position` by itself is out because "Programmer" appears twice. The same is true of `Location` and `Salary`. `Name` appears to be unique, but if you think about it there are lots of people with the same name, so that's probably not a good choice either. Hmm. We could concatenate all of the columns, making the primary key value for Anna Horton's row: "Anna HortonProgrammerLondon22000". This however makes searching for the data significantly harder as the RDBMS has to look at all the columns of the table instead of just one. Another reason for not using all the columns to make the primary key is that, unlikely as it may seem, there could just conceivably be two programmers in London called Anna Horton, each on 22000.

A better solution would be to invent an artificial unique identifier for each employee (perhaps an employee number, should such a thing not exist already). These "invented" primary keys are known as **surrogate keys** and will be covered in *Chapter 4*, where we will learn that they are often automatically generated by the RDBMS.

Every table has exactly one primary key, but that doesn't mean that a table must have only one unique identifier. For example, we might invent an employee number for the **EMPLOYEE** table, and also store the social security number. Either of these is a unique identifier for employees, but only one can be the primary key. The primary key is the "most important" unique identifier; the others are usually called **alternate keys**. From now on we'll underline the primary keys in our tables.

We'll say a little more about primary keys when we talk about data integrity below, but that's enough for the moment. Now let's look at how we can relate tables to each other.

Linking Related Tables: Foreign Keys

Things in the real world are related to each other in various ways: customers place orders, managers supervise employees, and books have authors. For a database to be useful, the same idea should apply to tables as well, so we need some way to link related tables to each other. We've actually already seen two examples of this (did you notice?) in the **SALE** table:

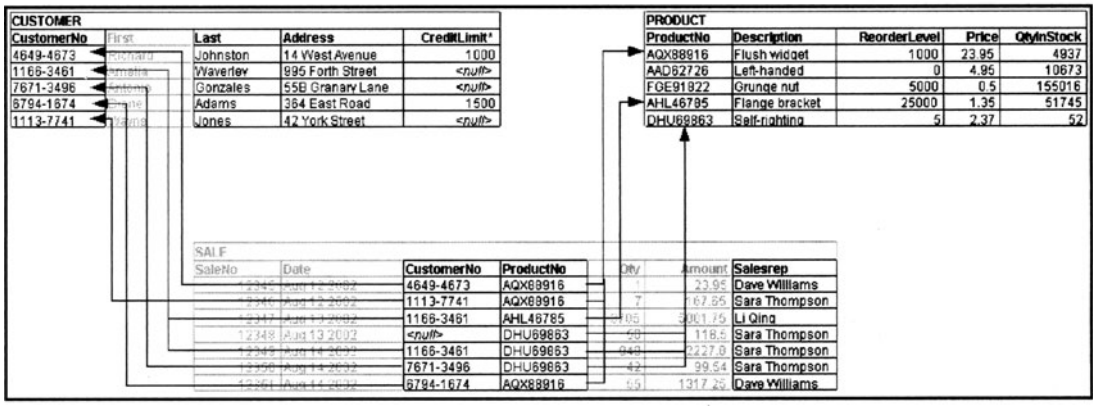
SALE						
SaleNo	**SaleDate**	**CustomerNo**	**ProductNo**	**Qty**	**Amount**	**Salesrep**
12345	Aug 12 2002	4649-4673	AQX88916	1	23.95	Dave Williams
12346	Aug 12 2002	1113-7741	AQX88916	7	167.65	Sara Thompson
12347	Aug 13 2002	1166-3461	AHL46785	3705	5001.75	Li Qing
12348	Aug 13 2002	*<null>*	DHU69863	50	118.50	Sara Thompson
12349	Aug 14 2002	1166-3461	DHU69863	940	2227.80	Sara Thompson
12350	Aug 14 2002	7671-3496	DHU69863	42	99.54	Sara Thompson
12351	Aug 14 2002	6794-1674	AQX88916	55	1317.25	Dave Williams

What do you think the `CustomerNo` column is doing here? Here's a clue:

CUSTOMER				
CustomerNo	**First**	**Last**	**Address**	**CreditLimit**
4649-4673	Richard	Johnston	14 West Avenue	1000
1166-3461	Amelia	Waverley	995 Forth Street	*<null>*
7671-3496	Antonio	Gonzales	55B Granary Lane	*<null>*
6794-1674	Diane	Adams	364 East Road	1500
1113-7741	Wayne	Jones	42 York Street	*<null>*

Do you see what's happening? We've taken the primary key of **CUSTOMER** (`CustomerNo`) and embedded it into **SALE**. When we look up a sale, we can grab the value of `CustomerNo` and use it as a primary key value to look up the correct row in **CUSTOMER**. `CustomerNo` in **SALE** is an example of what's called a **foreign key**. It's "foreign" because it's actually the primary (or sometimes the alternate) key of **CUSTOMER**, and therefore isn't "native" to **SALE**. Like primary keys, foreign keys may be either single columns or the concatenation of two or more columns. Unlike primary keys, there may be more than one foreign key in each table, a fact that is often taken advantage of. As a case in point, **SALE** has two foreign keys. The second is `ProductNo`, which is provides a link to **PRODUCT**.

Let's make this a little more explicit. The following diagram shows the **SALE** table and both its parent tables: **CUSTOMER** and **PRODUCT**. The arrows indicate how foreign and primary key values link a row in the child table to the corresponding row in the parent table.



You'll sometimes see foreign keys referred to as "references", although this term is less common than it used to be. The table containing the foreign key is also sometimes called the "child table" (**SALE** in our example), and the table it refers to is called the "parent table" (**CUSTOMER** or **PRODUCT**).

The `CustomerNo` column in **SALE** should only contain values that actually exist as primary key values in **CUSTOMER**. A non-existent customer number is totally useless, after all. The entry for sale 12348 is OK though, because it's null; that sale doesn't *have* a customer number, so we shouldn't expect to find a matching entry in **CUSTOMER**. More on this when we talk about referential integrity below.

Notice also that we can have products (left-handed doodads and grunge nuts in this example) that exist in the **PRODUCT** table, but whose primary key values don't appear in the **SALE** table. In other words, we can have rows in the parent table that have no matching entries in the child (foreign key) table.

Finally, notice that foreign key values aren't required to be unique in the child table. For example, customer number 1166-3461 appears twice (because that customer has been involved in two sales), and product number DHU69863 appears three times (because it has been sold to three different customers).

> *foreign key values aren't required to be unique in the child table*

Foreign keys are mostly used to join related rows from different tables. For example, we could join the row for **SALE** 12347 with the row for **CUSTOMER** Amelia Waverley to get all the information relating to that sale. We'll talk more about joining tables in *Chapter 3*, but right now we're going to look at the reason *why* we have to join tables in the first place.

Normalization

Why do we have to join tables? Why not just create one big table with all the data in it? Surely that would be more efficient than having to stitch together all these tables every time we want to find something out. Well, it does make it faster to retrieve data (up to a point), and this was a common criticism of the relational model in the early days. Unfortunately, any gains in speed from putting all the data in one table are more than offset by the problems caused by the resulting **data redundancy**. Suppose we put all the customer and sale data in one big table. We'd end up with something like this (ignoring products for the moment):

BIG_UGLY_TABLE										
SaleNo	**SaleDate**	**ProductNo**	**Qty**	**Amount**	**Salesrep**	**CustomerNo**	**First**	**Last**	**Address**	**CreditLimit**
12345	Aug 12 2002	AQX88916	1	23.95	Dave Williams	4649-4673	Richard	Johnston	14 West Avenue	1000
12346	Aug 12 2002	AQX88916	7	167.65	Sara Thompson	1113-7741	Wayne	Jones	42 York Street	*<null>*
12347	Aug 13 2002	AHL46785	3705	5001.75	Li Qing	1166-3461	Amelia	Waverley	995 Forth Street	*<null>*
12348	Aug 13 2002	DHU69863	50	118.5	Sara Thompson	*<null>*	*<null>*	*<null>*	*<null>*	*<null>*
12349	Aug 14 2002	DHU69863	940	2227.8	Sara Thompson	1166-3461	Amelia	Waverley	995 Forth Street	*<null>*
12350	Aug 14 2002	DHU69863	42	99.54	Sara Thompson	7671-3496	Antonio	Gonzales	55B Granary Lane	*<null>*
12351	Aug 14 2002	AQX88916	55	1317.25	Dave Williams	6794-1674	Diane	Adams	364 East Road	1500

Apart from the table being big and ugly, we can also see that Amelia Waverley's customer data appears twice in the table, once for each sale. Now, you might not think that this is much of a problem, but what if she keeps buying stuff? Her personal data will be repeated for every new sale. There are several problems that can result from redundant data like this:

- It wastes space. Amelia's customer data would probably occupy about 50 bytes or so on disk. Repeat that for 1,000 sales and that's 50KB of wasted space; repeat that for 10,000,000 sales and that's 500MB wasted. That doesn't sound like a lot in these days of 100GB disks, but why store data multiple times when you only need to store it once?
- More seriously, what would happen if Amelia changed her address? We would have to find every copy of her customer data and update it. Now that's inefficient, and if we miss one, we're in trouble. This is an example of an update anomaly.
- Even more serious, what if Richard Johnston canceled sale 12345? If we delete the row for that sale we also delete all of Richard's customer data. Similarly, we can't store any data for a new customer until they've actually bought something (although this might be normal depending on your business practices). These are examples of delete and insert anomalies respectively.

"The relational model says that every table is in at least first normal form"

Splitting up the data into separate related tables removes redundancy and makes these problems easier to deal with. This process is called **normalization**. Codd was working on this idea when Nixon went to China, and he heard a news item about Nixon "normalizing relations with China". Codd figured that if Nixon could normalize relations, so could he, and that's where the name comes from. (Obviously math geeks have just as strange a sense of humor as computer geeks.)

There's a whole separate mathematical theory that's grown up around normalization, but it all boils down to a set of rules on how to remove redundancy by splitting up tables. There are five **normal forms**, first (1NF) through fifth (5NF), each progressively removing more redundancy and splitting tables into smaller pieces. There are also a couple more obscure normal forms that don't quite fit into this progression, Boyce/Codd Normal Form (BCNF) and Domain-Key Normal Form (DK/NF), neither of which we need worry about for this book. The relational model says that every table is in at least first normal form, but most well-designed relational databases have tables that are in at least third or fourth normal form.

While normalization removes data redundancy, it can also reduce database performance, because the DBMS has to reassemble all the tables that were split up by the normalization process. This can be particularly bad in the web environment, where response times are critical (remember, users get twitchy after only three seconds). Not doing enough normalization, however, introduces a whole range of potential anomalies as noted above. It really depends on what you're doing with your data. If the data is mostly read-only, like catalog data for an e-commerce web site, then data redundancy probably isn't a major problem, and you can reduce the normalization level (this is known as **denormalization**). The **BIG_UGLY_TABLE** above is an example of a denormalized table.

If on the other hand you are dealing with mission-critical data that absolutely must stay consistent (like financial data), you probably want to minimize any redundancy even if it does mean that the database will run slower. Depending on the situation, it may make sense to duplicate the data that you need, and have one copy highly normalized while the other is not. This approach is fairly standard for data warehouses: data is extracted from the normalized operational database, transformed, and then loaded into a denormalized data warehouse.

An Example of Normalization

So how do we normalize a set of tables? Let's work our way through a simple example. Let's denormalize the example tables we have already, and then show how they can be normalized back to their current state (we'll leave the **EMPLOYEE** table out as it doesn't link directly to any of the other example tables). Here's what we might get if we just dumped all our data in one big table:

```
ALL_SALES(ProductNo, Description, ReorderLevel, Price, QtyInStock, CustomerNo,
First, Last, Address, CreditLimit, SaleNo, SaleDate, CustomerNo, ProductNo, Qty,
Amount, Salesrep)
```

First Normal Form (1NF)

The table **ALL_SALES** is already in 1NF because none of the columns contain multiple values (such columns are usually called **repeating groups**). This is in fact a requirement of the relational model: all columns must contain only scalar (that is, single-valued) values. Every table is therefore by definition in 1NF. We therefore don't need to do any more to this table other than choose a primary key. The only obvious unique combination here is `ProductNo` + `CustomerNo` + `SaleNo`, so we'll go with that for now (we shouldn't create a surrogate key at this stage because the table is not yet fully normalized). That is:

```
ALL_SALES(ProductNo, Description, ReorderLevel, Price, QtyInStock, CustomerNo, First,
Last, Address, CreditLimit, SaleNo, SaleDate, ProductNo, Qty, Amount, Salesrep)
```

Definition of 1NF: No repeating groups

Second Normal Form (2NF)

If we look at the columns of **ALL_SALES**, something becomes apparent: some of the columns are related. For example, {ProductNo, Description, ReorderLevel, Price, QtyInStock} form a natural group. So also do {CustomerNo, First, Last, Address, CreditLimit}, as do {SaleNo, Date, CustomerNo, ProductNo, Qty, Amount, Salesrep}. Now the interesting thing about these groups is that the values of the columns within each group are determined (that it, uniquely identified) by one of the other columns in that group. For example, in the first group, if we know the product number, then we also know the values of the other columns. This is known as a **functional dependency**, which is usually written as:

ProductNo → {Description, ReorderLevel, Price, QtyInStock}.

By definition, the value of every column in a table is determined by the value of the primary key. This is true for **ALL_SALES**, but the interesting thing is that some of the columns are also determined by parts of the primary key. For example:

ProductNo + CustomerNo + SaleNo → ReorderLevel

but

ProductNo → ReorderLevel

as well. In other words, some columns are determined by a subset of the primary key. We achieve second normal form by removing these kinds of structures.

A table is always in 2NF if it has a single-column primary key. This doesn't imply that all 2NF tables have only single-column primary keys however. A table can have a composite (multi-column) primary key and still be in 2NF.

To do this, we split off each of the three column groups into its own table and assign each a primary key:

```
PRODUCT(ProductNo, Description, ReorderLevel, Price, QtyInStock)
CUSTOMER(CustomerNo, First, Last, Address, CreditLimit)
SALE(SaleNo, Date, CustomerNo, ProductNo, Qty, Amount, Salesrep)
```

Aha! We've just arrived back at our original tables! We now know that these tables were in at least 2NF to start with.

Definition of 2NF: All columns in the table are determined by the whole primary key.

Third Normal Form (3NF)

Now we've hit a slight snag: we've already managed to normalize our tables back to their original forms. We'll have to modify them slightly so we can show you the later normal forms. First, let's modify the **PRODUCT** table as follows:

```
PRODUCT(ProductNo, Description, ReorderLevel, Price, QtyInStock, SupplierCode,
SupplierName, SupplierAddress)
```

We've added details of the supplier of each product. This table has a single-column primary key, so it must be in 2NF. If a supplier supplies multiple products, however, their details will appear once for each product, introducing data redundancy. The problem here is that some of the column values are also determined by something other than the primary key. In this case, not only are the supplier details determined by `ProductNo`, but they're also determined by `SupplierCode`. In effect:

`ProductNo → SupplierCode → {SupplierName, SupplierAddress}`

This is known as a **transitive** functional dependency. We achieve third normal form by removing such dependencies, again by splitting up the table:

```
PRODUCT(ProductNo, Description, ReorderLevel, Price, QtyInStock, SupplierCode)
SUPPLIER(SupplierCode, SupplierName, SupplierAddress)
```

Note that we've left `SupplierCode` in **PRODUCT** as a foreign key to the new **SUPPLIER** table.

Definition of 3NF: All columns in the table are determined by the primary key alone (that is, there are no transitive functional dependencies).

At this point, we can say that a table is in 3NF if all of its attributes are determined by the key, the whole key, and nothing but the key (with apologies to the legal profession).

Fourth Normal Form (4NF)

Suppose we decide that customers can have multiple addresses (this is fairly normal in a business environment; a customer's billing address often differs from their shipping address), and more important, that they can have any number of addresses (this means we can't just add another column to the table to support this requirement, because we don't know how many addresses they might have). If we do this with the **CUSTOMER** table as it stands, we'll once again end up with redundant data: the customer's details will be repeated for each different address.

Don't panic too much if you don't quite understand 4NF – many database experts have trouble with it too. Fortunately it's relatively unusual to have tables that are in 3NF but not 4NF. You'll often find that if tables are in 3NF they're also in 4NF (sometimes even 5NF).

The problem here is that `CustomerNo` no longer determines a single value of `Address`. Instead, it determines a *collection* of values of `Address` (this is not the same as a repeating group, because each column still contains single values). This is called a multi-valued dependency, and is written as:

`CustomerNo →→ Address`

We achieve fourth normal form when we remove such dependencies. Once again, we do this by splitting the table:

```
CUSTOMER(CustomerNo, First, Last, CreditLimit)
CUSTOMER_ADDRESS(CustomerNo, Address)
```

Now customers can have as many addresses as they like.

Definition of 4NF: The primary key determines only single values of each column, not collections of values (that is, there are no multi-valued dependencies).

Higher Normal Forms

We could carry on to fifth normal form at this point, but it's even less intuitive than 4NF. 5NF is actually the farthest we can go with normalization; it's the ultimate normal form. Practically speaking however, there's no real need to go beyond 4NF, especially if you're concerned about database performance. In fact, you might not want to go beyond 3NF, depending on your situation. Remember that with web databases performance is paramount so don't get too carried away with normalization.

Normalization is an important technique for designing effective databases (whether they're intended for use on the Web or not). It's really a "bottom-up" approach, starting from the data and working up to tables. In *Chapter 4* we'll see how to design databases from the "top-down" using conceptual modeling tools like entity-relationship diagrams. If you would like to learn more about normalization, any database theory book should cover it, but be warned that many of these books take a strongly theoretical approach. You'll probably find that most books describe the different normal forms in mathematical terms, rather than in plain English.

A Quick Recap

We've only looked at the structure aspect of the relational model so far. We'll cover the integrity aspect in the next section, and data manipulation (the operations aspect) is covered in *Chapter 3*.

We've seen that data in relational databases is structured as tables made up of rows and named columns. Each column is of a specific data type or domain, which determines the allowable values for that column. Missing or unknown values are dealt with by inserting null, or a default value.

Each row in a table must be uniquely identified, which is the job of the primary key. If we embed the primary key of one table in another table, we get a foreign key that links the two tables. Foreign keys are used to join corresponding rows from related tables when retrieving information from the database.

Finally, we looked briefly at why we need to break data up into multiple tables. If we don't, we end up with duplicated data, which can lead to all sorts of anomalies. Using normalization to split the data into multiple tables eliminates these problems.

In the remainder of this chapter we'll look at some concepts that apply to all database systems, relational or otherwise. First we'll look at the idea of data integrity, and then we'll look at how we can improve database performance by tuning the way that data are physically stored and accessed.

Data Integrity

Ideally, all data should have **integrity**. By "integrity" we mean that the data in our database should be valid and internally consistent. We can achieve this by defining a collection of integrity rules that we expect our data to meet. Note that this may not always be possible for some legacy data, or for data over which we have no direct control. In these situations the best we can do is identify how these data break the integrity rules that we've defined, so that we can then develop processes for dealing with it.

Integrity rules are normally identified during the design phase of database development (see *Chapter 4*), and are managed by the DBMS. Typical integrity rules could be things like:

- Two different employees can't have the same social security number.
- All employees must have a name (it's not allowed to be null).
- An employee's gender must be present (not null) and must be one of the values 'f' or 'm'.
- A customer's credit limit must either be between 0 and 5000, or it may be missing (null).

Integrity is a concept that applies to *any* database regardless of data model, so it applies equally as well to databases based on the hierarchical data model as it does to relational databases. There are two main types of integrity rule: those that are built into the data model we're using, and those that aren't. The former are generally mandatory and include things like ensuring that primary keys are unique (we'll explain shortly). The latter are generally optional and are usually referred to as "user-defined" integrity rules; the "user" here is you, the database developer. All of the example integrity rules listed above are user-defined integrity rules except for the first one, which is an example of the relational model's built-in entity integrity rule.

It's important to note that because user-defined integrity rules are optional, it's possible that they will not be implemented. Assuming that your database has appropriate integrity rules defined on it can lead to all sorts of problems. For example, it might be common knowledge in your company that no customer can get more than a 30% discount, but if this is not implemented as an integrity rule there is nothing to prevent an application program giving a customer a larger discount. All user-defined integrity rules should be thoroughly checked and rigorously documented, and where possible should be implemented as "close to" the data as possible.

Assuming that your database has appropriate integrity rules defined on it can lead to all sorts of problems

Ideally all user-defined integrity rules should be implemented within the database itself, as this makes it harder for rogue application programs to bypass them. This may not always be practical in a web environment however. If a customer is entering data into a form in their web browser and enters something that violates an integrity rule, you don't want them to have to wait until after they've submitted the form to tell them that something's wrong. The usual solution to this problem is to download some of the integrity rules to the user's machine as part of the web form (typically using embedded JavaScript). This should be fine as long as the integrity rules are downloaded from a single authoritative source such as an integrity or business rule server.

Let's return now to built-in integrity rules, which are rules built into the underlying data model itself. The relational model has three built-in integrity rules:

- **Entity Integrity**: This says that every row must be uniquely identifiable (that is, have a primary key). More specifically, it says that the primary key value must be unique across the entire table, and that no part of the primary key can be null. We haven't mentioned this second part before, but if you think about it, it makes sense: you can't guarantee the uniqueness of the primary key if it contains nulls, because you can't compare null with anything. The first rule in the list of examples above is an example of entity integrity (in this case for employees).

- **Referential Integrity**: This says that any foreign key value must either refer to an existing primary key value in the "parent" table, or it must be null. Remember the **SALE** example where sale 12348 had a null customer number? Now we can see why.
- **Domain Integrity**: This says that all values in a column can only be drawn from some particular set of pre-defined values. These values are the domain, which we discussed earlier in the **Domains and Data Types** section.

Note that integrity rules will ensure that any data entered into the database is valid, but not necessarily that it is correct. It's entirely possible to enter perfectly valid but totally incorrect data. For example, "Brain Watlers" might be a perfectly valid name for an employee as far as the integrity rules are concerned, but it's probably not correct. Sometimes such errors can have dire consequences, like entering valid but incorrect coordinates into a missile targeting system, but more commonly they just cause annoyances like bills being mailed to the wrong address. The best way to detect data entry errors like this is to double-enter or verify all data. The combination of integrity rules and data verification can be a powerful (but not foolproof!) way to ensure your data are correct. Remember, if you put garbage into your database, you should expect to get garbage out.

Metadata and the Data Dictionary

What do we store in a database? "Data," I hear you cry. That's true, but it's a little bit subtler than that. When we create a database we have to:

- Define all the database structures (tables, columns and their data types, primary and foreign keys, etc.)
- Populate the database with the data that we're interested in

When we do the first of these we are obviously creating information that's very important for the proper functioning of the database. This information is generally known as **metadata**, from the Greek "meta" (meaning "after" or "beyond") and the Latin "data" (the plural of "datum", meaning "something given"). Metadata is "data about the database" rather than "ordinary" data like sales or employee details. When you create a table in the database, the DBMS takes all the details about that table and files them away in a special area of the database called the **catalog** or **data dictionary**.

What kind of information gets stored in the data dictionary? Here are some examples:

- Table definitions (name, number of columns, primary key, physical storage attributes, etc.)
- Columns of tables (name, data type, size, what table it belongs to, etc.)
- Integrity rules
- User information (username, password)
- Access privileges: who is allowed to access what and how? (More on this in *Chapter 7*)
- Index details (see the next section)
- Various database statistics: sizes of tables, average number of queries, row sizes, etc.

Depending on the DBMS product you're using, the data dictionary could be large or small (Oracle's data dictionary contains literally hundreds of tables). It's usually structured using the same data model as the rest of the database, so if we're using a relational DBMS, the data dictionary will just look like a collection of tables. The data dictionary is mainly used by the DBMS to help run its internal operations.

Physical Data Access Methods

If you've ever used a database before, you may have heard of something called an **index**. You may have been told that they are used to somehow speed up the database, but you've never really been sure how or why. Performance is critical for any database system that's used by more than one person, but particularly so for web databases, which can have huge number of simultaneous transactions. We need to be able to tune the database to the types of operations most commonly performed by users. That's where indexes come in.

> *Database performance is determined at the physical storage level*

Indexes are just one part of a whole specialized area of database management: physical data storage and access methods. Up till now we've only talked about data in the logical sense, and pretty much ignored how it is actually physically stored. Database performance, however, is determined at the physical storage level, so it's important to look at how data is physically stored, and particularly how it can be retrieved efficiently from disk. Note that because of physical data independence, physical data storage mechanisms are (or should be) totally independent of the logical data model used. In other words, physical data storage is a level "below" things like the relational model. The relational model in particular says nothing about physical data storage.

The biggest bottleneck in any database system is how quickly we can retrieve data from disk (this includes both finding the data we want and actually reading it). Consider this: if a piece of data is stored in the main memory, the DBMS can usually retrieve it in a few nanoseconds. If the same piece of data is stored on disk, that time becomes a few milliseconds. We're talking thousandths of seconds vs. billionths here, so in theory reading something from RAM can be up to a million times faster than reading it off disk.

> *Research has shown that people start getting twitchy if response times are longer than three seconds*

Now you're probably thinking that a thousandth of a second isn't very long, so who cares? Well, if you're only retrieving one piece of data, it's probably fine. Scale that up to several million pieces of data, however, and you're starting to talk serious amounts of time (potentially several minutes) to retrieve them all. This scenario isn't far-fetched: a really popular web database might have millions of simultaneous transactions. Sure, each transaction might only be retrieving a few pieces of data, but add it all up and you're talking serious throughput. Research has shown that people start getting twitchy if response times are longer than three seconds; can you imagine how your users would react if it took three minutes to retrieve something from your database?

Despite the best efforts of the hard disk industry, we probably can't make disk drives much faster than they are already (there are limits to how fast you can physically move a piece of metal after all). There are tricks that we can do like putting multiple read/write heads on the disks or making multiple copies of the data across several disks to spread the load, but probably the most effective solution is to try to only access the disk when we absolutely have to.

One approach is to cache data that we've just read in RAM in case we need it again. A logical extension of this is to just store the entire database in RAM. The problem with this is that modern databases tend to get very large: in a recent survey, one telecommunications company was found to have over ten terabytes of data in their database (that's ten thousand gigabytes), and was expecting that to double within a year. Other companies have databases with upwards of forty terabytes (and would have more if they could afford it). Obviously you can't store that in RAM.

At the physical level of a database, data is usually stored in files on one or more disks. These files are divided up into (usually) fixed-sized chunks called blocks or pages, each of which may contain many records. The smallest unit that most DBMS products can read from and write to disk is one block. This is because block sizes are generally fixed, making blocks more predictable to deal with than records, whose sizes can vary considerably depending on the data stored within them. It's also more efficient (up to a point) to read bigger chunks of data than small ones, because we potentially have to move the read/write heads on the disk every time we read a new chunk of data.

The problem we now face is how do we find a particular record on disk? More importantly, how can we do this quickly, where "quickly" implies reducing the number of disk accesses? Usually we want to find a particular record based on some "key" value like an employee ID or a name (note that this is not the same thing as a primary key in the relational model, even though keys are often used as indexes). There are three main approaches to physically accessing data:

- Sequential access
- Indexed access
- Direct access

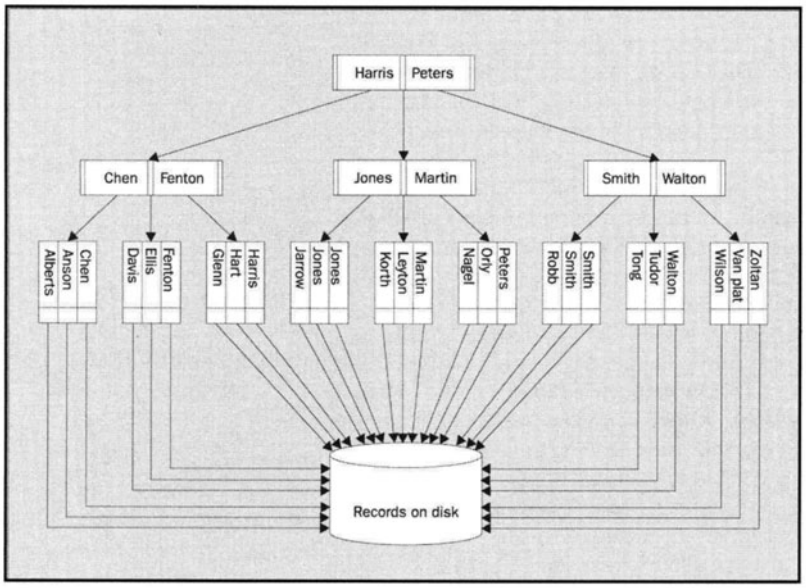
We'll now look at each of these in turn.

Sequential Access

Sequential access is the simplest way to retrieve data: take your key value, start at the beginning of the file and scan through each record one by one until you either find the key value you want or you reach the end of the file (some products call this a **table scan**). It's a bit like trying to find someone's phone number by reading the telephone book from cover to cover. If the records aren't sorted on disk then on average you'll have to search half the records. This is fine if there's only fifty records, but not so good if there are fifty million. Depending on the total number of records involved, it could take tens or hundreds of thousands of disk accesses to find a particular individual record.

Indexed Access

Indexed access provides a much faster means of finding a particular record by reducing the number of records we need to actually look at. An index is very much what it sounds like: a "directory" stored separately from the data that we can use to look up records on disk. Look up your key value in the index and it'll tell you where the corresponding record is on disk. The most common form of index used in database systems today (relational or otherwise) is the **B-tree** index. This reduces the number of records to be searched by breaking the list of key values up into small chunks and storing them in a hierarchical structure. The following diagram shows what a simple B-tree might look like (let's say it's an index on the surnames of clients or something similar):



Indexes can be unique (disallow duplicate key values) or non-unique (allow duplicate key values). The index in the diagram is obviously non-unique as there are two Joneses and two Smiths.

This index has three levels made up of collections of nodes. Each node in the first two levels contains two key values and pointers to nodes at the next level down (nodes can contain more than just two key values). All the key values are sorted from left to right across the tree. If we look at the first node at the second level, all the key values less than or equal to "Chen" can be found by following the leftmost pointer; those greater than "Chen" and less than or equal to "Fenton" are under the middle pointer; and all those greater than "Fenton" and less than or equal to "Harris" are under the rightmost pointer. The bottom-level nodes, known as leaf nodes, have pointers to the associated record on disk.

Suppose we want to find "Tudor". We start at the top-level node, and see that "Tudor" is greater than "Peters", so we follow the rightmost pointer. At the second level, we see that "Tudor" is greater than "Smith" but less than "Walton", so we follow the middle pointer. We are now in a leaf node, where we can scan from left to right until we find "Tudor". At worst we have made four disk accesses: one for each of the three nodes that we needed, plus one to retrieve the actual record. This index is so small however that the whole thing will probably get loaded into RAM, in which case we'd normally need only one disk access to retrieve a single record. In the absolute worst case (each record stored in a different block), sequential access would on average require fourteen disk accesses, as there are twenty-seven records. This would be extremely unlikely in practice though.

There are many varieties of the B-tree index, but they all share the property that key values are stored in the index tree in sorted order. This makes queries like "find all customers with surnames starting with C" much more efficient, because the DBMS can just find the first key value starting with "C", then scan through the index from left to right until it finds a key value starting with "D" (such queries are usually called range retrievals, because they are based on a range of consecutive key values). This doesn't necessarily imply that the corresponding records on disk are sorted in any way whatsoever. They could even be stored in random order, although this would be unusual in practice; it's more likely that records with similar key values will be stored physically close to each other. This is particularly nice with range retrieval queries, because when we retrieve one record, there's a very good chance that the next record in the sequence will be in the same block. That block will still be in RAM, so we don't need to go to the disk to get the next record.

Note that there's little point in placing indexes on very small tables. If there are only ten rows in a table, it's probably going to be just as fast to use sequential access, especially given that the entire table will probably fit into a single block anyway, and can thus be read with just one disk access.

Indexed access can easily be thousands or more times faster than sequential access – you can often retrieve a particular single record in less than ten disk accesses (especially if the index is loaded into RAM first). B-tree indexes also scale well to large numbers of records. Indexes do have a downside, however: if many rows are inserted or deleted, or key values are often changed, the DBMS has to do a lot of work maintaining the index tree structure in the correct order. If one part of the tree runs out of free space the DBMS will have to rebuild at least that part of the tree from scratch, which is a relatively slow operation, and while that's going on no one can use the index, leading to reduced database performance. When adding indexes to your database, you have to take into account the mix of operations that occur. If updates almost never occur, then you can be fairly free with indexes. If updates are common, you may find that adding an index actually reduces performance.

Note that adding an index to a primary key is usually safe, because primary key values shouldn't normally change (although there may still be issues if there is a high insert and/or delete rate). Many DBMS products automatically place indexes on all primary keys anyway.

We'll come back to indexes in *Chapter 8*, and show you some examples of how to create and use them.

Direct Access

Direct access is the fastest of the three physical access methods, and usually works through something called a **hash function** (some DBMS products implement these as "hashed indexes", but the process in general is called "hashing"). You feed key values into one end of the function, and it spits physical record locations out the other. Usually you can access any particular record in just one disk access. This sounds great, but direct access is most effective only in very specific circumstances: frequent single-record retrievals on *very* large tables. There's no hard and fast rule as to when a table becomes "very large" (it'll probably vary from one product to another), but below that point indexes are just as efficient (if not more efficient). Containing millions of records is probably enough to qualify a table as 'very large'.

Let's look at this in a little more detail. Direct access through hashing has an interesting side effect: it actually determines where a particular record will be placed on disk. Because the hash function returns a physical disk location corresponding to the input key value, the record *must* be placed at that location, otherwise the DBMS won't be able to find it again. Indexes in general don't do this; instead, they just catalog where the records happen to be already (clustered indexes are an exception, and we'll look at these in *Chapter 8*).

> The hash function attempts to distribute records fairly evenly

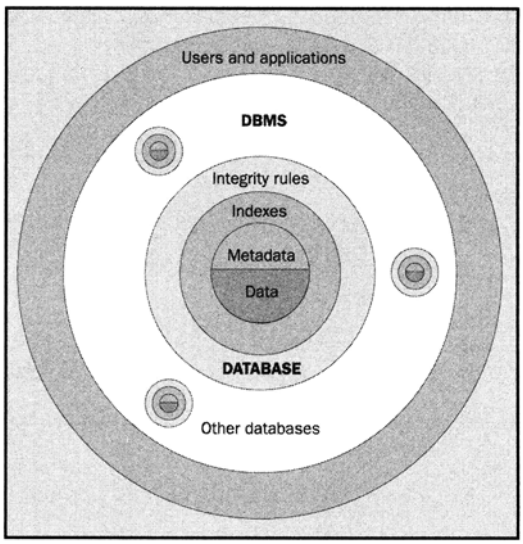
When we set up hashing on a table, the DBMS sets aside an area of disk in which to write the records for that table. The hash function then attempts to distribute records fairly evenly within this "hash space" as they are inserted. This means that records are more or less "randomly" scattered around the hash space, so the likelihood of records with similar key values being stored next to each other is therefore much lower than it would be normally. Hashing is therefore rather bad for range retrieval queries, because it's reasonably likely that the next record in the sequence will be in a different block, which may require another disk access.

You should only use hashing in situations where it performs best: frequent single-record retrievals on very large tables. Using it for anything else is a bit like using a sledgehammer to nail carpet tacks: it'll work, but it's not the best tool for the job (and it might punch holes in the floor if you're not careful). For most databases, indexes will give you all the performance you need.

Fitting It All Together

So far in this chapter we've shown you how to logically structure data using the relational model, and talked about data integrity, metadata, and physical data access methods (indexes). There are a lot of different pieces there, so how do we put them all together?

The diagram opposite shows how these concepts fit together. At the center we have the data and metadata. Wrapped around these we have layers of indexes and integrity rules. Together, all these items make up a single database. The DBMS manages this database and possibly several others, and controls access by users and application programs to all of these databases:



Obviously this diagram is a bit of a simplification. For example it doesn't include things like security, transaction management, application programming interfaces (APIs), data manipulation, and so on. These are all features that will be discussed in later chapters.

Summary

In this chapter, we introduced several fundamental concepts that underlie database systems. In particular, we looked at the concept of a (logical) **data model** and how it hides the physical representation of data from applications and users (**physical data independence**). Data models have three aspects: structure, integrity, and operations. We also looked at some aspects of database systems that are independent of the logical data model: **data integrity**, **metadata**, and physical data access methods (in particular **indexes**). Mostly however, we examined what is currently the predominant logical data model: the **relational model**.

The relational model is only one example of a logical data model, but it's the one that underlies most current DBMS products. It supplanted the earlier network and hierarchical models because it provided greater flexibility and simplicity for application programmers. Interestingly, though, both of these earlier models have been reborn in new forms: the hierarchical model has reappeared in the form of XML and the network model in the form of objects. Obviously the new models are not identical to the old (after all, the original models became unpopular for a reason), but the lineage is clear. The whole argument may become moot, however, as relational DBMS vendors have moved rapidly to support both object features and XML, and relational theorists are working on extending the relational model to support object-like capabilities.

The relational model structures data into **tables**, with **rows** and **columns**. Compare this with the object approach, which structures data into objects, with instances and properties. It turns out to be relatively easy to map from the relational model to the object approach and back. The main problem with the object approach however is that there is no single, widely accepted object model. The Object Data Management Group (ODMG, *http://www.odmg.org/*) has made the most successful attempt at defining such a model, but adoption of it has been very patchy.

The relational model has been stable for many years now. The only major controversy in the relational world was over the handling of missing information and the use of **null**. The debate split the relational world into two camps, those who support nulls, and those who think nulls are an abomination and favor **default values** instead. More recently, there has been some debate over how best to incorporate object features into the relational model, but this has been nowhere near as controversial (somewhat surprisingly).

The key concepts you should take forward from this chapter are:

- What logical data models are and how they provide physical data-independence.
- How the relational model structures data, particularly the concepts of tables, rows, and columns.
- How the relational model uses primary keys to uniquely identify individual instances of real-world items, and foreign keys to link related tables.
- How to handle missing data, the difference between nulls and default values, and when and where you should use each.
- How to use normalization to reduce data redundancy.

- What data integrity is, and how it can be enforced (we will look at this again in future chapters).
- What an index is, and how it can improve database performance.

So far we've only covered two aspects of the relational model: structure and integrity. In the next chapter we'll focus on the operations (data manipulation) aspect of the relational model. We'll introduce you to the relational database language SQL, and show you how to carry out several common database operations using this language.

3

- Getting information from the database
- Using SQL to narrow the results down
- Getting data from more than one table
- Sorting, grouping, and functions

Author: Nigel Stanger

Creating and Using Relational Databases with SQL

Introduction

In *Chapter 2* we looked at the core concepts of database systems. In particular we looked at the relational model of data, which is the data model underlying most database systems today. Having worked your way through the theory you're probably itching to get your hands dirty and try some of this stuff out. Your wish is granted: in this chapter we'll show you the basics of how relational databases can be created and used with a language called SQL. We'll give you enough information to create basic relational databases of your own and play around with them.

There's still a lot to learn though. By the time you reach the end of this chapter you'll know how to create and manipulate simple databases, but that's not enough to go off and create a good web database! We still haven't talked about how to design effective databases, which we'll do in *Chapter 4*, or how to plan and build databases in a web context, which we'll do in *Chapters 5* and *6*. We're also going to gloss over some more advanced stuff that will be covered in later chapters.

The main thing we will be looking at in this chapter is a language called **SQL**, which stands for **Structured Query Language**. This is *the* standard language for creating and manipulating relational databases. We'll tell you where the language came from, and show you how you can use it to create database structures and manipulate data in a database. We'll give you plenty of examples along the way (that's the main reason this chapter is so big!), most of which you should be able to try out yourself.

Most of the material in this chapter is fairly generic and applies to any kind of database, but we'll be pointing out items of interest to web databases as we go along. Note in particular that we don't normally use "raw" SQL in web applications. What usually happens instead is that we encapsulate our SQL code inside procedures that get stored in the database. This provides two main benefits: firstly we don't have to send long SQL strings over the network, so we can reduce bandwidth consumption, and secondly it's much more secure because the SQL code can't be intercepted and tampered with en route. We won't talk about these procedures specifically in this chapter, but keep them in the back of your mind: they're an important aspect of web databases that we'll come back to in *Chapter 8*.

A Note on the Examples

All the code examples in this chapter use the SQL-92 version of the SQL standard, which is the most widely supported version of SQL at present (we'll talk about the different versions of SQL shortly). They should work in any RDBMS that supports SQL-92, but you may find that some SQL features are not available in your RDBMS. SQL standard support varies from product to product – see the *Appendix* for details about individual products. All the examples in this chapter were tested in PostgreSQL, which has one of the better SQL-92 implementations.

Structured Query Language

All modern relational databases support some dialect of a language called Structured Query Language (SQL). The "query" part of the name is a slight misnomer because you can also use SQL to create and destroy tables, manipulate data, and many more things besides. Its primary use, however, is for retrieving data from databases, so the name makes sense.

> *SQL is declarative rather than procedural*

We should point out that SQL isn't a general-purpose programming language like Java or C++. It's a special-purpose language designed specifically for building, manipulating, and viewing databases, and nothing else. If you want to write application programs to access your database, you'll still be writing them in languages like Java, PHP, or ASP, but you can almost guarantee that your applications will use some variety of SQL to access the data in the database. Unlike most programming languages, SQL is declarative rather than procedural, meaning that when we use it we are telling the RDBMS what we want to achieve, not how to achieve it.

SQL has become such a powerful standard in the database world that even non-relational database technologies try to emulate it. For example, during the 1990s the Object Data Management Group (ODMG; see *http://www.odmg.org/*) defined a language for querying object databases called **Object Query Language** (**OQL**).

PostgreSQL is an open source relational DBMS that runs on a wide variety of platforms, and is available from *http://www.postgresql.org/*. It's not the most widely used RDBMS for building web databases, but there are many web sites that are built on PostgreSQL. (The most widely used RDBMS for high-performance web databases is probably MySQL, another open source RDBMS that is available from *http://www.mysql.org/*.)

OQL supports object-specific features like inheritance and method calls that don't exist in the relational model, but has been deliberately designed to look as similar to SQL as possible to ease the transition for developers already familiar with SQL. More recently the XML:DB initiative has begun defining a query language for XML data sources called **Simple XML Data Manipulation Language** (**SiXDML**). This too is deliberately based on SQL (see *http://www.xmldb.org/sixdml/*).

SQL is not perfect and from a theoretical perspective has some quite serious fundamental deficiencies. Despite this SQL is "sufficiently good" to have become the most widespread language for creating and using relational databases. SQL is now so entrenched that even if something better did come along, it would have great difficulty gaining any ground.

A Little History

A research group working at IBM developed the first version of SQL in 1974. Originally called **SEQUEL** (for **S**tructured **E**nglish **Que**ry **L**anguage), they had to change the name for unspecified legal reasons, so it was shortened to SQL (this is why most people pronounce "SQL" as "sequel"). SQL was developed for use with IBM's prototype relational DBMS, System R. SQL later became the query language for IBM's first relational product, SQL/DS, released in 1981.

The System R project had a very strong influence on early relational products, and companies like Oracle and Ingres also adopted SQL as their database access language (although Ingres initially developed their own query language, called QUEL). This very quickly led to several different incompatible dialects, so in 1982, ANSI chartered its Database Committee (X3H2) to develop a standard for SQL. The first version of the ANSI standard, released in 1986, was essentially just IBM's dialect of SQL, and was accepted as an international standard by ISO in 1987. Later versions of the SQL standard began to incorporate ideas from other vendors and interested parties. The current standard was released by ISO in 1999.

ANSI (American National Standards Institute, *http://www.ansi.org/*) is responsible for many standards, both in computing and in the wider world. ISO (International Organization for Standardization, *http://www.iso.org/*) is an international body that performs similar tasks.

Dialects

Every relational database product has its own dialect of SQL with its own proprietary extensions, but they all support some version of the ANSI/ISO SQL standard. There have been four versions of the standard, named after the year in which they were released: SQL-86, SQL-89, SQL-92, and SQL:1999. You'll sometimes see SQL-92 referred to as "SQL2" and SQL:1999 as "SQL3", but this is becoming less common.

The most widely supported version of the standard is SQL-92, and that's primarily what we'll be using in this book. The SQL-92 standard specifies three levels of compliance: Entry SQL, Intermediate SQL, and Full SQL. Entry SQL is the essential core of SQL-92; Intermediate and Full SQL add more features to this base.

Compliance with the standard tends to be patchy; for example, most modern products comply with SQL-92's Entry SQL, but very few if any products have claimed anything higher than this. Most products also add many extended features to the basic standard, diluting its usefulness. See the *Appendix* for details of some of the differences in SQL dialects across different RDBMS products.

SQL:1999 and the Future

The SQL standard is **BIG**. SQL-92 was impressive enough at about 500 pages, but SQL:1999 runs to over 2,100 pages, and is being added to on a regular basis. To give you some idea of how things have changed over time, the following table lists the number of keywords defined by each version of the standard (and just for interest's sake, Oracle8i's SQL dialect). For various technical reasons, SQL is also quite a difficult language for RDBMS vendors to implement, so it's no wonder that companies aren't rushing to implement full compliance in their products.

Version	Keywords
SQL-86	81
SQL-89	100
SQL-92	276
SQL:1999	473
Oracle8*i*	478

The SQL:1999 standard currently comprises 13 parts, but Part 2 (SQL/Foundation, which defines the core components of SQL:1999) is the most critical. SQL/Foundation is the SQL:1999 equivalent to SQL-92's Entry SQL, and is the minimum requirement for any vendor to claim that their RDBMS supports SQL:1999. Given the apparent apathy in the industry with regard to full SQL-92 compliance, it seems fairly unlikely that anyone will ever achieve official full compliance with SQL:1999.

Some of the more interesting additions in the SQL:1999 standard are:

- A separate general purpose programming language for writing database procedures, functions, and triggers (see Chapter 8). This language is suspiciously similar to an equivalent language in Oracle called PL/SQL.
- Support for structured data types like arrays and lists, and user-defined data types.
- Support for object storage (this was a major goal of the standard). It's rumored that OQL (Object Query Language) and SQL may one day be merged to provide full support for object databases, but this is speculative at best.
- A Boolean (True/False) data type. Rather astonishingly, this has been missing from SQL right from the very beginning!

Syntax Components

The commands in SQL (the statements with which we tell the RDBMS what we want) are often split into three groups:

- **Data Definition Language (DDL)**
  The DDL component of SQL covers the creation of database tables. It contains five main statements: `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`, `CREATE INDEX`, and `DROP INDEX`. This corresponds to the structure aspect of the relational model (apart from `CREATE` and `DROP INDEX`, which deal with physical data access).

- **Data Manipulation Language (DML)**
  The DML component of SQL covers the manipulation and viewing of data. It contains four main frequently used statements: `SELECT`, `INSERT`, `UPDATE`, and `DELETE`. This corresponds to the operations aspect of the relational model.

- **Data Control Language (DCL)**
  The DCL component of SQL covers managing access, authorization, and ownership of objects in the database, as well as procedural control statements. It contains such statements as `GRANT`, `REVOKE`, `COMMIT`, and `ROLLBACK`. We'll be looking at these aspects of SQL in *Chapters 7* and *8*.

We'll begin by looking at the most frequently used parts of the DDL.

Data Definition

How do we create a database using SQL? Oddly enough, exactly how you create an actual database is something that's not defined by the standard. Instead, it's one of many things that is left as "implementation-defined". This means that the procedure for creating a database varies from one product to the next. With Microsoft Access, you just select *New Database* from the *File* menu; with Oracle, you use the `CREATE DATABASE` command, either from the command line or from a graphical tool; and with PostgreSQL you enter the `createdb` command at the command line. Every RDBMS is different, so you'll need to check the documentation for the product you're using (also see the *Appendix*).

CREATE TABLE

Once we've created a database though, creating the tables within the database is pretty easy. Some products (like Access) have nice graphical tools for creating tables, but we'll ignore these and just work with plain ugly old SQL. This is partly because this technique is portable to any RDBMS product, and, perhaps more importantly, SQL can be used from within programs, or as part of a script, so we will be able to create (and later manipulate) tables from our web pages. To create tables with SQL we load up an SQL command line (see your RDBMS documentation to find out how to do this) and use the `CREATE TABLE` command.

> *the procedure for creating a database varies from one product to the next*

The CREATE TABLE command looks like this:

```
CREATE TABLE <name>
( <column-name>   <data-type> [DEFAULT <value>] [<column-constraints>],
  <column-name>   <data-type> [DEFAULT <value>] [<column-constraints>],
  ...
  [<table-constraints>]
);
```

Here are some notes on how to interpret this:

- The brackets [] indicate than an item is optional.
- *<data-type>* is one of the data types we talked about in Chapter 2.
- The optional **DEFAULT** keyword is used to specify a default value for a column.
- The "constraint" items are used to specify integrity rules. More on these shortly.
- Note that while we have used capitals for the keywords above, SQL is not case-sensitive. We use capitals to make the code easier to read.

CUSTOMER				
CustomerNo	**First**	**Last**	**Address**	**CreditLimit**
4649-4673	Richard	Johnston	14 West Avenue	1000
1166-3461	Amelia	Waverley	995 Forth Street	*<null>*
7671-3496	Antonio	Gonzales	55B Granary Lane	*<null>*
6794-1674	Diane	Adams	364 East Road	1500
1113-7741	Wayne	Jones	42 York Street	*<null>*

OK, let's look at an example; specifically, the **CUSTOMER** table from *Chapter 2*:
In SQL, this would look like this:

```
CREATE TABLE CUSTOMER
( CustomerNo    CHAR(9),
  First         CHAR(20)       NOT NULL,
  Last          CHAR(20)       NOT NULL,
  Address       VARCHAR(100)   NOT NULL,
  CreditLimit   SMALLINT       CHECK (CreditLimit >= 0),

  PRIMARY KEY (CustomerNo)
);
```

It should be fairly clear how most things correspond here. The table definition is enclosed in parentheses, commas separate all the individual elements, and a semicolon terminates the table definition (every SQL command is usually terminated by a semicolon so that the SQL processor can cleanly separate commands.)

Each column in the table is defined separately by giving it a name and a data type (more on this in a moment) and the primary key is defined by the clause at the end of the table definition, which is a type of **table constraint**. The NOT NULLs and the CHECK item are integrity rules defined by **column constraints**; we'll explain constraints shortly, but first let's look at data types in a little more detail.

Data Types in SQL-92

The SQL standard defines a fairly comprehensive selection of data types. For numeric data we have two sizes of integer (`SMALLINT` for numbers usually up to 32,767, `INTEGER` for bigger numbers up to about 2.1 billion), a `NUMERIC` type for decimal numbers, and floating-point numbers (`FLOAT`). Dates are created using the `DATE` data type. Character data can be either fixed length (`CHAR`) or variable length (`VARCHAR`).

Most vendors offer a superset of SQL-92 types, and these supersets vary considerably. Some of the sizes vary between vendors as well, with some allowing you to specify the number of bytes for `INTEGER` columns.

Several data types let you specify a size in parentheses after the data type name. These sizes do different things depending on the data type and what you specify:

Data type	What the size does
`CHAR(9)`	Character data *exactly* (always, no less, no more) nine characters long
`VARCHAR(50)`	Character data *up to* (can be less than) 50 characters long
`NUMERIC(6,2)`	A signed number of up to six digits in total, two of which always occur after the decimal point (legal values range from –9999.99 to 9999.99)

As well as offering extra data types beyond what SQL-92 supplies, some vendors may declare the data types with a different syntax or interpretation. For example, some products might interpret NUMERIC(6,2) as "a signed number with six digits before the decimal point and two after" (although this goes against the SQL standard). Some products even use their own data types in preference to the standard ones; for example, Oracle prefers its own NUMBER data type for all numeric data, and maps all of the standard numeric types onto this.

So looking at the **CUSTOMER** table above, we can see that customers' first and last names are exactly 20 characters long, while the address can be anything up to 100 characters. If a customer's first or last name is less than 20 characters long, the extra space at the end is padded with blanks. The decision whether to use `CHAR` or `VARCHAR` for a column is a bit of a tradeoff. `CHAR` columns can waste space, but they're faster for the RDBMS to deal with because they're fixed length. Conversely, `VARCHAR` columns save space but are slower to work with. For web databases, you're probably better off using `CHAR` for columns that are roughly the same length all the time (like phone numbers), or for columns that are commonly searched on (like customers' names). Use `VARCHAR` for larger text columns that aren't accessed as often.

Column and Table Constraints

Integrity rules are defined in SQL using **constraints**. A constraint is something that places limits on the data in a database. In SQL, constraints may be attached to a particular column (**column constraints**), or attached to a whole table (**table constraints**). Column constraints can only directly refer to the column that they are attached to, whereas table constraints may directly refer to any column in their associated table (so if you want a constraint to compare the values of two different columns, it must be specified as a table constraint).

You can define keys as column constraints as well, but it can be easier to see them as a table constraint.

Using constraints, we can define most of the kinds of integrity rules that we covered in *Chapter 2*. For example, entity integrity rules are specified using the `PRIMARY KEY` table constraint, and referential integrity rules are specified using the `FOREIGN KEY` table constraint (you'll see an example of this shortly). We've seen an example of the `PRIMARY KEY` table constraint already:

```
PRIMARY KEY (CustomerNo)
```

This says that the primary key of the **CUSTOMER** table is the `CustomerNo` column. It enforces the entity integrity requirement that `CustomerNo` must be unique and cannot be null.

You can define composite (multi-column) primary or foreign keys by listing multiple column names in the constraint, separated by commas, for example:

```
PRIMARY KEY (A, B, C)
```

This example forces the concatenation of columns `A`, `B`, and `C` to be unique, and ensures that none of `A`, `B`, or `C` can be null.

SQL provides several other constraints in addition to primary and foreign keys. The most commonly used ones are:

- `NOT NULL`: the column cannot contain nulls. If you don't say otherwise, the default is to allow nulls in columns, that is, you have to explicitly include a `NOT NULL` constraint to disallow nulls. (Many RDBMS products also let you say `NULL` to explicitly *allow* nulls, which can be very useful for documentation purposes, but this is non-standard syntax.)
- `CHECK` (*<condition>*): values in the column must satisfy the True/False expression *<condition>*. The <condition> must always be inside parentheses. (See later for more on True/False expressions.)

If we look at the **CUSTOMER** table, we can see several `NOT NULL` constraints

```
CustomerNo    CHAR(9),
First          CHAR(20)        NOT NULL,
Last           CHAR(20)        NOT NULL,
Address        VARCHAR(100)    NOT NULL,
CreditLimit    SMALLINT        CHECK (CreditLimit >= 0),
```

There's also a `UNIQUE` constraint that says that a column cannot have duplicate values. This is mainly useful for defining alternative keys.

This means that `First`, `Last`, and `Address` cannot be null; you must supply a value for these columns. You might be wondering why `CustomerNo` doesn't have a `NOT NULL` constraint (given that it's the primary key and the primary key can't be null), but that would be redundant because the `PRIMARY KEY` constraint does this already. In fact, adding a `NOT NULL` constraint to `CustomerNo` could slow the RDBMS down because it would be checking it twice.

CreditLimit doesn't have a NOT NULL constraint, so it's allowed to be null. It also has a CHECK constraint saying that if there is a value for CreditLimit, it can't be negative. Note that the CHECK constraint only kicks in when we actually supply a value, so nulls are still allowed by this (remember that null isn't a value).

You can name any column or table constraint using the optional CONSTRAINT keyword, for example:

```
CreditLimit   SMALLINT        CONSTRAINT CreditCheck CHECK (CreditLimit >= 0),
```

> *you should try not to allow nulls in columns that might be used in True/False expressions*

This associates the name CreditCheck with the CHECK constraint on CreditLimit. If you don't supply a name yourself, the system will automatically generate a (usually cryptic) name. Constraint names are mainly useful should you later want to remove a specific constraint from a table or column (see the section on ALTER TABLE).

Note that nulls can throw a major spanner in the works of Boolean logic, so you should try not to allow nulls in columns that might be used in True/False expressions. (Yes, we know that CreditLimit in **CUSTOMER** probably falls into this category. We'll show you how to fix this when we get to the ALTER TABLE command.)

Some More Tables

Let's look at the rest of the tables we used from *Chapter 2*, which are defined in SQL. First, the **PRODUCT** table:

PRODUCT				
ProductNo	**Description**	**ReorderLevel**	**Price**	**QtyInStock**
AQX88916	Flush widget	1000	23.95	4937
AAD62726	Left-handed doodad	0	4.95	10673
FGE91822	Grunge nut	5000	0.50	155016
AHL46785	Flange bracket	25000	1.35	51745
DHU69863	Self-righting socket	5	2.37	52

```
CREATE TABLE PRODUCT
( ProductNo      CHAR(8),
  Description    VARCHAR(50)    NOT NULL,
  ReorderLevel   INTEGER        NOT NULL  CHECK (ReorderLevel >= 0),
  Price          NUMERIC(5,2)   NOT NULL
                                  CHECK ((Price >= 0) AND (Price <= 999.99)),
  QtyInStock     INTEGER        NOT NULL  CHECK (QtyInStock >= 0),

  PRIMARY KEY (ProductNo)
);
```

Here we can see the use of `NUMERIC` for `Price`. Prices have a total of five digits, including two decimal places, so the maximum price that can physically be stored is $999.99 (we've added a column constraint to reinforce this).

You should always use either `NUMERIC` or `INTEGER` for monetary values, not `FLOAT`. `FLOAT` is a "floating point" number, which means that the number of decimal places will vary as necessary while doing calculations. Monetary amounts always have a fixed number of decimal places (usually two), so you should store and manipulate them that way. Extraneous decimal places have the potential to cause calculations to come out slightly off, especially if the extra decimal places are just truncated.

Next we have the **SALE** table:

SALE						
SaleNo	**SaleDate**	**CustomerNo**	**ProductNo**	**Qty**	**Amount**	**Salesrep**
12345	Aug 12 2002	4649-4673	AQX88916	1	23.95	Dave Williams
12346	Aug 12 2002	1113-7741	AQX88916	7	167.65	Sara Thompson
12347	Aug 13 2002	1166-3461	AHL46785	3705	5001.75	Li Qing
12348	Aug 13 2002	*<null>*	DHU69863	50	118.50	Sara Thompson
12349	Aug 14 2002	1166-3461	DHU69863	940	2227.80	Sara Thompson
12350	Aug 14 2002	7671-3496	DHU69863	42	99.54	Sara Thompson
12351	Aug 14 2002	6794-1674	AQX88916	55	1317.25	Dave Williams

```
CREATE TABLE SALE
( SaleNo       SMALLINT,
  SaleDate     DATE            NOT NULL,
  CustomerNo   CHAR(9),
  ProductNo    CHAR(8)         NOT NULL,
  Qty          INTEGER          NOT NULL  CHECK (Qty > 0),
  Amount       NUMERIC(6,2)    NOT NULL  CHECK (Amount >= 0),
  Salesrep     VARCHAR(50),

  PRIMARY KEY (SaleNo),
  FOREIGN KEY (CustomerNo) REFERENCES CUSTOMER,
  FOREIGN KEY (ProductNo) REFERENCES PRODUCT
);
```

Ah, this is a little more interesting: we have a `DATE` column and two foreign keys. As we noted earlier, foreign key definitions are specified using a table constraint. We tell the RDBMS which columns make up the foreign key (`CustomerNo` for the first one and `ProductNo` for the second), and which table the corresponding primary key is in (**CUSTOMER** and **PRODUCT** respectively).

Some of you may have been thinking that there's something slightly wrong with the **SALE** table. You're right: customers can buy only one product per sale. This isn't very realistic, but we did it this way to reduce the complexity of the example. In reality we'd allow multiple products per sale. Before we can do this we need to learn about a structure called **header-lines**, which is a little premature at this stage. Don't worry; we'll come back to the header-lines structure in *Chapter 4*, where we deal with this kind of many-to-many relationship. Also, the `Salesrep` column should really be a foreign key to some other table, but once again, we've omitted this for the sake of simplicity.

Finally let's look at the **EMPLOYEE** table:

EMPLOYEE			
Name	**Position**	**Location**	**Salary**
Jane Smith	Manager	London	35000
Richard Jones	Analyst	New York	35000
Anna Horton	Programmer	London	22000
Alex de Tocqueville	Programmer	Paris	21000

You'll remember that in *Chapter 2* we said that the only really viable primary key for this table is to concatenate all the columns. That's not ideal, and also rather inefficient, so we'd be much better to introduce a surrogate or artificial primary key. Let's invent an `EmployeeID` column and use that:

EMPLOYEE				
EmployeeID	**Name**	**Position**	**Location**	**Salary**
617258	Jane Smith	Manager	London	35000
125611	Richard Jones	Analyst	New York	35000
916201	Anna Horton	Programmer	London	22000
384460	Alex de Tocqueville	Programmer	Paris	21000

In SQL, this looks like:

```
CREATE TABLE EMPLOYEE
( EmployeeID  INTEGER,
  Name        VARCHAR(50)  NOT NULL,
  Position    VARCHAR(20)  NOT NULL,
  Location    VARCHAR(20)  NOT NULL,
  Salary      INTEGER      NOT NULL
                CHECK (Salary BETWEEN 10000 AND 50000),

  PRIMARY KEY (EmployeeID)
);
```

The only new thing here is the condition in the `CHECK` constraint attached to `Salary`. The `BETWEEN` operator is a convenient shorthand for saying:

```
(Salary >= 10000) AND (Salary <= 50000).
```

OK, we're done creating tables for now. Let's now look at some other things we can do with tables.

DROP TABLE

The opposite of `CREATE TABLE` is `DROP TABLE`, which is used as follows:

```
DROP TABLE CUSTOMER;
```

This will delete the **CUSTOMER** table and all the data in it, usually without warning, so be careful! Normally only the person who created a table can drop it again, assuming that your RDBMS has a proper security model. We'll discuss database security in *Chapter 8*. If you do manage to accidentally delete a table and all the data in it, you'd better hope that you have good backups! We'll discuss this and other database maintenance issues in *Chapter 9*.

ALTER TABLE

Database structures are never static; requirements change over time, new data needs to be stored, and integrity rules come and go. Obviously it's horribly inefficient to have to drop an entire table just so we can add a new column or constraint (not to mention having to take the database offline). We need some way to make small changes without rebuilding the whole table. The `ALTER TABLE` command provides this. It comes in three main forms, which we'll look at separately. The forms are:

- `ADD` form
- `ALTER` form
- `DROP` form

ADD Form

```
ALTER TABLE <table-name>
ADD [COLUMN] <column-definition>;

ALTER TABLE <table-name>
ADD <table-constraint>;
```

> *you can't add a NOT NULL column to a table unless you specify a default value*

The `ADD` form of `ALTER TABLE` lets you add new columns or table constraints to a table. New columns will initially be filled with nulls, or with the default value if you've supplied one. This implies that you can't add a `NOT NULL` column to a table unless you specify a default value (which may not always be possible), and the SQL standard defines no way to add column constraints to existing columns (but see the *ALTER Form* below).

For example, suppose that after a restructuring, some of our employees now manage other employees, and that we want to store this information in the database. The easiest way to do this is to add a `ManagerID` column to the **EMPLOYEE** table:

```
ALTER TABLE EMPLOYEE ADD COLUMN ManagerID DEFAULT 0 NOT NULL INTEGER;
```

We're assuming that everyone has a manager, and that the people at the "top of the tree" manage themselves. The default of `0` allows us to create the column with a `NOT NULL` constraint. We now need to go through the entire table and assign every employee the correct `ManagerID` (using the `UPDATE` command, which will be introduced shortly; see also the code download).

To be really useful, this new column should be a foreign key back into the **EMPLOYEE** table. (This is quite legal and very useful at times. It's called a **unary** or **recursive relationship** – we'll talk about these in *Chapter 4*.) In other words, the value of `ManagerID` contains some other employee's `EmployeeID`. We therefore need to add a `FOREIGN KEY` table constraint to the table as well:

```
ALTER TABLE EMPLOYEE ADD FOREIGN KEY (ManagerID) REFERENCES EMPLOYEE;
```

ALTER Form

```
ALTER TABLE <table-name>
ALTER [COLUMN] <column-name> SET DEFAULT <value>;

ALTER TABLE <table-name>
ALTER [COLUMN] <column-name> DROP DEFAULT;
```

The ALTER form lets you add and remove default values from columns. Many RDBMS products also extend this form with the ability to add or remove column constraints, or even to change the data type of a column. Specifying the default value is the same as for adding a column with a default value. Suppose we hadn't specified a default value for the newly added ManagerID column; we could remedy that simply as follows:

```
ALTER TABLE EMPLOYEE ALTER COLUMN ManagerID DEFAULT 0 NOT NULL INTEGER;
```

DROP Form

```
ALTER TABLE <table-name>
DROP [COLUMN] <column-name>;

ALTER TABLE <table-name>
DROP CONSTRAINT <constraint-name>;
```

The DROP form lets you remove an entire column or table constraint from a table. Any data contained in a removed column will be lost, so you should use this with the same caution that you use DROP TABLE. You also can't drop a column if it's referenced by a constraint like a foreign key; you'll need to drop the offending constraint first. Note that you can only drop a constraint if you know its name.

> *Any data contained in a removed column will be lost*

Suppose we've decided that we no longer want to allow nulls in the CreditLimit column of the **CUSTOMER** table; people with no credit will instead be assigned a zero credit limit. Under standard SQL-92 we have to follow this somewhat cumbersome procedure:

- Add a new TempLimit column with a NOT NULL constraint and a default value of 0.
- Copy the existing data (ignoring the null entries) from CreditLimit to TempLimit. (Again, using the UPDATE command we mentioned above.)
- Drop the CreditLimit column.

```
ALTER TABLE CUSTOMER
ADD COLUMN TempLimit SMALLINT DEFAULT 0 NOT NULL CHECK (TempLimit >= 0);

UPDATE CUSTOMER SET TempLimit = CreditLimit WHERE CreditLimit IS NOT NULL;

ALTER TABLE CUSTOMER DROP COLUMN CreditLimit;
```

We can't stop at this point because all of our application programs are expecting a column called CreditLimit, not TempLimit. So we then have to do the whole procedure again in reverse to get back to where we want to be:

- Add a new CreditLimit column with a NOT NULL constraint and a default value of 0.
- Copy the existing data from TempLimit to CreditLimit.
- Drop the TempLimit column.

```
ALTER TABLE CUSTOMER
ADD COLUMN CreditLimit SMALLINT DEFAULT 0 NOT NULL CHECK (CreditLimit >= 0);

UPDATE CUSTOMER SET CreditLimit = TempLimit;

ALTER TABLE CUSTOMER DROP COLUMN TempLimit;
```

Phew! Obviously this will be much easier if the RDBMS allows us to add column constraints to existing columns. All we'd have to do then is change all the null entries to `0`, and then add a `NOT NULL` constraint to the column.

Obviously `ALTER TABLE` is somewhat limited in what it can do, and every RDBMS product seems to have its own variation on what it's capable of, but it's an extremely useful feature when you need it.

CREATE INDEX, DROP INDEX

These commands allow you to create and drop an index, respectively (technically they're not part of the SQL standard, but we'll let that slide). We'll learn more about indexes in *Chapter 8*, but here is the basic syntax for these commands:

```
CREATE [UNIQUE] INDEX <index-name> ON <table-name> (<key-columns>);

DROP INDEX <index-name>;
```

`CREATE INDEX` attaches an index to the specified table, with the index key made up of the specified columns. For example, suppose we wanted to create an index on customer names:

```
CREATE INDEX CustomerNameIndex ON CUSTOMER (Last, First);
```

This isn't a unique index because the names of people can't be guaranteed unique. If you need a unique index, just add the `UNIQUE` keyword:

```
CREATE UNIQUE INDEX EmployeeIDIndex ON EMPLOYEE (EmployeeID);
```

Many RDBMS products automatically create unique indexes on all primary keys anyway, so the above example could well be redundant. Check your RDBMS documentation.

> *Many RDBMS products automatically create unique indexes on all primary keys*

You now have enough knowledge to create some simple tables of your own, so feel free to experiment. As noted earlier, we'll look at how to properly design databases in *Chapter 4*, and some more advanced aspects of SQL data definition in later chapters. For now, we'll finish with data definition and move on to how we can manipulate data with SQL.

Data Manipulation

There are four operations we can perform on the data in a database:

- Create (insert) new data.
- Read (query) existing data.
- Update (modify) existing data.
- Delete (remove) existing data.

These four operations are collectively referred to by the somewhat unfortunate mnemonic acronym CRUD, and cover all the typical operations that we would perform on a database during normal operation. SQL defines commands for each of these operations. Creating, updating, and deleting data are relatively simple, so we'll look at these first. There are so many extra features associated with reading data that it gets its own section.

Creating Data: INSERT

If we want to add new data to our database, we use the `INSERT` command. The easiest way to see how this command works is to show you an example. Suppose we've just finished creating all the tables outlined earlier, and we want to add Jane Smith to the **EMPLOYEE** table. Here's how we do it:

```
INSERT INTO EMPLOYEE (EmployeeID, Name, Position, Location, Salary)
VALUES (617258, 'Jane Smith', 'Manager', 'London', 35000);
```

First, we tell the RDBMS which table we are inserting into (`EMPLOYEE`). Next, we provide a list of the column names that we want to insert values into, and finally a list of values to be stored in each column. As long as the values we've provided don't violate any constraints attached to the columns or to the table, they'll get stored in the table.

Technically the list of column names in the `INSERT` command is optional; if we omit them the RDBMS will attempt to insert values into columns in the order they're defined in the table. Of course, if we get the order of the data different to the order of the columns, we'll be in trouble, as we will if the order of the columns is changed (if another column is added, for example). This is why it's much safer to explicitly list the column names when inserting data.

Note the use of single quote marks around the character values. Be careful about using double quote marks; these can sometimes mean something quite different. Two single quotes normally mean include the single quote character in the string, so 'Fred"s fish' would include the string "Fred's fish" in the table.

There are however two valid reasons for omitting one or more column names from the list:

- The value of one of the columns (for example EmployeeID) might be automatically generated by the RDBMS. If that's the case, we're going end up butting heads with the RDBMS if we try to supply a value ourselves. This is a common technique for primary keys.
- We might want to leave a column null, or if a column has a default value, to use that default.

We can cater for both of these situations by omitting the offending column name(s) from the list. For example, let's assume that the value of `EmployeeID` is automatically generated:

```
INSERT INTO EMPLOYEE (Name, Position, Location, Salary)
VALUES ('Heather Ralston', 'Manager', 'Melbourne', 33000);
```

This will insert a record for Heather into the database, and the RDBMS will automatically assign her an employee ID. (Of course, if the `EmployeeID` wasn't automatically generated, we'd quite rightly get an error for attempting to insert a row without a primary key value.)

Here's an example of the second scenario, leaving a column with null or its default value:

```
INSERT INTO CUSTOMER (CustomerNo, First, Last, Address)
VALUES ('5794-3467', 'Eric', 'Wilberforce', '955B Great South Road');
```

This time we're trying to insert a customer without specifying a credit limit. If the **CUSTOMER** table were in its original form (allowing null credit limits), then the RDBMS would simply insert null into the `CreditLimit` column. We've since used `ALTER TABLE` to disallow nulls in `CreditLimit`, and added a default value of `0`. The RDBMS therefore inserts `0` for Eric's `CreditLimit` value.

The inserts for the rest of **EMPLOYEE**, and for **PRODUCT** and **CUSTOMER** are fairly conventional, so we'll not show them here (they're all available in the code download).

The inserts for **SALE** are a little more interesting, as one of the sales has a null customer number and they also contain dates. We've already seen how we can implicitly insert a null by omitting the column from the list, but we can also explicitly insert a null using the keyword `NULL` where we would normally put a value:

> *we can implicitly insert a null by omitting the column from the list*

```
INSERT INTO SALE (SaleNo, SaleDate, CustomerNo, ProductNo,
                  Qty, Amount, Salesrep)
VALUES (12346, '2002-08-12', '1113-7741', 'AQX88916',
        7, 167.65, 'Sara Thompson');

INSERT INTO SALE (SaleNo, SaleDate, CustomerNo, ProductNo,
                  Qty, Amount, Salesrep)
VALUES (12348, '2002-08-13', NULL, 'DHU69863',
        50, 118.5, 'Sara Thompson');
```

We use single quotes around the date values, and the dates in this example are formatted using ISO date format (year-month-day). Every RDBMS handles date formats differently however, so you'll need to check the default date format for your product.

For reference, here's the syntax for the INSERT command:

```
INSERT INTO <table> [(<column-names>)]
VALUES (<column-values>);
```

Updating Data: UPDATE

We now have a bunch of data in our database. Great, but what if we want to change something? Suppose Jane Smith gets a pay rise, or we decide that Richard Johnston is no longer allowed credit. To change data already in the database, we use the UPDATE command, which looks like this:

```
UPDATE <table-name>
SET <column-name> = <value>, <column-name> = <value>, ...
[WHERE <condition>];
```

Remember that the brackets [] mean that the WHERE clause is optional. The UPDATE command works fairly much how you might expect. You tell it which table you're updating, and then give it a list of column names and associated values or expressions. We'll explain the WHERE clause in a minute. First, let's look at an example. Let's change everyone's salary:

```
UPDATE EMPLOYEE SET Salary = 37000;
```

This will set **every** employee's salary to $37,000. Obviously that's not very useful in real life (unless you actually *want* to pay all your employees the same salary), so let's just pretend that update never happened. What we really need to be able to do is target specific rows with our UPDATE commands. That's the job of the WHERE clause.

The WHERE clause tells the RDBMS which rows we want to update by specifying a Boolean condition that the rows to be updated must meet. So if we wanted to restrict our UPDATE to just Jane Smith, we could do one of the following:

```
UPDATE EMPLOYEE SET Salary = 37000
WHERE EmployeeID = 617258;

UPDATE EMPLOYEE SET Salary = 37000
WHERE Name = 'Jane Smith';
```

The first UPDATE is preferable because we're restricting based on the value of the primary key, which we know to be unique. Names aren't unique, so the second UPDATE could affect more than one employee. .

Now let's look at the second example we mentioned at the start of the section, canceling Richard Johnston's credit. While we're at it, let's also change his address:

```
UPDATE CUSTOMER SET CreditLimit = 0, Address = 'Iowa State Penitentiary'
WHERE CustomerNo = '4649-4673';
```

This changes Richard's CreditLimit to 0 and his Address to "Iowa State Penitentiary".

This is a good point to mention a handy trick for resetting tables to a known state. Suppose you have a table whose data doesn't change much, and one day you inadvertently do something to mess up the data in that table. Because the data doesn't change much, you can have an SQL script file stored away with all the INSERT commands needed to rebuild it from scratch. What we want to do is quickly dump the current contents of the table and regenerate it from the script.

While technically not part of the SQL standard, many RDBMS products have a command called TRUNCATE that empties out a table very quickly. You can achieve the same effect using the DELETE command described next, but TRUNCATE is much faster, especially for large tables, because DELETE logs all the deletions as they occur, whereas TRUNCATE doesn't. All you need to do is TRUNCATE the table, run your INSERT script, and presto, you're back where you started. This is particularly handy if you're doing development work and want to play around with the database.

If we were still using the old version of the **CUSTOMER** table that allowed null credit limits, we could cancel Richard's credit this way:

```
UPDATE CUSTOMER SET CreditLimit = NULL, Address = 'Iowa State Penitentiary'
WHERE CustomerNo = '4649-4673';
```

This is the only time you'll ever see null used with the "=" operator; for comparison operations we use the special `IS NULL` syntax that we saw briefly in the `ALTER TABLE` section, and will see again in the next section.)

What about something a bit more general? What if we want to increase the prices of all products by 10%? We want to update all the rows, so we can leave off the `WHERE` clause, but how do we say that we want to increase the prices based on the existing values? Here's how you do it:

```
UPDATE PRODUCT SET Price = Price * 1.1;
```

We're using the `Price` column not only as the destination of the `UPDATE`, but also as the source of the initial values. You can read the `UPDATE` above as "for every row, get the existing price value, multiply it by 1.1, then put it back again".

Deleting Data: DELETE

Finally, let's look at how to remove data from the database, using (no surprise here) the `DELETE` command, which looks like this:

```
DELETE FROM <table-name>
[WHERE <condition>];
```

Again this works much as you would expect. Tell it which table to delete from and give it a `WHERE` clause to control which rows get deleted (as with `UPDATE`, if you leave the `WHERE` clause off, it will delete **all** the rows in the table). For example:

```
DELETE FROM PRODUCT WHERE ProductNo = 'AAD62726';
```

This deletes the row for left-handed doodads from **PRODUCT**. We could also have said `WHERE Description = 'Left-handed doodad'`, but as with names of people there's no guarantee that the description is unique, so it might delete more than one row. Next example:

```
DELETE FROM EMPLOYEE WHERE Location = 'New York' AND Salary >= 40000;
```

This deletes all employees in New York who also earn greater than or equal to (>=) $40,000. Here we've included more than one condition in the `WHERE` clause and combined them using the Boolean `AND` operator. Another example:

```
DELETE FROM SALE WHERE CustomerNo IS NULL;
```

This deletes all sales that don't have an associated customer. Note the special syntax `IS NULL` instead of saying `= NULL`. (Earlier on in the `ALTER TABLE` section we also saw `IS NOT NULL`, which is the inverse of this.) One last example:

```
DELETE FROM PRODUCT;
```

This deletes (or at least attempts to delete) **all** products. Note that this operation should fail because there are several sales that reference rows in **PRODUCT** through the `ProductNo` foreign key. Recall from *Chapter 2* that a foreign key must either reference a valid primary key value or be null (referential integrity). If we deleted a product that was referenced by a sale that would break referential integrity because the foreign key would no longer be referencing a valid product. As long as appropriate foreign keys have been defined, the RDBMS will automatically prevent such violations from occurring.

> *If we deleted a product that was referenced by a sale that would break referential integrity*

The WHERE Clause

As we've already seen, the `WHERE` clause lets us control which rows get updated or deleted, a process also known as **filtering**. If we omit the `WHERE` clause from an `UPDATE` or `DELETE`, it affects **every** row in the table. If the table is large, this could take a very long time to complete – fatal in a web environment (the fact that you're probably affecting the wrong data can be fatal in other ways too). Unless this is what you want to do, you must **always** include an appropriate `WHERE` clause to limit the effects of an `UPDATE` or `DELETE` to only those rows you want affected. Forgetting the `WHERE` clause is one of the most common mistakes made by novice SQL users.

Including a `WHERE` clause is only half the battle, however; incorrect `WHERE` clauses can be just as bad. You need to test all `WHERE` clauses thoroughly before trusting them on your operational system. This means testing, testing, and yet more testing. You can never test enough, especially with `WHERE` clauses intended for destructive operations like `UPDATE` and `DELETE`. The best way to test a `WHERE` clause is to attach it to a `SELECT` command (introduced next) and try it on a backup database with good, representative test data. This lets you see exactly which rows will be affected in a non-destructive manner, and if something does go wrong, you haven't lost anything. Never, never, **never** test `WHERE` clauses on your operational environment!

> `WHERE` *is irrelevant for the* `INSERT` *command because we're adding new data – there's nothing to filter on even if we wanted to.*

As noted earlier, the expression in the `WHERE` clause is a Boolean (True/False) expression. In general this expression gets evaluated for each row in a table. If the values in a row cause the expression to return "True", that row is passed through to the operation in progress. If the expression evaluates to "False", that row is ignored and not affected by the operation.

We can in theory put as many sub-conditions in the `WHERE` clause as we like, although some SQL dialects place limits on how long a command can be. These sub-conditions are combined using the usual Boolean `AND`, `OR`, and `NOT` operators. All the examples we've seen so far have been fairly simple, so let's look at a couple of examples of more complex `WHERE` clauses:

```
UPDATE PRODUCT
SET Price = Price * 2
WHERE ((QtyInStock > 5000) OR (Price <= 10))
  AND (ReorderLevel < 1000);
```

This `UPDATE` doubles the price of all rows in **PRODUCT** that meet *all* of the following sub-conditions:

- The reorder level is less than 1000.
- Either the number of items in stock is over 5000, or the price is at most $10 (or both).

If we look at the data in the table, we can see that two rows match the first sub-condition (products AAD62726, DHU69863), and four rows match the second sub-condition (products AAD62726, FGE91822, AHL46785, and DHU69863). The first set of rows is totally contained by the second; so four rows are affected, leaving the row for product AQX88916 untouched. Another example:

```
DELETE FROM SALE
WHERE (      (SaleDate BETWEEN '2002-08-13' AND '2002-08-14')
         AND (Salesrep = 'Li Qing')
       )
   OR (ProductNo <> 'DHU69863')
   OR (Qty > 50);
```

This will delete rows from **SALE** that meet *any* of the following sub-conditions:

- Sales made by Li Qing on August 13th and 14th 2002.
- Sales of product DHU69863.
- Sales for more than 50 items.

"<>" is the standard SQL operator for "not equal to". Some RDBMS products may support other forms like "!=" as well.

Looking at the data, we can see that only one row in the table (sale 12347) meets the first sub-condition, three rows (sales 12348, 12349, and 12350) meet the second, and three rows (sales 12347, 12349, and 12351) meet the third. We're left with two rows that don't match any part of the `WHERE` clause: sales 12345 and 12346. The rest are all deleted.

The `WHERE` clause is also very important when reading data from the database, as it allows us to specify exactly which rows we want retrieved, and is usually also used to join tables.

Reading Data from the Database

The process of reading data from a database is called **querying**. A query in the database sense is just a set of instructions that tell the RDBMS what data you want to retrieve. Some examples of queries (in English) are:

- Show me all the employees in London.
- How many grunge nuts do we have in stock?
- Are there any customers with a credit limit higher than $6,000?
- How many sales did we make on August 12th and 13th 2002 and how much revenue did they generate?
- Show me all the sales made to Amelia Waverley.
- Show me all products, and the value of each sale that they've been involved in (if any).

These can all be formulated in SQL, as indeed can pretty much any query we might want to use.

When Codd invented the relational model, he devised a couple of mathematical query languages to complete the operations aspect of the model. The first of these languages is called **relational algebra**, which is made up of a set of operators that work on relations in much the same way that +, −, ×, and ÷ work on numbers. Relational algebra expressions provide step-by-step instructions on how to manipulate the data to get the result you want.

> *Relational algebra expressions provide step-by-step instructions on how to manipulate the data*

The other language that Codd defined is **relational calculus**, which is based on mathematical logic. This also has a set of operators, but relational calculus expressions don't tell you *how* to get the result you want; instead they tell you what the result should look like, and leave the detail of figuring out how to generate that result up to someone (or something) else.

The part of SQL that deals with querying the database is based mostly on the relational algebra, but also includes features of the relational calculus. This makes SQL a very powerful general query language. Unfortunately certain decisions made early on in the development of SQL mean that some queries are unnecessarily difficult to express in SQL, but most typical queries can be expressed relatively easily.

> *relational calculus expressions tell you what the result should look like*

Querying in SQL: SELECT

All querying in SQL is done using the `SELECT` command. The basic form of the command looks like this:

```
SELECT <columns>
FROM <tables>
[WHERE <conditions>];
```

For example, to list the names and salaries of all employees in Paris:

```
SELECT Name, Salary
FROM EMPLOYEE
WHERE Location = 'Paris';
```

The result of this query will look something like this:

Name	Salary
Alex de Tocqueville	21000

(Note that we're showing the output in generic form. The output produced by your RDBMS product will almost certainly be formatted differently, but the result will be the same.)

The best way to understand what a `SELECT` query will do is to look first at the `FROM` clause. This tells you which tables you are querying (in this case, **EMPLOYEE**). Next, look at the `WHERE` clause (if there is one) to figure out which rows will be included in the output (in this case, only rows that have a `Location` of "`Paris`"). Finally, look at the `SELECT` clause to see which columns will be included in the output (in this case, only `Name` and `Salary`). If you see a "`*`" in the `SELECT` clause, this means, "include all the columns from the table":

```
SELECT * FROM EMPLOYEE;
```

EmployeeID	Name	Position	Location	Salary	ManagerID
617258	Jane Smith	Manager	London	35000	617258
125611	Richard Jones	Analyst	New York	35000	617258
916201	Anna Horton	Programmer	London	22000	617258
384460	Alex de Tocqueville	Programmer	Paris	21000	916201

This time we've also left off the `WHERE` clause, so the query lists all the rows in the table. As you'll realize, if you use `SELECT *` on a large table, you'll get a lot of information. Because of this, using `SELECT *` is regarded as bad practice, and most people in the industry are more specific about what they want from their tables. Returning all the data in a table with one million rows is going to take quite some time, and is particularly wasteful of server resources if you only want the names of one type of person. Using `SELECT *` can also be a security hole: if you add a new column to the table that you don't really intend users to see, any queries that use `SELECT *` may be able to see it immediately. (We'll discuss other ways of ensuring database security in *Chapter 7*.)

Spending a little of your time learning how to use effective and efficient SQL queries will save your visitors time, and your site money as there will be less information transferred over the Internet, meaning less bandwidth will have to be paid for. Remember also the "three second rule of user twitchiness" from *Chapter 2*; if the query that tells the visitor what CDs by their favorite artist are in stock takes more than 30 seconds to a minute, they're likely to look elsewhere – efficient queries can reduce lost revenue.

Now let's see if we can devise SQL versions of the queries we listed earlier.

Show me all the employees in London.

This is just a variation on the first query we did above:

```
SELECT Name FROM Employee
WHERE Location = 'London';
```

Name
Jane Smith
Anna Horton

You may have noticed in two of the previous examples that the `WHERE` clause referred to columns that weren't in the output. That's OK, because the `WHERE` clause is processed before the RDBMS figures out what the output should look like. It would in fact be very annoying for web developers if SQL forced us to include possibly unwanted columns from the `WHERE` clause in the output, because in the web environment we want to reduce bandwidth usage as much as possible.

How many grunge nuts do we have in stock?

Grunge nuts are products, so we need to query the **PRODUCT** table. Because we only want grunge nuts, we'll also need to tell the RDBMS that we only want the row for that product (number FGE91822). Finally, we're only interested in how many there are in stock, so we only need the `QtyInStock` column (and probably also `ProductNo` and `Description`):

```
SELECT ProductNo, Description, QtyInStock
FROM PRODUCT
WHERE ProductNO = 'FGE91822';
```

ProductNo	Description	QtyInStock
FGE91822	Grunge nut	155016

If we didn't happen to know what the product number was, we could have said `WHERE Description = 'Grunge nut'`, but remember that the description might not be unique.

Are there any customers with a credit limit higher than $6,000?

We want to query the **CUSTOMER** table for rows that have a `CreditLimit` value greater than 6000:

```
SELECT CustomerNo, First, Last FROM CUSTOMER
WHERE CreditLimit > 6000;
```

CustomerNo	First	Last

There aren't any – good.

How many sales did we make on August 12th and 13th 2002 and how much revenue did they generate?

The first part of this sounds easy: just query the **SALE** table for rows that have a `SaleDate` of either August 12 or August 13 2002:

```
SELECT SaleNo, SaleDate, Amount FROM SALE
WHERE SaleDate = '2002-08-12'
   OR SaleDate = '2002-08-13';
```

SaleNo	SaleDate	Amount
12345	2002-08-12	23.95
12346	2002-08-12	167.65
12347	2002-08-13	5001.75
12348	2002-08-13	118.50

Bingo, four rows. Incidentally, this isn't the only way we can write this query in SQL. Here are a couple of alternative ways of writing it (the variations are highlighted in **bold**):

```
SELECT SaleNo, SaleDate FROM SALE
WHERE SaleDate >= '2002-08-12'
   AND SaleDate <= '2002-08-13';

SELECT SaleNo, SaleDate FROM SALE
WHERE SaleDate BETWEEN '2002-08-12' AND '2002-08-13';

SELECT SaleNo, SaleDate FROM SALE
WHERE SaleDate IN ('2002-08-12', '2002-08-13');
```

We've seen the `BETWEEN` operator before; it's particularly handy when dealing with an inclusive range of values. The third alternative introduces a new operator, `IN`, which tests whether a value occurs in a supplied list of values. This ability to write queries in multiple ways is one of SQL's greatest strengths, but also one of its greatest weaknesses because it can make it harder for the RDBMS to figure out the most efficient way to execute a query.

Always be careful to use the correct date format for your RDBMS. We're using ISO date format in our examples, but there's no real standard across RDBMS products. For example, Sybase uses the format "12 Aug 2002", while Oracle defaults to "12-AUG-2002". Some products allow you to specify the date format on the fly.

Although our original query asked how much revenue these sales generated, for the moment we'll have to add that up by hand. For small tables like **SALE**, this isn't so bad, but for a table with thousands of rows it would be nice if the RDBMS would do the sums for us; in the section on *Aggregation*, we'll see how SQL can help us out.

Show me all the sales made to Amelia Waverley.

Oh, this is easy. We know what Amelia's customer number is, don't we?

```
SELECT SaleNo, SaleDate, Amount, Salesrep FROM SALE
WHERE CustomerNo = '1166-3461';
```

SaleNo	SaleDate	Amount	Salesrep
12347	2002-08-13	5001.75	Li Qing
12349	2002-08-14	2227.80	Sara Thompson

What happens though if we *don't* know Amelia's customer number? The customer name isn't included in the **SALE** table, so we can't use that. Or can we? Let's have a look at that syntax for the `SELECT` command again:#

```
SELECT <columns>
FROM <tables>
[WHERE <conditions>];
```

Aha! It says that the `FROM` clause can contain "tables"! Let's put both **CUSTOMER** and **SALE** in the `FROM` clause, and use Amelia's name in the `WHERE` clause to filter out the irrelevant rows.

```
SELECT SaleNo, SaleDate, Amount, Salesrep
FROM CUSTOMER, SALE
WHERE First = 'Amelia' AND Last = 'Waverley';
```

SaleNo	SaleDate	Amount	Salesrep
12345	2002-08-12	23.95	Dave Williams
12346	2002-08-12	167.65	Sara Thompson
12347	2002-08-13	5001.75	Li Qing
12348	2002-08-13	118.50	Sara Thompson
12349	2002-08-14	2227.80	Sara Thompson
12350	2002-08-14	99.54	Sara Thompson
12351	2002-08-14	1317.25	Dave Williams

Oops, that's not right at all! For one thing, Amelia's only been involved in two sales, so we shouldn't be getting seven rows in the output. We'll have to try again, but what happened? On closer examination it looks very much like the RDBMS has combined Amelia's **CUSTOMER** row with every row in **SALE** regardless of whether they're Amelia's sales or not. Now why on earth did it do that?

It turns out that this is normal behavior. If you list multiple tables in the `FROM` clause without any further instructions, any RDBMS will do exactly what's just happened here: it'll generate every possible combination of rows from *all* the tables, even if there are fifteen tables in the query. The RDBMS produces a mathematical **product** of the two tables (like 18 is the product of 6 and 3). This may sound like a weird thing to do, but doing it this way provides us with greater flexibility in determining exactly how to combine rows from the different tables.

What we really want to do for this query is to *join* related rows from the two tables using the foreign key/primary key link from **SALE** to **CUSTOMER** (you may recall that we talked about joins in *Chapter 2*). A similar thing is also required for the last example query that shows details of all products, and the sales they're involved in (if any). Joins are such a common operation that they rate their own section.

Joins

Because of the way relational databases are designed using normalization, most of the interesting queries we're going to want to do will involve two or more tables, as we've begun to see. A **join** operation is one that combines rows from two tables in some way, and there are a few ways we can join tables:

> *Joins are a common operation*

- Natural joins
- Condition-based joins
- Outer joins

Natural Joins

Natural joins are the most fundamental type of join; the other join types just add features. A natural join is based around the idea that if two tables have one or more columns in common (columns with the same name and data type), we can attempt to combine rows from each table that have the same value(s) in the shared column(s). This can be dangerous however; just because two tables share an `ItemNo` column, this doesn't mean that it represents the same thing in both tables. It's generally much safer to explicitly specify how all joins are to be made. We'll look at how to do this in a moment, but first let's look at a quick example of a natural join just to set the scene:

Always give your columns descriptive titles, and make sure that columns in different tables that have the same name represent the same attribute.

Show me all the sales made to Amelia Waverley.

```
SELECT SaleNo, SaleDate, Amount, Salesrep
FROM CUSTOMER NATURAL JOIN SALE
WHERE First = 'Amelia' AND Last = 'Waverley';
```

SaleNo	SaleDate	Amount	Salesrep
12347	2002-08-13	5001.75	Li Qing
12349	2002-08-14	2227.80	Sara Thompson

This looks more like what we expected to get from this query. What happens here is that the NATURAL JOIN takes the column in common to both **CUSTOMER** and **SALE** (CustomerNo), and only combines rows from each table that have the same CustomerNo value (of course, none of the data from the **CUSTOMER** table is included in the output because we've not asked for it). The natural join works for this example because CustomerNo happens to be a foreign key from **SALE** to **CUSTOMER**. SQL doesn't require this, but in general, it's best to only join over foreign key/primary key links.

We've only included natural joins here for completeness, and many RDBMS products don't support the syntax for them anyway. In practice you're much better off (and safer) using the condition-based joins described next. Natural joins really fall into the same category as SELECT *; they are potentially dangerous in the wrong circumstances.

Condition-Based Joins

The more usual way to do joins is to use an explicit Boolean condition to specify how to combine rows from the tables we're joining. There are two main syntaxes for doing this: pre-SQL-92 style and SQL-92 style. We'll look at the pre-SQL-92 style first because it still tends to be more commonly used than the SQL-92 syntax (and not all RDBMS products support the SQL-92 style syntax yet, although this is improving).

You may have already guessed at how we can do a condition-based join: simply embed the condition in the WHERE clause of a SELECT statement. The pre-SQL-92 syntax for doing this with two tables is:

```
SELECT <whatever>
FROM <table1>, <table2>
WHERE <table1>.<join-column1> = <table2>.<join-column2>;
```

This might be a bit clearer if we apply it to the Amelia Waverley query:

Show me all the sales made to Amelia Waverley.

```
SELECT SaleNo, SaleDate, Amount, Salesrep
FROM CUSTOMER, SALE
WHERE CUSTOMER.CustomerNo = SALE.CustomerNo
  AND First = 'Amelia' AND Last = 'Waverley';
```

*The CUSTOMER.CustomerNo and SALE.CustomerNo notation above is used to distinguish between the CustomerNo column in **CUSTOMER** and the CustomerNo column in **SALE** (otherwise the RDBMS wouldn't be able to tell which was which). The column names have been **qualified** with the name of the table that contains them. This wouldn't strictly be necessary if the columns had different names, but it's good practice to do it anyway for documentation purposes (remember that you may not be the person maintaining the code in future). In fact, it's very good coding practice to qualify all column names, so we'll adopt this style from now on.*

As you can see, we've explicitly told the RDBMS to only combine rows from each table that have the same value for CustomerNo. (We've used CustomerNo because it's the foreign key linking **SALE** with **CUSTOMER**; as we noted in the previous section, you should normally only join tables over foreign/primary key links.) It's much clearer with this syntax what's actually happening; one of the biggest disadvantages of a natural join is that it's harder to figure out exactly what's going to happen. Conditional joins are nicely self-documenting.

The part in **bold** above is called a **join condition**, and as you can see, it's just embedded in the WHERE clause along with any other filtering conditions we might want to include. We're not restricted to a single join condition; you can join multiple tables in the same SELECT command by listing them all in the FROM clause and adding appropriate join conditions. For example, consider the following query:

For all sales placed on August 14th 2002, show me the quantity and value of the product bought (showing the product code and description), and the name of the customer it was sold to.

```
SELECT SALE.SaleNo, SALE.Qty, SALE.Amount,
       PRODUCT.ProductNo, PRODUCT.Description,
       CUSTOMER.First, CUSTOMER.Last
FROM SALE, PRODUCT, CUSTOMER
WHERE SALE.ProductNo = PRODUCT.ProductNo
  AND SALE.CustomerNo = CUSTOMER.CustomerNo
  AND SALE.SaleDate = '2002-08-14';
```

Once again, the join conditions are indicated in **bold**. The first condition joins **SALE** with **PRODUCT** over the ProductNo foreign key, and the second joins **SALE** with **CUSTOMER** over the CustomerNo foreign key. The output should look something like this:

SALE. SaleNo	SALE. Qty	SALE. Amount	PRODUCT. ProductNo	PRODUCT. Description	CUSTOMER. First	CUSTOMER. Last
12349	940	2227.80	DHU69863	Self-righting socket	Amelia	Waverley
12350	42	99.54	DHU69863	Self-righting socket	Antonio	Gonzales
12351	55	1317.25	AQX88916	Flush widget	Diane	Adams

The SQL-92 syntax for joins separates the join conditions from the filtering conditions and introduces an explicit JOIN keyword. The most logical place to specify joins is in the FROM clause (because that's where you're saying what table you want anyway). This is exactly what SQL-92 does:

```
SELECT <whatever>
FROM <table1> JOIN <table2> ON <table1>.<join-column1> = <table2>.<join-column2>
WHERE <filtering conditions>;
```

The join condition is introduced by the ON keyword. Here's the Amelia Waverley query again in SQL-92 format:

```
SELECT SaleNo, SaleDate, Amount, Salesrep
FROM CUSTOMER JOIN SALE ON CUSTOMER.CustomerNo = SALE.CustomerNo
WHERE First = 'Amelia' AND Last = 'Waverley';
```

Things get somewhat more involved when joining more than two tables:

```
SELECT SALE.SaleNo, SALE.Qty, SALE.Amount,
       PRODUCT.ProductNo, PRODUCT.Description,
       CUSTOMER.First, CUSTOMER.Last
FROM (CUSTOMER JOIN SALE ON SALE.CustomerNo = CUSTOMER.CustomerNo)
     JOIN PRODUCT ON SALE.ProductNo = PRODUCT.ProductNo
WHERE SALE.SaleDate = '2002-08-14';
```

Can you see what we've done here? We've joined **CUSTOMER** with **SALE** first, and then joined the result of that first join with **PRODUCT**. We could just as easily done it the other way around for this example, but for some queries the order in which you do the joins might be important, especially when dealing with outer joins (described next).

Here are the two different forms of the queries we've done, placed side by side so you can compare them more easily:

SQL-92 style	Pre-SQL-92 style

`SELECT SaleNo, SaleDate, Amount, Salesrep`
`FROM CUSTOMER, SALE`
`WHERE CUSTOMER.CustomerNo = SALE.CustomerNo`
`  AND First = 'Amelia'`
`  AND Last = 'Waverley';`
	`SELECT SaleNo, SaleDate, Amount, Salesrep`
	`FROM CUSTOMER JOIN SALE`
	`  ON CUSTOMER.CustomerNo = SALE.CustomerNo`
	`WHERE First = 'Amelia'`
	`  AND Last = 'Waverley';`

`SELECT SALE.SaleNo, SALE.Qty, SALE.Amount,`
`  PRODUCT.ProductNo,`
`  PRODUCT.Description,`
`  CUSTOMER.First, CUSTOMER.Last`
`FROM SALE, PRODUCT, CUSTOMER`
`WHERE SALE.ProductNo = PRODUCT.ProductNo`
`  AND SALE.CustomerNo = CUSTOMER.CustomerNo`
`  AND SALE.SaleDate = '2002-08-14';`
	`SELECT SALE.SaleNo, SALE.Qty, SALE.Amount,`
	`  PRODUCT.ProductNo,`
	`  PRODUCT.Description,`
	`  CUSTOMER.First, CUSTOMER.Last`
	`FROM (CUSTOMER JOIN SALE ON`
	`  SALE.CustomerNo = CUSTOMER.CustomerNo)`
	`  JOIN PRODUCT ON`
	`  SALE.ProductNo = PRODUCT.ProductNo`
	`WHERE SALE.SaleDate = '2002-08-14';`

As we noted at the beginning of this section, not all RDBMS products support the SQL-92 join syntax. This is somewhat of a pity, because the SQL-92 syntax tends to be easier to understand than the older alternative. If you have a RDBMS product that doesn't support SQL-92 join syntax (such as any version of Oracle prior to Oracle9*i*), you won't have any option other than to use the pre-SQL-92 syntax.

Outer Joins

Now let's try out our newfound knowledge on the last of our list of queries:

Show me all products, and the value of each sale that they've been involved in (if any).

At first glance, we might think that the following would give us the correct answer:

```
SELECT PRODUCT.ProductNo, PRODUCT.Description, SALE.SaleNo, SALE.Amount
FROM PRODUCT JOIN SALE ON PRODUCT.ProductNo = SALE.ProductNo;
```

> *Some information has been ignored by the RDBMS because it doesn't meet the join condition*

PRODUCT. ProductNo	PRODUCT. Description	SALE. SaleNo	SALE. Amount
AHL46785	Flange bracket	12347	5001.75
AQX88916	Flush widget	12346	167.65
AQX88916	Flush widget	12351	1317.25
AQX88916	Flush widget	12345	23.95
DHU69863	Self-righting socket	12348	118.50
DHU69863	Self-righting socket	12349	2227.80
DHU69863	Self-righting socket	12350	99.54

Compare the output to the requirements of the query though; the query asks for *all* products, but the SQL query has retrieved only the three products that have actually been sold, not the five products that we want. Some information has been ignored by the RDBMS because it doesn't meet the join condition, so we need something beyond what a normal join provides.

What we really want to do is include all the rows from the **PRODUCT** table that don't have any associated rows in the **SALE** table (all **PRODUCT** rows that don't meet the join condition). This is known as an **outer join**, because we're including some rows that would normally be "outside" the join. All the joins we've looked at so far are **inner joins**, because they only include in the output rows from each table that meet the join condition.

Let's look at an example first, and then we'll explain how this works. Here's our query again, using an outer join:

```
SELECT PRODUCT.ProductNo, PRODUCT.Description, SALE.SaleNo, SALE.Amount
FROM PRODUCT LEFT OUTER JOIN SALE ON PRODUCT.ProductNo = SALE.ProductNo;
```

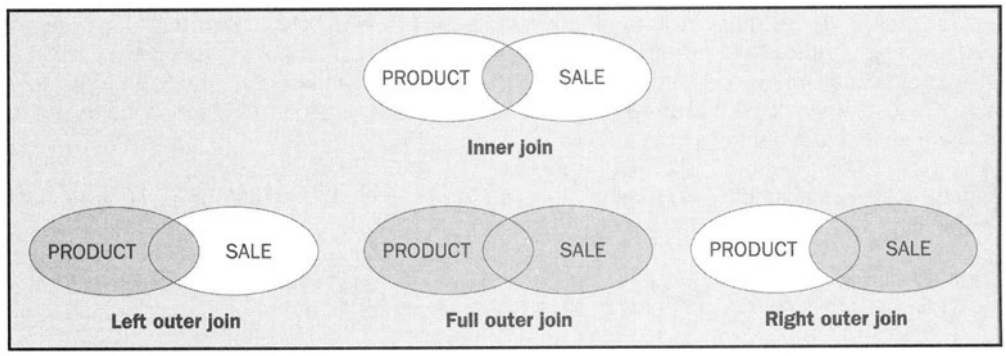
PRODUCT. ProductNo	PRODUCT. Description	SALE. SaleNo	SALE. Amount
AAD62726	Left-handed doodad	*<null>*	*<null>*
AHL46785	Flange bracket	12347	5001.75
AQX88916	Flush widget	12346	167.65
AQX88916	Flush widget	12351	1317.25
AQX88916	Flush widget	12345	23.95
DHU69863	Self-righting socket	12348	118.50
DHU69863	Self-righting socket	12349	2227.80
DHU69863	Self-righting socket	12350	99.54
FGE91822	Grunge nut	*<null>*	*<null>*

That looks more like what we wanted. All the products have been listed, regardless of whether they've actually been sold. Notice that because the two "extra" products (left-handed doodads and grunge nuts) don't have any sales associated with them, there are no values to go in the `SaleNo` and `Amount` columns for those products. The RDBMS therefore does the sensible thing and fills the `SaleNo` and `Amount` columns for these two products with nulls. (You might be wondering how this can work given that these columns all have `NOT NULL` constraints attached to them. That's a reasonable point, but these are temporary columns in a temporary result table – the `NOT NULL` constraints only apply to the "permanent" rows in the original table.)

So what does the `LEFT` keyword signify? Outer joins come in three varieties:

- **Left outer joins** work the same as a normal inner join, but they also preserve (include in the output) any rows in the table to the left of the JOIN keyword that don't meet the join condition. The example above uses a left outer join, where **PRODUCT** is the "left" table.
- **Right outer joins** work the same as left outer joins except that rows are preserved from the table to the right of the JOIN keyword (**SALE** in the example above).
- **Full outer joins** preserve rows from both the left and right tables.

We can visualize joins as being a bit like set operations. The following diagram uses this analogy to show how each of the different inner and outer join types work. Imagine that each set represents the associated table, and that the "intersection" represents a join condition between the two tables. The gray-filled areas indicate which rows from which tables are included in the output for each type of join:



Note that some RDBMS products don't support outer joins of any sort, while others might support left and right outer joins but not full outer joins. Those that do support outer joins may use the SQL-92 join syntax, or they may use their own non-standard syntax. For example, this is what the above query would look like in Oracle8*i*:

You can normally omit the `OUTER` *keyword, so* `RIGHT JOIN` *is equivalent to* `RIGHT OUTER JOIN`.

```
SELECT PRODUCT.ProductNo, PRODUCT.Description, SALE.SaleNo,
SALE.Amount
FROM PRODUCT, SALE
WHERE PRODUCT.ProductNo = SALE.ProductNo (+);
```

The (+) operator signifies the table from which rows *aren't* preserved (so it's sort of backwards from what we've seen already). Check your RDBMS documentation to see what support it has for outer joins.

Joins in the Web Environment

While joins are obviously a very useful and powerful operation, you need to be careful using them in web-based applications. As with many things in the web environment, it comes back to performance and how quickly you can return data to the user. Joins can take some time to process, especially if you're dealing with large tables. If we can eliminate joins, the database (and hence our web applications) will run faster.

> *normalization is generally considered a Good Thing because it reduces data redundancy*

As you'll recall from *Chapter 2*, joins are needed mainly because of normalization, and normalization is generally considered a Good Thing because it reduces data redundancy. So, what do we do? Optimize for performance and just live with redundant data, or normalize and lose customers because of poor web site performance? In a commercial situation, it's really no contest; customers (and thus revenue) must come first, but data redundancy can have some very nasty side effects.

Fortunately, it's not completely black and white. There are situations where normalization isn't so critical, especially if we have data that changes very little. In such cases we can "pre-compute" joins, thus saving considerable processing time on the database server and improving response times for the user. Let's look at an example of how we can do this.

A handy feature of many e-commerce web sites is the ability for customers to look at their past order history. With our example tables as they currently stand, the obvious way to do this is to join **CUSTOMER**, **SALE**, and **PRODUCT**, and then filter this based on the customer number, for example (using pre-SQL-92 join syntax):

```
SELECT SALE.SaleNo, SALE.SaleDate, SALE.Qty, SALE.Amount,
       PRODUCT.ProductNo, PRODUCT.Description, PRODUCT.Price,
       CUSTOMER.First, CUSTOMER.Last, CUSTOMER.Address
FROM SALE, PRODUCT, CUSTOMER
WHERE SALE.ProductNo = PRODUCT.ProductNo
  AND SALE.CustomerNo = CUSTOMER.CustomerNo
  AND CUSTOMER.CustomerNo = '1166-3461';
```

SALE. SaleNo	SALE. SaleDate	SALE. Qty	SALE. Amount	PRODUCT. ProductNo	PRODUCT. Description	...	CUSTOMER. Last	CUSTOMER. Address
12347	2002-08-13	3705	5001.75	AHL46785	Flange bracket	...	Waverley	995 Forth Street
12349	2002-08-14	940	2227.80	DHU69863	Self-righting socket	...	Waverley	995 Forth Street

If we have hundreds of thousands of customers and millions of sales, this result could take some time to compute. Fortunately, the **SALE** data has an important property once the product has been shipped; the data for that sale are extremely unlikely to change. In other words, this data is relatively **static** once the sale is completed. If data is (almost) never updated, then data redundancy is much less of an issue.

Rather than denormalize the entire database, however, a more common technique is to pre-compute the result of the join for all customers, and then copy that result into a separate, denormalized table. So, we might create a table that looked something like this:

```
CREATE TABLE SALE_HISTORY
( CustomerNo    CHAR(9),
  First         CHAR(20)       NOT NULL,
  Last          CHAR(20)       NOT NULL,
  Address       VARCHAR(100)    NOT NULL,
  ProductNo     CHAR(8),
  Description   VARCHAR(50)    NOT NULL,
  Price         NUMERIC(5,2)   NOT NULL
                               CHECK (Price >= 0 and Price <= 999.99),
  SaleNo        SMALLINT,
  SaleDate       DATE          NOT NULL,
  Qty           INTEGER        NOT NULL  CHECK (Qty > 0),
  Amount        NUMERIC(6,2)    NOT NULL  CHECK (Amount >= 0),

  PRIMARY KEY (SaleNo)
);
```

We can then use a handy variation on the INSERT command to copy the data from the original tables to the new table:

```
INSERT INTO SALE_HISTORY  (SaleNo, SaleDate, Qty, Amount, ProductNo,
                           Description, Price, CustomerNo, First, Last,
                           Address)
  SELECT SALE.SaleNo, SALE.SaleDate, SALE.Qty, SALE.Amount,
         PRODUCT.ProductNo, PRODUCT.Description, PRODUCT.Price,
         CUSTOMER.CustomerNo, CUSTOMER.First, CUSTOMER.Last,
         CUSTOMER.Address
  FROM SALE, PRODUCT, CUSTOMER
  WHERE SALE.ProductNo = PRODUCT.ProductNo
    AND SALE.CustomerNo = CUSTOMER.CustomerNo;
```

Rather than provide actual data values to insert into each individual row as we did earlier, (which would be incredibly tedious for large tables), we've told the RDBMS to execute the sales history query and insert the result of that straight into **SALE_HISTORY**. (Notice that we've left the CustomerNo filter out of the WHERE clause, as we want this table to store sale history details for *all* customers.) All we need to do now is ensure that we refresh the **SALE_HISTORY** table at regular intervals to capture new sales. We can achieve this using something called a **trigger**; we'll tell you about these in *Chapter 8*. This technique is sometimes called a **materialized view**; we'll look at views in *Chapter 8* as well.

Some RDBMS products allow you to create an entire table from scratch based on the output of a SELECT query (so you wouldn't need the CREATE TABLE command above). This is non-standard and will vary from product to product. Some products use SELECT INTO, although this means something different in the SQL standard.

Another example of where this technique might be useful is a student accessing their academic record through a college web site. Once again, this data is relatively static once the student has completed a course, so we can pre-compute the joins required and store the result in a separate table designed specially for this purpose. Many colleges and universities use exactly this technique.

We can also use this technique with frequently updated data if the users of the data don't require it to be absolutely up-to-date.

Useful Reporting Features

The features we've looked at so far are all well and good, but what about typical reporting features like sorting, grouping, and aggregation? The SELECT command supports these too. The full form of the SELECT command looks like this:

```
SELECT <columns>
FROM <tables>
[WHERE <conditions>]
[GROUP BY <columns>
   [HAVING <conditions>]]
[ORDER BY <columns>];
```

Sorting: ORDER BY

You may remember from *Chapter 2* that the relational model doesn't say anything about the order of rows in a table. Rows usually come out of the database in whatever order they happened to be inserted, which may not always be what we want. For example, let's do a simple query on **EMPLOYEE**:

```
SELECT  EMPLOYEE.EmployeeID, EMPLOYEE.Name, EMPLOYEE.Position,
        EMPLOYEE.Location, EMPLOYEE.Salary
FROM EMPLOYEE;
```

EMPLOYEE. EmployeeID	EMPLOYEE. Name	EMPLOYEE. Position	EMPLOYEE. Location	EMPLOYEE. Salary
617258	Jane Smith	Manager	London	35000
125611	Richard Jones	Analyst	New York	35000
916201	Anna Horton	Programmer	London	22000
384460	Alex de Tocqueville	Programmer	Paris	21000

Here, the rows are not sorted in any particular order. Suppose we want the listing in ascending alphabetic order of position. We can do this with the optional `ORDER BY` clause:

```
SELECT  EMPLOYEE.EmployeeID, EMPLOYEE.Name, EMPLOYEE.Position,
        EMPLOYEE.Location, EMPLOYEE.Salary
FROM EMPLOYEE
ORDER BY EMPLOYEE.Position;
```

EMPLOYEE. EmployeeID	EMPLOYEE. Name	EMPLOYEE. Position	EMPLOYEE. Location	EMPLOYEE. Salary
125611	Richard Jones	Analyst	New York	35000
617258	Jane Smith	Manager	London	35000
916201	Anna Horton	Programmer	London	22000
384460	Alex de Tocqueville	Programmer	Paris	21000

The output is now sorted the way we wanted. Notice that we only specified the column to sort on; this is because ascending is the default order. How about descending order of salary?

```
SELECT  EMPLOYEE.EmployeeID, EMPLOYEE.Name, EMPLOYEE.Position,
        EMPLOYEE.Location, EMPLOYEE.Salary
FROM EMPLOYEE
ORDER BY EMPLOYEE.Salary DESC;
```

EMPLOYEE. EmployeeID	EMPLOYEE. Name	EMPLOYEE. Position	EMPLOYEE. Location	EMPLOYEE. Salary
617258	Jane Smith	Manager	London	35000
125611	Richard Jones	Analyst	New York	35000
916201	Anna Horton	Programmer	London	22000
384460	Alex de Tocqueville	Programmer	Paris	21000

The `DESC` keyword indicates that we want the rows sorted in descending order. There's also an `ASC` keyword for sorting in ascending order, but since this is the default it's usually omitted.

You can also sort by multiple columns. For example, sort in descending order of position, and for those employees with the same position, sort in ascending order of salary:

```
SELECT  EMPLOYEE.EmployeeID, EMPLOYEE.Name, EMPLOYEE.Position,
        EMPLOYEE.Location, EMPLOYEE.Salary
FROM EMPLOYEE
ORDER BY EMPLOYEE.Position DESC, EMPLOYEE.Salary;
```

EMPLOYEE. EmployeeID	EMPLOYEE. Name	EMPLOYEE. Position	EMPLOYEE. Location	EMPLOYEE. Salary
384460	Alex de Tocqueville	Programmer	Paris	21000
916201	Anna Horton	Programmer	London	22000
617258	Jane Smith	Manager	London	35000
125611	Richard Jones	Analyst	New York	35000

You can see in this example that the two programmers are sorted in ascending order of their salary.

Removing Duplicate Values

Consider the following query:

What dates have we made sales on?

```
SELECT SALE.SaleDate FROM SALE;
```

SALE.SaleDate
2002-08-12
2002-08-12
2002-08-13
2002-08-13
2002-08-14
2002-08-14
2002-08-14

> *As usual, SQL has an answer*

While this gives us the answer we want, it's a little messy because some dates are repeated. It would be nicer if we could remove the duplicates. As usual, SQL has an answer – we can use the `DISTINCT` keyword to tell the function to remove duplicate values first:

```
SELECT DISTINCT SALE.SaleDate FROM SALE;
```

SALE.SaleDate
2002-08-12
2002-08-13
2002-08-14

This has obvious utility in a web environment for reducing the amount of output produced by certain classes of query.

Aggregation

Another common reporting operation that we want to do is to apply some sort of aggregate operation to a collection of rows, like a sum or an average. SQL has five built-in aggregate functions: `COUNT`, `SUM`, `MIN`, `MAX`, and `AVG`. The following example shows you how these functions work:

How many employees do we have, how big is the total salary bill, and what are the minimum, maximum, and average salaries?

```
SELECT  COUNT(*), SUM(EMPLOYEE.Salary), MIN(EMPLOYEE.Salary),
        MAX(EMPLOYEE.Salary), AVG(EMPLOYEE.Salary)
FROM EMPLOYEE;
```

COUNT	SUM	MIN	MAX	AVG
4	113000	21000	35000	28250.00000

(As usual, the exact format of the output will vary from product to product.) Aggregate functions take either a single column or expression as their argument (in this case the `Salary` column in the **EMPLOYEE** table), and always produce a single number result. The function is applied over the set of rows specified by the `WHERE` clause (or all the rows in the table if there's no `WHERE` clause).

Because aggregate functions only return a single number, you can't normally mix aggregate functions with "ordinary" columns in the `SELECT` clause. That is, `SELECT COUNT(*), CUSTOMER.CustomerNo FROM CUSTOMER;` should produce a syntax error. You can only mix aggregate functions and "ordinary" columns if you're doing a `GROUP BY`, which is coming up next.

Here's another example that shows you how to use a more general expression in an aggregate function:

What's the total value of our inventory?

```
SELECT SUM(PRODUCT.QtyInStock * PRODUCT.Price) FROM PRODUCT;
```

SUM
318559.49

(Note that we're using `*` here in its role as the multiplication operator, not to say we want all the columns from a table.)

We can tell the aggregate functions to ignore duplicate values in a column using the `DISTINCT` keyword. For example, to find out how many distinct employee positions we have:

```
SELECT COUNT(DISTINCT Position) FROM Employee;
```

COUNT
3

"Programmer" occurs twice, so there are only three unique values in the `Position` column.

> *The COUNT function is the only one that doesn't need to use the actual values in a table*

The COUNT function is the odd one out in this collection of functions. In the first example we saw the somewhat odd construction COUNT(*). What does this mean? The COUNT function works slightly differently from the rest because it's the only one that doesn't need to use the actual values in a table. If we're just counting the number of rows, then it's pretty much irrelevant which column we count, so SQL abbreviates this to COUNT(*). The only times this isn't useful are when:

All the aggregate functions except COUNT ignore nulls by default.

- We want to count the number of distinct values in a particular column (as shown above); or
- We don't want to count rows that have null in a particular column.

For both of these situations, we use the COUNT(DISTINCT *<column>*) form shown above.

Aggregate functions really come into their own when used with the GROUP BY clause.

Grouping Rows: GROUP BY

Grouping is a very common reporting operation. The idea is to group all rows that have the same value in some column or set of columns, usually so that you can then apply an aggregate function to each group. This is achieved using the GROUP BY clause. For example:

How many employees do we have in each position?

```
SELECT EMPLOYEE.Position, COUNT(*) FROM EMPLOYEE
GROUP BY EMPLOYEE.Position;
```

EMPLOYEE.Position	COUNT
Analyst	1
Manager	1
Programmer	2

What happens is that the RDBMS first sorts the table in order of Position. It then goes through and divides the table up into groups so that all the rows in a particular group have the same value for Position. Finally it applies the COUNT(*) to each group in turn. Each group thus produces a single row in the output, regardless of how many rows are actually in the group. In the example above, there are three distinct values of Position ("Analyst", "Manager" and "Programmer"), so we get three separate groups: two with one row each, and one with two rows.

Just like ORDER BY, you can GROUP BY multiple columns. This causes the rows in the table to be grouped by the concatenation of the values from the specified columns. For example, if we group employees by position and location, we get four groups instead of three:

```
SELECT EMPLOYEE.Position, EMPLOYEE.Location
FROM EMPLOYEE GROUP BY EMPLOYEE.Position, EMPLOYEE.Location;
```

EMPLOYEE.Position	EMPLOYEE.Location
Analyst	New York
Manager	London
Programmer	London
Programmer	Paris

This is because there are four distinct values that can be formed by concatenating the two columns: "AnalystNew York", "ManagerLondon", "ProgrammerLondon", and "ProgrammerParis". Notice also that we've included Position and Location in both the SELECT and GROUP BY clauses. If an "ordinary" column appears in the SELECT clause, then it **must** also appear in the GROUP BY. (Some RDBMS products also require the inverse: if a column appears in the GROUP BY, it must also appear the SELECT. This varies from product to product however.) This doesn't apply to aggregate functions; they only ever appear in the SELECT clause and the HAVING clause (see below).

Which sales representatives have made over $2,000 worth of sales?

We can do this "manually" using what we've learned already:

```
SELECT SALE.Salesrep, SUM(SALE.Amount) FROM SALE GROUP BY SALE.Salesrep;
```

SALE.Salesrep	SUM
Dave Williams	1341.20
Li Qing	5001.75
Sara Thompson	2613.49

Since there are only three results it isn't too onerous to scan through them and figure out the answer, but what if there were 300 results, or 3,000? Plus we're not necessarily interested in the actual amount of sales, as long as it's over $2,000 (remember that in the web environment we want to reduce bandwidth usage wherever possible, so the smaller we can make the result, the better). Fortunately we can get the RDBMS to give us only the results we want. You might be thinking "Aha, we can use a WHERE clause". Not so fast!

```
SELECT SALE.Salesrep FROM SALE
WHERE SUM(SALE.Amount) > 2000
GROUP BY SALE.Salesrep;
```

ERROR: Aggregates not allowed in WHERE clause

Curses, foiled again! The WHERE clause only applies to individual rows, not groups (probably because there's enough ambiguity in SQL already); what we need is something like a WHERE clause that applies to groups. SQL provides the HAVING clause for exactly this purpose:

```
SELECT SALE.Salesrep FROM SALE
GROUP BY SALE.Salesrep
HAVING SUM(SALE.Amount) > 2000;
```

SALE.Salesrep
Li Qing
Sara Thompson

Excellent. What happens here is that the RDBMS builds the groups as normal, then throws away any groups that don't meet the HAVING condition(s).

As with joins, grouping operations on large tables can take some time. If you have a commonly used GROUP BY, you can apply the same performance technique we showed you for joins: pre-compute the GROUP BY and store the result in another table. Again, this is only really feasible if the data is relatively static and/or absolutely up-to-date data isn't required.

Other Miscellaneous Useful Stuff

We've now covered the core aspects of SQL, but there's a whole bunch of miscellaneous little features that make life easier when writing queries. We'll look at some of these here. Some more advanced features will be covered in *Chapter 8*.

Comments

Anything that's preceded by two hyphens (--) is considered a comment and is ignored by the SQL command interpreter. Comments can occur anywhere in a line of code. These are more important when we're storing the code in the database to be run later, in the form of stored procedures, which we'll cover in *Chapter 8*

Expressions and Constants in the SELECT Clause

So far we've only put column names, *, or aggregate functions in the SELECT clause, but we can also include expressions and constants. For example:

How far away is each product from its reorder level?

```
SELECT PRODUCT.Description, PRODUCT.QtyInStock - PRODUCT.ReorderLevel
FROM PRODUCT;
```

PRODUCT.Description	Expr
Flush widget	3937
Left-handed doodad	10673
Grunge nut	150016
Flange bracket	26745
Self-righting socket	47

Here we've included an arithmetic expression in the SELECT clause. The result of this expression becomes a new column in the output. (We'll explain how to give this column a more meaningful name in the next section.)

List the names of sales representatives and the customers they have sold to.

```
SELECT SALE.Salesrep, 'has sold to', CUSTOMER.First, CUSTOMER.Last
FROM SALE JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo;
```

SALE.SaleDate
2002-08-12
2002-08-13
2002-08-14

This time we've embedded a string constant (`has sold to`) in the SELECT clause. This then gets included as a new column in every row of the output.

Table and Column Renaming

Sometimes it's useful to be able to define temporary names for tables or columns. At the very least we can use table renaming, or aliasing, to reduce the amount of typing we need to do:

```
SELECT S.SaleNo, S.SaleDate, S.Amount, C.First, C.Last
FROM SALE AS S JOIN CUSTOMER AS C ON S. CustomerNo = C.CustomerNo;
```

S.SaleNo	S.SaleDate	S.Amount	C.First	C.Last
12346	2002-08-12	167.65	Wayne	Jones
12347	2002-08-13	5001.75	Amelia	Waverley
12349	2002-08-14	2227.80	Amelia	Waverley
12345	2002-08-12	23.95	Richard	Johnston
12351	2002-08-14	1317.25	Diane	Adams
12350	2002-08-14	99.54	Antonio	Gonzales

(The `AS` keyword is optional.) In this example we've temporarily renamed **SALE** to **S** and **CUSTOMER** to **C**. Actually, there's a little more to it than that: what we're effectively doing is declaring a pair of "table variables" called **S** and **C**. **S** can range over all the data in **SALE**, and **C** can range over all the data in **CUSTOMER**. This distinction only becomes important when we want to join a table to itself (sometimes called a "self-join"). Recall that we added a `ManagerID` column to the **EMPLOYEE** table back in the section on `ALTER TABLE`. `ManagerID` is a foreign key back into the **EMPLOYEE** table itself (a recursive relationship – see *Chapter 4*), so we can use this to join **EMPLOYEE** to itself.

Now let's try the following query:

List the names of all employees along with that of their managers.

What we have here are two different "roles" for **EMPLOYEE**, one representing the employees and one representing the managers. We effectively want a different **EMPLOYEE** "variable" for each of these roles so that we can then join them over the `ManagerID/EmployeeID` foreign/primary key link.

The following `SELECT` obviously won't work:

```
SELECT EMPLOYEE.Name, EMPLOYEE.Name
FROM EMPLOYEE JOIN EMPLOYEE ON EMPLOYEE.ManagerID = EMPLOYEE.EmployeeID;
```

ERROR: Table name "EMPLOYEE" specified more than once

This is because the RDBMS can't tell which **EMPLOYEE** "role" is which. Table renaming allows us to create two independent variables, one for each "role" of **EMPLOYEE** with different names:

```
SELECT EMP.Name, MGR.Name
FROM EMPLOYEE EMP JOIN EMPLOYEE MGR ON EMP.ManagerID = MGR.EmployeeID;
```

EMP.Name	MGR.Name
Jane Smith	Jane Smith
Richard Jones	Jane Smith
Anna Horton	Jane Smith
Alex de Tocqueville	Anna Horton

> **!** Don't take this to mean that the RDBMS temporarily creates two copies of the **EMPLOYEE** table! Imagine what would happen if the RDBMS took your six billion-row orders table and made a copy of it every time you renamed it. There is only ever one actual copy of the table, no matter how many times we rename it.

We can also rename columns. This can be particularly useful when using aggregate functions, for example:

What is the average price and average number of products in stock?

```
SELECT AVG(PRODUCT.Price), AVG(PRODUCT.QtyInStock) FROM PRODUCT;
```

AVG	AVG
6.62400	44484.60000

Without column renaming the output provides no clues to differentiate the two AVG columns (some RDBMS products may include more detail in the output than this, but you can't count on it). We can fix this by renaming the columns:

```
SELECT  AVG(PRODUCT.Price) AS AveragePrice,
        AVG(PRODUCT.QtyInStock) AS AverageQty
FROM PRODUCT;
```

AveragePrice	AverageQty
6.62400	44484.60000

Column renaming is also useful when we embed expressions in the SELECT clause (repeating the example from earlier):

How far away is each product from its reorder level?

```
SELECT  PRODUCT.Description,
        (PRODUCT.QtyInStock - PRODUCT.ReorderLevel) AS QtyRemaining
FROM PRODUCT;
```

PRODUCT.Description	QtyRemaining
Flush widget	3937
Left-handed doodad	10673
Grunge nut	150016
Flange bracket	26745
Self-righting socket	47

Set Operations

You probably remember sets from school: given two different sets of objects, you can intersect them (that is, find the objects that occur in both sets), union them (combine both sets into one), and take the difference of two sets (find all the things in one set that aren't in the other). SQL provides similar operators for combining sets of rows. You don't need them very often, but occasionally they can be very useful.

> *SQL provides operators for combining sets of rows, occasionally they can be very useful*

Most RDBMS products support the UNION and INTERSECT operators, and some also support the difference (EXCEPT) operator. These operators work with sets of rows (each produced by a separate query) in exactly the same way that they would work with sets of numbers or letters. There's one important restriction to remember, though: you can only apply a set operation to two sets of rows if the rows have the same structure. In other words, the two sets of rows should have the same number of columns, with the same data types (CHAR and VARCHAR are considered the same for this purpose, and sizes are usually ignored). If this is True, the two sets are said to be **compatible**.

Let's look at an example of using the UNION operator (the others work exactly the same way, just with different results). Consider the following query:

List the number of sales for each customer (in order of customer name).

That doesn't sound too difficult. The only real complication is that cash sales have a null customer number, so we'll need to do an outer join:

```
SELECT CUSTOMER.CustomerNo, CUSTOMER.First, CUSTOMER.Last, COUNT(*) AS NumSales
FROM SALE LEFT JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
GROUP BY CUSTOMER.CustomerNo, CUSTOMER.First, CUSTOMER.Last
ORDER BY CUSTOMER.Last, CUSTOMER.First;
```

CUSTOMER.CustomerNo	CUSTOMER.First	CUSTOMER.Last	NumSales
6794-1674	Diane	Adams	1
7671-3496	Antonio	Gonzales	1
4649-4673	Richard	Johnston	1
1113-7741	Wayne	Jones	1
1166-3461	Amelia	Waverley	2
<null>	*<null>*	*<null>*	1

This looks fine apart from those pesky nulls. We haven't mentioned this before, but nulls can be quite a nuisance for application programs to deal with. Because most programming languages don't have a concept of null like SQL does, programs have to test every value coming in from the database to see whether it's null or not before trying to do anything with it. If an application can assume that no nulls will be arriving, the coding can be much simpler and more efficient. It would be nice if we could replace those nulls with actual values.

> *If an application can assume that no nulls will be arriving, the coding can be much simpler and more efficient.*

So what does that have to do with the UNION operator? Well, there's no obvious way to eliminate the nulls from the query that we've already got, but what we can do is break the original query into two: one that retrieves the sales for "real" customers, and one that retrieves the cash sales. As long as the results of both queries are compatible, we can then use a UNION to combine them into a single result.

To get the non-cash sales, we just take the original query and turn the outer join into an inner join:

```
SELECT CUSTOMER.CustomerNo, CUSTOMER.First, CUSTOMER.Last, COUNT(*) AS NumSales
FROM SALE JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
GROUP BY CUSTOMER.CustomerNo, CUSTOMER.First, CUSTOMER.Last;
```

(We'll leave the ORDER BY out for the moment – we can put it back at the end.) The output from this is the same as the original query, minus the last row.

Now for the cash sales; we don't need a join to do this second query because all we need to do is retrieve sales with a null customer number. We do however want the result of this query to be compatible with that of the previous query, otherwise we won't be able to union them. So the result of this query has to have four columns: three character columns and one number. The simplest way to do this is to just embed appropriate constants in the SELECT clause:

```
SELECT ' ', ' ', 'Cash sale', COUNT(*)
FROM SALE WHERE SALE.CustomerNo IS NULL;
```

The result of this query is now compatible with that of the first, so all we need to do now is UNION the two queries:

```
SELECT CUSTOMER.CustomerNo, CUSTOMER.First AS FirstName,
       CUSTOMER.Last as LastName, COUNT(*) AS NumSales
FROM SALE JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
GROUP BY CUSTOMER.CustomerNo, CUSTOMER.First, CUSTOMER.Last
UNION
SELECT ' ', ' ' AS FirstName, 'Cash sale' AS LastName,
       COUNT(*) AS NumSales
FROM SALE WHERE SALE.CustomerNo IS NULL
ORDER BY LastName, FirstName;
```

CUSTOMER.CustomerNo	FirstName	LastName	NumSales
6794-1674	Diane	Adams	1
		Cash sale	1
7671-3496	Antonio	Gonzales	1
4649-4673	Richard	Johnston	1
1113-7741	Wayne	Jones	1
1166-3461	Amelia	Waverley	2

Notice that the ORDER BY comes after all the queries, and that we had to rename several columns so that they were the same across both queries. (This is only necessary because we wanted to sort the output.)

Techniques for Building SELECT Queries

So how do you go about converting a question in English into a SELECT query, and more important, how do you know that the SELECT you've created is giving the right answer? There are some useful techniques you can use to make life easier.

You may recall that earlier on we showed you how to interpret what a SELECT command does:

- First, look at the FROM clause to see which tables are being queried
- Next look at the WHERE clauses to figure out which rows will be retrieved
- Finally look at the SELECT clause to find out which columns will be included in the output.

We can apply a similar approach to building SELECT commands as well.

The first thing to do is figure out which tables you'll need to query. If you need to access more than one table, you'll probably be doing a join somewhere along the line (unless you've pre-computed it already). Make sure you've got the join conditions correct, and that you're using the right kind of join. Depending on the query, you may need to consider table renaming (especially if you're going to join a table to itself). Once you know which tables you need and how they will be joined, you can build the FROM clause.

> *Make sure you've got the join conditions correct*

Now that you know which tables you're retrieving from, you need to figure out which rows you want from these tables. Decide the criteria for selecting these rows (if any), and express this as a Boolean condition. This gives you the WHERE clause. Remember, you have to test WHERE clauses thoroughly and repeatedly before putting them into production. Insufficient testing is one of the main reasons that buggy software gets released.

Next, you need to decide whether to group and/or do an aggregate operation. Decide which columns to group on, and put them in the GROUP BY clause (remember that they'll also need to go in the SELECT clause). If you need to filter out some of the groups, figure out the condition(s) for this, and put it in the HAVING clause. As with WHERE clauses, make sure you test these thoroughly too.

That's the hard stuff out of the way. Next, you need to decide what columns and/or expressions you want in the output (within the limitations imposed by what you've done already), and put them in the SELECT clause. Once you have this, you can then decide whether you need to sort the output, and if so, on which column(s). Use this to build the ORDER BY clause.

> *Make sure you have good test data*

You'll often find that these steps don't proceed in linear order. Sometimes the result of one step will feed into and affect an earlier or later step. Probably the best piece of advice for building effective SELECT queries is to test everything at every stage, again and again. Make sure you have good test data, figure out what you expect the answer to be, and then verify that your query is producing that answer. Break complex queries down into pieces and test each piece separately. For example, if you have a complex WHERE clause with five sub-conditions, test each condition in a separate query first, then gradually combine them, continually testing as you go. Test, test, and test again. That's the only way to ensure that your SELECT queries produce the right results.

A good description of this style of approach can be found in the article "An Incremental Approach to Developing SQL Queries" in the July/August 2000 issue of *Oracle Magazine* (*http://www.oracle.com/oramag/oracle/00-jul/index.html?o40sql.html*).

A Detailed Example

Suppose that you're developing a web-based information system to enable the company management to extract various bits of business information from the database (sort of like a rudimentary data warehouse). One particular area that they're interested in is the value of sales to credit customers vs. non-credit customers over various time periods. For example, a typical query might look like this:

What is the total value of sales to non-credit customers, on a day-by-day basis during August 2002?

The complexity inherent in this query is quite surprising, as you'll see shortly.

First, which tables do we need? We need to calculate the total value of sales; the data we need to calculate that is in the **SALE** table. We also need to know whether a given sale was to a non-credit customer; this information is in the **CUSTOMER** table. We can join these over the customer number primary/foreign key link. Our `FROM` clause should therefore look like this:

```
FROM SALE JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
```

Let's just test that. We know there are seven sales, so we should get seven rows in the result:

```
SELECT CUSTOMER.CustomerNo, SALE.SaleNo
FROM SALE JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo;
```

CUSTOMER.CustomerNo	SALE.SaleNo
1113-7741	12346
1166-3461	12347
1166-3461	12349
4649-4673	12345
6794-1674	12351
7671-3496	12350

Wait a minute; there are only six rows in the result. One of the sales has a null customer number, which is why it wasn't included in the join (it didn't meet the join condition). We need to tell the RDBMS to preserve all the rows from the **SALE** table regardless of whether they meet the join condition. That is, we need to do a left outer join (because **SALE** is on the left-hand side of the `JOIN` keyword):

```
SELECT CUSTOMER.CustomerNo, SALE.SaleNo
FROM SALE LEFT JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo;
```

CUSTOMER.CustomerNo	SALE.SaleNo
1113-7741	12346
1166-3461	12347
1166-3461	12349
4649-4673	12345
6794-1674	12351
7671-3496	12350
<null>	12348

That's better. Now let's look at which rows we want out of this. We want rows that meet *both* of these sub-conditions:

- The sale date is during August 2002.
- The customer has no credit (recall that non-credit customers have a zero credit limit).

Let's test these sub-conditions separately before we combine them, starting with the date:

```
SELECT CUSTOMER.CustomerNo, SALE.SaleNo, SALE.SaleDate
FROM SALE LEFT JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
WHERE SaleDate BETWEEN '2002-08-01' AND '2002-08-31';
```

CUSTOMER.CustomerNo	SALE.SaleNo	SALE.SaleDate
4649-4673	12345	2002-08-12
1113-7741	12346	2002-08-12
1166-3461	12347	2002-08-13
<null>	12348	2002-08-13
1166-3461	12349	2002-08-14
7671-3496	12350	2002-08-14
6794-1674	12351	2002-08-14

Looking good so far. (Of course our current data set doesn't actually have any sales outside August, but feel free to add some to your version of the database if you want to see how that affects things.) Notice that we've added the `SaleDate` column to the output to get a quick visual double-check that the correct rows are being retrieved.

Now let's test the `CreditLimit` sub-condition. The immediately obvious approach would just be to test whether `CreditLimit` is `0`, but this misses something important. Cash sales are effectively sales to non-existent customers (the sale has a null `CustomerNo`), and by definition a non-existent customer can't have credit. (Although this seems obvious, it should be documented somewhere as a business rule and not just assumed.) So not only do we need to check for a zero credit limit, we also need to check for a null credit limit.

Wait a minute, the credit limit *can't* be null! What's the point in doing that? Remember how an outer join works: rows preserved by an outer join don't have any corresponding row in the other table. Because of this, the RDBMS fills the "unmatched" columns with nulls in the result table. In this example, the **CUSTOMER** columns for cash sale rows (including `CreditLimit`) will be filled with nulls by the outer join, as we saw above. So we're actually testing for a null credit limit in the result of the join, not the original **CUSTOMER** table.

There are four customers with no credit, and one cash sale, so we should expect to get five rows in the output.

```
SELECT CUSTOMER.CustomerNo, SALE.SaleNo, CUSTOMER.CreditLimit
FROM SALE LEFT JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
WHERE (CreditLimit = 0) OR (CreditLimit IS NULL);
```

CUSTOMER.CustomerNo	SALE.SaleNo	CUSTOMER.CreditLimit
1113-7741	12346	0
1166-3461	12347	0
1166-3461	12349	0
7671-3496	12350	0
<null>	12348	*<null>*

OK, now things are starting to come together. Let's combine the two conditions and see what we get. Because we want all of the sub-conditions to be met, we combine them using AND:

```
SELECT CUSTOMER.CustomerNo, CUSTOMER.CreditLimit, SALE.SaleNo, SALE.SaleDate
FROM SALE LEFT JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
WHERE ((CreditLimit = 0) OR (CreditLimit IS NULL))
  AND (SaleDate BETWEEN '2002-08-01' AND '2002-08-31');
```

CUSTOMER.CustomerNo	CUSTOMER.CreditLimit	SALE.SaleNo	SALE.SaleDate
1113-7741	0	12346	2002-08-12
1166-3461	0	12347	2002-08-13
<null>	*<null>*	12348	2002-08-13
1166-3461	0	12349	2002-08-14
7671-3496	0	12350	2002-08-14

With our incremental testing approach we're now sure that we've restricted the output to only those sales that meet the specified criteria. The next thing to figure out is whether we need to do any grouping. The query says, "total value of sales to non-credit customers, on a day-by-day basis". In other words, for each day, what is the total value of sales? This implies grouping the output based on sale date, and we want to know the total, so we'll need to apply the SUM aggregate function to Amount in **SALE**:

```
SELECT SALE.SaleDate, SUM(SALE.Amount)
FROM SALE LEFT JOIN CUSTOMER ON SALE.CustomerNo = CUSTOMER.CustomerNo
WHERE ((CreditLimit = 0) OR (CreditLimit IS NULL))
  AND (SaleDate BETWEEN '2002-08-01' AND '2002-08-31')
GROUP BY SaleDate;
```

SALE.SaleDate	SUM
2002-08-12	167.65
2002-08-13	5120.25
2002-08-14	2327.34

And there you have it (you can add it up by hand if you want to double-check, but it is correct). About the only refinement we might want to make is to rename the SUM column to something more useful.

You now have a working query that you can drop into your system; all that remains to be done is to generalize it so that it'll work for any date range rather than just for August 2002. The way to do this is to create something called a **stored procedure**, which gets stored in the database alongside the data. This procedure would take a date as its argument and return the total sales to non-credit customers for that date. The body of the procedure would contain the query that we've just written. As we noted in the *Introduction* to this chapter, stored procedures are particularly useful in the web environment because they can encapsulate arbitrarily complex SQL code into a single procedure call, thus removing the need to send long SQL strings over the network. They're also much more secure than sending "raw" SQL over the network, where it can be intercepted and tampered with. We'll come back to stored procedures in *Chapter 8*.

Summary

In this chapter, you've learned how to use SQL to create and manipulate databases. In particular, we showed you how to create and manipulate table structures, insert data into them, and then update and delete that data. The most common operation on a database however is querying, so not surprisingly we've spent most of the chapter looking at SQL's `SELECT` command, which allows you to query the data in your database in quite complex and powerful ways.

It's been a (very!) long chapter, but even then we haven't covered everything you can do with the `SELECT` command. We've deliberately omitted some of the more esoteric features in order to keep things relatively simple (although we will come back to some of these in *Chapter 8*). While these advanced features are useful in certain circumstances, we've tried to stay focused on the most commonly used features of the `SELECT` command.

If you want to learn more about SQL, there are books available by the truckload. Be warned, however, that you probably won't become an expert overnight. Creating tables and getting data into them is fairly easy, but it usually takes at least a couple of years of experience to become truly proficient at writing queries in SQL.

From a web perspective, the thing to take away from this chapter is that you need to be very aware of the effects of the different types of operations you can perform on your data, both in terms of performance, and in terms of the data returned. For example, are you attempting to join several million-row tables? Are you retrieving more columns or rows than you actually need? Either of these can have severely detrimental effects on your web applications.

We've spent the last couple of chapters looking at relational database theory and practice, and to some extent how this relates to web-based systems. As we stated at the start of this chapter, you now have enough knowledge to create and use *simple* relational databases. These databases however will probably not be adequate to run a business or to drive a web site. You still need to learn how to design *effective* databases for the web, and that's what the next chapter's all about.

4

- Modeling basic structures
- One-to-Many relationships
- Many-to-Many relationships
- Sample data

Author: Chris Auld

Fundamentals of Database Design

We looked at the relational data model back in *Chapter 2*. In this chapter we will look at the fundamentals of applying that model to real-world situations. We'll start by looking at the concept of data modeling and why it is important. Then, once we have got to grips with these basic modeling ideas, we'll look at some of the common data structures that arise in our applications. We will also look at the practical implications of data integrity – the rules that determine the validity of data in our database. Finally we'll look briefly at using a software-diagramming tool to help us design our databases

Data Modeling

Data modeling concentrates on creating an abstract representation of the real world. It aims to remove extraneous information while retaining the information that is important. Modeling allows us to simplify the database design process by helping us to concentrate on the crucial elements of data and the relationships between them. Once we have developed a model of our data, we can use it to generate the appropriate structures in our RDMBS (relational database management system).

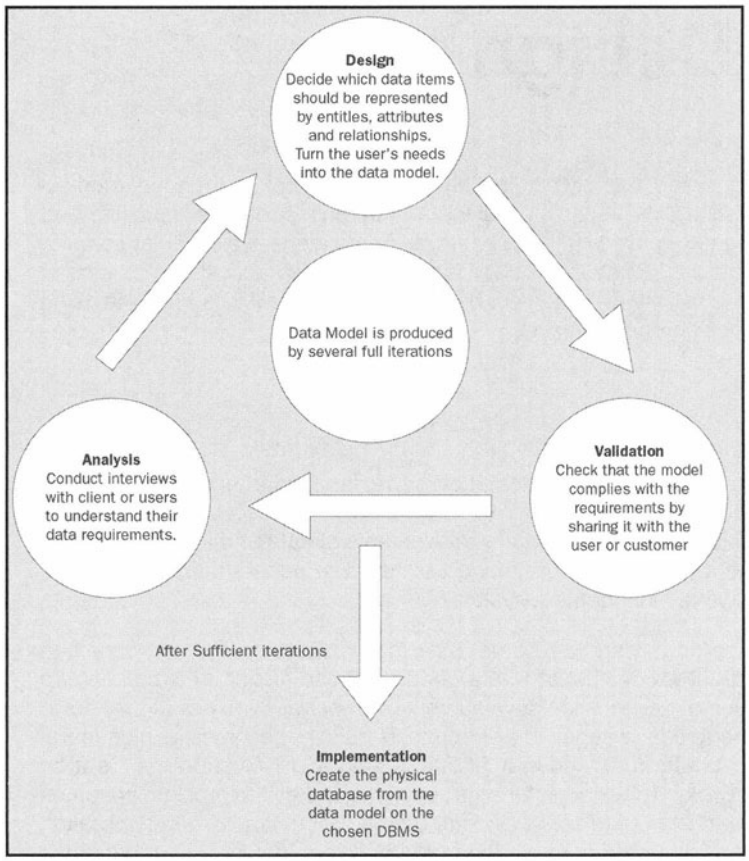
Our applications place enormous reliance on having good data: data that is useful to the end user and allows our application to operate efficiently. If we don't start with a good database design we won't end up with a good database, and chances are that our application development process will fly to bits further down the road. It's a bit like program documentation: without it maintenance quickly becomes a nightmare. Unfortunately, the graphical tools that ship with most modern RDBMSs lend themselves to the 'do it by hand' approach to database design. This works well enough for small systems, but as the problem becomes more complex we need to get a broad overview of the entire system. Ultimately our whole web application will rely on a good database design. It is very important for us to start out on the right foot – our database may be small to start off with, but with the right design it will be infinitely extensible as the requirements of our application grow and change.

Why Bother Modeling?

Modeling is part of the planning process that we undertake when building a system. Even if we don't write our plans down on paper, somewhere in our mind we are building a model, a plan of action on how to approach the problem. Of course the advantage of getting our model down in hard copy is that we can share it with others (including ourselves after the weekend). This is important as we go through the process of refining the requirements for our systems. By sharing our models with other developers and even the end user or customer we can help them to get an understanding of the problem without needing to explain everything in full detail.

It is important that we get our model right. As with an architect's drawings, one small item out of place can become a major problem and have consequences for the final success of a project. It is much easier to change our plans at the modeling stage than several months down the track when the application is live and serving pages to users.

The modeling process is iterative: that is we will generally go through the same process several times as we refine our model. The following diagram shows how the data modeling process will generally flow:



We are trying to distill all of the business requirements for the system. Often the user or customer will not have a complete grasp on what their needs actually are. By sharing our data models, they will gain a better understanding of their problem and hopefully will be in a better position to explain their needs to us. This is why it is very important to make multiple iterations in the design phase before you actually implement the database in the RDBMS. Often we will not know what questions to ask our customers until we have started modeling. The model will help us crystallize their requirements in our mind and hopefully turn up areas that we may have overlooked.

Once we have completed our database model we can implement it on the database platform of our choice. The beauty of modeling our data in an abstract fashion is that after the modeling phase we can choose to implement the database on any number of different platforms. The model is independent of the database.

This means that certain requirements that may have emerged in the analysis phase can be taken into account when choosing the database platform, such as the storage required for the data, target query response times, etc. Another advantage of this model independence is that we can prototype our system on a simple database, say on our desktop PC, and then migrate it to a production-capable platform when we are ready to go live.

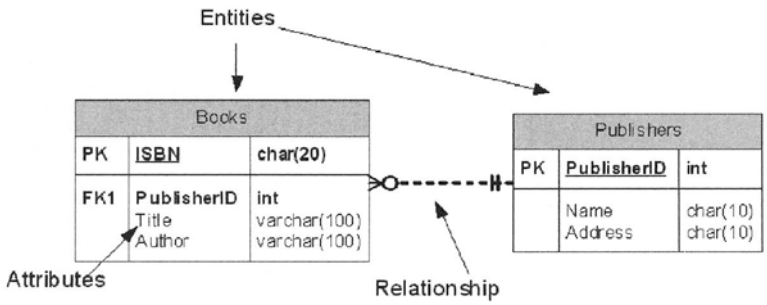
The final situation in which modeling can be useful is when we need to get to grips with a pre-existing system such as a legacy database that we need to modify some years later. By extracting the structure of the database out into some sort of model we, as developers, can better come to grips with the original design of the database, something that is seldom obvious from looking at reams of DDL code.

Modeling Entities, Attributes, and Relationships

While there are a number of different approaches to data modeling, the most common is Entity Relationship Modeling and this is the approach that we will use in this book. There are three elements to the Entity Relationship Model (ER Model).

- Entities
- Attributes
- Relationships

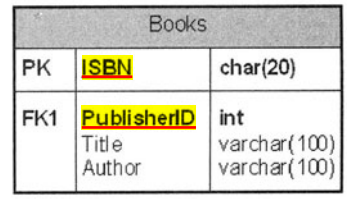
Let's take a look at these in the context of a little example. Say we are modeling an online bookstore. We would probably want to store information about publishers and books. We would need to store specific properties about books and publishers, things like author, title, publisher, and ISBN number for the books. For the publishers we would probably want to know their name, address, and the number of books that we have in our catalog from them. Popping all of this information into a diagram we get something like the following:



Now many of you may not have seen an ER Diagram like this before. The actual symbols used in database diagrams generally vary somewhat depending on the drawing package used but they all share some common characteristics in the way that they set out the elements I listed above. The notation that we are using in this book is called 'Crows Foot' notation. The diagrams have been drawn in a database modeling and diagramming tool called Visio. Let's discuss each of the elements in the diagram above in more detail.

Entities

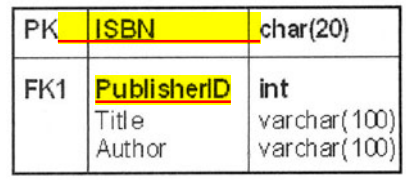
An entity is basically an abstract representation of a real-world thing, and is represented in a database by a table. This diagram has two entities, **Books** and **Publishers**; each of the large boxes represents one entity, as indicated by the name in the top gray panel. Most diagramming tools use a box of some sort to indicate an entity. The diagram also uses some formatting to tell us a bit more about the entity:



It groups the primary key attributes in a separate section at the top and it also breaks the information on those attributes down into three columns – we will look at these shortly. Most diagramming tools allow the amount of information shown in entities to be configured, which is very useful when dealing with large models. We might want to display a diagram that shows all of the entities, but only shows the entity names, not the attributes.

Attributes

Attributes are the properties of entities, and are represented in the database by columns (fields to give them their proper name). So for example a book has an ISBN number, a title, a length, physical dimensions, an author, and a publisher. A publisher has a name, an address, a number of employees, and a field of publishing. In the example above, **Books** has four attributes (*ISBN*, *Publisher*, *Title*, and *Author*) and **Publishers** has three attributes (*Publisher ID*, *Name*, and *Address*). In the diagram notation we are using the attributes will be listed inside the entities.

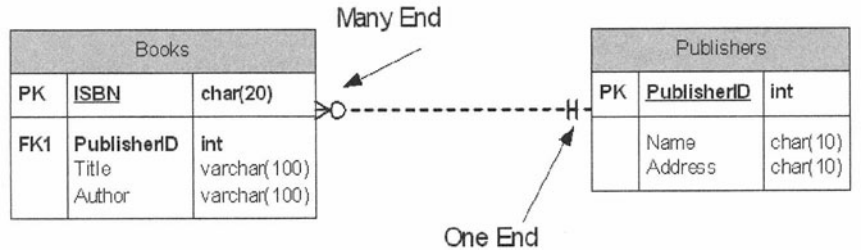


Most ER Diagrams will also include some information about the attributes. For example the notation we are using splits this information into three columns inside the entity. From left to right, the first column contains information on whether the attribute is either a primary or foreign key. The second column contains the name of the attribute, and the third column contains the data type of the attribute. So in the diagram above, `ISBN` is a `char` type of length 20 and it is also the Primary Key. Primary Keys are annotated with a 'PK' in the first column. Foreign keys are annotated with a 'FK' followed by a number – the number is arbitrary and serves only to allow us to easily differentiate between the various foreign keys. This diagram also shows whether or not an attribute is allowed to be null; bold-type attributes are not allowed to be null.

Something important to note is that not all of the information that we listed at the start is explicitly represented in our diagram. Where is the attribute for the number of books that a publisher has listed with us? This raises an important issue that we will discuss later in the chapter – redundancy. Storing the number of books that a publisher has listed with us would be redundant; we can easily obtain this information from the database using the `COUNT` function that you saw in *Chapter 3*.

Relationships

Relationships define the interactions between entities. In our example above, a publisher might publish many books. The diagram shows this relationship between **Books** and **Publishers**, as indicated by the line drawn between the two entities.



We will look at exactly how relationships work a bit later in the chapter, but in terms of ER Diagrams, the forked end indicates the 'many' end of the relationship, the straight end with the cross indicates the 'one' end of the relationship – this is known as the cardinality of the relationship. The relationship line also displays what is known as the optionality of the relationship. A circle indicates an optional end and double vertical lines indicate a mandatory end. So in the above example, a book **must** have a publisher, but a publisher **needn't** necessarily have any books.

Common Structures

Most database models are made up of a number of common structures of related tables. These structures are generally known by the cardinality of their relationships. This section will examine each of the five most common structures that you will encounter when designing databases. The aim is to give you a toolkit of data model pieces that you can easily identify and apply to real-world data problems. Some of the structures below are very common; others you will only see on very rare occasions.

In order of most common to least, the structures that we will be looking at are:

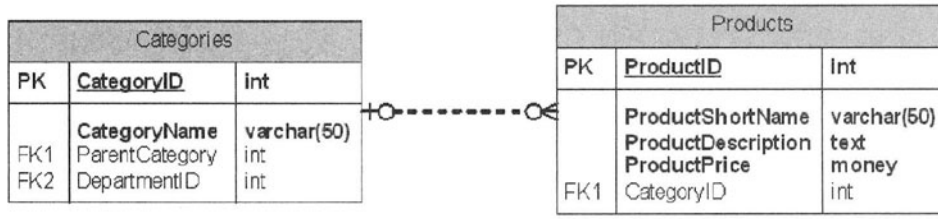
- One-to-many
- Many-to-many
- One-to-many Recursive
- Many-to-many Recursive
- One-to-one

> *we will look at a fully worked e-commerce example in the following two chapters*

We are going to use a large number of examples to illustrate each of the structures. Most of the examples we are going to use are from a hypothetical e-commerce store. The full database diagram is included at the end of this section, and it is also available in MS Access and Visio format with the sample code for this chapter should you wish to modify it and use it yourself. It is a very simple example, but we will look at a fully worked e-commerce example in the following two chapters. We will also be using examples from other types of systems where necessary.

One-to-Many

The one-to-many relationship will raise its head in pretty much every relational database you build. In a one-to-many relationship, one row in a parent table is related to many rows in a child table. Let's take a look at some examples to illustrate how a one-to-many relationship might be used.



In this case a category can contain many products. We use a one-to-many relationship when the parent can be associated with many instances of the child, but the child will only ever be associated with one instance of the parent. Take a look at the following sample data for these two tables:

Categories : Table

CategoryID	CategoryName
1	Running Shoes
2	Hiking Boots
3	Sweatshirts
4	Hooded Sweats
5	V Neck Sweats
6	Tracksuits
7	College Teams

The simple contents of the Categories table

Technically the many end of a one-to-many relationship can be zero, one, or many. Calling it a one-to-zero-one-or-many relationship would be too long-winded for something so common, so it's generally referred to as a one-to-many.

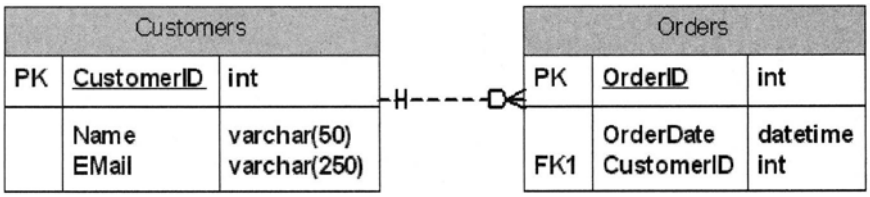
Products : Table

ProductID	ProductShortName	ProductDescription	ProductPrice	CategoryID
1	Super Sprint Shoes	The fastest running shoe on earth.	$89.95	1
2	Alpine Boots	Super waterproof. Accept crampons.	$160.00	2
3	Summer Tracksuit	Cool and loose fitting.	$75.00	6
4	Winter Tracksuit	Warm, for those cold days jogging.	$90.00	6
5	Tracksuit Combo Deal	Buy Summer and Winter and save $$$	$150.00	6

The contents of the Products table

A product is associated with a category by the `CategoryID` foreign key in the **Products** table. You will see that the last column in the **Products** table 'references' the category that the product sits in. So for example 'Super Sprint Shoes' are in the 'Running Shoes' category. The fact that the Tracksuits category contains three products shows that this is indeed a one-to-many relationship. If we look back at the diagram of these two tables we can see that the relationship line has the *one* end pointing at **Categories** and the *many* end pointing at **Products**.

Another example of a one-to-many relationship is **Customers** and **Orders**.



A customer can place many orders, but each order will only ever be placed by one customer. Once again the data for these two tables is fairly easy to understand:

Customers : Table

	CustomerID	Name	EMail
▶ +	1	John Smith	jsmith@smith.com
+	2	Bill Young	byoung@billsboats.com
+	3	James Jones	james@jonesshoes.com
*			

The data in the Customers table

Orders : Table

	OrderID	OrderDate	CustomerID
▶ +	1	11/7/2002	1
+	2	12/9/2002	1
+	3	9/2/2013	2
*			

The data in the Orders table

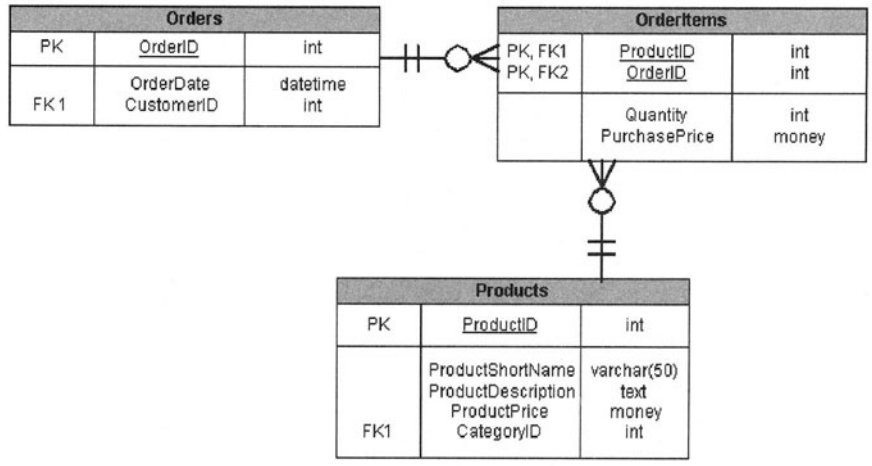
Orders are associated with customers by the `CustomerID` foreign key column in the **Orders** table. You can see that John Smith has two orders and Bill Young has one order. James Jones doesn't have any orders.

One-to-many relationships are very easy to understand and, because of the way that the relational database model works, very easy to implement in modern RDBMSs.

Many-to-Many

The many-to-many relationship is almost as common as the one-to-many relationship, but it is more difficult to implement in a relational database. As humans we can conceptualize a many-to-many relationship fairly easily. A student takes many classes, and a class can contain many students. At the database level we can't implement a many-to-many relationship directly. The reason for this is that we would need to have repeating groups of foreign key columns in one of our tables – but even this approach wouldn't give us a true many-to-many relationship as we would be limited to the number of repeating foreign key columns we included.

Instead we rely on the relational model to help us to implement a many-to-many relationship correctly. To do this we use what is commonly known as an associative entity or joining table. This is best illustrated by an example:



In this example the **OrderItems** table is our intermediary table. It allows us to associate many products with many orders. If we look at the data for the three tables you can see how this association is made:

Orders : Table

OrderID	OrderDate	CustomerID
1	11/7/2002	1
2	12/9/2002	1
3	9/2/2013	2

The Orders table, detailing when and by whom our orders were made

OrderItems : Table

OrderID	ProductID	Quantity	PurchasePrice
1	2	1	$160.00
2	4	2	$90.00
2	5	1	$150.00
3	4	1	$90.00

The OrderItems table, detailing which product is in what order

Products : Table

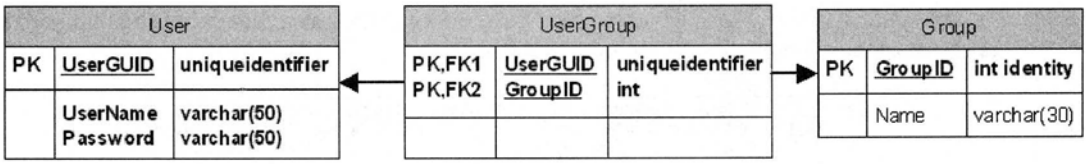
ProductID	ProductShortName	ProductDescription	ProductPrice	CategoryID
1	Super Sprint Shoes	The fastest running shoe on earth.	$89.95	1
2	Alpine Boots	Super waterproof. Accept crampons.	$160.00	2
3	Summer Tracksuit	Cool and loose fitting.	$75.00	6
4	Winter Tracksuit	Warm, for those cold days jogging.	$90.00	6
5	Tracksuit Combo Deal	Buy Summer and Winter and save $$$	$150.00	6

The Product table, detailing the products for sale

Products are associated with the order by inserting rows into the associative table, **OrderItems**. So you can see that the order with an `OrderID` of `2` contains two products – that is there are two rows in the **OrderItems** table. Conversely, the product with a `ProductID` of `4` has been included in two different orders – there are two rows in the **OrderItems** table referencing that product. Each **OrderItems** row also lets us specify some parameters for the association, namely the quantity of the product that has been ordered and the actual price at the time the order was placed.

As you can see from the four rows in the **OrderItems** table it is possible to have multiple products in an order (`OrderID 2`) and it is also possible to associate the same product with multiple orders (`ProductID 4`).

In many situations there is no need to have additional columns in the associative table. That is, the associative table is used purely to facilitate the many-to-many relationship. For example, in the following diagram we can see that the associative table is simply used to create a many-to-many relationship. Note the name of this intermediary table is a concatenation of the two tables that it is being used to join. It is a common style among database designers to do this, especially where the joining table only uses two columns, that is, the foreign keys:



Another issue that often arises when designing a database is when to use a many-to-many as opposed to a one-to-many relationship. While a many-to-many structure is able to represent one-to-many data fairly easily, the inverse is not true. The easiest way to decide which relationship to use is to verbalize what is happening, in both directions. Then take these phrases back to the end customer as questions. So for the above example, the two questions would be:

- Can a user belong to many groups?
- Can a group contain many users?

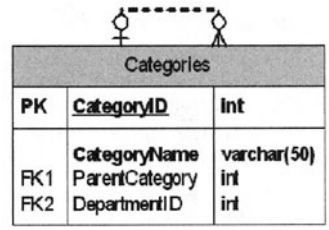
One-to-Many Recursive

Now that we have the simple relationships out of the way we can dig a bit deeper into the ER modeling toolbox to take a look at some of the more sophisticated relationship types. The first two that we'll look at are what is called recursive relationships, although some people prefer reflexive. These occur when a table is related to itself. Now this might seem a little confusing so, again, let's dive straight into an example to get some real-world context.

A very common problem in web-based applications is sorting various 'things' into categories, such as categories of products, or categories of users. In our e-commerce example we need to place products into categories. In most e-commerce stores the product range is too large to store it in a flat set of categories; they really need to be nested, that is categories within categories. For example, on Amazon this book would probably sit inside a category tree like the following:

- Computers
  - Databases
    - Database Design

Recursive relationships provide us with an easy way to model these tree type structures in a database. Let's have a look at how we could model a category tree.



As you can see from this diagram, the **Categories** table has an optional one-to-many relationship with itself. If we look at the data in the categories table we will get a better understanding of how this recursive relationship works to build our tree structure.

Categories : Table

	CategoryID	CategoryName	ParentCategory	DepartmentID
⊞	1	Running Shoes		1
⊞	2	Hiking Boots		1
⊞	3	Sweatshirts		2
⊞	4	Hooded Sweats	3	2
⊞	5	V Neck Sweats	3	2
⊞	6	Tracksuits		1
⊞	7	College Teams	4	2
*				

The data in the Categories table, showing parent categories

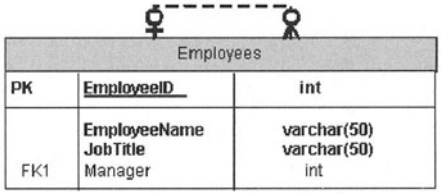
The two columns that we are most interested in are `CategoryID` and `ParentCategory`. The `ParentCategory` column is the foreign key to `CategoryID`. The rows that have a blank cell contain a null entry for their parent category and so are at the top of the tree. Those rows that have a 'real' value for `ParentCategory` sit further down in the tree. So for example `Hooded Sweats` is a subcategory of `Sweatshirts`. If we break the data above down into a tree-form we get something like the following:

- Running Shoes
- Hiking Boots
- Sweatshirts
  - Hooded Sweats
    - College Teams
  - V Neck Sweats
- Tracksuits

> *It is important to check before inserting or updating rows in a hierarchical table*

You can see how powerful this approach is. We can potentially nest our categories as deeply as we want to. Hierarchical tree structures arise in all sorts of situations and recursive relationships give us a very elegant approach to modeling them in a database. There are some pitfalls with this approach though. We cannot rely completely on the database to ensure the integrity of our data – for example it would be possible for us to accidentally set the parent of a node to be a node that is actually below it in the tree. For example, if we accidentally set the parent of `Sweatshirts` to be `College Teams` the database would not recognize this error even though we would have effectively corrupted the tree. It is therefore important to check for this situation before inserting or updating rows in a hierarchical table. One approach is to do this in your application code. An even better approach is to use a special database mechanism called triggers. We discuss triggers in more detail in *Chapter 8*, but they effectively allow us to run procedural code within the database when a row is inserted, updated, or deleted from a table.

Another common example of using a one-to-many recursive relationship is when we want to set up the management hierarchy of an organization. Take a look at the following example. It uses the same approach that we saw above with categories.



In this case the hierarchy has a single top level, the CEO.

An important thing to note with recursive relationships is that the foreign key column (`ParentCategory` and `Manager` in the examples) must always permit null. The reason for this is that there must always be a top level to the tree. That top level will have no parent record to point to and so its foreign key column must be set to null. If we used a value other than null to indicate the top level of the tree, we would break one of the key database rules called referential integrity – we will look at referential integrity in more detail later in this chapter.

Employees : Table

	EmployeeID	EmployeeName	JobTitle	Manager
+	1	Simon Smit	CEO	
+	2	James Bacon	Footware Manager	1
+	3	Robert Jones	Clothing Manager	1
+	4	Richard Smith	Shoes Warehouse Storeman	2
*				

Programming for Recursive Relationships

While this chapter is focused more on the general concepts of database design, it is still important to quickly show how to work with recursive relationships in our applications. We can navigate the tree from either direction. We can start at the top level of the tree and work down, or we can start with a given node and work back up the tree. The latter approach is fairly trivial, but to retrieve the whole tree, say to draw a menu, requires the top-down approach, which is a bit more complex. The following pseudo-code sections illustrate how this can be done for the **Categories** table above. Note the use of a recursive function – that is a function that calls itself:

By 'pseudo code' I mean that this code is not in any particular language, rather it is a semi-plain English description of how the program should run.

```
//Fetch the top-level Recordset
Recordset = DBQuery("SELECT * FROM CATEGORIES")

//Call the recursive function passing the top-level Recordset
traverseTree(Recordset,0)

Function traverseTree(pRecordset,pDepth)
  For each CurrentRow in Recordset
```

```
    //Output this rows details
    Print CurrentRow
    //Select all children of this row
    newRecordset = DBQuery("SELECT * FROM CATEGORIES WHERE ParentCategory=" +
                           CurrentRow("CategoryID"))
    //Call this function again passing the Recordset of children
    traverseTree(newRecordset,pDepth + 1)
  End For
End Function
```

Note that we also pass the current depth to the recursive function. This is often useful for determining how much to indent each level. The code above retrieves the entire tree. We could just as easily return just one branch by only selecting a single item from the top level – we would just have to change the first line of code to something like:

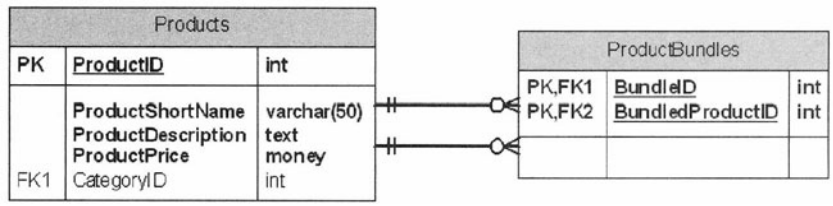
```
Recordset = DBQuery("SELECT * FROM CATEGORIES WHERE CategoryID = 3"
```

You should be **very** careful about where you use this type of code in your web-based applications. Depending on the depth of the tree, it has the potential to make tens or even hundreds of queries against the database. If you have hundreds of users hitting your site and running the code at the same time you may run into problems. A good way to get around this is to cache the data in a more tree-friendly format such as XML. For more information on using XML in your web applications take a look at *Practical XML for the Web*, from glasshaus, ISBN 1904151086.

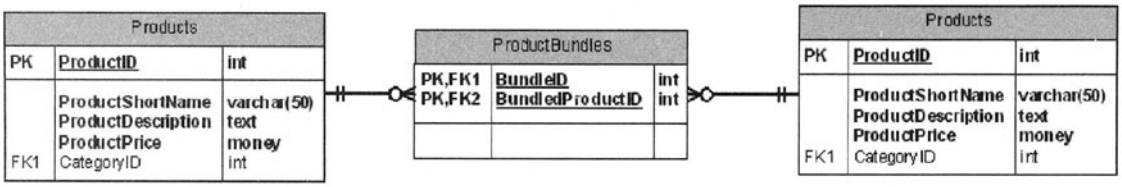
Another good method for retrieving data from a recursive structure is to extract it from the database in a flat format and then process it using code to turn it into a tree structure. Once again XML and XSLT are a good choice because XSLT is specifically designed to efficiently deal with recursive algorithms like the one we saw above.

Many-to-Many Recursive

The many-to-many recursive relationship takes things a step further. In this case a table is related to itself through an associative entity. Many-to-many recursive relationships are great for situations where something is an aggregate of other things of the same class. So, for example a computer is a product, but it is also an aggregation of other products sometimes know as a parts explosion or bill of materials. Take a look at the following example; in it we are using a many-to-many recursive relationship to help us represent bundles of products:



We are using a simple many-to-many relationship here, as we do not need to store any additional information about the relationship. Many-to-many recursive structures can be a bit confusing, so a good way to visualize it is to imagine that we are really dealing with two tables – or rather the same table twice. Note that the following is not a valid diagram; rather, it shows how the **ProductBundles** table really is just an intermediary table similar to those that we saw when we looked at many-to-many structures.



If we take a look at the data in the two tables we will get a better understanding of how the relationship is working:

Products : Table

	ProductID	ProductShortName	ProductDescription	ProductPrice	CategoryID
▶ +	1	Super Sprint Shoes	The fastest running shoe on earth.	$89.95	1
+	2	Alpine Boots	Super waterproof. Accept crampons.	$160.00	2
+	3	Summer Tracksuit	Cool and loose fitting.	$75.00	6
+	4	Winter Tracksuit	Warm, for those cold days jogging.	$90.00	6
+	5	Tracksuit Combo Deal	Buy Summer and Winter and save $$$	$150.00	6
*					

The data in the Products table; note the ProductID column

ProductBundles : Table

	BundleID	BundledProduct
▶	5	3
	5	4
*		

The ProductBundles table links products into bundles, using ProductID to identify the bundle and its products

Our `Tracksuit Combo Deal` bundle is a product just like any other product, but it is actually a product that is made up of other products. We are using the recursive many-to-many relationship to allow us to associate many products with many other products. We need to be able to have many bundles, and each bundle must be able to contain many products. We add products to the bundle by entering rows into the **ProductBundles** table. As you can see, our `Tracksuit Combo Deal` contains both the `Summer Tracksuit` and `Winter Tracksuit` – this is because the associative table contains a reference to both `ProductID 3` and `ProductID 4` for the bundle. This approach to aggregation gives us an enormous degree of flexibility. Not only can we have bundles of products, we can have bundles of bundles. This is probably a bit overboard for our current example, but for companies with complex stock lines, like a computer hardware company, this powerful aggregation functionality is very useful.

> *This approach to aggregation gives us an enormous degree of flexibility*

As we saw above with one-to-many recursive relationships it is very important to avoid circular references. For example it should not be possible for a bundle to contain itself.

One-to-one

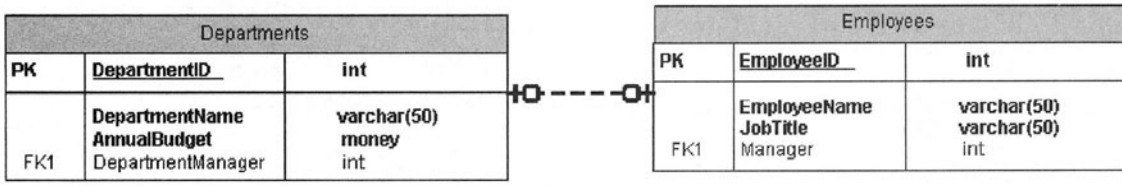
The final structure that we will look at is the one-to-one relationship. One-to-one relationships are not particularly common – where they do exist they usually take one of two forms:

- Actual one-to-one Relationships
- Sub-type/Super-type Relationships

We'll look at them separately as they have quite different design implications.

Actual One-to-One Relationships

The first type is for occasions where there is an actual one-to-one relationship between two things in the real world. Take a look at the following example:



Each department is managed by one and only one manager, and each manager manages one and only one department.

Employees : Table

	EmployeeID	EmployeeName	JobTitle	Manager
+	1	Simon Smit	CEO	
+	2	James Bacon	Footware Manager	1
+	3	Robert Jones	Clothing Manager	1
+	4	Richard Smith	Shoes Warehouse Storeman	2

The data in the Employees table, showing EmployeeID

Departments : Table

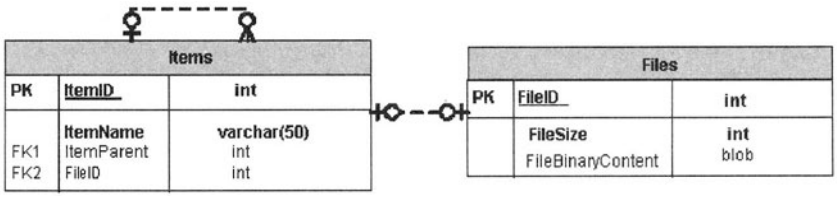
	DepartmentID	DepartmentName	AnnualBudget	DepartmentManager
+	1	Shoes	$500,000.00	2
+	2	Clothing	$1,500,000.00	

The data in the Departments table showing which employee manages which department

In terms of the data in the tables it is very similar to a one-to-many relationship, but of course we can only ever have one row in the child table. You should be careful when using actual one-to-one relationships – there are no efficiency gains over a similar one-to-many relationship and the latter affords a greater degree of flexibility. If you think that you may need an actual one-to-one relationship in your model make sure that you go back and check with the client. In this case, it might actually be more sensible to use a one-to-many relationship even if there will generally only be one-to-one data stored in it.

Sub-type/Super-type Relationships

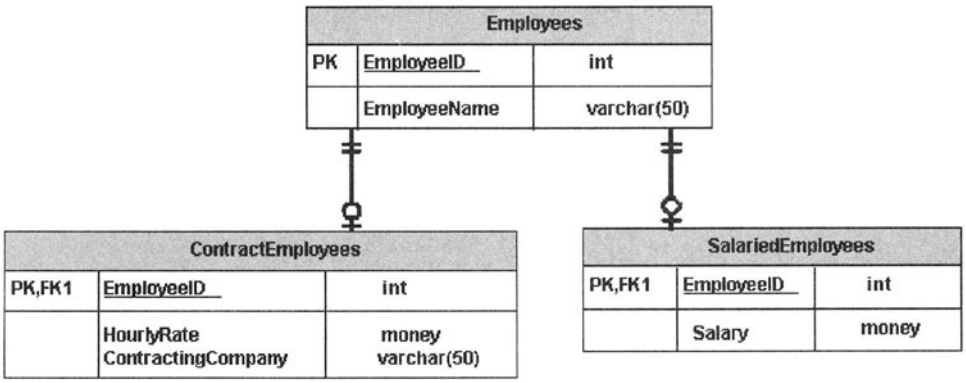
The more common form of a one-to-one relationship is the sub-type/super-type relationship. These sorts of relationships are used where entities in a database share some common attributes, but also have distinct attributes of their own. There isn't a sub-type example in the e-commerce example, but take a look at the following example, which is used to model a file system-like structure.



The **Items** table represents files or folders in the file tree. As you can see it has a one-to-many recursive relationship to form a tree structure. Where an item is a folder, the `FileID` foreign key column in **Items** will be null. Where an item is a file there will also be a corresponding row in the **Files** table and as such the `FileID` foreign key column will reference this row. In this way we are able to keep the file-specific information in a separate table but still treat files as items when we are building the tree for display.

> *In models where we have more than one sub-type, we need to place the foreign key on the sub-type side of the relationship*

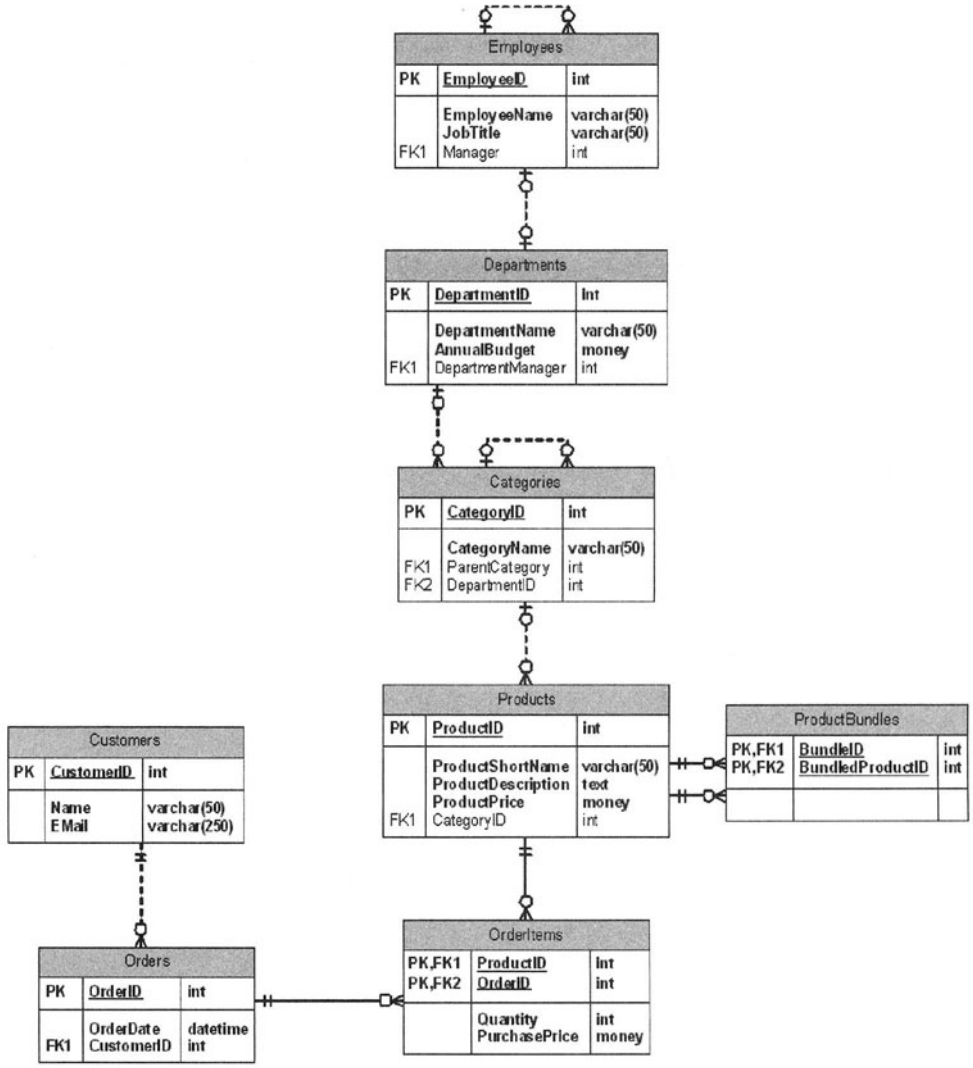
Which side of the relationship we place the foreign key on will depend on whether we have more than one sub-type or not. In the file systems example above we have a single sub-type and so we can place the foreign key in the super-type table. The advantage of this is that we do not need to perform a join between the two tables to determine whether an item is a file or folder – we can simply check if the `FileID` foreign key column is null. In models where we have more than one sub-type, we need to place the foreign key on the sub-type side of the relationship. For example, take the following example:



We can't store the foreign key on the super-type side of the relationship in this case because it would mean having two foreign keys, one for each of the sub-type tables. In the model above it is theoretically possible for an employee to be both a contract type and a salaried type. A good way to prevent this from happening would be to use a check constraint on the foreign key columns on each of the sub-type tables. We will look at check constraints a bit later in this chapter.

E-Commerce Example Database Diagram

As promised, here is the full database diagram for the e-commerce system:



Integrity Constraints

Integrity constraints are the rules that ensure that the data in our database is valid. These rules restrict the actions that applications can perform against our databases. By putting the data integrity rules into the database rather than the application layer, they will be enforced regardless of the particular application that accesses them. This is especially important when you have limited control over the applications or application developers who will be using your database.

There are three key areas where we can enforce integrity constraints in the database.

- Entity Integrity
- Attribute Integrity
- Relational Integrity

In this section we will look at each of these three types of integrity constraints.

Entity Integrity

Entity integrity is based around the idea that every row in a database must be unique. Specifically, the primary key must be unique – the remainder of the columns in the row may well be like other rows. The reason for this is that any time that an entity is to be related to another, that relation will be based around the primary key. If the primary key is not valid then no relationship can be created. For the primary key to be valid it has to meet the following criteria.

For single-column primary keys:

- The values in the primary key column must be unique.
- The primary key column may never contain null.

For concatenated or composite primary keys:

- The concatenation of all of the primary key columns must be unique.
- None of the primary key columns may contain null.

The reason that nulls are never allowed as part of the primary key goes back to the nature of null that we saw back in *Chapter 2*. Null is not a value, and because it is not a value, it cannot be a unique value. It may seem a little pedantic, but if you remember that null does not equal null then you will realize that we cannot compare null values in the column in order to determine uniqueness.

Another thing to note is that for concatenated keys, the columns that make up the key needn't be unique – only the combination of the columns is required to be unique. So, for example in the following table none of the three key columns (in bold) is unique in and of itself, however, the concatenation of all three does produce a unique key value:

Author	**Title**	**Publisher**	Price
J. Smith	Advanced Spoonbending	XYZ Press	23.95
J. Smith	Elementary Spoonbending	XYZ Press	23.95
R. Jones	Top Golf Holes	ABC Books	18.5
S. Robinson	Top Golf Holes	Greenkeeper Publishing	22.95

Keys that are made up of data that is meaningful to the end user are generally referred to as intelligent keys or natural keys. There has always been a lot of debate about whether or not to use intelligent keys. Whenever we use intelligent keys for our primary key we are taking a risk. The risk is that at some later point we may find an item of data that compels us to break the uniqueness requirement mandated by the primary key. For example, say XYZ Press decided to publish a special hardback edition of *Advanced Spoonbending* at $53.95. How would our database cope with this change? We could change the column or columns that make up the primary key, but this would require finding another column in the database that did not contain nulls and would give us a unique key. Of course there is still the risk that we may find a row that breaks the entity integrity rule once again.

Most database theory texts will go out of their way to discourage the use of surrogate keys wherever possible as they are in some way "impure". In the real world, however, everybody uses them all the time (sometimes exclusively).

Even values that we would think should always be unique can cause problems. For example, the obvious value to use with the book example above would be the ISBN number, which publishers use to uniquely identify titles. On the face of it this is a good choice: it is a unique number generated by the publishers specifically for identifying individual titles, just what we need in our database. There are some potential pitfalls, though: what about books that don't have ISBN numbers? Maybe they were published many years ago or are from a publisher that doesn't use ISBN numbers. What if the users of our database suddenly decide that they want to be able to store second-hand copies of the book as well as new copies? This would lead to duplicate primary key values. The best way for us to solve these potential risks is through the use of surrogate keys.

The most common argument that is made in favor of using intelligent keys is that they allow users to more easily understand the relationships in the database when using ad-hoc query tools (tools that allow the end user to write their own queries for reports and so forth). Support for ad-hoc queries in web applications is very rare; most web applications have to assume the lowest common denominator in terms of user ability and adhoc querying is not going to be one of those basic skills.

Using Surrogate Keys to Guarantee Uniqueness

A surrogate key is some sort of artificially generated unique value. Surrogate keys allow us to use a single column key where we otherwise have had to use a concatenated key, which is far less efficient in operation.

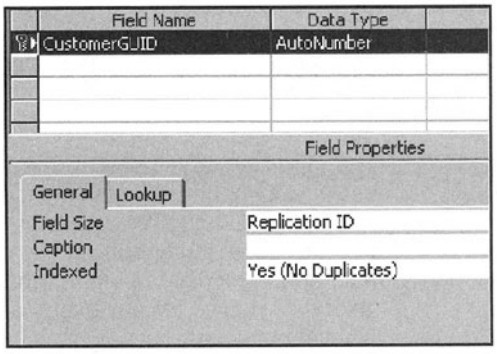
While most database systems allow surrogate keys to be generated within the database itself, it is also possible to generate surrogate keys in your application. There are two common types of surrogate key:

- Numeric sequence keys
- Globally Unique Identifiers (GUIDs)

Numeric sequence keys are based around a simple sequence of integers, normally starting at 1. Numeric sequence keys are available in most of the major database products. In Microsoft Access, sequence columns are called `autonumber` columns; in Microsoft SQL Server they are called `identity` columns. In MySQL, `auto_increment` columns, and in Oracle, you create a special database object called a `sequence`, which you can then use to generate unique key values. In PostgreSQL, you can either take the Oracle approach and use a `sequence` object to build your surrogate keys, or you can use the built-in `serial` data type which implements all of the sequence functionality for you behind the scenes. We will take a look at using numeric sequence surrogate keys in *Chapter 6* when we build our sample database application.

Globally Unique Identifiers are less commonly available. They are natively supported in SQL Server and Access. SQL Server uses the `uniqueidentifier` column type whereas in Access you need to create an `autonumber` column and set its type to *Replication ID*, as in the following screenshot:

Some developers like to use a timestamp as a surrogate key, although there could be problems if your database inserts two pieces of data in the same table within a second, unless the timestamp is more accurate.



Postgres is able to support a `uniqueidentifier` column type using an extension library, see *http://archives.postgresql.org/pgsql-announce/2002-07/msg00001.php* for details. Oracle supports GUIDs for identification of object types, but not relational types. Oracle does expose its GUID generation function for you to use in your own code via the `SYS_OP_GUID()` function, as follows:

A common naming convention with surrogate keys is to suffix the column name with either 'ID'or 'GUID' depending on the surrogate key data type.

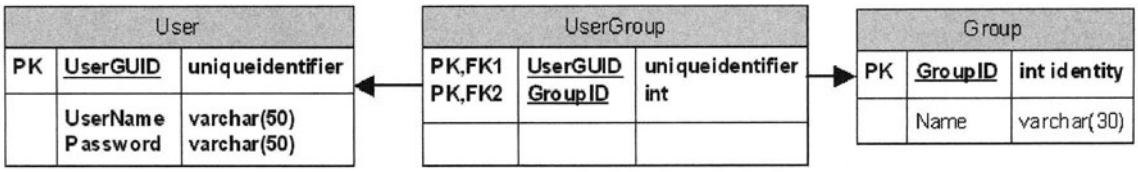
```
SELECT SYS_OP_GUID() FROM DUAL;
```

Unique identifier columns, while often a bit more difficult to implement, do have some properties which make them well suited to certain situations. They are often used by databases for managing replication; because they are globally unique they guarantee a unique key value even across different databases. As a general rule you should only use GUIDs in your application if you require the key to be unique even outside your particular database or where you want to hide the keying mechanism from malicious users as described in the note below.

Unique identifier columns are especially well suited to use in some areas of web applications. This is because they help prevent the user from being able to guess the key to look up a record. Take a site which allows users to view the profile of others by going to a page such as *view_user.asp?userid=463*. It would be fairly easy for a malicious user to work out that we have used a sequential key and to then write a program that attempted to fetch all the user records by simply scanning across the key range. If we use unique identifier columns there is no possibility for a user to do this – even guessing a single key is all but impossible.

When Not to Use a Surrogate Key

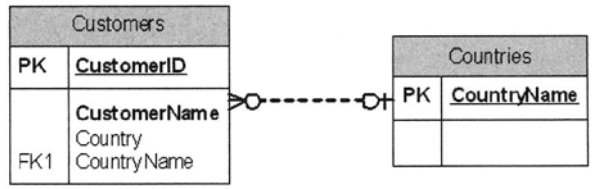
There are times when the use of a surrogate key becomes redundant. For example, take a look at the following database diagram.



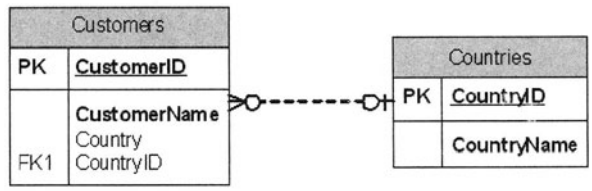
> *minimize the number of indexes on the table*

The purpose of the **UserGroup** table is to associate users with particular groups. It is an intermediary table for resolving the many-to-many relationship between **User** and **Group**. As you can see, instead of having a single-column primary key on the **UserGroup** table we have used a concatenated key consisting of the two foreign keys. By taking this approach we minimize the number of indexes on the table (as you will see in *Chapter 8*, the more indexes a table contains the slower it will be to `INSERT` rows into, although indexes generally make it faster to `SELECT` rows). Adding a new surrogate key to the **UserGroup** table would be pointless, a user can only ever belong to a group once and we can compose a perfectly suitable primary key out of the two foreign keys, which are themselves surrogates.

In some situations intelligent keys can also reduce the number of joins we have to make when querying our database. For example, say we used a lookup table of countries in our application. We might then reference this lookup table from our **Customers** table. Take a look at the following two diagrams and the primary key that we have used on the **Countries** table, the first uses an intelligent key:



The second uses a surrogate key:



When we come to select data from our **Customers** table we want the `CountryName` for each customer. With the intelligent key approach, the query is fairly simple:

```
SELECT CustomerName, CountryName FROM Customers;
```

Because the value in the `CountryName` foreign key column is the actual `CountryName` value that we are after, we do not need to join with the **Countries** table in our query. Using the surrogate key approach the above query (with `CountryID` replacing `CountryName`) would only return the `CountryID` of the corresponding country; we would have to use a join in order to retrieve the data:

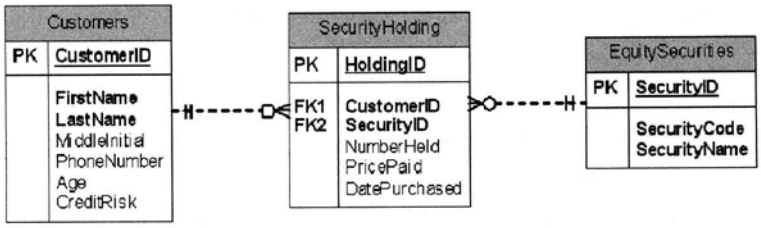
```
SELECT Customers.CustomerName, Countries.CountryName
FROM Customers
INNER JOIN Countries ON Customers.CountryID = Countries.CountryID;
```

Used sensibly, intelligent keys can help us avoid this unnecessary overhead. But as a general rule surrogate keys are very well suited for use in web-based databases.

Attribute or Domain Integrity

Attribute or domain integrity is concerned with the validity of the data stored in our attributes. The domain of an attribute is the full set of values that attribute is allowed to have.

Like most of the important aspects of designing our databases, we will discover most of the things we need to know about the domain by talking to the customer or end user of the system. The sort of questions you need to be asking them are things like – 'Do you need to be able to store a value of unknown for a customer's gender?', 'How long will a product description need to be?', 'To how many decimal places do you need to store the price of a stock?'. Let's take a look at a couple of entities and think of some hypothetical domain integrity rules that we might need to apply to them. Say we have the following tables in our database for managing stock portfolios for customers online:



It is a pretty simple schema really, a many-to-many relationship between **Customers** and **EquitySecurities**: customers can hold many securities, and a security can be held by many customers.

The customer might have the following set of business rules that we need to accommodate in this database:

- We need to store the age of our customers.
- We need to keep tabs on whether a certain customer presents a credit risk.

- We want to give a customer the option of keeping a variety of information about each of their holdings
- Securities in the market we are trading on are represented by a unique security 'ticker' code that is exactly 4 characters long.

On top of this we will have some rules of thumb that we've developed in our time as database designers that we'll want to apply to each of the attributes in order to determine their domains.

Most RDBMSs don't support domains explicitly; rather the domain of an attribute will be determined by a number of different settings that can be applied to that attribute. We looked at some of these settings in *Chapter 2*. They are:

- Data Type
- Length
- Constraints
- Optionality
- Default Values

Let's look at each of these settings in more detail and how we might use it to apply the correct domain integrity rules to our above example.

Data Type and Size

Determining the data type of a column is probably the most important setting that we have to decide on. When we are picking data types we are basically balancing and trading off three key factors:

1. We want the data storage format that we choose to be as efficient as possible. Both in terms of space usage, and speed of storage and retrieval. Generally we want to use the smallest data type possible for the data we are storing while still leaving enough leeway for unexpectedly large values.
2. We want the data to be in an appropriate format for our application as soon as it comes out of the database. We don't want to have to convert our data between formats before we can make use of it in our application.
3. We want to maintain as much flexibility as possible. Even if the current requirements don't necessarily mandate a certain type of data, it is good to think of potential changes that require it in the future and weigh up the costs of using that data type from the outset.

Most of the database platforms that we use today come with a myriad of data types available for use. Some of these are covered in the *Appendix*, but the most common ones are outlined in the sections below. These data types are supported by all of the major database platforms but their names may differ, for more information on a specific database platform see the *Appendix* or search for 'Data Types' in the documentation for your database. Some modeling tools allow you to design with 'generic' data types and then have the modeling tool choose the correct type for the chosen platform when you forward engineer the database. We will look at forward engineering at the end of this chapter.

INTEGER

A fixed precision number that has no decimal places. The standard size in most databases is 32 bits, which gives a range of -2147483648 to +2147483647. Most databases also support smaller and larger variants of the integer. One important note with integers is to make sure that your programming language representations line up with your database representations. For example a VBScript Integer is only 16 bit (+32000 to -32000) so trying to convert database integers to VBScript integers can result in overflow errors once the numbers go over 32000. The solution is to make sure you understand the size limitations of both your programming language and database and how they map with each other in terms of size and capability.

CHARACTER

The SQL92 standard defines three types of character types:

- `CHAR`, which is a single character.
- `CHAR(n)`, which is a fixed-length, character data of length n.
- `VARCHAR(n)`, which is a variable-length character data with a maximum length of n.

The difference between `CHAR(n)` and `VARCHAR(n)` is how they handle data which is shorter than `n`. `CHAR` attributes will pad the data with spaces at the right-hand end to make up the correct length, n, `VARCHAR` attributes will only store the actual data. There are trade-offs with both approaches. With say a `VARCHAR(30)` attribute, updating data of 20 characters in length to data of 21 characters in length will require more resources than doing the same to a `CHAR(30)` field because it needs to find new space on the disks to store the data, whereas with the `CHAR` attribute it just changes some of the padding characters to data characters. In many cases, however, the extra storage we would use to store `CHAR(n)` attributes outweighs the potential efficiency benefits in terms of updating. For example if an attribute could be any length between 1 and 100 it would probably be stupid to always store 100 characters even if the data was only 1 character in length.

Some older database platforms are very restrictive in terms of the length they allow for `CHAR` and `VARCHAR` columns – often it is as little as 255 characters. You should check the documentation for your DBMS to see what the maximum allowable length is. Most databases have a data type for storing very large text strings, but they are generally nowhere near as efficient as `CHAR` and `VARCHAR` columns.

DECIMAL

Decimal numbers are the numeric equivalent of a `CHAR`. They are a packed decimal number with an implicit decimal point. The actual size of the column is determined by variables called precision and scale. **Precision** is the number of digits in the figure, and **scale** determines the position of the decimal place (how many digits are after the decimal place). So for example the number 157.4568 has a precision of 7 and scale of 4.

To create a table containing this decimal column we would use the following DDL:

```
CREATE TABLE SomeTable (
        DecimalNumber decimal(7, 2)
)
```

FLOAT

Most databases also allow for floating point data types. These are numbers with a decimal point but where the scale is not fixed. Examples include such types as `REAL`, `DOUBLE`, `FLOAT`, `MONEY`, and `CURRENCY`. The size limitation of these types tends to vary greatly between different databases and even different platforms. For example, the `MONEY` data type in PostgresSQL is a `DECIMAL(9,2)` type whereas in SQL server it is a `DECIMAL(19,4)` type.

DATETIME

The SQL standard defines separate `DATE` and `TIME` data types. Most RDBMSs define their own `DATETIME` combination type that stores both the date and the time in a single attribute. This is generally the best choice for attributes that require either type.

Let's look at how we should apply data types to the attributes in our example.

Simple Choices

For the name attributes we will use a combination of `CHAR` and `VARCHAR`. `FirstName`, `LastName`, and `SecurityName` will be `VARCHAR`. `SecurityCode` and `MiddleIntial` will be `CHAR`. Lengthwise I usually follow a couple of rules:

1. For `CHAR` columns, we should set the length to be equal to the maximum length of data we'll be including.
2. For `VARCHAR` columns I have a series of column sizes I like to use – 20, 50, 100, 250, 1000, 2000. I simply decide on the maximum column length of the column and then choose the next size above this. In this way, when it comes to coding time and I need to know how long a column is allowed to be, I can easily remember the size of the column without having to go back and look at the data model.

If you prefer to keep your `VARCHAR` lengths closer to the maximum length of the data, you could include the length of the column in the name, like `LastName50`

For phone number we will use a `CHAR`. This might surprise you a little, using a `CHAR` for a 'number'. The reason for this comes back to the factors that I listed above (specifically the second point). In terms of how our application uses phone number data it is not numeric. We don't add two phone numbers together, or multiply them. Most of the time we only ever want to display the number to the user. If we are actually going to be 'using' the number in our code it will probably be to drive some sort of number dialing code – this task will work just as well with character data. While an `INTEGER` column would be more space efficient, we lose a lot of flexibility in requiring it to be a number. For example, we would not be able to store numbers that require special dialing codes such as **999* or *2377544#44*, codes which are becoming increasingly important in this age of mobile telephony. As a general rule, we should only use numeric column types when we will have to deal with the data as a number in our application, such as performing mathematical operations or logical comparisons.

More Complicated Choices

All of the above data type choices were pretty obvious. For some of the others we probably need a bit more information from the customer. How does their business classify credit risk? Do they have some sort of sliding scale? What about the price of securities? What level of detail do we need to store these to? What is the highest price that a security could take? How accurately will users need to store their date of purchase? Will they want to store the time that they made the purchase as well as the date? This is an example of why database modeling must be an iterative process; sure we collected some information at our initial interview with the client, but in many ways the information we received just raised more questions. Let's say we put those questions to the client and they answered with the following:

- We simply store credit risk as a yes or no. That is, a customer is either a credit risk or they are not a credit risk.
- Because securities are generally traded in high volumes the price of a security needs to be stored to quite a high degree of precision. For example, 0.1 of a cent on a four-cent stock is going to be quite significant if the customer owns 4 million of that stock. We need to be able to store stock price in dollars to four decimal places – that is a stock price could take the form $3.6525. Realistically we have never seen a stock worth more than a couple of thousand dollars, but given the volatility of the stock markets we would like you to leave us with some room for growth there.
- Because stocks can change so much over the course of a day we need to be able to store the time of purchase down to the minute level as well.

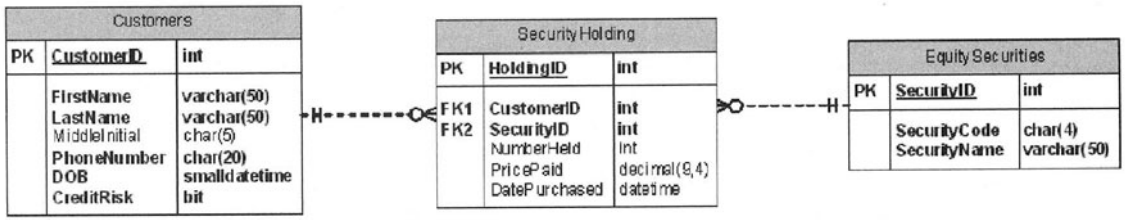
With this refined information in hand it is easy for us to decide on the data types. The `CreditRisk` attribute really just needs to hold an on/off value. Some RDBMSs support `BOOLEAN` or `BIT` types for this purpose. In others we would probably use the smallest `INTEGER` type available. If you are using a good modeling tool it will probably let you choose to use portable data types. What this means is that the modeling tool stores the fact that you want a True/False column even though it doesn't know which database you are ultimately going to use. When the time comes to forward engineer your database the tool will choose the most appropriate column type for your selected database platform. We'll see more about modeling tools later in the chapter.

For the `PricePaid` attribute we have used a `DECIMAL` data type. Because it is very unlikely that we will ever go beyond four decimal places in this column the `DECIMAL` data type is more than adequate. It also offers us the advantage that our application will not have to deal with rounding errors as a result of using say a `FLOAT` data type. Once again, if we configure our modeling tool to use portable data types the correct data type will be chosen at the forward engineering stage.

Finally, we will use a `DATETIME` type for the `DatePurchased` attribute.

The only attribute outstanding is `Age`, and this one requires a bit of a rethink. We could simply store the age of the customer as an integer in the column, but because people's ages change over time, our data would be inaccurate the moment it was entered. It would be much better to store their date of birth and then calculate the age from this whenever we need it. Temporal (time-based) data like this can be a real gotcha when converting a customer's requirements into a database: it's the time that we database designers get to show off experience and expertise. So, let's change age to be `DOB` and use a `DATE` type.

The final result, with data types in place will look something like this:



Optionality

The optionality of an attribute is what determines whether it is allowed to be null or not. You will remember we looked at null in *Chapter 2*. Different database platforms have different ways of implementing optionality. Some have native support, others use check constraints like those we are going to look at below. If we are using a modeling tool we can leave the tool to take care of all of those details. The modeling tool will choose the approach intelligently based on the database platform that we are targeting. If we choose to target a different platform it will choose a different approach. We just need to decide which of our columns should be allowed to be null. Looking at the diagram above, the weight of the typeface shows optionality. Bold for required, plain type for optional (allows null).

We need to be a little careful with null. As we saw in *Chapter 2* it is not possible to use traditional comparison operators with null. Nulls are, however, very useful as a special data placeholder for a column to indicate that no value exists. The other option to nulls is the use of default values such as 0 or "" (empty string). This approach can have its drawbacks as well which we will discuss below. In our example we have used nullability for the three optional attributes in **SecurityHolding**. We do this because we want to know if a customer has chosen not to enter information for any of those attributes.

Allowing nulls is also good when using `Char` columns. For example we have allowed nulls in the `MiddleInitial` column. Because many people do not have any middle initials we can avoid filling the `CHAR` column with padding characters by allowing nulls.

Default Value

As I mentioned above, default values are effectively a practical alternative to allowing nulls for an attribute. If you insert data into a table and do not specify a value for a certain column, the default value will be inserted. Some databases will also allow us to specify a function for the default value – this is great for using on columns that require the date of data entry.

> *You should use default values carefully*

You should use default values carefully; where you want to tell whether a user has chosen not to insert a value it is best to use nulls. For example, we could have used a default value of 0 for the `PricePaid` attribute in our example. It is foreseeable that a user may want to enter a value of 0 for this attribute – say if they had received the stocks as part of an employment package.

In the end we have only used a default value for one of the attributes in the example, for credit risk. We have used a default value of 0 to indicate no credit risk exists. Having this default value is important because it means we should never end up in the indeterminate position of not knowing whether a credit risk exists. Some modeling tools allow you to set a default value for the column. We can set the default value in DDL fairly easily; the following code shows how to use a simple default value.

```
,
CREATE TABLE SomeTable (
       SomeColumn varchar(50) DEFAULT ('This is the default value')
)
```

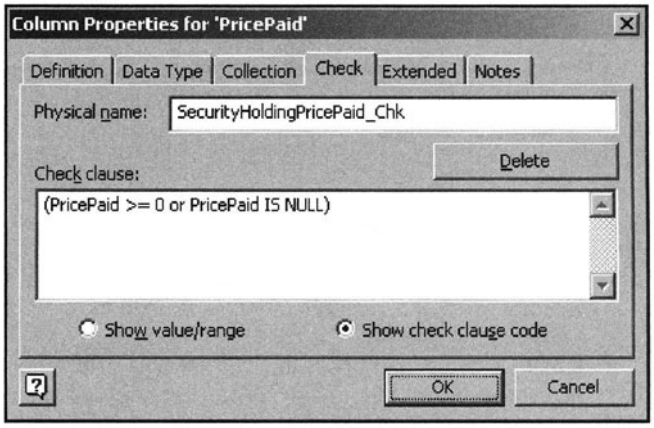
Most databases also allow us to use functions for the default value. A good use for this functionality is when you wish to populate a `DATETIME` column with the current date and time. For example the following syntax shows how to insert the current date as a default using SQL Server:

```
CREATE TABLE SomeTable (
       SomeDate datetime DEFAULT (getdate())
)
```

Check Constraints

As you have seen above, data type, size, and optionality give us a pretty good degree of freedom to restrict the domain of an attribute. Most RDBMSs also allow us to take this further by defining **Check Constraints**. Check Constraints (sometimes known as check clauses) allow us to set up specific rules against which the database will check values that are inserted or updated into the column. For example we can specify that the `SecurityCode` attribute only contains letters, and that the `PricePaid` column is a positive number. Ideally we should do this in our modeling tool (as below in Visio) then it can be responsible for creating the constraint in the actual database:

Notice how we have had to explicitly make provision for nulls in our check clause. If we had not done that the check clause would have overridden our decision to allow nulls for this attribute. This may puzzle you, but it comes back to the properties of null that we saw in *Chapter 2*. Null cannot be compared to anything, for example `NULL >= 0` returns `NULL`. This is what the `SecurityHoldingPricePaid` check constraint looks like in DDL – again the specific syntax is from SQL Server:




```
CONSTRAINT SecurityHoldingPricePaid_Chk CHECK (PricePaid >= 0 or PricePaid IS NULL)
```

Referential Integrity

Referential integrity (**RI**) ensures that the relationships between data in the database are valid. RI ensures that every foreign key attribute is either null or that it references a valid primary key in the related table. Referential integrity is usually enforced in the database by a special type of constraint. Fortunately CASE tools and the graphical editing tools that ship with most databases shield us from having to create these constraints ourselves. If you do need to do it in DDL the syntax will vary across platforms. The following code gives an example of how SQL Server approaches referential constraints – in this case for an intermediary table.

```
CREATE TABLE PropertyFeature (
        feature_code CHAR (5) NOT NULL ,
        property_id CHAR (8) NOT NULL ,
        CONSTRAINT propertyfeature_pk PRIMARY KEY  CLUSTERED
        (
                feature_code,
                property_id
        ),
        CONSTRAINT [propertyfeature_fk_featurecode] FOREIGN KEY
        (
                feature_code
        ) REFERENCES Feature (
                feature_code
        ),
        CONSTRAINT propertyfeature_fk_propertyid FOREIGN KEY
         (
                property_id
        ) REFERENCES Property (
        property_id
        )
)
```

> *If you do it in DDL the syntax will vary across platforms*

Referential integrity places checks against three potential problems:

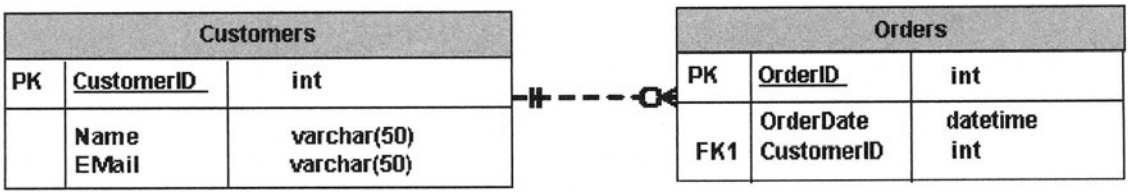
- Insert Anomalies
- Update Anomalies
- Delete Anomalies

Each one involves there being an orphaned row in a child table, that is, a row whose foreign key does not point at a valid primary key value in the parent table.

Note that some databases do not allow you to enforce referential integrity. Particularly, MySQL allows you to create relationships, but the database server will never actually enforce those relationships. This means that you need to be far more careful with checking appropriate values in your code.

Insert Anomalies

An insert anomaly occurs when a row is inserted into a child table that does not correctly reference a valid value in the parent table. Consider the following example.



Referential integrity requires that any records in the **Orders** table have a corresponding record in the **Customers** table. Without referential integrity being enforced we could potentially insert rows into **Orders** without a matching customer, thus creating an orphaned row. Referential integrity prevents insert anomalies from occurring by preventing us from inserting rows where there is not a corresponding row already in the parent table.

Update Anomalies

Update anomalies occur when we change a primary key value in the parent table leaving one or more rows in the child table pointing at the non-existent old primary key value. Referential integrity prevents update anomalies from occurring by not allowing us to change the parent primary key.

Delete Anomalies

Delete anomalies are similar to update anomalies. They occur when we delete a row from the parent table leaving child table foreign keys pointing at non-existent values. Referential integrity prevents delete anomalies by not allowing us to delete parent rows while they are still referenced by child rows.

Implications of Referential Integrity

Referential integrity has some fairly significant practical implications when we come to access our database in our web applications (or any application, for that matter). The first is that we should always insert into the parent table of a relationship first. In our stock example from above we would need to insert a record into the **Customer** table before we could create any **SecurityHolding** records for that customer.

The second and more significant limitation occurs when we want to delete a customer record. Because the foreign key attribute in the **SecurityHolding** table must always point to a valid primary key value in the **Customer** table we must always delete all of a customers **SecurityHolding** records before we can delete a customer.

Finally we can't update the primary key value while child rows still reference it, but neither can we update the child rows until we have updated the parent row: a stalemate position that prevents us from ever changing the primary key.

We can obviously write some code in our application to get around these last two issues, but it seems somewhat redundant to do so – what we really need is a way to tell the database what to do when we perform an action that might break the referential integrity of the database. Most RDMBSs allow us to do this by using Referential Actions.

Referential Actions

Referential actions are basically instructions to the DBMSs as to what to do when we try to break an integrity rule. We can't instruct the database to simply let us get away with breaking the integrity rules, but we can tell it to perform some remedial action that we might otherwise have to perform in code. The default behavior for referential integrity that we saw above (not letting us make the changes) is actually a specific type of referential action – `RESTRICT`.

We can apply referential action to two types of situation – updating and deleting records in the parent table. There are three referential actions commonly available in most DBMS packages.

Referential Action	Update Parent Key	Delete Parent Record
`RESTRICT` (Default action in most RDBMSs)	Updating parent records is prevented by RDBMS.	Deleting of parent records is prevented by RDBMS.
`CASCADE`	The updated parent key value will be cascaded down to the child records – that is, the child records will also have their foreign key values updated.	Deleting a parent record will also delete any associated child records.
`SET NULL`	The child foreign key will be set to null.	The child foreign key will be set to null.

> *Without referential actions we would have to manually delete all of the children before deleting the parent*

Because it is generally regarded as a less than ideal approach, some RDBMSs do not make explicit provision for the `SET NULL` referential action. If your design really needs this feature it is possible to mimic the `SET NULL` action by using triggers. We will look at triggers in *Chapter 8*.

Correct use of referential actions is very important for performance reasons. In many cases it is quite natural to be able to delete a parent and all of its children. We see this every day when we delete a folder in a file system and the files within that folder are also deleted. By using a `CASCADE` in our database we can achieve these parent/child deletions by executing a single statement against the database deleting the parent row. Without referential actions we would have to manually delete all of the children before deleting the parent. This would generally require at least three commands to be executed – one to check if any children exist, a second to delete those children, and a third to delete the parent.

In many situations, however, it is most inappropriate to use `CASCADE` deletes. For example consider a situation where an employee manages many projects. In this case, if we defined an `ON DELETE CASCADE` we would end up losing valuable project information anytime an employee left the organization. In this case it is more appropriate to use the default `RESTRICT` action. This would force us to either delete the projects ourselves if that was what we wanted, or more likely, we could update the relevant projects or assign them to another employee.

Denormalizing for Performance

Over the past few chapters you have had the importance of proper normalized database design drilled into you. Now it is time to talk about bending some of those hard and fast rules in the name of performance. If you remember back to the very start of this chapter I briefly mentioned the issue of redundancy and how we should really be determining the number of books that a publisher has in our catalog by using a `COUNT GROUP BY` function at runtime. In reality the decision is not quite that clear-cut. While storing the count of a publisher's books in the **Publishers** table introduces redundancy into our database, it also saves the database engine from having to count the number of matching book rows every time that we select a **Publisher** row. This can be extremely important in web-based applications because the load on the database server is often much higher than in more traditional systems.

So, it may in some situations be better to introduce redundancy into our database and store our data in a denormalized form. This should, however, be the exception to the rule and that is why I have left it until the end of this chapter. Denormalization should always be left until after you have built a properly normalized structure **and** tested it under a realistic working load equivalent to what it would receive in production. With web applications this often means leaving a decision about denormalization until the development is complete.

Denormalization is particularly common in web-based applications because it tends to favor applications that do a large amount of data reading compared to data writing. Denormalized data models require less processing when selecting data from the database, but more processing at insert and update time. In most web applications there is a much smaller proportion of insert and update transactions executed against the database compared to select transactions.

> *Denormalized data models require less processing when selecting data from the database, but more processing at insert and update time*

In some situations, however, this is not the case. For example most large e-commerce sites offer some sort of 'customers who purchased X also purchased Y' functionality. Often this is implemented by recording and summarizing past customers buying habits in the database. In this case, while denormalization is appropriate it would not be a good idea to do it when the actual insert occurred, as this would have an adverse impact on the user browsing the site or making purchases. Instead the summary data should be calculated and inserted into the denormalized tables either as a background task or as a batch task, once a night or once a week.

Achieving high performance in relational databases usually requires trade-offs to be made – particularly between speed of inserts and updates versus speed of retrieval. We will discuss these trade-offs in more detail when we look at indexing in *Chapter 8*.

Using CASE Tools for Modeling

As we saw just above, diagramming is a great way of expressing our data models in an easy to understand form. **CASE** tools (**Computer Aided Software Engineering**) allow us to build our diagrams in software, and then use those diagrams to automatically generate the database for us at the implementation stage.

CASE is a generic term for what we're doing here. More specifically, this is entity modeling, which is something to look out for as well – not all CASE tools will let you model a database.

There are a variety of CASE tools on the market. They range from freeware products through to very high-end enterprise applications with a corresponding range in price from nothing to many thousands of dollars. The following list gives some examples and where they sit in the price range. When choosing a CASE tool you should ensure that it supports the database(s) that you intend to use for development:

- Dia (Free)
- Datanamic DeZign (Low End)
- Model Creator ERCreator (Low End)
- Charonware Case Studio and Case Studio Lite (Low to Mid Range)
- Oracle Designer (Mid Range)
- Microsoft Visio (Mid Range)
- Quest Software Q Designer (High End)
- Sybase Power Designer (High End)
- Embarcadero ER/Studio (High End)
- Computer Associates ER Win (High End)

It is beyond the scope of this book to cover a variety of CASE tools, but we will briefly look at just one application to explore the opportunities that CASE tools present.

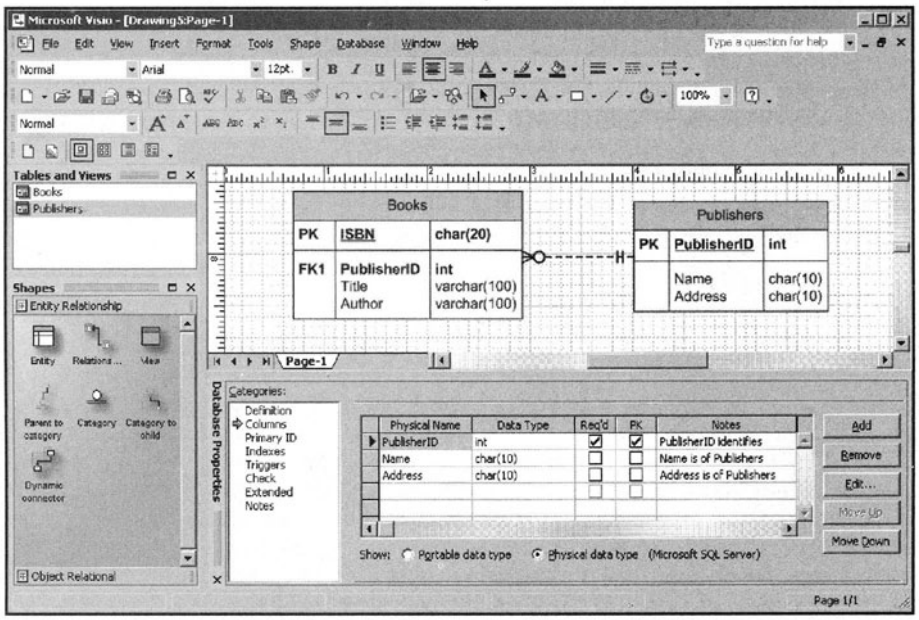
CASE Demonstration – Visio 2002

The CASE tool we will look at briefly is Microsoft Visio 2002. Visio falls into the middle of the price range. It is a mature product in the market having had several versions that supported database design prior to the Microsoft acquisition of Visio.

Visio is bundled with several of the Microsoft Developer tools; a free evaluation version on CD is available to order at *http://www.microsoft.com/office/visio/*, and from the same site there is an online trial version as well. While Visio works best for users of SQL Server and Oracle, it is also able to work with almost any ODBC-compatible database, though for some of the more advanced features your mileage may vary.

Diagramming

Visio is at its heart a diagramming tool. As such it has a very powerful drawing interface.



Visio 2002, using the Database Model Diagram template

The most recent versions of Visio have specific templates for database design. Looking at the screenshot above there are 4 main parts to the window, which have been defined by the 'Database Model Diagram' template.

Tables and Views

The tables and views section (middle left) lists all the entities in our model. This is important because in very large systems we may want to have multiple diagrams, each showing a different part of the system. The underlying model remains the same; we just have several diagrams that give us a different view of the data.

Shapes

The shapes section (bottom left) contains the various types of objects that we can drop onto our diagrams. When we drop entities onto the drawing area Visio will automatically create that object in the underlying data model and they will appear in the *Tables and Views* window. The shapes that we have dealt with in this chapter are the *Entity* and *Relationship* shapes. Some of the other shapes that are available allow us to diagram views (something that we will look at in later chapters) and subclass Parent/Category relationships.

Database Properties

The database properties section (bottom right) allows us to define the properties of entities and relationships in more detail.

Drawing Surface

The drawing surface (middle right) is where we lay out the diagrams of our model.

Forward Engineering

Most CASE tools support forward and reverse engineering of models into actual databases, with varying degrees of success. What this means is that given the model, the CASE tool will automatically communicate with the database server and build the database for you. Visio has the advantage that it can also merge changes into an existing database.

> *The level of model validation varies between different CASE tools*

Visio can either output a Data Definition Language (DDL) Script (`CREATE TABLE` statements that we saw in *Chapter 3*) or it can generate the database for us (or do both). Visio will validate our model, to check that we haven't broken any database design rules, and generate the correct objects in our database. The translation from diagram to database can be initiated through a wizard. Most database tools ship with a variety of different drivers for forward engineering a database model into different databases. Depending on the driver we choose, Visio will output the appropriate DDL for us. The level of model validation varies between different CASE tools, but most will at least ensure that the model that we have drawn will indeed produce a valid database by ensuring that the data types of foreign keys and their matching primary keys are the same.

Once you start working with models that contain tens or even hundreds of tables, the ability to quickly implement the database with just a few mouse clicks becomes quite appealing.

What forward engineering really does is take the pain out of modeling. It used to be the case that once we had developed a model in a drawing package, we would print it out and then generate all of the tables by hand in the database, crossing them off on our printout as we went along. Effectively we were doubling our workload by having to enter the same information into both the diagramming tool and the database – it was all too easy to simply flag away the formal modeling stage and jump straight into the database. Using a CASE tool such as Visio allows us to take advantage of all of the gain from modeling without the traditional pain that we would encounter at the implementation stage.

Once we have implemented our databases, however, and the development team have got their hands on them there will inevitably be changes. Often those changes will be made in the database, but not reflected in the model. A model that doesn't reflect the true state of the database is all but useless, but fortunately most tools allow us to reverse engineer a database back into a model form. Some even have special functionality to manage and detect changes to the design as the lifetime of the database progresses, often called **round trip engineering**.

Reverse Engineering

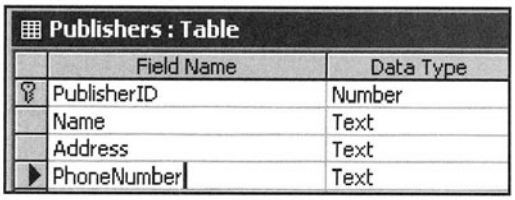
Sometimes we might want to generate a model from a pre-existing database. For example, we may want to get an understanding of some legacy database system, or we may have started building a system directly in the RDBMS only to find that it was more complex than we had originally estimated.

> *the generation of models and diagrams from pre-existing databases is called reverse engineering*

Entity relationship diagrams make great documentation for database systems. Any competent database designer should be able to pick up an ER diagram and quickly gain an understanding of the system that it represents. Almost all database CASE tools support the generation of models and diagrams from pre-existing databases: this process is called reverse engineering. Some tools also support round trip engineering, in that they support both forward and reverse engineering and provide some synergies between the two. We saw Visio make explicit provision for round trip engineering when it stored the database image in the wizard above.

Let's look at how Visio is able to perform reverse engineering on our database. Say we have taken our two-table database to a full production environment, but along the way we have had to make some changes to the structure of the database. It is now time to document the system and we would like to include an uptodate ER diagram with our documentation.

Before we look at how to do this in Visio we need to make some changes to our database in Access. Let's add a phone number column to the **Publishers** table:



Now let's work through the *Reverse Engineering Wizard*. We need to reverse engineer into a new *Database Model Diagram* document. Alternatively we could have chosen the *Update* option in the menu to merge the changes with our existing diagram.

First we choose the data source, the database we want to diagram.

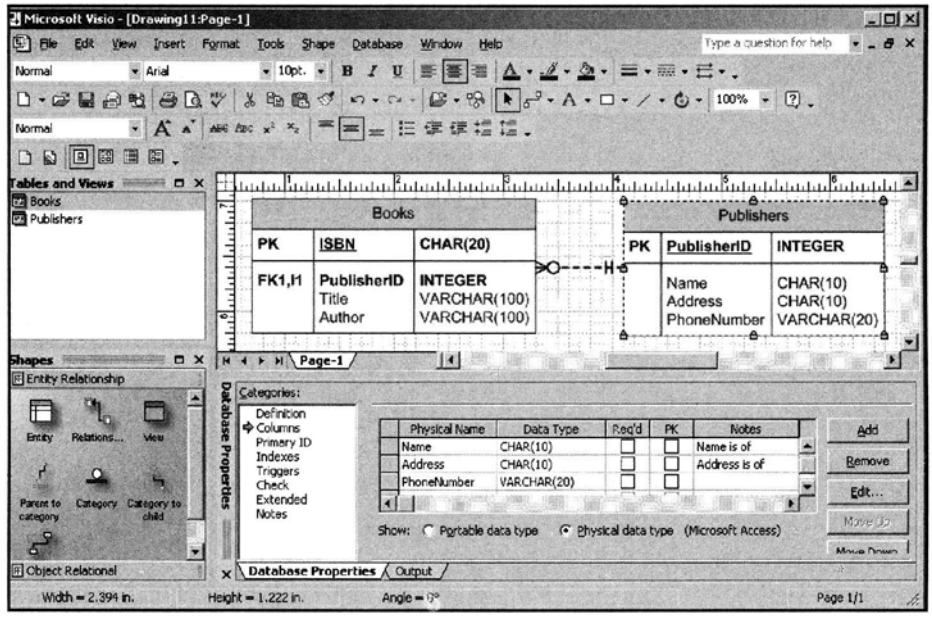
The second step in the wizard allows us to choose the types of objects to reverse engineer. With more advanced database platforms such as SQL Server and Oracle we are able to reverse engineer things such as stored procedures and triggers. Don't worry about what these are at the moment; we'll be taking a look at them in *Chapter 8*.

At the next step in the wizard we can select the particular objects to reverse engineer. In Visio it is generally best to reverse engineer all of the objects in the database. Because diagrams in Visio are independent of the underlying data model, we can choose to only draw some of the objects once we have them in Visio.

Finally we tell Visio whether the objects that we are reverse engineering into the model should be added to the current diagram. If you are dealing with a large database (more than about 30 tables) it is best to choose *no* at this stage and add the objects to one or more diagrams yourself. If we choose not to add the objects to a diagram, they will still be available in the *Tables and Views* panel in Visio for us to drag and drop onto the drawing surface. When we do so, Visio will automatically work out what any of the relationships between the objects are and draw in the appropriate relationship lines.

> *If you are dealing with a large database it is best to and add the objects to one or more diagrams yourself*

The result of our reverse engineering task should now be visible on the drawing surface. Note that the column that we added during the development phase of the project has been correctly imported into the data model along with the rest of the columns, entities, and relationships:



This chapter makes reference to the use of a CASE tool and specifically of MS Visio in several places. A CASE tool is not essential to a good database design but it does make life a lot easier, especially with large database models. You needn't necessarily do your design in a computer at all; many designers are more comfortable working with pen and paper or brainstorming a model with teammates on a whiteboard.

Importance of Sample Data

As part of the ongoing process of analysis and modeling, you should be constantly validating your model and the resulting database against a set of sample data. Ideally, sample data should be provided by the client and drawn from the real-world processes that your system is being designed to solve.

You should try and get the sample data in a format that the client can deal with. In this way you can better assess how your decision to normalize the data in a given way correlates with the way that the client deals with their information.

Sample data can generally be broken down into three layers, which tend to cover different requirements of testing and model validation:

1. The first layer of sample data should always come from the client. If there is no client, someone who was not directly involved in modeling the database should be sought. This layer of sample data should be used to assess that the model is indeed able to fully represent the data needs of the client and their workflows and processes.
2. The second layer of sample data should be added once the system has moved beyond the model stage and into a real-life database. This sample data should be used to check the integrity of the database. It should be a combination of client-supplied data and deliberately corrupted data that is intended to test the ability of the implemented database to enforce the integrity rules required by both the business processes and the data model being used.
3. The third layer of sample data is added as the actual web application is developed. As the programming of the web application nears completion the database should contain a sufficient amount of realistic sample data for a load-testing application to be run against the system. This will help test for weaknesses in the model and aid in physical database tuning. We will discuss tuning in more detail in *Chapter 8*.

Stress testing is an essential part of any testing process. With web-based systems it is especially important as the potential for traffic to fluctuate unpredictably means that you must know the 'breaking' point of your application so that you can determine hardware and hosting requirements **before** you run into problems.

Microsoft provides a superb free web stress-testing tool at *http://webtool.rte.microsoft.com/*. While it is designed for testing ASP applications it is a very flexible tool and can easily be used for testing web applications on any platform.

Summary

In this chapter we have looked at using database modeling and database diagrams as part of the development process. Models and diagrams are useful because they help us better clarify the requirements of the database in our minds, and we can share our models and diagrams with the end user to validate our understanding of their needs.

We have also looked at the key structures that generally arise when we model real-world data in a relational database model. We have tried to give you a number of examples of where these structures might be used. When you are working on your own databases you should be able to look back through these examples to spot similarities; they will help you identify ways of modeling the complex data relationships that arise in realworld development.

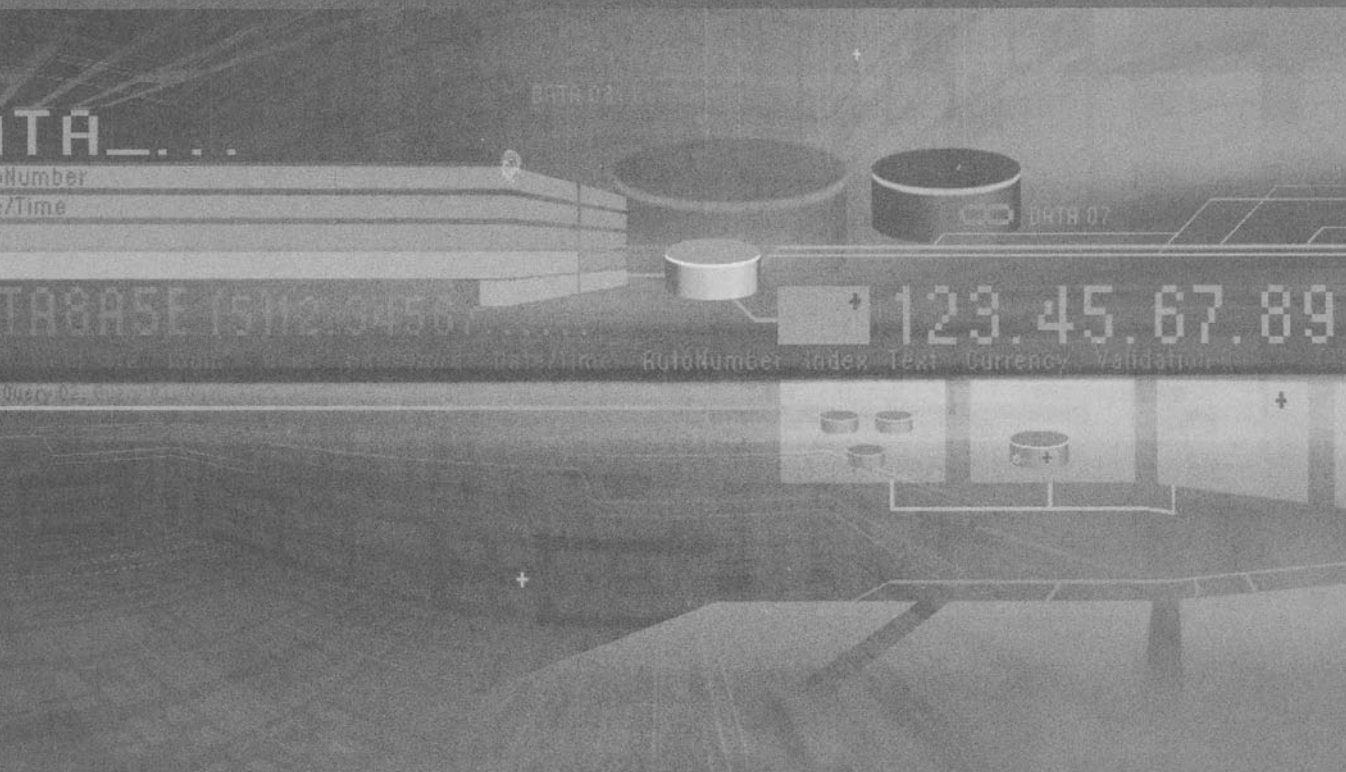
Finally we looked at integrity constraints, the rules that keep our databases valid and well organized. By putting as many as we can of the rules that govern our data in the database we will ensure that they are enforced whatever application accesses our system.

In the next two chapters we will apply the knowledge that we have learned so far to a real-world e-commerce application. We will look at the whole database design process, from interviewing the end users and customers through to designing the model, and then implementing that model into a physical database.

5

- How web databases differ
- What will your database do?
- What data do you need?
- Getting the answers

Author: Rudy Limeback



Planning the Database

Planning and designing the database are critical steps to ensure the success of any project using a database. Albert Einstein was once asked what he would do if he had only one hour to save the world, and reportedly remarked that he would spend the first fifty minutes studying the problem. Planning and designing the database should receive appropriate attention, although maybe not five-sixths of the project.

In this chapter and the next, we will study the steps necessary first to plan and then to design and implement a web database. We will draw on the lessons of the previous three chapters, and put them into practice with a real-life project.

Web databases today have a variety of purposes. Many web sites use a content management system, driven by a database, to allow producers to maintain site content without having to be a web designer or developer. Search engines have large databases of URLs and links between sites. Discussion forums use databases to store member information and forum postings and replies. E-commerce sites have databases that contain product information, from which selections can be made and purchased.

To illustrate the database planning and design process, we will choose an e-commerce example, and set out to build the database for a **web store**.

In developing the web store example, we will want to take into consideration the needs and requirements of the main **stakeholders** involved, namely:

- The web store (web site) owner
- The web store customer

Preliminary requirements for the web store will be:

- To allow people to select from products, place an order, and track their order status, all from their browser
- To allow the store owner to manage product inventory in the web store
- To provide a reporting capability to the store owner on the success of the e-commerce venture.

In this scenario, we play the role of database designer and developer. Other skills will also be needed on the project, such as web designer, web developer, project manager, and so on. In this chapter we concentrate on the planning and design of the database, and for this purpose we will assume that discussions with the stakeholders have already taken place, to uncover detailed information requirements. The store owner will have outlined how products are to be displayed in the web store. Perhaps potential customers have been interviewed, or experts in customer behavior have been consulted about what customers expect when using an e-commerce site. In any case, as database designers we need access to this information.

Let us now proceed with the initial planning process, which consists of the following steps:

- Gather information
- Identify entities and attributes
- Identify entity relationships
- Prepare the conceptual data model

We're following these steps in this particular order because we won't know what's going to be in the conceptual data model until we know what entities and relationships exist, which we won't know until we know what the database will be representing, and what information it will store. These steps have been mentioned in previous chapters, and each is critical to the success of the database. This chapter will examine each step in detail, using the web store as a case study. So let's start by gathering all the information about requirements for the web store as given by the stakeholders.

Gather Information

The stakeholders involved in the web store are the **store owner** and his/her **customers**. During the business analysis and general web design stage of our web store project, stakeholder requirements are made known, and our job now, to begin planning for the database design, is to collect this information and identify the data requirements.

Let's start the information gathering process with the store owner.

Store Owner

The store owner provides the majority of requirements that influence the planning and design of the database. As database designers, we need answers to many general questions. What products will be sold? What information about those products must be maintained in the database? How should customer orders be stored?

Many of these requirements are related to the processing of customer orders. Let's not underestimate the size of the application design task, which involves designing the web pages that allow for product selection and ordering. On the other hand, let's not be too concerned with *how* the web store will work, and concentrate instead on the details of the product and customer data that will need to be stored and processed.

> *let's not be too concerned with how the web store will work, and concentrate instead on the details of the data that will need to be stored*

Let's look first at product data requirements.

Product Details

The first requirement is that the web store must **display the products** that are available, so that customers can see them, and thus want to purchase them.

The web store must be able to show detailed information about each product, so at this point, let's list everything the store owner has identified as required product information:

- Product name
- Short description
- Long description
- Size, color, weight
- Price
- Availability (quantity on hand)
- Picture or image
- Comments from satisfied customers
- Special discounts
- Shipping cost
- Product category or categories

> *some of these data requirements, like product categories, are not product attributes, but rather relationships*

This list of product data requirements is the starting point for the planning and design of the web store database. No matter what actual physical database table design we end up with, those items of product data need to be in there somewhere. Most of them are the basic attributes of the **product entity**, which will be examined in detail later on in this chapter in the step to identify entities and attributes. Furthermore, some of these data requirements, like product categories, are not product attributes, but rather relationships. For now, our main purpose is simply to gather this information.

The store owner has also indicated a requirement that product information be **searchable**. Customers should be able to visit the web store and easily find products that they wish to purchase. While this is also a customer requirement, it's probably more of a store owner requirement, because if product search is cumbersome, customers may purchase elsewhere, and our web store will not be successful. In other words, we need to anticipate effective search mechanisms, which may involve additional data requirements. For example, one way to make product information searchable is to provide a **keyword search**, which might allow customers to choose selected keywords rather than entering strings such as "men's blue t-shirt." *How* searching works is not really germane to the planning and design of the database at the initial requirements gathering stage, but it may be important in the physical design of database tables later on (for example, it may require keyword tables and relationships). For now, we make a note of it.

Another way to allow customers to find desired products easily is to present our products in **categories**. Product categories quickly allow customers to zero in on categories that are likely to contain the products they seek. To that end, we will need products to belong to categories and we will need the following information about the categories themselves:

- Category name
- Description
- Subcategories

The concept of subcategories should be familiar to anyone who has used a search engine such as Yahoo (*http://www.yahoo.com/*) or the Open Directory Project (*http://dmoz.org/*).

One caveat to keep in mind with regard to subcategories is to **limit the number of subcategory levels where possible**. Studies of customer behavior have shown a reluctance to "drill down" through too many subcategories and sub-subcategories. Therefore, while we need to provide for product categorization, we also want to keep the structure simple.

Product Administration

Another set of requirements given to us by the store owner is **administrative**. The web store developers will need to provide administrative functions to add new products, delete products that aren't selling, tweak product descriptions, offer special discounts, and so on. Most of these requirements are processing-oriented. The reason we give them a quick review in the information-gathering stage is to make note of anything that might involve storing pieces of data in the database, for this may affect the database design.

For example, we will want an *add product* administrative web page that lets the store owner enter all the relevant bits of information for a new product. While this fact does not in itself uncover any additional requirements for product information, it does alert us to the need for the web store to recognize the store owner, because this *add product* web page is *not* one we wish to make available to customers. So we will need something in the application that can recognize an **owner ID and password** so that the store owner can be authorized for access to these administrative functions. We will discuss this issue in more detail in *Chapter 7* on security. For the sake of our example web store database, we will not include this, even though in a real project, the owner ID and password may well be stored in the database. You should note this in passing, and keep in mind that this chapter's purpose is to describe the design process, rather than to identify every possible item in a real-world database.

On the "change product" administrative web page, the store owner will be able to change descriptions, set different prices, move a product to a new category, and so on. One special task that needs discussion is what to do when a product must be removed from the web store. If a product has had any sales at all, we cannot just delete it, because that would affect historical information, and we wouldn't want to do that. Therefore we will need one additional item of product information: **status (active/deleted)**. By changing the status indicator, the store owner can effectively remove a product from being listed in the web store, at least as far as the customers are concerned.

> *If a product has had any sales at all, we cannot just delete it, because that would affect historical information*

Finally, included in the store owner's administrative requirements for products are **reporting** requirements. Assuming the web store is humming along nicely, selling products to customers, the store owner will want to be able to extract information such as:

- Which (categories of) products sold the most/least?
- Which (types of) customers bought which products?
- Were special discount offers effective?

These are only some of the many reporting requirements the store owner may have given us. Each must be examined against the information gathered about the data that will be stored. What we're looking for here is any mention of reporting something that we cannot produce from the stored data. These reporting requirements will be passed on to web developers for detailed analysis later.

Order Details

The database will need to store detailed information about **customer orders**. This might seem obvious, but consider a different type of web store where the customer order is passed to a third party for fulfillment. In our web store, however, the store owner will ship the orders, and so we want to capture and record as much information about the customers' orders as possible, so that sales tracking and analysis reports will be possible.

What information should we capture about customer orders? It's easy to nail down the product information:

- Product(s) ordered
- Quantity of each product
- Discounts
- Total shipping cost
- Taxes, if any
- Total cost

These items of product information are essential, because without them, the customer order cannot be fulfilled. So product-related order information is easy to capture.

What about customer information? Here we need to tread carefully.

The store owner would like to capture who ordered which product, and this we can do because every order will be associated with a specific customer. What, though, do we know about a customer? In other words, what customer information can we reliably capture and store in our database? In practice, this comes down to a question of privacy.

To see why, let's look at the customer database requirements from the perspective of the customer.

Customer

A web store customer will search through products and select one or more items to purchase. At the time of purchase, the database can be expected to record product-related order information as described above. The customer expects the web store to remember what was ordered, in order to ship the right products.

Customer Details

To complete a purchase, the customer will need to provide a minimum of two salient pieces of information:

- Valid payment authorization
- Where and perhaps how to ship the product(s)

Either or both of these may include the **customer name**, and the second will provide a shipping **address**. Perhaps the web store owner has indicated a requirement to store the customer name and address in the web store database, in order to produce reports showing which customers are purchasing which products. If this is the case, be sure to check the relevant privacy legislation in the web store's jurisdiction. For example, in Europe Data Protection Legislation (for example, see *http://www.dataprotection.gov.uk/*) allows only the bare minimum of information to be stored for a very limited time, unless the customer actively agrees to allow you to hold it.

A good rule of thumb is that all items of customer information should be optional. Customers do not give up personal information easily. They may offer it if asked, but they'll want the option. For example, if the web store sells jewelry, it may wish to ask the customer "Is this a gift or for personal use?" The answer may be a useful piece of information to the web store owner, but the customer has no incentive to answer, or to answer honestly. If forced to answer, by an unfriendly order form that will not submit unless all questions are answered, the customer just might *abandon the order*; this is a major concern for both the customer and web store owner.

> *A good rule of thumb is that all items of customer information should be optional*

One item that can safely be asked for is the customer's **e-mail address**. This allows the web store to send a confirmation e-mail when the order has been shipped. Unless the web store deals in products that can be immediately downloaded from the web store, such as software or electronic documents, we should expect this type of follow-up as an option. Even better would be the ability to *track the order* through the web store. This requires that the customer be able to come back to the web store and identify themselves as a particular customer, to view their order(s). This in turn requires that the database must store some type of **customer id and password.**

Payment Processing

While the web store can ask for and capture a minimum of customer order information, best practice dictates that payment authorization information not be stored, but be passed instead to a **third-party payment processor**. The details of this procedure can vary, but in general the customer visits the payment processor's secure site, enters payment information, and then a response comes back from the payment processor, either "payment approved" or "payment denied". Then the web store application can store the outcome in the customer order. If payment is denied, the customer might like the option of either abandoning or saving the order, perhaps to return at a later time with different payment authorization.

For more information on connecting with payment processors, visit one or more of the following web sites:

- Authorize.Net (*http://authorize.net/*)
- LinkPoint International (*http://linkpoint.com/*)
- VeriSign Inc. (*http://www.verisign.com/*)
- PayPal (*http://www.paypal.com/*)
- 2CheckOut.Com (*http://www.2CheckOut.Com/*)

Disclaimer: glasshaus does not necessarily endorse any of these sites, and cites them purely as common examples.

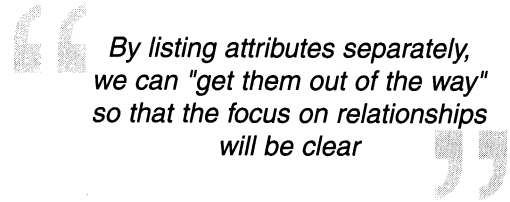
This completes our process of gathering information about data requirements. We now move on to assembling what we have gathered into entities and attributes.

Identify Entities and Attributes

As we learned in *Chapter 4*, the way to start a conceptual or ER (entity-relationship) design is to list the entities and their attributes.

It's important to distinguish between attributes and relationships, so in this step we carefully exclude anything that looks like a relationship between entities, knowing that we will pick it up shortly. By listing attributes separately, we can "get them out of the way" so that the focus on relationships will be clear.

Our list of entities and attributes is as follows:



- **Product**
  - Product name
  - Short description
  - Long description
  - Size
  - Color
  - Weight
  - Price
  - Availability (quantity on hand)
  - Picture or image
  - Comments from satisfied customers
  - Special discounts
  - Shipping cost
- **Category**
  - Category name
  - Description
- **Customer**
  - Customer name
  - E-mail address
  - Ship-to address
  - Customer userID (login ID)
  - Customer password

- **Order**
  - Product(s) ordered
  - Quantity of each product
  - Discounts
  - Total shipping cost
  - Taxes, if any
  - Total cost

From stakeholder requirements, we have identified not only our preliminary web store entities and attributes, but also an initial idea of how they might relate to each other. For example, products are classified by category, and customers have orders.

Identify Entity Relationships

Now that we have listed the entities and their attributes, let's move on to analyzing entity relationships, for that is where the real action is.

This will be our first opportunity to discuss relationship **cardinality** in detail. Cardinality is determined by how many instances of one entity relate to an instance of another. We will examine the relationship between two entities by looking at the relationship in both directions, once for the relationship from X to Y and once for the relationship from Y to X.

The way to do this is to create a statement that has the general form "each X is **related** to **how many** Y". In these statements, "related" will be a verb of some kind, and "how many" will give the cardinality of the relationship, in the direction indicated, from X to Y. Let's break these down into more manageable groups.

Product Relationships

The first entity for which we will examine relationships is Product. Initially, we can see that products will be related to the following other entities:

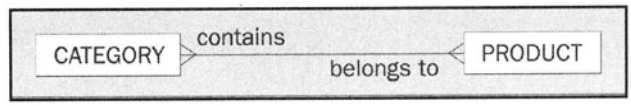
- Category
- Order

You might think that the Product entity is also related to Customer. With a moment's reflection, though, we can see that this relationship is manifested only via Order, in that products are related to orders and orders are related to customers. In other words, products are related to customers only through the orders that the customers create.

So let's start our analysis with the relationship between Category and Product.

Category and Product

Using ER diagramming, here is what this relationship looks like:



Notice that the relationship between Category and Product entities is initially drawn as a general many-to-many relationship. This is done using the "crow's feet" diagramming convention, which, as we learned in *Chapter 4*, indicates that the entity on that end of the relationship has a "many" cardinality. There are several different conventions for showing this cardinality in ER models, but "crow's feet" is one of the more recognizable ones.

Notice also that the relationship is labeled with two verb phrases. These verb phrases help us understand the relationship when viewed in one direction or the other. We say a category **contains** products, and a product **belongs to** categories.

Referring now to the information gathered in the previous step from the stakeholders, vague though it may be, we need to confirm this relationship's cardinality. Specifically, we need to generate the "each X is **related** to **how many** Y" statements.

With categories of products, it's reasonable to assume that each category will contain multiple products. Here we have our first design decision. Must a category contain any products? If it doesn't, then customers may become frustrated when they find no product in it. Therefore we will say yes, and stipulate that a category must contain at least one product. Why, you may be wondering, does this matter? Because the application developers may need to implement special processing so that categories that are empty are not visible on the web store. The ER diagram can communicate this requirement succinctly.

Thus we have "**each category contains one or more products**".

In the other direction, we have another design decision. Must a product belong to any category at all? If it doesn't, then it will not be listed on the web site page that lists products by category. Therefore we will say yes here as well, and stipulate that a product must belong to at least one category.

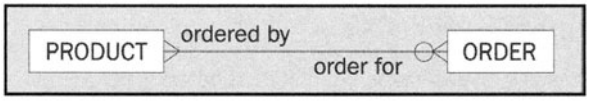
A further decision is whether to allow a product to belong to more than one category. As might be expected, this will have an impact on the database table structure that will support this relationship. Not only that, but it will have an effect on the processing required to report sales by category. Which category should get credit for the sale of a product that belongs to more than one category? We can solve this problem by designating a **default category** for the purpose of tracking sales. This default category is an example of **relationship data**. It is a special type of attribute that we will discuss again in the next chapter. So complexity notwithstanding, let's allow a product to belong to more than one category.

Thus we have "**each product belongs to one or more categories**".

So we have confirmed that the category-product relationship is a many-to-many relationship. As we learned in *Chapter 4*, however, databases do not implement many-to-many relationships directly. We know we will have to break the many-to-many product-category relationship down into two one-to-many relationships using an **associative entity**; we'll see these in action in *Chapter 6*.

Order and Product

Now we come to the relationship between Order and Product. Clearly, this is also a many-to-many relationship. Here is the ER diagram for this relationship:



A product can be ordered by multiple orders. An order can be for multiple products. As we did with category and product, we examine the many-to-many order-product relationship in both directions, and look at the cardinalities.

> *the fact that zero is allowed is reflected in the ER diagram by the little circle on the Order end of the relationship line*

Must a product be ordered by at least one order? Not necessarily, since a new product just introduced into the system will not be on any orders yet. Can a product be ordered by more than one order? Yes, in most cases. There might be "one of a kind" products of which only one exists, but in general products can be ordered over and over, and that is the design we want. If there are unique products, they will also fit into this structure, and will appear on only one order simply because their on-hand quantity will immediately fall from 1 to 0.

So the first cardinality statement is "**each product can be ordered by zero, one, or more orders**". The fact that zero is allowed (that a product can exist without being on any order) is reflected in the ER diagram by the little circle on the Order end of the relationship line. Again, ER diagram conventions will vary, and it's not crucial to get the diagrams perfect according to some modeling standard, but it is important to confirm the cardinality and make sure it is properly recorded for reference in later stages.

Must an order be for at least one product? Yes, since without a product, it isn't really an order. Can an order be for multiple products? Yes, since if we allowed only one product per order, the procedure to order multiple products would be seen by customers as very unfriendly.

So our second cardinality statement is "**each order is an order for one or more products**".

Again, since this is a many-to-many relationship, we anticipate that it will be broken down into two one-to-many relationships with an associative entity.

Now let's move on to categories.

Category Relationships

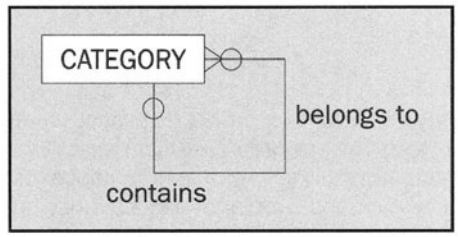
The Category entity is related to:

- Product
- Subcategory

We've already covered the relationship between Category and Product, so we'll move straight on to the relationship of category to subcategory.

Category and Subcategory

The main question to answer right away is whether Subcategory is actually a separate entity. The clue to reaching a solution is whether a subcategory is intrinsically different from a category. It isn't. Both categories and subcategories have a name and description, and that's it. More telling is the fact that we can have levels of subcategories and sub-subcategories, and one good way to handle a "sub" structure like this is with the one-to-many recursive relationship introduced in *Chapter 4*. Here's the ER diagram for the recursive relationship:



There is, however, a limitation in a recursive relationship, namely that it is not many-to-many. We need to confirm this by examining the category-subcategory relationship in both directions.

Can a category contain more than one subcategory? Yes. Must a category contain a subcategory? No, because going down the category-subcategory hierarchy, eventually you reach subcategories "at the bottom level".

So the first cardinality statement is "**each category can contain zero, one, or more (sub)categories**".

Can a subcategory belong to more than one parent category? We will make a design decision and say no, if only to preserve our sanity. Subcategories that belong to more than one parent category are certainly possible in a database structure, but are notoriously difficult for users to grasp conceptually.

Even in cases where a subcategory appears to belong to more than one category, it often does not. For example, do a search on the Open Directory Project (*http://dmoz.org/*) for "Programming", and you will find many categories, including:

- Computers: Programming
- Computers: Education: Programming
- Computers: Parallel Computing: Programming

These are not the same (sub)categories, although they all have the same name! In each case, it is a different category, belonging to a different parent category.

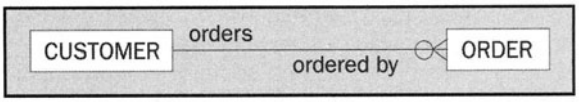
Finally, must a subcategory belong to any parent category? No, because categories "at the top level" do not belong to parent categories.

So the second cardinality statement is "**each (sub)category belongs to zero or one category**". The grammar may not be perfect but the intention should be clear.

Customer Relationships

A customer, for our purposes, is related to only one other entity, Order.

Order and Customer



The ER diagram for the customer-order relationship is as follows:
This is a one-to-many relationship, which we confirm as follows.

Can a customer have more than one order? Yes (and the web store owner will be happy to see repeat business). Can we really have a customer without an order? Yes, because a customer can register on the web store prior to placing their first order. We might additionally maintain a mailing list, and for this purpose, all we need is the customer's e-mail address, which we have previously identified as an attribute.

So the first cardinality statement is "**each customer orders zero, one, or more orders**".

Must an order belong to a customer? Yes, because the application should not allow an order to be created without knowing which customer placed it. As for orders that might be "shared" between multiple customers, we won't allow that either, because it's too complex (and unrealistic).

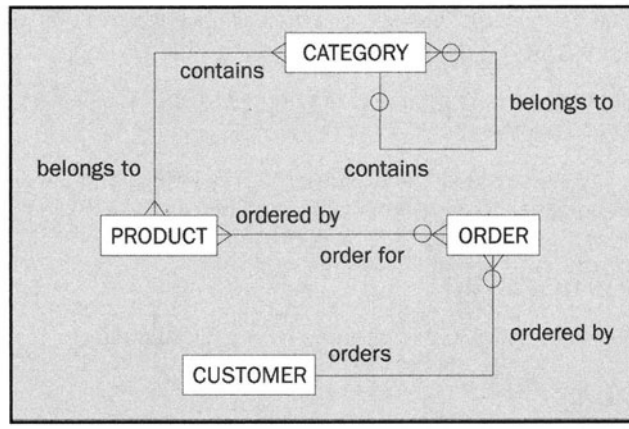
> *"the application should not allow an order to be created without knowing which customer placed it"*

So the second cardinality statement is "**each order is ordered by exactly one customer**".

We are now ready to assemble the various entities and relationships into our conceptual data model.

Prepare Conceptual Data Model

The following figure shows the conceptual data model of our web store as an ER diagram. A conceptual data model should include all the entities of interest, and identify the relationships between them. By assembling the smaller ER diagrams we created along the way, we get an overall view of the database at a conceptual level:



All of the relationships we examined in detail are shown in this diagram. Relationships have "crow's feet" at the "many" end of a relationship, for example, product-category is many-to-many, but customer-order is one-to-many. The little circles on some of the relationships indicate where zero cardinality is permitted, for example, a customer can have zero, one, or many orders.

Summary

In this chapter we began to design a database for a web store. We started by gathering information from the stakeholders of the project, with the intention of identifying the data items that will need to be stored in the database. Without gathering requirements in this manner, we would just be guessing about what's important enough to store in the database, and what isn't. What may be a good guess for one project could easily turn out to be disastrously wrong for another.

Having talked to the stakeholders and figured out what entities are involved, and what their attributes will be, we made a list of entities and their attributes. Next we began to look at how these entities will be related to each other, and used ER diagramming to model those relationships.

As we looked at the relationships, we noted that some would eventually require the use of associative entities, to create pairs of one-to-many relationships out of each many-to-many relationship (products and categories, products and orders). This, however, is best done in physical database design, which follows in the next chapter.

Finally, after seeing the entity relationships between themselves, we combined the diagrams for each relationship into one for the database as a whole.

In *Chapter 6* we will build on this conceptual data model as we examine the implementation implications for these entities and relationships.

6

- Turning the planning into reality
- Designing tables
- Choosing data types
- Picking queries

Author: Rudy Limeback

Implementing the Database

In the previous chapter, we planned and designed an e-commerce database, for products, categories, customers, and orders. This process culminated in a **conceptual design** embodied in an **Entity-Relationship** (**ER**) model, which allowed us to identify, and verify the cardinality of, the relationships between the entities.

In this chapter we will take the conceptual design, extend it by adding **physical characteristics**, and conclude by preparing the **data definition language** (DDL) that will be used to create our database tables.

To transform the conceptual design into a physical database, we will consider the following:

- Target database system
- Naming conventions
- Associative entities for many-to-many relationships
- Data types and nullability
- Natural and surrogate primary keys
- Foreign keys and relational integrity
- Table and column definitions
- Database indexes
- Sample DDL
- Sample SQL

Target Database System

A project to implement a web database consists of many steps in addition to designing and creating the database tables. In fact, the procedure to create the database tables will require some customization, albeit minor, which depends on the target database system.

In order to set up a web site, certain choices will be made regarding the host (Internet provider), operating system, scripting language, and database system to be used. In many projects, these choices are made long before the database design has begun. For example, if we're told that the web store site will be hosted on a Windows NT server running ASP against a SQL Server database, then we will tailor the DDL to SQL Server's dialect of SQL. In other projects, the target database system might be MySQL, Microsoft Access, DB2, Sybase, Oracle, or PostgreSQL. Fortunately, the differences between database systems are relatively minor when it comes to the DDL for creating tables.

> *In many projects, these choices are made long before the database design has begun.*

In addition to creating database tables, other tasks required to implement a web database include reserving server file space, connecting the database and web servers with the operating system, installing the web application, setting security controls. These important tasks are affected by the decisions about which host, operating system, scripting language, and target database system will be used even more than the DDL for creating tables is.

Some of these tasks will be mentioned in later chapters. As far as the transformation of our conceptual design into database tables is concerned, we simply note that the choice of target database system must be made before we can create the final DDL.

Naming Conventions

It's surprising how contentious the issue of naming conventions can be. Data modelers of course prefer clear, descriptive names, using complete English words. Developers and database administrators on the other hand, prefer brevity, acronyms, and other naming-convention gizmos.

Unless you are building both the web application and the database completely on your own, you should meet with the developers and database administrators and any other people involved with the database, and agree on a naming convention up front. Rest assured that as long as you use a bit of common sense, you cannot really make a bad choice, because one naming convention is as good as another when it comes to whether or not the web application works, the site succeeds, and the orders come in.

Insofar as database creation and application development are concerned, naming conventions have only a minor impact on the effort required. It's more work to type a longer name than a shorter name, but on the other hand, a name that is too short could become unrecognizable.

Choosing a naming convention really comes down to two factors: clarity and preference. Let's keep these in mind as we prepare to define our table and column names.

Note that the opinions expressed here are the opinions of the author, and are useful in representing the kind of thought processes you need to go through when choosing a naming convention. If you disagree with any of these opinions, there is nothing to stop you choosing a different syntax for your own database projects, if you wish.

Table Names

The entities that we identified in *Chapter 5* will be implemented in the database as tables. Could we use those entity names as table names? Let's look at a couple of table naming conventions.

One convention recommends using the prefix **tbl** as part of the table name, so that we know it's a table (in this convention, queries start with the prefix **qry**). In our e-commerce example, the tables would be named **tblProduct** and **tblCustomer** and so on. As far as clarity is concerned, however, this convention of using a prefix to label something according to the type of thing it is seems redundant, so we will not use it for our table names.

Another table-naming convention recommends using plural rather than singular names. In our web store example, the tables would be named **Products** and **Customers** and so on. The idea behind this convention is to remind us that tables contain sets of rows. Still, a singular table name has appeal: perhaps it's just a matter of preference, but the SQL statement `SELECT ... FROM Product` sounds more natural than `SELECT ... FROM Products` because only one table is being selected from. So let's go with singular table names.

There is one caveat, however. Every database system has a set of **reserved words**, and we should not use a reserved word as the name of a table or a column.

Thus, we can have **Product**, **Category**, and **Customer**, but "order" is a reserved word in every database system, being used in the `ORDER BY` clause of SQL. Therefore, in this case we will use the plural, and call it the **Orders** table. A good naming convention is nothing if it is not flexible!

Column Names

Generally speaking, we can use the attribute names previously chosen in the conceptual design as column names, with the same caveat about reserved words. So, for example, we would not be allowed to have a column called `Date`. However, an alternative name like `ShippedDate` not only avoids the reserved word problem, but also is more descriptive, and that's desirable.

Another naming convention involves whether to incorporate the table name into the column name. For example, compare this:

```
SELECT ID
     , Name
  FROM Customer
 WHERE City = 'Eureka'
```

with this:

```
SELECT CustomerID
     , CustomerName
  FROM Customer
 WHERE CustomerCity = 'Eureka'
```

The prefix seems redundant, right? So why would this even come up as a something to decide? Because the same column names can be used in different tables, and any SQL statement that involves more than one of those tables will require coding a certain way, depending on the naming convention. For example, compare this:

```
SELECT CustID
     , CustName
     , ProdID
     , ProdName
  FROM Customer
INNER
  JOIN Orders
    ON CustID = OrdCustID
INNER
  JOIN OrderItem
    ON OrdID = OrdItemOrdID
INNER
  JOIN Product
    ON OrdItemProdID = ProdID
 WHERE CustCity = 'Eureka'
```

with this:

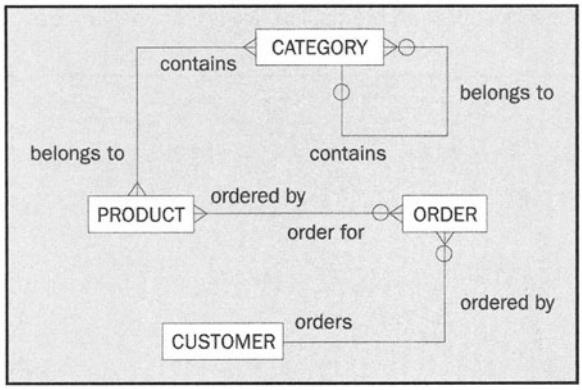
```
SELECT Customer.ID
     , Customer.Name
     , Product.ID
     , Product.Name
  FROM Customer
INNER
  JOIN Orders
    ON Customer.ID = Orders.CustomerID
INNER
  JOIN OrderItem
    ON Orders.ID = OrderItem.OrderID
INNER
  JOIN Product
    ON OrderItem.ProductID = Product.ID
 WHERE Customer.City = 'Eureka'
```

In the first SQL query, the column names contain a prefix to indicate the table they are found in. In the second, the table names themselves are used to distinguish like-named columns.

The pros and cons of each method are somewhat subtle, and of interest mainly to database specialists, yet a choice still must be made. For our web store example, then, let us adopt a naming convention that incorporates a prefix into the column name to identify its table. The prefix will be abbreviated to preserve some brevity. Thus we will have `CustID` and `ProdID` as column names, rather than `ID` in the **Customer** table and also `ID` in the **Product** table.

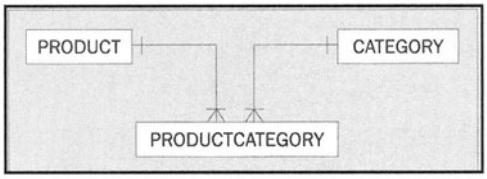
Associative Entities for Many-to-Many Relationships

In *Chapter 5*, we identified entities and relationships, including a couple of many-to-many relationships. Here again is the ER model we developed:



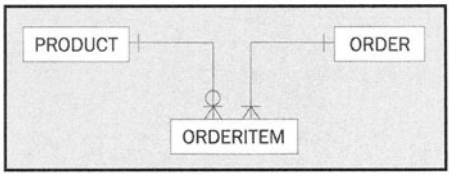
A necessary step to proceed from conceptual to physical is to check which relationships between entities are many-to-many. Recall from *Chapter 4* that we need to create **associative entities** to achieve this objective.

The first many-to-many relationship we'll look at is the **Product-Category** relationship. The associative entity needed here will be called **ProductCategory**. The revised model will look like this:



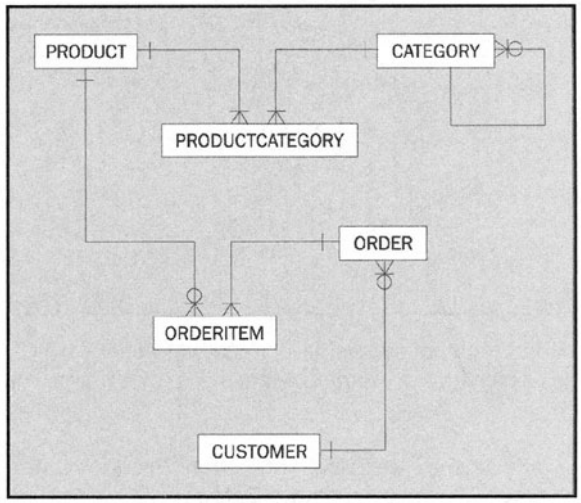
The **Product-ProductCategory** relationship is one-to-many, and allows a single product to belong to multiple categories. The **Category-ProductCategory** relationship is also one-to-many, and allows a single category to contain multiple products. Both of these one-to-many relationships will be implemented using primary and foreign keys, which we'll cover in the next couple of sections.

The other many-to-many relationship that requires an associative entity is **Product-Order**. Here, we will name the associative entity **OrderItem**:



Why the name **OrderItem** and not **ProductOrder**? No real reason: it could as easily be **OrderProduct** too. **OrderItem** is a common name, and seems to suggest the items that belong to an order. Giving the name **ProductOrder** to the associative entity would be just as correct, even though it suggests all the orders for a product. In fact, this associative entity performs both roles; it associates all the orders for a specific product, and it associates all the products on a specific order. The name **OrderItem** merely uses common terminology while placing slightly more emphasis on the product items that make up a given order.

Having added the associative entities to the design, we have a new model:



The model that includes associative entities, is usually the final ER model produced before creating the database tables. We have identified all the tables to be created, and we can now pick a prefix for each, to use in column names:

Table Name	Prefix
Product	Prod
Category	Cat
ProductCategory	ProdCat
Customer	Cust
Orders	Ord
OrderItem	OrdItem

Before we can create physical column definitions for each table, we need to cover the considerations in choosing column data types and nullability.

Data Types and Nullability

The first physical characteristic to consider for table columns is data type. Generally speaking, a column will hold **character**, **numeric**, or **date** data. Standard SQL provides special data types for these types of data. Since most database systems implement standard SQL, we will use standard data types throughout, and make note of additional choices that might be available in certain database systems. This has the advantage that our database design is **portable**, meaning that it can be implemented in any target database system with minimal effort.

> *we will use standard data types throughout*

Character Data Types

varchar(n) is one of the standard data types used for character columns. A `varchar` or varying character column has a varying length, with a maximum length given by the number n. Actual occurrences of column values are only as long as the character strings that they hold.

By contrast, a **character(n)** or **char(n)** data type has a fixed length. Fixed-length character columns are not used as often as `varchar` columns, for the simple reason that a `char(n)` column always contains its maximum length of n characters. For columns that are larger than their contents, this means that any unused space will be filled with spaces, and are, in a sense, wasted.

> *There is a small overhead in processing varchar columns*

The lengths, or widths if you prefer, of individual rows are determined by the sum of the defined lengths of fixed-size columns, plus the lengths of the actual values in `varchar` columns. The shorter the rows are, the more efficient the processing of rows will be. There is a small overhead in processing rows containing `varchar` columns, but given today's hardware speeds, this is negligible. Therefore, using `varchar` for longer, descriptive fields is the best choice.

There is another standard data type for character data, called **clob** (**Character Large Object**), but support for this is not consistent across database systems, and our web store database will not require it.

Numeric Data Types

The next set of data types to consider is for numeric columns.

smallint is a standard data type implemented using 16 binary digits. A `smallint` column can therefore hold numeric values between -32,768 and 32,767.

An **integer** column, implemented using 32 binary digits, can hold numeric values between -2,147,483,648 and +2,147,483,647.

MySQL offers several special numeric data types, for example **tinyint,** which can hold values between –128 and 127; however, `tinyint` is not a standard data type, and the benefit in physical space saved (each `tinyint` is half the size of a `smallint`) is offset by the loss in portability.

As their names suggest, `smallint` and `integer` data types hold only whole numbers. If fractions of numbers are required, a good choice is the standard data type **numeric(p,s)**, where p and s are numbers indicating the data type's **precision**, the number of significant digits it can hold, and **scale**, the number of decimal places. For example, monetary columns are often defined as `numeric(7,2)`. This allows a price amount as large as 99,999.99 to be accommodated. Note that this is a number, not a string, and that the commas and decimal point are not stored. Neither is the currency indicator (for example, $, £, or).

Microsoft SQL Server offers a special data type called **money**, specifically intended to hold monetary amounts, so that the currency indicator is handled automatically by the database when the amount is displayed. If you plan to support multiple currencies, you may need additional columns and processing logic, for example to offer an up-to-date exchange rate. In our web store database, we will use one currency only, define monetary amounts with `numeric(7,2)`, and rely on the application to display the amounts formatted properly.

Other standard numeric data types are **float** and **real**, used mainly for scientific applications. These data types are implemented with significant digits and exponents, and will not be used in our web store.

Date Data Types

The third category of data type is used to store temporal data, data about dates and times. The two main data types are **date** and **time**. Microsoft SQL Server combines these into a data type called **datetime**, which, while not portable, is the correct data type to use if SQL Server is the target database system. The nuances of handling dates and times alone could fill an entire chapter, but two recommendations are important: use a `date` or `time` data type whenever possible (as opposed to, say, declaring it as a character column), and let the database do the date and time calculations.

All database systems have comprehensive functions for handling dates and times, and we should never have to check for February or divide the year by 4 with application logic.

Nullability

Nullability, sometimes called nullity, refers to a special characteristic of a column, namely whether the column may or may not be **null**.

Null, as we've already seen, is not a special value; it is the absence of a value. Null is a missing value, a value that is not known.

Null is even more contentious than naming conventions. There are arguments both for allowing and for not allowing a column to be null. If a column is nullable (that is, if it can contain null) then this fact has a certain known implication on the nature of the SQL used to handle the column values. In particular, correct results may depend on testing whether the column is, in fact, null. Some people see this as a needless complication, and prefer to use a default value such as zero (for numeric columns) or a zero-length string (for character columns). This is dangerous and sometimes flat-out wrong, as can be seen in examples involving date columns.

A good example of a valid use for null is the date an order was shipped. If a particular order hasn't shipped yet, what value goes in that column? Some anti-null people will use a "dummy" date such as December 31, 2999. It is this kind of thinking that resulted in the Y2K problem. Furthermore, this type of dummy value still needs special handling, to prevent it from taking part in, say, the calculation to determine average number of days between order placement and shipment. Nulls are excluded from aggregate functions by definition.

In our e-commerce database, null will be chosen whenever a data value is optional. When a value is always required, the column will be defined as **not null**.

Natural and Surrogate Primary Keys

In this section we will have a look at the difference between natural and surrogate primary keys.

We'll begin with the definition of a primary key. A primary key is a column, or set of columns, with two essential characteristics:

- The primary key is never null.
- The primary key values are unique.

A **natural** primary key is an existing column or set of columns. The columns of a natural key were identified as entity attributes during conceptual design

A **surrogate** primary key is used in a table where there is no suitable natural key to use as the primary key. Surrogate keys are most often implemented as numbers. Note that standard SQL does not recognize the difference between surrogate and natural keys.

Depending on which database system you use, a surrogate key will usually be defined as an **identity**, **autonumber**, **auto_increment**, **serial**, or **sequence** column. These special data types are attractive because of the way the database system assigns values to such a column when rows are inserted. Unless it is using **random** assignment, the database system will assign an **initial** value to the column for the first row inserted, and subsequent inserts will be assigned a value equal to the last number assigned plus an **increment**. The actual numeric data type used by most database systems for this type of column is integer. Let us refer to this type of column as an **automatic column**, a neutral name that shows no favoritism for any database system.

A surrogate key defined as an automatic column easily meets the two criteria for being a primary key: all values are unique and all have a value. The values are unique and not null by the very nature of their being incremented. Furthermore, because they are integers, it will be a huge database indeed where the incremented numbers might bump up against the upper limit of 2,147,483,647.

Most database systems will allow you to set the initial and increment numbers. So, for example, you could define the initial number as 815 and the increment as 27. *Why such weird numbers?* Because they make just as much sense as defining surrogate keys that start with 1 and are assigned sequentially. Just remember that if you assign a large increment, don't make it so large that you hit the upper limit any time soon.

The main rule to remember when working with surrogate keys is: **never reveal surrogate key values needlessly** to the user. By their very nature, they have no meaning, so they do not need to be sequential, but neither should they be revealed.

> *never reveal surrogate key values needlessly*

It may not always be possible to adhere to the "never reveal surrogate key values" rule. For example, if you have an HTML form that allows a customer to change their address, this form will have various fields and a *Submit* button. The customer enters new data, clicks on the *Submit* button, and the form is transmitted to the web server: the application running on the web server then pulls out the various bits of address information from the form fields and passes them to the database. How does the database know which customer row is to be updated? This is usually done by having the form submit the value of the `CustID` primary key as a `type="hidden"` form field.

This in itself is not sufficient protection for privacy, however. A savvy user can "view source" and see the value of their CustID field in the hidden form field. Therefore, to ensure privacy it is necessary to implement a **secure login** feature as well. The customer must log in with a username and password before they are allowed to update their address, and indeed, before they are allowed even to see their address, because the e-commerce site will not reveal any addresses unless the customer is logged in, and then only their own. For more information on privacy and security, see Chapter 7.

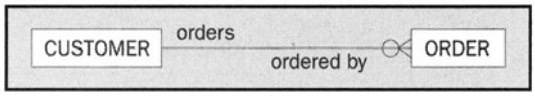
Foreign Keys and Relational Integrity

Foreign keys are the "links" that implement the relationships between related rows in database tables. Foreign keys go hand in hand with primary keys. Together, foreign keys and primary keys are used by the database system to implement **relational integrity**. Relational integrity is a much desired database feature that ensures, for example, that an order will never exist for a customer who doesn't exist. This sounds trivially obvious at first blush. Still, in practice many database developers do not implement database-enforced relational integrity, preferring to do the same function themselves in application logic.

Our e-commerce store will have basic relational integrity, as enforced by the use of foreign keys. In situations where the target database system does not support relational integrity (like versions of MySQL before 4), we have no choice but to use application logic. It is a fallback, rather than a preferred strategy.

To understand the difference between primary and foreign keys, remember that a foreign key contains the value of a primary key. The primary key is usually in some other table (unless the relationship is recursive) and always in some other row. That explains the "foreign" aspect: the foreign key links its row to a primary key that is not located in that row. For one-to-many relationships, the rule to remember is that **the primary key is on the "one" side of the relationship, and the foreign key is on the "many" side**.

Let's take as an example the **Customer-Order** relationship:



A customer may have many orders, while an order belongs to only one customer. Thus the foreign key is in the order. No real surprise. Let us call this column `OrdCustID`. Every row in the **Order** table will have a value in its `OrdCustID` foreign key column that matches the value of the `CustID` column in exactly one existing **Customer** table row. It is exactly one, because `CustID` is a primary key and therefore contains unique values.

Is it possible that a foreign key be null? Yes – we'll see an example in a moment. For `OrdCustID`, however, we stipulate that it must be not null. We need to know which customer placed every order. More than that, it must be an existing customer.

The **relational integrity constraint** enforced by the database ensures that every foreign key value of `OrdCustID` in the `Order` table will match some existing value of `CustID` in the **Customer** table. We will never have an order for a non-existent customer.

An immediate consequence of this is that the `CustID` primary key value must exist in the database before a row containing the matching `OrdCustID` foreign key is inserted. In short, the customer must be added to the database before any order for that customer can be added. If you ever see an HTML form where customer and order information are collected simultaneously, rest assured that on the server, the information is added in two steps: first the customer, then the order for that customer.

Another aspect of relational integrity is that it will not be possible to remove a customer row as long as there is even one order row that refers to it. This would leave the `OrdCustID` foreign key value "pointing to" a `CustID` value that no longer exists, and that's not permitted.

Table and Column Definitions

We have covered naming conventions, data types and nullability, natural and surrogate primary keys, and foreign keys and relational integrity. Now we are finally ready to define our tables and columns.

Taking each entity from *Chapter 5* in turn, the procedure consists of the following steps:

- Identify the primary key, either by selecting one of the attributes as a natural key, or creating a surrogate
- Assign to each attribute a column name, data type, and nullability
- Add foreign keys for relationships as required

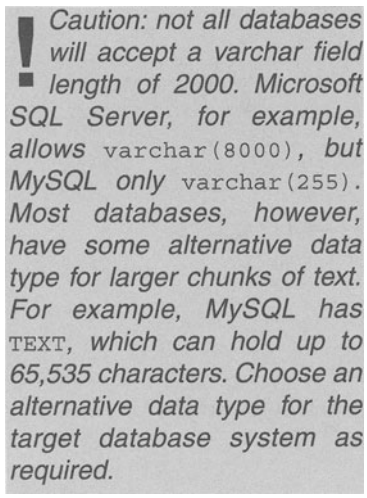
We'll begin with the **Product** table. The "table layout" consisting of columns and their characteristics is shown first, followed by comments on the choices made.

Product

Column	PK	Data type	Nullability	FK	Refers to PK
ProdID	PK	char(10)	**not** null		
ProdName		varchar(100)	**not** null		
ProdDescr		varchar(500)	null		
ProdLongDescr		varchar(2000)	null		
ProdSize		varchar(10)	null		
ProdColour		varchar(10)	null		
ProdWeight		smallint	null		
ProdPrice		numeric(7,2)	**not** null		
ProdOnHand		smallint	**not** null		
ProdComments		varchar(1000)	null		
ProdImage		varchar(100)	null		
ProdDiscount		numeric(7,2)	**not** null		
ProdShipCost		numeric(7,2)	**not** null		

The primary key for the **Product** table is `ProdID`, defined as a `char(10)` natural primary key. The purpose is to allow product codes to be used to identify our e-commerce store products. This means that every product code we use must be unique. Our intention is to use the existing product codes as defined by our suppliers. The only difficulty we anticipate will be when different suppliers use the same product code.

Now, either these will be the same product, in which case we may not need to distinguish them by the supplier they came from, or they will be different products that by sheer chance have the same code, in which case we need to assign some different product code to one of them. In either case, the use of a surrogate key does not really provide any advantage, as we would likely want to carry the supplier's product code as well.



In the **Product** table, the maximum lengths chosen for the next three `varchar` columns range from 100, to 2000 for a "long description."

`ProdID` and `ProdName` are defined as **not** null, meaning that a value must be supplied for these columns for every product added to the table. `ProdDescr` and `ProdLongDescr` are optional, so they are defined as null, that is, their values may be missing.

`ProdSize` and `ProdColour` will be character columns so that values like "XXL" and "Taupe" can be entered into them. For columns of this size, the total savings in row length between `varchar` and `char` is hardly worth the effort, but they too are defined as `varchar`. Define them large enough to hold the longest size or color descriptions.

`ProdWeight` is chosen to be a numeric column. Yes, this is quite an arbitrary decision, but a reasonable one. We want this column to record the product weight in pounds, in cases where weight is used to determine shipping cost. So `ProdWeight`, representing weight in pounds, is defined as `smallint`. An integer field would be twice as big and the likelihood that a product on our e-commerce site will have a weight exceeding 32,767 pounds is remote. Again, the safest way is to choose a data type and unit of measure that makes sense to you. If you plan to ship products via courier, and your courier measures weights in kilograms and fractions of kilograms, then perhaps a different numeric data type is more appropriate.

`ProdPrice` is a monetary data type, so we use the standard `numeric(7,2)`. Further, it is defined as not null, because every product must have a price.

`ProdOnHand` is `smallint`, and not null, because we always need to know how many we have in stock. Note that a zero is not the same as a null, and instead indicates that we have none in stock – time to reorder!

`ProdImage` is the URL pointing to the location of an image of this product. Almost all database applications that deal with images will store the URL instead of the image itself, which will be located on the web server. Although many databases can handle storing images (using a data type such as `OBJECT` or `BLOB`), it is simply easier and more efficient to do it this way.

`ProdComments` is a character column, intended for optional customer testimonials (for example, "'This product has changed my life!' – Mrs. A, Eureka, Kansas").

`ProdDiscount` and `ProdShipCost` are like `ProdPrice`, monetary amounts that may not be null.

Finally, note that the **Product** table has no foreign keys to other tables.

Category

Column	PK	Data type	Nullability	FK	refers to PK
`CatID`	PK	`integer`	**not** null		
`CatName`		`varchar(100)`	**not** null		
`CatDesc`		`varchar(1000)`	null		
`CatParentID`		`integer`	null	FK	`CatID`

`CatID` will be a numeric surrogate primary key, so it will be `integer` and not null.

`CatName` is `varchar`, and not null because every category must have a name.

`CatDesc` is a longer description, optional so not null.

The **Category** table will also have an additional column for the foreign key for its recursive relationship. `CatParentID` implements the category-subcategory hierarchy. Notice that this foreign key may be null. This does not violate relational integrity. It simply means that if a category has a parent category, then that primary key must exist. Obviously, the category at the "top" of a category hierarchy has no parent, so its `CatParentID` column will be null.

ProductCategory

Column	PK	Data type	Nullability	FK	Refers to PK
ProdCatProdID	PK	char(10)	**not** null	FK	ProdID
ProdCatCatID	PK	integer	**not** null	FK	CatID
ProdCatDefault		char(1)	**not** null		

The **ProductCategory** table has two foreign keys, because it participates in two one-to-many relationships. It is the **associative table** that implements the many-to-many relationship between products and categories.

The primary key of the **ProductCategory** table is a **composite** or **concatenated primary key**, consisting of `ProdCatProdID` and `ProdCatCatID` together. While a given product can belong to multiple categories, and a given category can have multiple products, each combination of product and category must be unique. In other words, a given product can belong to a given category only once.

Each of the components of the primary key is also a foreign key. Notice how the data types of the foreign keys match the data types of the primary keys that they refer to.

The `ProdCatDefault` column, defined as `char(1)`, indicates that a particular product-category combination is the default. It defines the "default category" for a given product. The values `Y` and `N` will serve this purpose, although there are other choices we could as easily make, such as a numeric data type to hold a `0` or `1`, or a `Boolean` data type to hold a True or False value. However, not all database systems support `Boolean`, and in keeping with our desire to make the design portable if possible, `char(1)` is a good choice.

Customer

Column	PK	Data type	Nullability	FK	Refers to PK
CustID	PK	integer	**not** null		
CustUsername		varchar(20)	**not** null		
CustPassword		varchar(20)	**not** null		
CustEmail		varchar(100)	**not** null		
CustFName		varchar(50)	null		
CustLName		varchar(50)	null		
CustAddress1		varchar(100)	null		
CustAddress2		varchar(100)	null		
CustCity		varchar(100)	null		
CustProv		varchar(100)	null		
CustCountry		varchar(50)	null		
CustPostal		varchar(10)	null		

`CustID` will be a numeric surrogate primary key, so it is `integer` and not null.

CustUsername and CustPassword are defined as varchar(20) to allow for strings of adequate length, and they are defined as not null, because they are an essential part of the security of the application. CustEmail is also a character string, and a maximum size of 100 is usually sufficient (given that domain names can now be 64 characters long).

CustFName and CustLName are defined as null, because either may be missing. While few in number, there are people who legally have only one name, and we do not want to drive them (and their money) away by requiring two names.

As for the address fields, the choices here are not as straightforward as they seem. Designing these fields to be flexible enough to support international customers raises many questions, which are beyond the scope of this chapter. For example, it is common practice to use a drop-down list to select a country. Next time you see one of these on a web site, check whether it contains an option for Bouvet Island; many web forms do, despite the fact that nobody lives there.

For further information on country codes and usable form design, see Web Forms and ISO 3166 country codes at http://sites.encompassnow.com/idude/articles/web_forms_and_iso_3166.asp, *and Usable Forms (for an international audience) at* http://evolt.org/article/UsableForms/4090/15118/index.html

For our purposes, we will simply declare the address fields as varchar, rather than using country codes or similar techniques. Note that all address fields may be null, because we will permit customers to register on our online store to receive our e-mail newsletter without necessarily placing an order, which would require an address.

Orders

Column	PK	Data type	Nullability	FK	Refers to PK
OrdID	PK	integer	**not** null		
OrdTotal		numeric(7,2)	**not** null		
OrdDiscount		numeric(7,2)	**not** null		
OrdTax		numeric(7,2)	**not** null		
OrdShipping		numeric(7,2)	**not** null		
OrdDate		date	**not** null		
OrdShipDate		date	null		
OrdCustID		integer	**not** null	FK	CustID

OrdID will be a numeric surrogate primary key, so it is integer and not null.

OrdTotal, OrdDiscount, OrdTax, and OrdShipping are all monetary columns, defined as not null because we need to know them for every order, even if one or more of them is zero on any given order.

OrdDate is the date the order was placed, and so it requires a date data type. It is not null because the date must be known. The value will typically be inserted using the database system's "current date" function when the order row is inserted.

OrdShipped is the date the order was shipped, and may be null, for those orders that have been placed but not yet shipped.

OrderItem

Column	PK	Data type	Nullability	FK	Refers to PK
OrdItemOrdID	PK	integer	not null	FK	OrdID
OrdItemProdID	PK	char(10)	not null	FK	ProdID
OrdItemQty		smallint	not null		
OrdItemPrice		numeric(7,2)	not null		
OrdItemDiscount		numeric(7,2)	not null		
OrdItemTax		numeric(7,2)	not null		
OrdItemShipping		numeric(7,2)	not null		

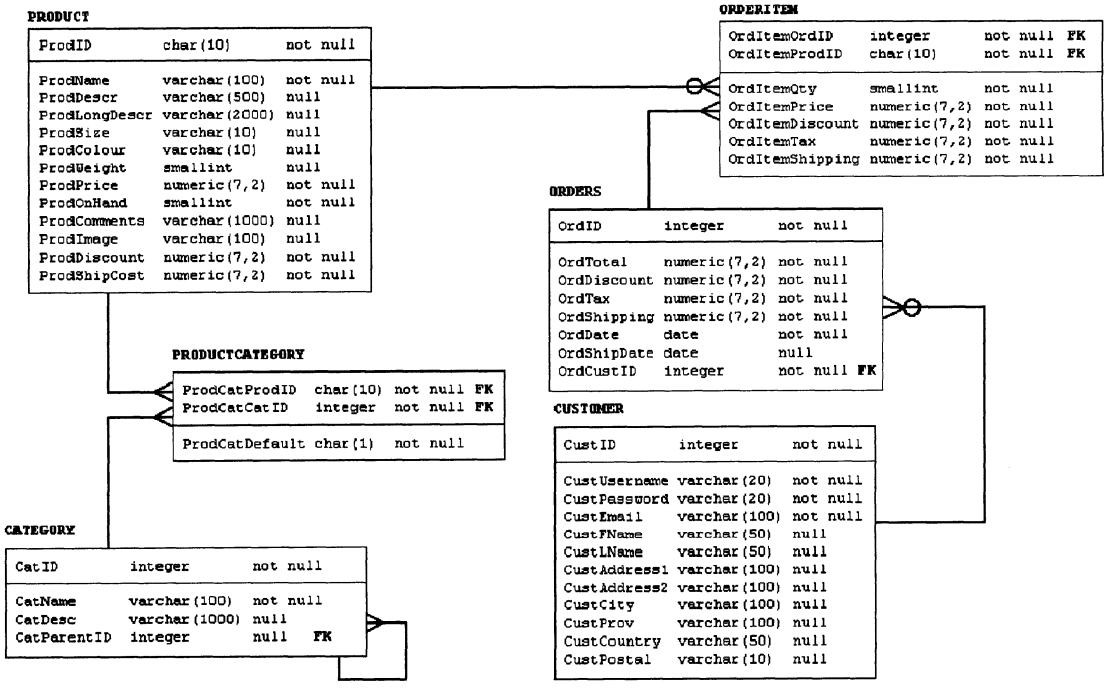
The **OrderItem** table has two foreign keys and a concatenated primary key. By now, this should be easy to recognize and need no further explanation.

`OrdItemQty` is the quantity of this particular product on this particular order. It is a `smallint` but could as easily be another numeric data type, for example if the product is perhaps sold in meters and fractions of meters, like wallpaper.

`OrdItemPrice`, `OrdItemDiscount`, `OrdItemTax`, and `OrdItemShipping` are monetary columns, allowing for overrides on this particular order for the corresponding normal product amounts (in case of a special offer).

The Overall Database

Bringing all these tables together into one diagram, we can now see how it all fits together:



Database Indexes

An important step in implementing a database is to create indexes. Indexes are used by the database system to improve the efficiency of read operations.

There are two sets of indexes that should be created – those for keys, both primary and foreign, and those for other columns frequently used in SQL queries.

Primary keys, it turns out, will always be indexed, because database systems use a unique index to implement them.

Foreign keys require indexes only if the database needs to search a table for a particular foreign key. This is best illustrated with an example. Suppose we want to return all products ordered by customers in Eureka. This is the sample query we saw earlier:

```
SELECT CustID
     , CustName
     , ProdID
     , ProdName
  FROM Customer
INNER
  JOIN Orders
    ON CustID = OrdCustID
INNER
  JOIN OrderItem
    ON OrdID = OrdItemOrdID
INNER
  JOIN Product
    ON OrdItemProdID = ProdID
 WHERE CustCity = 'Eureka'
```

In order to satisfy this query, the database would most likely start with the **Customer** table and search it by `CustCity`. If we planned to run this query often, then it makes sense to have an index on `CustCity`. Let's say that the result of this search is a small number of **Customer** rows.

Next, each **Customer** row that satisfied the search condition is then joined with its **Orders** records. But how does the database join **Customer** and **Orders** records? The answer is, by the foreign key `OrderCustID` in the **Orders** table. The only way that the database can avoid scanning the entire **Orders** table to find rows with `OrdCustID` values that match one of the `CustID` values from the rows that satisfied the search condition, is if there's an index defined on `OrdCustID`.

Once the matching **Order** records are determined, the **OrderItem** records can quickly be found through the primary key index on `OrdItemOrdID`. This is a happy coincidence, because we defined `OrdItemOrdID` as the first of the two columns in the concatenated primary key (more on this in a moment). Finally, the `OrdItemProdID` values from the selected **OrderItem** rows are used to match with `ProdID` in the **Product** table, using the primary key index on `ProdID`.

If we were to run a different query, to find all the customers who ordered a particular product, the search would proceed in the other direction, from **Product** to **OrderItem** to **Order** to **Customer**. In this case, the primary key index on the `OrderItem` concatenated columns `OrdItemOrdID` and `OrdItemProdID` would likely not be used, because the index entries are maintained in `OrdItemOrdID` sequence. The database might do an index scan, or it might just do a table scan, which would be much less efficient.

> *a good strategy is to define an index for any foreign key that isn't the first column of a primary key*

Thus, a good strategy is to **define an index for any foreign key that isn't the first column of a primary key**.

It's worth knowing that numeric data is indexed much more efficiently than character data. We'll see more about indexes, how to use them, and how to improve them, in *Chapter 8*.

Sample DDL

Below is an example of **Data Definition Language** (**DDL**), used to declare a table and its indexes. Remember that DDL is not a separate language, but a subset of SQL. It's merely more descriptive than "structured query language" when we're dealing with `CREATE` statements. (SQL isn't a terribly accurate name either, since besides `SELECT`, it includes the `INSERT`, `UPDATE`, and `DELETE` statements.) The language used below is standard SQL; minor variations may be required for different database systems.

```
CREATE TABLE OrderItem
      ( OrdItemOrdID      integer        NOT NULL
      , OrdItemProdID     varchar(10)    NOT NULL
      , OrdItemQty        smallint       NOT NULL
      , OrdItemPrice      numeric(7,2)   NOT NULL
      , OrdItemDiscount   numeric(7,2)   NOT NULL
      , OrdItemTax        numeric(7,2)   NOT NULL
      , OrdItemShipping   numeric(7,2)   NOT NULL
      , PRIMARY KEY
          (OrdItemOrdID, OrdItemProdID),
        FOREIGN KEY (OrdItemOrdID)
          REFERENCES Orders,
        FOREIGN KEY (OrdItemProdID)
          REFERENCES Product
      )
CREATE INDEX OrdItemProdIndex
  ON OrderItem (OrdItemProdID)
```

Notice how the composite primary key needs to be declared in a separate clause. If the primary key is a single column, it can also be declared on the column, for example:

```
CREATE TABLE Product
      ( ProdID  char(10)  PRIMARY KEY  NOT NULL
        ...
```

Since primary keys are not null by definition, it seems redundant to add the not null specification, but it doesn't hurt, either.

Notice also that the **OrderItem** foreign key specifications, to the **Orders** and **Product** tables, name the table as the referenced target. By definition, this links the foreign key to the primary key of the referenced table. It is not necessary to name the target table's primary key, because that's the default. It is possible, albeit unusual, for a foreign key to reference a column that isn't the primary key in its table; in that case the target specification is:

```
REFERENCES tablename (columnname)
```

The index `OrdItemProdIndex` is declared on the second of the two primary key columns. Notice that there is no user-defined index declared on `OrdItemOrdID`, the first of the two primary key columns. As mentioned previously, most databases create a unique index on the primary key as their means to ensure uniqueness, and to provide a quick means to check relational integrity. If you declare an index on the primary key or on the first column of a composite primary key in addition to the one that the database creates, then there will be two indexes on the first column of the primary key. While this would not hurt retrieval (since only one of them would be used) it will hurt inserts and deletes, as both indexes would need to be maintained. Some databases will detect this and warn you, some may not.

> *most databases create a unique index on the primary key as their means to ensure uniqueness*

For complete DDL samples in the particular dialect of several popular databases, see the Appendix.

Sample SQL

Here are some sample queries that will be required for our e-commerce site.

Search for Products

Suppose a customer visits our e-commerce site and wants to buy a T-shirt. One of the basic functions we should provide is a search facility. The following query retrieves product information based on a search string (*T-shirt*) that the customer has entered into the search field of the web form:

```
SELECT ProdID
     , ProdName
     , ProdDescr
     , ProdLongDescr
     , ProdSize
     , ProdColour
     , ProdWeight
     , ProdPrice
     , ProdOnHand
     , ProdComments
     , ProdImage
     , ProdDiscount
     , ProdShipCost
  FROM Product
 WHERE ProdName LIKE '%T-shirt%'
    OR ProdDescr LIKE '%T-shirt%'
    OR ProdLongDescr LIKE '%T-shirt%'
```

There are a number of important considerations here. The first is to note that the query selects every column of the **Product** table. There is another way to write this, namely:

```
SELECT *
  FROM Product
 WHERE ProdName LIKE '%T-shirt%'
    OR ProdDescr LIKE '%T-shirt%'
    OR ProdLongDescr LIKE '%T-shirt%'
```

The "select star" (`SELECT *`) syntax is shorthand for selecting all columns. In general it is **not** a good idea to do this. Listing the individual columns is a better programming strategy, because it is self-documenting, and because the number of situations where you really want all columns is rare. For instance, in the above example, it is much more likely that the verbose `ProdLongDescr` will not be shown in search results, since many products might satisfy the search criteria; the long description would be shown only on a separate page reached by following a link such as *Show product* details. Furthermore, returning large fields such as `ProdLongDescr` for multiple products is quite inefficient if they are not going to be used, and that's what would happen with the "select star" syntax.

The next point to note about the sample query is the use of the `LIKE` comparator against several columns. `LIKE` uses wildcards (in the example above, the percent signs) to allow for the designated string to be found anywhere within the searched column. So how does the string `T-shirt` get into the `LIKE` field between the wildcards? The e-commerce application will be driven by some scripting language such as Perl, ASP, PHP, JSP, or ColdFusion. It is the function of the scripting language to capture the contents of the form field and insert it into the SQL statement, pass the SQL statement to the database, and then format the results into the web page.

Add a Product to a Customer's Order ("add to cart")

Suppose a customer has already selected one or more items to purchase, that is, an **Order** row for this customer has already been created. Now the customer clicks on the "*Buy Now*" button or link beside a product. Assume there's a quantity field on the web form such as a drop-down list or input field. The online store application script would then construct an `INSERT` SQL statement similar to the following:

```
INSERT INTO OrderItem
      ( OrdItemOrdID
      , OrdItemProdID
      , OrdItemQty
      , OrdItemPrice
      , OrdItemDiscount
      , OrdItemTax
      , OrdItemShipping
      )
SELECT OrdID
      , ProdID
      , form.qty
      , ProdPrice
      , ProdDiscount
      , calculation
      , ProdShipCost
  FROM Product
 WHERE ProdID = 'form.prodid'
```

This `INSERT SELECT` statement allows values from a row in the **Product** table to be used to construct the new row of the **OrderItem** table. The `OrdID` value, which is the value of the customer's current **Order**, must be inserted into this statement, as indicated by italics in the sample above. This value is supplied by the application that is controlling the current order.

The particular product is located by the `WHERE` clause using the value passed to the application by a hidden form field, *form.prodid*. The quantity ordered, *form.qty*, also comes from the web form, and the tax might be a calculation based on product price and perhaps the customer's location.

This is necessarily a superficial treatment of the order process, meant only to illustrate the SQL.

Summary

In this chapter we have taken the conceptual design of our database from *Chapter 5*, and turned it into a real database.

We began by looking at some simple naming conventions to follow when creating tables and columns. We then reviewed our existing entities, and created additional associative entities for the implementation of many-to-many relationships. We reviewed data types and nullability, natural and surrogate primary keys, and foreign keys and relational integrity.

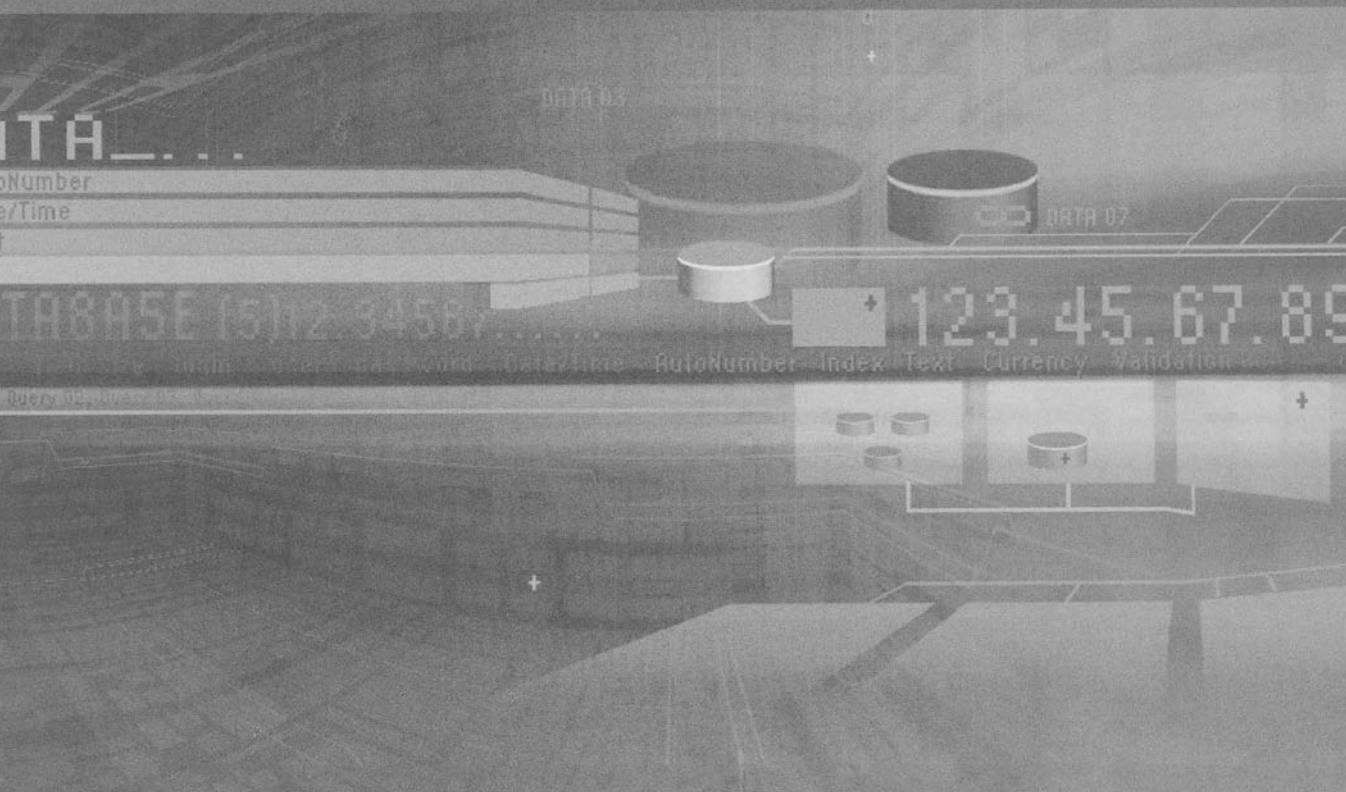
Then the real work of turning design into reality began, as we began to define our tables and their columns. At this point, we took the lists of entities and their attributes that we created in *Chapter 5* from our discussions, including the additional associative entities we needed, and began to choose appropriate data types and nullability for each of the attributes. We also specified which columns would be primary keys, and which columns would be foreign keys.

Finally, we took a brief look at adding indexes to the tables to make searching them easier. We rounded off by looking at a couple of queries that we'll want to use: one for finding a product and another for adding products to a customer order.

7

- Secure databases by design
- Different access for different users
- Using views and stored procedures

Author: Thearon Willis



Database Security

In the last few years, we have seen a significant increase in the publicized attacks on major web sites such as *eBay*, *Amazon*, and even Microsoft. While most of these attacks focus on denial of service to, or defacement of these web sites, attacks on the databases used to run these web sites have risen too.

The security of the data that is stored in databases and the database servers used by web sites is becoming increasingly important especially for e-commerce sites, which not only store data about products but also store sensitive information about their customers such as addresses, phone numbers, and credit card information.

The network and database administrators at your company are concerned about security issues and do everything possible to prevent attacks. You as a developer can help them by focusing on the database-related security issues up front when you first set out to design your application and database.

This chapter examines the techniques and best practices that you as a developer can use to secure your database and data.

In this chapter, we will cover:

- Designing a security model
- Securing your database
- Using Inline SQL, stored procedures, and views
- The importance of data validation
- Network security for your database systems]

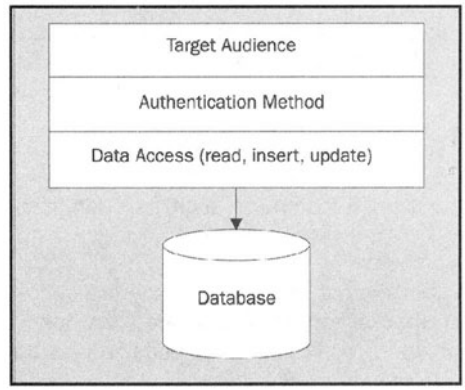
A note about the examples: as with other chapters in the book, there are times when we have to pick a specific database in order to illustrate something. In this chapter, we've picked Microsoft SQL Server 2000.

Designing a Security Model

When you first set out to design your database, you probably thought about security at some point, but it really didn't fit into designing the structure of your database. At that point, you were most likely concerned with the types of data to be stored, and how you would organize and store that data for the most efficient access and processing. Making your database into an accurate representation of the entities you were modeling, including the right attributes in the right places, and the right relationships modeled in the right manner, was the key element in the design of your database at that point.

Now is the time to design, develop, and implement the necessary security measures to help protect your data and database from attacks, while you are developing the tools that you will be using for your data manipulating and retrieval. This involves not only choosing an access method for your logins – which might be operating system (OS) authentication, database authentication, or both – but also determining your target audience, who needs access to the data and what type of access they need.

The following diagram shows the major components of a typical security model. Each of these components, starting from the top down, affects how you proceed to the next level. This section of the chapter covers these topics and provides some best practices where applicable.



Identifying Your Target Audience

The first thing that should be taken into consideration when designing a database security model is who your target audience is. This will affect the various areas of your security model design such as determining what type of authentication to use as well as what types of permissions to grant to the various users and groups.

For example, if yours is an e-commerce web site, the majority of your database access will be read-only while a B2B (business to business) web site will not only need to query data, but will also update it on a regular basis.

> *Identifying the target audience will help you design your database security model more efficiently*

When you have identified your target audiences, the type of data access they need should become apparent from examining the workflows of your application. Identifying the target audience will help you design your database security model more efficiently as you will only be providing the levels of access needed for the tasks being performed.

Don't forget to take into consideration everyone who will be using your application and how they will be using it. For example, suppose you have an e-commerce web site. Most of your access will come from customers shopping at the site, but don't forget about the administrators. They need access to update the contents of the database for the products and services being sold. This could be through a separate web interface or through another application.

Choosing an Authentication Method

What you must decide next is what types of authentication the RDBMS you are using supports, and which is the best method to use. For example, Microsoft SQL Server supports both Windows Authentication and SQL Server Authentication.

When a RDBMS uses **Windows Authentication** it uses your Windows NT or 2000 domain account to check you have permission to access the RDBMS and databases. The DBA will add your Windows user account to the RDBMS and grant you a specific type of access to selected databases.

SQL Server Authentication uses a login and password defined in the RDBMS itself to check that you have permission to access the RDBMS and the databases. Using this method, as most RDBMSs do, means storing the user ID and password in the database itself. The database administrator (DBA) will have to define these in the RDBMS before you can access it or any given database.

Again, your target audience will help you to determine the type of authentication method that needs to be used. For example, if you have an e-commerce web site, you obviously do not want to use OS authentication as your customers will not have user accounts defined for your system. Also, your company is likely to have strict security policies in place on who can add and control user accounts on your system.

If you are a B2B web site, however, depending on your size, business being performed, and how the web site is used, you might define and require OS authentication to be used. This would limit access to those individuals or businesses that are defined on your system.

Microsoft SQL Server allows you to set up SQL Server with Windows Authentication only or use a mix of Windows Authentication and SQL Server Authentication. This allows a DBA to set up Windows domain accounts for internal users, such as developers and administrators, and SQL Server logins for external users and applications. Most other RDBMSs don't use OS authentication, relying on their own authentication procedures – checking a user ID and password against previously created user accounts.

Your company security policies and the type of RDBMS you use will play an important role in deciding the type of authentication method that you use. As a best practice, you should use a mixed mode authentication model when using SQL Server databases that are accessed by the Web. This allows you to use Windows Authentication for all internal users and SQL Server Authentication for external users and web site applications.

Identifying the Type of Data Access Needed

Having designed your database already, you know what type of data is to be stored and accessed and have probably already written some queries for manipulating the data. Now you must decide what type of data access is required by the users and the application.

Let's look at an example for an e-commerce web site to determine the type of access that would typically be needed. Let's assume that you want to buy a new DVD player for your television. You would probably go to *Amazon.com* and do a search for DVD players. You would see the most popular DVD players with a brief description of each along with their prices. If you follow the link for a particular one, you will see more details, along with customer reviews. All of this information will help you decide if you want to purchase a particular item or do some more comparison shopping.

The point is the application has only **read** data from the database to build the pages that are displayed to you. This is probably the most typical scenario at *Amazon.com* and the application would need read-only access to the data most of the time.

Let's take this one step further and assume that you have decided to purchase an item. For the sake of argument, let's also assume that the shopping cart feature of the web site uses cookies instead of storing the shopping cart items in the database or by some other means.

> *Using two logins is a best practice*

You now want to proceed to the checkout pages of the web site where you are taken to a secure link within *Amazon.com* for checkout. At this point the application will be collecting data about you such as your shipping address and phone number as well as credit card information. Only now does the application need insert and update access to the database to store this information.

As you can see from the example above, there are times when your web application will need read-only access to the data in the database, and times when it will need insert and update access to the data. You have to determine when each is appropriate, design your web pages accordingly and use **two logins** for accessing the database. Using two logins is a best practice, although you could use a single login that has the appropriate permissions to perform all actions that your web site needs.

Now you may be asking yourself, why have two separate logins. The answer is simple, security. We want to take all the necessary precautions to prevent a hacker from accessing our data. If you use one login throughout your application that has select, insert, update, and delete access to the database, it makes it much easier for a hacker to access and wreak havoc with your database. Keep in mind that using one single login for your web site exposes a security risk and should be used with caution. We will see more about this later in this chapter.

Choosing the Type of Access Needed

Now we have established that it is better to use two separate logins to access the database, you must next determine when read-only access is needed and when insert and update access is needed. You'll need to go through the workflows of your application carefully and determine when to use a login with read-only access and when to use a login with insert and update access.

As we saw in our example with *Amazon.com* previously, most of the typical access will be read-only, as most of the visitors to the site will just be browsing. Only when a visitor decides to become a customer and purchase something do we take them to a secure link and provide the application with insert and update access to the database.

Defining Users and Groups

While our discussion thus far has been about the access needed by our application, we don't want to ignore the fact that administrators and developers will also need access to the database. The levels of access granted and to which databases (development, test, and production), will be determined by the policies set by your company.

At this point it makes sense to examine the use of users and groups within a database. A user represents a single login in the database and is typically assigned to one person, while a group typically represents a group of logins. Permissions are assigned to a login defined in the database for a single user and also for a group defined in the database. Then every user who is assigned to a group inherits the permissions of that group.

Let's take a look at a single user for a moment. A single person would typically use one login defined in the database, and that person would be responsible for keeping the password associated with that login secure. That login would be granted certain rights in the database depending on the purpose it is intended for and the person to whom it is assigned.

For example, a developer accessing the development database would be using a login that could manipulate not only the data within the tables themselves, but also the structure of the database by adding and changing tables. A login setup for an application to access the database would only need access to modify the data within a table and not the actual table structure itself. This matter would be different for a production database, where the developer will probably not have the same privileges as all the changes they would want to make should be made in the development database first.

Never give administrative rights to the login for an application unless it is absolutely necessary, and then keep the use of that login limited to its intended function. If a hacker were able to gain access to a login with administrative rights, they could do untold damage to your database.

A group login is typically associated with a Windows domain group, which means that the RDBMS would need to support Windows Authentication. Once the group has been defined in the RDBMS and the appropriate permissions granted to a specific database, all users who belong to that group would be allowed to access the database with the permissions specified in the RDBMS using Windows Authentication.

Group logins for a database are often different from group logins for operating systems.

Let's assume that you have six developers working on a project and those six developers are all members of a Windows domain group called *Developers*. The DBA could define six separate logins in the database, or they could define the one group called *Developers*. Defining the Windows group in the RDBMS is administratively easier because if one person left the company or moved to another group, their user account would be removed from the *Developers* group at the network level, which automatically means they would no longer have access to the database. It is also easier to assign and maintain privileges in the RDBMS on a group than to have to update six separate logins.

The bottom line here is that if your RDBMS supports Windows Authentication, it makes sense where applicable to use groups, such as developers, administrators, and users who would typically need read-only or restricted access to the data.

Using Strongly Typed Passwords

Different companies have different policies when it comes to the length of passwords, the use of upper- and lowercase letters, numbers, or special characters in a password. A **strongly typed password** is one that contains upper- and lowercase letters, numbers, and special characters, is sufficiently long in, and doesn't represent a word or name.

Bear in mind that there is little point in enforcing a 128-character password for something as simple as registering for a newsletter.

A password that matches these characteristics will be harder for a hacker to crack compared to a simple password that spells a name or word. Your company policy should dictate the minimum requirements for passwords, but I encourage you to use strongly typed passwords.

A typical best practice would be to use a password that is at least seven characters in length, containing both upper- and lower-case letters, at least one number, and one special character. An example of a good strongly typed password might be something like: lZ@oQ2$.

This, however, is not practical for most people as a hard-to-remember password will be written down by a user, which defeats the purpose of a strongly typed password. Users could employ a combination of special characters and numbers with their passwords to make it easier to remember. For example a password of **Waterhouse** could be broken up to be **w@t3r#H0u$e9**. This is easier to remember and contains upper- and lowercase letters as well as special characters and numbers. The # symbol in the middle makes it even more secure, as many hackers will work on substituting letters with symbols or numbers of similar appearance.

Encrypt Your Passwords

Encrypting the login passwords is typically done automatically by the RDBMS when the password is added to it, and is also done automatically by Windows operating systems. A lot of applications, however, come up with their own security scheme that usually consists of a user table containing a list of users who are authorized to access the application, along with their passwords. In the e-commerce example we have been looking at, a user wanting to buy something will have an account with a user ID and a password (they'll have to register the first time they want to buy something).

If the passwords in this table are not encrypted, then anyone with access to the database can view the user logins and their passwords in this table. You may be thinking that only internal users would have access to the database and this is not a big deal, but consider this: suppose a hacker was able to gain access to your database. They would then have access to all that data, and, depending on the type of application you have, could use those logins and passwords to gain credit card information about your customers. They could even go as far as logging in as a customer, changing their address, ordering merchandise, and having it sent to themselves (changing it back afterwards so they don't get caught).

Auditing database changes would be a way to combat the situation described here.

You should use a sound and proven encryption routine to encrypt and decrypt all sensitive data including passwords. The Windows operating systems have the CryptoAPI and there are also many third-party vendors that provide sound, proven, and reliable encryption algorithms.

The bottom line here is that it is best practice to always encrypt the passwords and any other sensitive data in your database.

Another way to help keep sensitive information secure is to use Secure Sockets Layer (SSL) on your web site. Using SSL will keep your information secure and encrypted to prevent it from being intercepted and read by unauthorized persons.

How to Secure Your Database

Now that we have had a look at the types of data access that are needed by your application, it's time to move on and look at what can be done to secure your database. In most RDBMSs you define a login that has permission to log in to the RDBMS but that login doesn't automatically have access to any specific database until the appropriate permissions are granted. In other words, you have to tell the RDBMS which database or databases that login will be able to access.

Once you specify a database that the login can access, you typically specify a role that the login will have in that database, such as reading data, being an administrator in that database, etc.

Along with assigning a role to a login, most major RDBMSs provide the capability to apply permissions at the table level. This allows you to further restrict the actions that a login can take

This section will take a look at applying permissions and help you decide what type of access a login should have in your database.

Those RDBMS products that allow you to specify table-level permissions will generally allow stored procedure- and view-level permissions. We'll see more on stored procedures and views shortly in this chapter, and again in Chapter 8.

Table Permissions

Different RDBMSs have different roles that are assigned to a login when it is granted access to a database, although some don't have roles at all. For example, Microsoft SQL Server has the **db_datareader** role that can be assigned to a login. This role specifies that the login has read-only access to all tables in the database and cannot update any data.

A login that has access to all the tables in your database may not be appropriate given the nature of your database and application. Let's go back to our *Amazon.com* example for a moment and assume that not only do we have product-related tables in our database but we also have customer-related tables in our database. We wouldn't want the login that has read-only access to be able to access the customer tables.

Ideally you would have a separate database for your customer information to keep it distinct and secure. This would force us to have different logins and passwords to access the information.

Some RDBMSs, like Oracle, allow you to go as far as applying permissions at the column level, further restricting access to sensitive information within a table. This type of filtering allows you to provide access to certain columns in a table while restricting access to other columns.

When you grant or deny permission for a login to access a table, you typically have the option of specifying what type of access the login can have, for example, `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and even DRI permissions depending on the RDBMS that you are using. DRI permissions allow a login to drop and create **Declarative Referential Integrity** constraints such as foreign key constraints.

Setting the permissions for each table allows you to restrict a login to the specific activity that it was created for. You will have to go through your database table by table and set the specific permissions for each login. This may seem like a lot of work given the number of tables and logins defined in your database, but think of the consequences if you don't and a hacker gains access to your database through a login that has blanket select and update capabilities.

Stored Procedure Permissions

A stored procedure is a sequence of SQL statements compiled into a single object within the database under a single name. When you execute a stored procedure it can return data or insert, update, and delete data depending on the SQL statements it contains. Stored procedures will be covered in depth in the next chapter, but we do look briefly at them in this chapter.

Like table permissions, stored procedures in your database also allow permissions to be granted or denied to a login. Unlike table permissions, a stored procedure can only have grant or deny execute access – there's nothing else you can do with one, other than create or delete it.

> *keep in mind what task the stored procedure is performing*

This is another area that you will need to address and determine which logins should be able to execute the stored procedures created. Again, keep in mind what task the stored procedure is performing and whether or not that task is appropriate for the login you are dealing with. For example, it probably isn't appropriate for a login that was created for updating product information to be able to execute stored procedures to update customer information. Likewise a login that is created for read-only access shouldn't be granted access to stored procedures that update data.

Stored procedures are a good way to enforce the security of your database. You needn't grant a login permission to access tables directly: only the stored procedures that are relevant to the login. This prevents a login from accessing any data that isn't accessed by the stored procedures.

View Permissions

A view is similar to a stored procedure in that it contains SQL statements that can select, insert, update, and delete data. A view is, however, more like a virtual table in that it can return data to the user that can be updated. We will talk more about views later in this chapter and they will also be covered in more depth in the next chapter.

> *Most views are created as read-only*

View permissions are a lot like table permissions in that you can grant select, insert, update, and delete permissions for a single view. You really need to take a look at the views created and determine what type of access should be given to each login based on the tasks that it will be performing. Most views, however, are created as read-only: they typically select data from one or more tables but don't allow updates to be made.

While views are a great way to shield the user from the complexities of your database structure, they can also be dangerous because they can modify the data in your tables. This becomes increasingly important for complex views that return a result set from multiple tables. If a hacker were to gain update access to a complex view, they could modify the data in multiple tables at one time.

Again, you need to analyze each view and determine the level of access to be granted to each login based on the role it was created for.

Setting Permissions

We talked about setting permissions on each table (and column), stored procedure, and view created in your database. This may seem and is a lot of work for a database that has already been created, especially if the database contains many tables, stored procedures, views, and logins. This task is simplified for databases that have defined groups with many assigned users, and it is the group that is granted permissions.

For databases in the process of being developed, however, you can minimize the amount of work with a little planning up front. In an ideal world, you should create all of your logins when you create your database. Then as you create a table, stored procedure, or view, you can apply the appropriate permissions for each.

This can be done using a SQL statement to grant permissions for the table, stored procedure, or view that you are creating. The example below grants various permissions on different objects to the login called `MyLogin`. You place these after the SQL statements that create your table, stored procedure, or view:

```
GRANT SELECT, INSERT, UPDATE ON MyTable TO MyLogin

GRANT EXECUTE ON MyStoredProcedure TO MyLogin

GRANT SELECT ON MyView TO MyLogin
```

Along with granting permissions, there may come a time when you will want to revoke the permissions assigned to a user or group. This can be done using the same syntax as above substituting the `REVOKE` statement instead of the `GRANT` statement as shown below:

```
REVOKE SELECT, INSERT, UPDATE ON MyTable TO MyLogin

REVOKE EXECUTE ON MyStoredProcedure TO MyLogin

REVOKE SELECT ON MyView TO MyLogin
```

Inline SQL vs. Stored Procedures

This section of the chapter will take a look at using inline SQL and stored procedures in your web applications. We will show you the dangers that they can present and how to handle or avoid those dangers.

The most common of these dangers is **SQL injection**. SQL injection is a term used to describe the insertion of extra SQL statements into an inline SQL stream by manipulating the data input of the application. You may think that this sounds hard to do but is actually quite easy and we'll take a look at this in detail and the danger it presents shortly.

> *Imagine the damage a hacker could cause, knowing the structure of your database*

Another common danger is the lack of error handling or the use of generic error handling which can expose the structure of your database to the user. Imagine the damage a hacker could cause, knowing the structure of your database and how the tables relate.

We'll examine these topics and more in this section so let's get started.

Inline SQL

Inline SQL refers to the use of SQL statements in your code. The following code fragment shows how this is typically used. This code fragment uses ADO and VBScript to build a SQL string and open an ADO `Recordset`. The important thing here is not what method we are using to retrieve the data but how the SQL string is being built. We are merely substituting the data received from a web page and inserting it into our SQL string. We've all seen and done this before; it's the easiest way to retrieve data and requires the least amount of code:

```
strSQL = "SELECT * FROM Users WHERE LoginName = '" & _
         Request.Form("txtLogin") & "';"
objRS.Open strSQL, objConn, adOpenForwardOnly, adLockReadOnly, adCmdText
```

Let's forget about the lack of data validation in this code for a moment. Did you know that this type of code is very dangerous and is susceptible to SQL injection? Let's examine this in more detail.

Remember that SQL injection occurs when a hacker injects extra SQL statements into your query. You may be wondering how that could happen since they can't get to and modify the source code.

> *in practice you should always require a password*

Let's say you have a login form that just requires a user to enter their login name in a textbox and then click on the *Submit* button. For the simplicity of this example we won't require a password, although in practice you should always require a password. Let's further assume that the database login used to execute this query has administrative rights to the database; if we've followed the guidelines above, of course, we won't be in this situation.

Now the hacker can enter the following code in the textbox on your HTML form and then click the *Submit* button:

```
foo'; drop table user --
```

Before we move on, let's examine the data that was entered. Keeping in mind that the login name is a string, the hacker knows that the code for our inline SQL string will have to delimit the string with a single quote at the end, which they have done here in the text that they have entered. Next, they have entered a semicolon indicating that this is the end of the first SQL statement. They have then entered the valid syntax to drop a table followed by two dashes, which is the SQL syntax for a comment. Any code following the comment is ignored, which would be the final delimiter that your code is inserting. Because any code following the comment is ignored, such as the single quote delimiter that our code adds, it makes it easier for a hacker to inject his SQL statements without having any errors occur.

Assuming you have generic error handling that returns the error messages from the database, the hacker would see the following message returned.

```
Incorrect syntax near the keyword 'user'.
```

At this point the hacker knows that the name of our user table is not **User** and can proceed to try different names. Let's assume that they try this syntax next in which **Users** is the correct name for our user table.

```
foo'; drop table users --
```

Because we have used a login with administrative privileges to execute our inline SQL statement, the **Users** table is dropped preventing any other user from accessing the system.

*There are no warnings that the **Users** table is going to be dropped, and no messages are returned: the table is simply dropped.*

Ways to Stop Inline SQL Abuse

As we can see from the code above, there are multiple dangers present, all of which we could have prevented. First, we should **never use a login that has administrative privileges** to execute the code from our web pages.

In a previous section of this chapter, we examined what type of access was needed by an application and came to the conclusion that, in the best interest of security, a common login for an application would have read-only access to the data for most types of operations performed by the user.

A login that has read-only access to the database cannot drop tables or update and delete data.

Next, we should always validate the data coming in, on the server rather than at the client. There are several things that we should validate:

- We should validate that the data is of the correct data type for the parameter that is being used. For example, if the parameter expects numeric data then we should validate that the data the user entered for that parameter is numeric.
- We should validate the length of the data. If a parameter expects a maximum length of 12 characters, then we should ensure that a maximum length of 12 characters has been entered and no more.

- We should escape all single quotes within the input. This is required to ensure the syntax of the SQL string is correct.
- We should validate the data to ensure no extra SQL statements are being passed. This can be as simple as validating that there are no semicolons or we can take it even further and validate the data for keywords such as `DROP TABLE` or the names of the tables in our database.

If you want to capture the detailed error messages returned from the database to assist you in debugging a problem a user may be having, then you could set up some type of logging mechanism where the detailed error messages returned are logged in a file.

Finally, we should provide better error handling and not just return the messages received from the database to the user. Returning the database errors to the user can reveal the structure of our database to that user. We should handle the common errors that are expected to be returned, interpret them, and send a meaningful error message to the user. For the errors that are unexpected, a generic friendly error message should be sent to the user. We'll cover more on error handling shortly.

As we can see, there are several actions that we can and should take to prevent SQL injection. All of these are commonsense precautions that every developer should be taking, and they go a long way in preventing attacks on our data.

As a best practice, you should always use stored procedures whenever possible. They are more secure than inline SQL, execute faster, and there is less data to be sent across the network when invoking a stored procedure.

Stored Procedures

Stored procedures can also be susceptible to SQL injection if you use a SQL string to build the SQL statement to execute the stored procedure along with its parameters. It all depends on how you implement the stored procedure in the SQL string. Let's take a look at a couple of examples.

In these next few examples, the hacker has entered the same data as before which is shown again in the following code fragment:

```
foo ; drop table users --
```

Our SQL string for executing the stored procedure is shown in the following code fragment, which again uses ADO and VBScript. Notice the last parameter (`adCmdText`) on the `Open` method of the `Recordset`. We have failed to change this parameter to indicate that we want to execute a stored procedure and the database will interpret this SQL string as a text command when it gets executed.

```
strSQL = "exec usp_parmsel_login '" & Request.Form("txtLogin") & "'"
objRS.Open strSQL, objConn, adOpenForwardOnly, adLockReadOnly, adCmdText
```

When our code executes, the SQL string is built and looks like that shown below:

```
exec usp_parmsel_login 'foo'; drop table users --'
```

When our code executes, the stored procedure successfully executes too but so does the next command, which drops the **Users** table.

If we were to update the last parameter in the `Open` method of the `Recordset` and change it to a command type of `adCmdStoredProc`, as shown in the following code, it will be next to impossible for the hacker to inject extra SQL into our SQL string. Notice that I said next to impossible, not totally impossible. Hackers are clever people and they will find a way to do something if they want to do it badly enough.

```
strSQL = "exec usp_parmsel_login '" & Request.Form("txtLogin") & "'"
objRS.Open strSQL, objConn, adOpenForwardOnly, adLockReadOnly, adCmdStoredProc
```

So again, the SQL string that gets built with the user input looks the same as it did before, as shown below:

```
exec usp_parmsel_login 'foo'; drop table users --'
```

When we try to open the `Recordset` this time, however, we receive an error from the database as shown below. This time the database saw that we wanted to execute a stored procedure and performed some validations against our SQL string and that stored procedure:

```
Syntax error or access violation
```

We can take this code one step further and provide parentheses around our stored procedure parameters, making it even more difficult for a hacker to inject extra SQL into our SQL string as shown in the code below:

```
strSQL = "usp_parmsel_login ('" & Request.Form("txtLogin") & "')"
objRS.Open strSQL, objConn, adOpenForwardOnly, adLockReadOnly, adCmdStoredProc
```

When this code runs, our SQL string is built and looks like the code shown below. We will receive the same error as in the last example when we try to open the `Recordset`:

```
usp_parmsel_login ('foo'; drop table users --')
```

This method provides an extra level of protection, as even if we used the value `adCmdText` as the options parameter of the `Recordset`, we would receive the error shown below.

```
Incorrect syntax near 'foo'
```

To make it really hard and next to impossible for a hacker to inject extra SQL statements into our code, we can use the ADO `Command` object in conjunction with the ADO `Recordset` object in our ASP pages to execute our stored procedure. Yes, using this method requires more code, but think of the consequences if you don't do everything possible to stop a hacker.

The code fragment below shows how we can use the `Command` object to specify each parameter that the stored procedure expects and then to return our `Recordset`:

```
objCmd.CommandText = "usp_parmsel_login"
objCmd.CommandType = adCmdStoredProc
objCmd.Parameters.Append objCmd.CreateParameter("LoginName", adVarChar, _
        adParamInput, 20, Request.Form("txtLogin"))
Set objRS = objCmd.Execute
```

Using this method to execute stored procedures provides two significant benefits. First, it performs a certain level of validation for you. The first thing that it will do is to properly escape all login names that contain a single quote, O'Malley for example. Next, it is going to validate that the length of the parameter being passed does not exceed the length that is specified in the parameter. It will also ensure that the data type being passed matches what the stored procedure expects.

> *It will ensure that the data type being passed matches what the stored procedure expects*

Do not think just because certain validations are being performed for you that you don't need to perform any validations yourself: this is far from the truth. You should always perform data validation on the input you receive from the user. This way you are ensuring that you are receiving the type and length of data that you expect and can possible head off an attack by branching to other logic in your code if you find any malicious data such as `DROP TABLE`. This approach will also help to keep rubbish out of your database, when otherwise benign users include the wrong things in their input.

> *You should always perform data validation on the input you receive from the user*

Enough about SQL injection and data validation, let's move on and talk some more about error handling.

Error Handling

The worst thing that you can do is to provide generic error handling such as shown in the code below. This code has the potential of exposing your database structure to the user when an error occurs:

```
Dim objErr As ADODB.Error
For Each objErr In objConn.Errors
    MsgBox objErr.Description
Next
```

To make this point, let's assume you have two tables in your database named **Employees** and **Payroll**. These tables are linked together with a foreign key constraint on `PayrollID`. When you insert a record into the **Employees** table you must also insert a corresponding record into the **Payroll** table.

Depending on the values entered you could receive one of two messages as shown below. Both of these messages expose some structure of your database to the end user.

```
INSERT statement conflicted with COLUMN FOREIGN KEY constraint
'FK_Employees_Payroll'. The conflict occurred in database 'TestDB', table
'Payroll', column 'PayrollID'.

Violation of PRIMARY KEY constraint 'PK_Employees'. Cannot insert duplicate key
in object 'Employees'.
```

This information in the hands of a hacker allows them to exploit your database further, and could cause some serious harm. Notice that the first error message not only exposed the foreign key constraint name, but also exposed the database name, the table name, and the column name.

One way to handle this is to catch as many expected errors as possible. In the code shown below, we are using the `NativeError` property of the ADO `Error` object to select the known errors and display a related message to the user. We also have a procedure in place that logs the detailed error messages to assist us in debugging the errors received.

Notice that we also handle any unexpected errors in the last `Case` statement:

```
For Each objErr In objConn.Errors
   Select Case objErr.NativeError
      Case 2627 'Violation of PRIMARY KEY constraint
         LogError (objErr.Description)
         MsgBox "An employee already exists with the ID specified"
      Case 547 'INSERT statement conflicted with COLUMN FOREIGN KEY constraint
         LogError (objErr.Description)
         MsgBox "A payroll record already exists with the ID specified"
      Case Else
         LogError (objErr.Description)
         MsgBox "Unexpected database error, please contact support"
   End Select
Next
```

While this error-handling procedure might not fit everyone's need, you get the idea of how we want to handle the errors returned from the database. The bottom line here is that, as best practice, you should interpret the errors returned from the database and display a message to the user that is not only meaningful, but one that does not expose the structure of your database. You are logging the real errors that are returned which can then be used by your DBA or network administrator to determine the cause of the problem.

Using Views to Restrict Data Access

Views are a great way to restrict access to your database and to also shield the complex design of the database from the end user. A view is just like a query or stored procedure in that it can select and update data in multiple tables. Views allow you to restrict access to data in your tables at the column or row level providing even more security for your data.

Permissions are applied to views just as they are to database tables. You can grant `SELECT`, `INSERT`, `UPDATE`, and `DELETE` permissions, and even grant permissions at the column level depending on the RDBMS that you are using. This type of security model allows you to further restrict data access to your tables. For example, you can deny a login access to your tables but grant it access to a view that selects data from your tables. This allows you to permit the user to see only the data that the view retrieves without providing access to your tables.

Because views shield the underlying structure of your database from the end user, they give the user the perception that the data they are viewing is coming from one table. While this may be the case for simple database designs, databases that are normalized to the third and fourth normal form typically require joining multiple tables to retrieve the data that the end user wants to see.

> *Best practice in web database design is to define logins with read-only access to the database*

Whether you use views or stored procedures will largely depend on the type of access you want the end user to have, and the level of security you apply to the logins that you define. Best practice in web database design is to define logins with read-only access to the database, and to use views to select and display data.

As we discussed earlier, a separate login can and should be defined for those circumstances where data needs to be inserted and updated and it should only be used to insert and update data.

Network Security for Your RDBMS

This section of the chapter will discuss the security of your RDBMS from an internal standpoint. That is, what you can do to secure the RDBMS internally on your network. In most large companies, a DBA (or team) is responsible for installing, securing, and managing the RDBMS. This is great if you work in a large company that has a DBA, but a lot of small companies depend on a few key developers to play the role of both developer and DBA.

Earlier in the chapter we talked about authentication modes and using Windows Authentication and SQL Server Authentication. We determined that logins for web use should be defined using SQL Server Authentication while logins for internal use should be defined using Windows Authentication (if supported by the RDBMS.)

Let's recap on what Windows Authentication does for you. Windows Authentication uses your Windows user account to authenticate you to the RDBMS. When you first try to access the RDBMS, it will validate your user account and password by calling into the security features of the Windows operating system. Once authenticated, you will have access to the databases and permissions in those databases as defined by the DBA. Windows uses encryption between the machine where you enter your password and the machine where it is stored, thereby providing a secure environment for validation, ensuring that all password data is protected and properly authenticated.

Why should you choose to use Windows Authentication where available? Primarily because it facilitates stronger security and eases the user administration tasks within the RDBMS. When using Windows Authentication, you rely on your company's security policies such as the minimum length of passwords, the use of special characters and numbers and how soon passwords expire. This helps ensure that users choose strongly typed passwords and change them regularly.

Security permissions are also retrieved from your Windows user account and applied automatically in the database. For example, if your login is a member of the *Enterprise Administrators* group, then your user account will automatically have database administrative privileges in the RDBMS provided it has been appropriately mapped in the RDBMS.

You can use Windows groups as logins in your RDBMS and any user belonging to the group can then log into the RDBMS using Windows Authentication. This helps simplify the user administrative tasks for large departments. For example, suppose you had a development staff of 20 developers all belonging to the Windows group *Developers*. You would define this group in the RDBMS and specify which databases this group should have access to and the level of access in each database. Then any user in this group could log into the RDBMS using Windows Authentication and access those databases.

If you're not using Windows on your network, you'll still have some sort of login requirements. In this case, the machine the RDBMS is installed on should only be accessible by people who have a genuine need to access it, like the DBA and developers.

Administrator Accounts

Most RDBMSs come with a built-in administrator account that has global access to the RDBMS. For example, Microsoft SQL Server has the SA administrator account that is installed with the RDBMS and this account has global access to the RDBMS. Unfortunately this account was installed by default with no password in versions prior to SQL Server 2000. SQL Server 2000 allows you to specify a password when installing the RDBMS, but does not require one.

The important thing here is to **assign a password to the administrator account for your RDBMS**. If your RDBMS is installed with a default password for the administrator account, change it. Better still, remove the default administrator account if possible, or rename it. Restrict who has access to the administrator account because the fewer people who have access to this account, the better control you will have over your RDBMS.

Typically there should be two to three people who have access to the administrator account for your RDBMS: one or two who are usually the people primarily responsible for maintaining your RDBMS, and one other just as a backup. Typically the backup person should be a network administrator.

The RDBMS administrator can set up logins for other users and assign them administrative privileges for the database or databases that they will be responsible for (but not RBDMS administrator privileges). This is typically done in development situations, so that developers can add and modify tables as the development process progresses.

The alternative to this is to store the password in a secure place that is both accessible by the backup person in an emergency, and makes it obvious when it has been used (for example, in a sealed envelope in the company safe which needs the MD's key). The password must be changed immediately after such a use.

Summary

This chapter has taken a look at the different aspects of securing your databases. We have examined authentication modes and determined what works best for internal network security and what works best for web security. To recap, Windows Authentication, if supported by your RDBMS, is the best authentication mode to use for securing your RDBMS internally and RDBMS authentication is the best authentication mode to use for databases that are accessed from the Web.

We have also talked about what types of access should be granted to logins that are used from your web application. We have seen the dangers presented by SQL injection and have determined that this not only can be prevented through good coding practices, but also through the use of a login that has read-only access to your data.

Having talked about SQL injection as a security threat, we have seen how this can occur and how to prevent it in both your inline SQL statements and stored procedures. You should now realize how important it is to validate the data that is being entered before using it in code that accesses your database. At the same time, we also covered error handling and how you should interpret the error messages from the database to shield the structure of your database from the end user.

In short, we have covered the following topics and you should understand how each plays an important role in your database security plan:

- Identifying your target audience and choosing an authentication mode
- Identifying the type of data access needed (`SELECT`, `INSERT`, `UPDATE`)
- Defining users and groups to facilitate administrative tasks
- Securing your database objects (tables, stored procedures, views)
- Protecting your code against SQL injection
- Handling database errors
- Using views to shield the database structure and to maximize security of your data
- Securing your administrator account

8

- Triggers and transactions
- Views and stored procedures
- Advanced indexing
- Advanced queries

Author: Chris Auld

Advanced Database Features

In this chapter we will examine some of the more advanced concepts of database design for the Web. Some of the topics we will look at in some depth, others would require very platform-specific details, which are beyond the scope of this book. The goal of this chapter is not to teach you all you need to know about these topics but rather to give you an overview of some of the more advanced features of RDBMSs and where you might use them in your projects. My recommendation is that if you need to do a significant amount of work with any of these features, you should buy a book that deals specifically with your database platform of choice and then combine the technical instructions from that book with the more theoretical and practical web-based ideas from this chapter.

In this chapter we will be covering a lot of ground. After the first couple of sections, though, it should be fairly easy going, as we won't be dealing with excessive amounts of code or technical detail. First we will look at some advanced query types that allow us to set more sophisticated restrictions on the data returned in our `SELECT` statements. Then we will look at a thing called a view. This allows us to create a virtual table that is really just another `SELECT` statement, potentially from lots of other tables. After that we will take a look at using procedural code within our database as stored procedures and triggers.

We will then look at some of the ways we can improve the performance of our databases by physically tuning them. And finally we will look at the concept of transactions, which allow us to ensure the consistency of the data in our database if our applications or hardware fail partway through a crucial piece of processing.

Advanced Queries

We have looked at simple SQL queries in earlier chapters. In *Chapter 3* we looked at the theory of SQL queries and how to perform basic queries on our data. In *Chapter 6* we looked at applying our knowledge of SQL queries to our real-world database example. Now it's time to dig deeper into SQL and look at some of the more advanced techniques for querying relational databases.

Sub-Queries

Sub-queries allow us to use a `SELECT` statement within other database query types to provide the contents of a `WHERE` clause. Basically sub-queries allow us to apply more complex restriction rules than we can otherwise achieve with a simple `WHERE` clause. The results of the sub-query are used to determine which rows are worked upon by the main query.

Let's take a look at an example. Say we want to select all the products from the following table that are more expensive than the average product price:

Products		
PK	ProductID	int
	ProductShortName	varchar(50)
	ProductDescription	text
	ProductPrice	money
FK1	CategoryID	int

We could probably do this using some combination of `GROUP BY` and `HAVING` clauses like we saw in *Chapter 3*, but we would end up with a very complex looking query that would be hard to understand for anyone working on the project in the future. If we think about the problem it seems reasonably straightforward on the face of it. We can easily select all of the products whose price is greater than a certain number, and we can also easily get the average product price for all of our products. The following two SQL statements demonstrate this, first getting products greater than a given price:

```
SELECT Products.ProductShortName, Products.ProductPrice
FROM Products
WHERE ProductPrice > X;
```

and then getting the average price:

```
SELECT Avg(ProductPrice)
FROM Products;
```

> *We want to do as much of our data manipulation at the database level as it is always faster*

Obviously we could use the result of both of these queries in our application and do the filtering there, but by doing that we lose all of the efficiencies that the database engine can offer us in dealing with our data. We want to do as much of our data manipulation at the database level as it is always faster than working with the data in our application, not to mention the extra traffic on the network from passing both query results to the client. We need some way to combine the two simple queries above into a single query.

Basically, we need to replace `X` with the result of the second query. This sort of nesting of `SELECT` statements is what sub-queries allow us to do. We can achieve our desired result using the following `SELECT` statement:

```
SELECT Products.ProductShortName, Products.ProductPrice
FROM Products
WHERE ProductPrice > (SELECT Avg(ProductPrice) FROM Products);
```

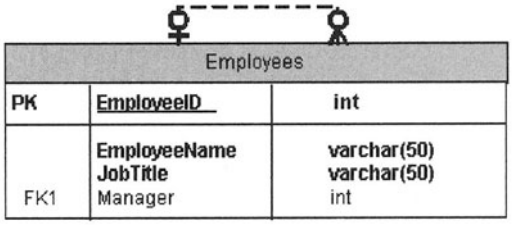
As you can see we have simply nested the second query (the **inner query**) within parentheses and slotted it into the position of `X` in our `WHERE` clause of our **outer query**. We can also nest sub-queries in the `HAVING` clause of `GROUP BY` statements. For example the following query will get all of the categories whose average product price is greater than the average price of all products:

```
SELECT Products.CategoryID, Avg(Products.ProductPrice) AS AvgOfProductPrice
FROM Products
GROUP BY Products.CategoryID
HAVING Avg(Products.ProductPrice) > (SELECT Avg(ProductPrice) FROM Products);
```

As well as the simple, single-level sub-query shown above, some database platforms will permit further nesting of sub-queries – that is, a sub-query within a sub-query. Some, on the other hand, won't allow any sub-queries at all. Exactly how the database deals with our queries internally is something that is determined by the database engines query processor. Strictly speaking it shouldn't matter how we write our SQL, the query processor should come to the same conclusion as to the most efficient way to do what we want. This is not always the case though and, particularly with sub-queries, we can run into performance issues. We will discuss how the query processor determines how to execute our queries later in this chapter. For essential and common queries, it is worth testing several formats of the query.

Sub-Query Set Operators

The sub-query above is fairly simple in that it uses one of the basic comparison operators: greater than, ">". Sub-queries also allow us to use operators that can deal with multi-valued comparisons. Let's say we wanted to select all of the employees who manage others in our organization, in order to send them on a human resources training seminar. We have the following table, which uses a recursive relationship to define who manages whom in the organization:



It's worth looking at the data in this table again just to refresh your memory as to how it works:

Employees : Table

	EmployeeID	EmployeeName	JobTitle	Manager
▶ +	1	Simon Smit	CEO	
+	2	James Bacon	Footware Manager	1
+	3	Robert Jones	Clothing Manager	1
+	4	Richard Smith	Shoes Warehouse Storeman	2
*				

If you remember from our discussions of one-to-many recursive relationships in *Chapter 3* the `Manager` column references the manager for each employee. So Richard Smith is managed by James Bacon, who is in turn managed by Simon Smit. In terms of our query, to select all of the managers we need to select all of those employees who either have a null in their `Manager` column, or whose `EmployeeID` value is in the `Manager` column of another employee. So, for the above table we could use a query something like the following:

```
SELECT Employees.EmployeeName
FROM Employees
WHERE (Manager IS NULL) OR (EmployeeID=1) OR (EmployeeID=2);
```

This query will work for the above data, but we have hard coded the contents of the manager column values into the `WHERE` clause. This means that whenever we change the number of managers we will need to go back and change our query – not an ideal solution. What we really need is to be able to have some sort of clause that allows us to query whether a value is in a list of other values. The `IN` operator allows us to do just this:

```
SELECT Employees.EmployeeName
FROM Employees
WHERE (((Manager) IS Null) OR (EmployeeID IN (1,2)));
```

We'll explain exactly what the `IN` operator does in the *IN* section, but for now it's enough to know that it checks if `EmployeeID` is in the group specified. This syntax removes the need for so many `OR` operators, but we still have the hard-coded values 1 and 2. In order to solve the problem at hand we need to combine sub-queries with the `IN` operator so that the list of values from the `Manager` column is generated dynamically, like this:

> *This syntax removes the need for so many OR operators*

```
SELECT Employees.EmployeeName
FROM Employees
WHERE (Manager IS Null) OR (EmployeeID IN (SELECT DISTINCT Manager FROM
Employees));
```

A self join is where we join a table to itself in a similar fashion to that which we saw with recursive joins in Chapter 4.

We have a solution. We could probably have solved this problem in a different way by using some scary self joins, but the advantage of our sub-query solution is that it is very readable by anyone who comes along later to look at our code, and it isn't hard coded so we won't have to change it after the next management reshuffle.

`IN` is just one of the operators that we can use in sub-queries. In total there are four. They are:

- `IN`
- `ALL`
- `ANY`
- `EXISTS`

We'll look at each of these shortly, but first we need to look at a way of sharing information between the inner and outer sub-queries. This is a concept called correlation.

Correlated Sub-Queries

Correlated sub-queries allow us to write a sub-query that relies on the outer query for one or more of its values. What this means is that for every row in the outer query result set, the inner query may return a different value. With the sub-queries we have looked at so far the inner query will always return the same result – even if we run it outside the context of a sub-query. Let's take a look at an example of a correlated sub-query.

We will extend the example that we used above, and this time instead of getting products whose price is greater than average, we will get products whose price is greater than the average of the category that they are in:

```
SELECT ProductsOuter.ProductShortName, ProductsOuter.ProductPrice
FROM Products ProductsOuter
WHERE (((ProductsOuter.ProductPrice)>(SELECT Avg(ProductPrice) FROM Products
WHERE Products.CategoryID = ProductsOuter.CategoryID;)));
```

This is a very similar query to our first example, except that the inner query now references the outer query. To do this we need to give the tables unique names for the context of the whole query. You can see that we have to alias the **Products** table in the outer query to be **ProductsOuter**. Once we have done this we can now reference the values of the outer query when we perform our inner query.

Because we are dealing with the same table, once in our outer query and once in our inner query, we need to give one of them a different name temporarily. This is called an alias. In this case I have chose to give the products table an alias in the outer query so I am in fact selecting '`FROM Products ProductsOuter`' that is **Products** *referred to as* **ProductsOuter**.

The way this correlated sub-query will be parsed by the database is that it will select all of the rows from the outer query. Then, for each row in the outer query it will calculate the result of the inner query to do the comparison. Remember that this is **what** the database will do, not **how** it will do it – exactly how the database executes the query will be determined by the query processor. Let's look at the data for the **Products** table and work out what the database will be doing:

Products : Table

	ProductID	ProductShortName	ProductDescription	ProductPrice	CategoryID
+	1	Super Sprint Shoes	The fastest running shoe on earth.	$89.95	1
+	2	Alpine Boots	Super waterproof. Accept crampons.	$160.00	2
+	3	Summer Tracksuit	Cool and loose fitting.	$75.00	6
+	4	Winter Tracksuit	Warm, for those cold days jogging.	$90.00	6
+	5	Tracksuit Combo Deal	Buy Summer and Winter and save $$$	$150.00	6
*					

The database will execute the sub-query for each row in the **Products** table. So for the first row the inner query will be:

```
SELECT Avg(ProductPrice) FROM Products WHERE Products.CategoryID = 1;
```

This query returns the value $89.95 as there is only one product in that category. The outer query will then do the comparison to see if the product that it is currently processing has a price greater than the average for its category. In the case of the first row it doesn't: 89.50 is not greater than 89.50 and so the first row is not returned. The second, third, and fourth rows are in a similar position.

The only row that is returned is the fifth row. Let's look at how the database would return this row. First it would perform the inner query:

```
SELECT Avg(ProductPrice) FROM Products WHERE Products.CategoryID = 6;
```

The result of this would be the average of the three products in category 6: $105. It would then compare the price of product five, $150, with the average, and determine that the price was greater than the average and so should be returned. The results of our query should be as follows:

	ProductShortName	ProductPrice
▶	Tracksuit Combo Deal	$150.00
*		

IN

`IN` returns a Boolean value (True or False) that indicates whether a value is contained in a sub-query or list of values. The example we saw above used `IN` with both a list `(1,2)` and a sub-query `SELECT DISTINCT Manager FROM Employees.`

Using `IN` with lists is particularly useful for web development when you want to use checkboxes to filter a query. Say we wanted to use the following HTML form to allow a user to choose which products to display based on categories:

> *Using IN with lists is particularly useful for web development*

We should give all of the `<input type="checkbox">` elements the same name, and give them a value which corresponds to their category. Then when the user posts back the form, all we will need to do is gather up the checkbox values that they have selected as a comma-separated list, and insert that list into our `IN` statements. In most web scripting languages this is very easy, as the value of the above checkboxes will be returned to our scripting code as a comma-separated list when the user submits the form.

EXISTS

EXISTS returns True if a sub-query contains any rows, and False if the sub-query contains no rows. EXISTS is always used for correlated sub-queries, and in all cases the equivalent query could also be written using IN, or ANY, or ALL as well. The advantage of EXISTS is that it is much better suited to making efficient use of any indexes on the tables in the inner query. For example we could rewrite our query to find all the managers using EXISTS:

```
SELECT EmployeesOuter.EmployeeName
FROM Employees EmployeesOuter
WHERE (Manager IS NULL) OR EXISTS
       (SELECT * FROM Employees
        WHERE Employees.Manager = EmployeesOuter.EmployeeID);
```

While the query processors in different databases may handle this query differently, in theory the EXISTS version should be more efficient because the database can perform an existence query simply by using the indexes. That is, it can check if any rows will be returned from the inner query without actually having to read any of the inner query rows off the disk.

ANY, ALL

These operators are used in combination with the more traditional scalar operators (<, >, =, etc.) to allow set-based comparisons using those operators. The ANY operator will return True if any of the items in the inner query or list would return True if compared with the parent query individually using the scalar operator. The ALL operator returns True if all of the items in the inner query or list would return True if compared with the parent query individually using the scalar operator.

Take a look at the following examples of WHERE clauses, focusing on the ANY or ALL operator:

```
3 > ANY (1,2,3);
```
Evaluates to True. 3 is greater than two of the items in the list.

```
0 > ANY (1,2,3);
```

Evaluates to False. 0 is not greater than any of the items in the list.

```
2 > ALL (1,2,3);
```

Evaluates to False. 2 is only greater than one of the items in the list.

```
4 > ALL (1,2,3);
```

Evaluates to True. 4 is greater than all of the items in the list.

Let's take a look at an example using the data sets that we have been using earlier in this chapter. The following query selects all of the products that are more expensive than all of the products that are in category 1.

```
SELECT Products.ProductShortName, Products.ProductPrice
FROM Products
WHERE ProductPrice > ALL (SELECT ProductPrice FROM Products WHERE CategoryID =1);
```

Views

A view is basically a virtual table that is made up of another SELECT statement. Once we have created it, we can use a view in the same way that we would use any other table in our database. Let's look at an example that takes advantage of some of the concepts we saw in the last section.

> *we can use a view in the same way that we would use any other table in our database*

```
CREATE VIEW CategoriesAndSummary
AS
SELECT Categories.CategoryID,
       Categories.CategoryName,
       Avg(Products.ProductPrice) AS AveragePrice
FROM Categories INNER JOIN Products ON Categories.CategoryID =
                                       Products.CategoryID
GROUP BY Categories.CategoryID, Categories.CategoryName
```

Here we have created a view that lists all of the categories, as well as summary information about that category, specifically the average price of products in that category. Once we have created the view we can select from it in the same way that we would select from a normal table:

```
SELECT * FROM CategoriesAndSummary;
```

And the result we get is as if we had run the full SELECT statement stored in the view:

	CategoryID	CategoryName	AveragePrice
▶	1	Running Shoes	$89.95
	2	Hiking Boots	$160.00
	6	Tracksuits	$105.00

Views offer us a number of advantages. The first is that when a SELECT statement is stored in a view, the database is able to compile a query plan for that view the first time a SELECT is performed against it. When subsequent SELECT statements are run, the view will reuse this precompiled query plan and thus execute a lot faster than it would otherwise run if we had re-sent the whole SQL statement.

As we saw in the last chapter, a view is also a good mechanism for limiting access to certain items of data. For example you could restrict access to the **Employees** table to managers only and then create a view called say, **BasicEmployees** which would be a SELECT statement containing just some of the data in the **Employees** table and omitting confidential information like salary and home phone number.

The above example is a common use of views whereby we have used the view to merge the normal content of a table with some aggregate information about that table.

Once created, views behave just like tables, meaning that you can create a view that is a SELECT statement against another view. Often it is not possible to obtain the results you want from a single SELECT statement. By using views it is sometimes possible to chain together several statements to get the desired result.

Stored Procedures and Triggers

Most databases support the creation of procedures and/or functions within the database itself. Database stored procedures are similar to procedures in any other programming language. Unlike the rest of SQL, however, there is no broadly supported syntax for stored procedures; each database platform has its own syntax.

> *there is no broadly supported syntax for stored procedures; each database has its own*

The specific details of how to implement stored procedures and triggers on every database are beyond the scope of this book.

This section will focus on a very broad introduction to the concept of stored procedures and triggers. I will also give some examples of where I have used each of these technologies in recent projects – the idea being that you will be able to more easily spot situations in which triggers or stored procedures may give you an advantage.

Let's take a look at a couple of simple stored procedures to get an idea of some of their properties – the syntax in this case is Transact SQL (Microsoft SQL Server). For example the following procedure takes two parameters and inserts a row into a table:

```
CREATE PROCEDURE AddOrganization
(
  @Name varchar (50),
  @Email varchar (50)
)
AS
if exists (SELECT * FROM organization WHERE organization.Name = @Name)
  RETURN
INSERT INTO Organization
(
  Name,
  Email
)
VALUES
(
  @Name,
  @Email
)
```

Let's look at some of the parts of the query in more detail. It takes two parameters, `@Name` and `@Email`. Note that the parameters are strongly typed, both as `varchar(50)`. The actual procedural logic begins after the `AS` keyword. The logic is fairly simple; it checks if an organization of the specified name (`@Name`) already exists. Note the use of the `EXISTS` keyword that we saw in the last section. If that organization already exists the query simply exits using the `RETURN` keyword. If there is not a pre-existing organization then we perform an `INSERT` query using our parameters in the `VALUES` list.

Stored procedures are able to return values as well. Most often this is done by using `OUTPUT` parameters. These are similar to passing parameters by reference in a programming language; the calling code declares a variable, passes it into the procedure as a parameter, the procedure modifies the parameter, and when the procedure returns that modified value it is then accessible by the calling code. For example the following procedure populates an `OUTPUT` parameter (`@AvgPrice`) with the value of the average product price for a given category (`@CategoryID`):

```
CREATE PROCEDURE GetCategoryAvgPrice (@CategoryID int,@AvgPrice float OUTPUT) AS
SELECT @AvgPrice = Avg(Products.ProductPrice)
FROM Products
WHERE (((Products.CategoryID)=@CategoryID))
GROUP BY Products.CategoryID;
```

Depending on your database platform it may also be possible to return record sets from stored procedures. Some databases also support stored functions, which can return a value directly in the same way that functions in other programming languages return values.

Stored Procedures

The first time the RDBMS runs a SQL statement, it has to parse the text to find out what it's being asked to do, then it has to work out how it's going to do what it's being asked to do, and finally it can execute the statement and return the results. For stored procedures, the RDBMS will parse the statement and generate its plan once, and use that each subsequent time it is called. Not all RDBMSs will keep the stored procedure in memory.

Stored procedures (commonly referred to as sprocs or stored procs) can be executed much more quickly than SQL strings sent straight to the database. This is because the database will compile the stored procedure and store it in its memory. This compilation process includes the query processor building most of the query plan for a given set of operations. The next time the procedure is called the database will use the compiled version, whereas when SQL strings are sent they are generally recompiled every time. The compiled stored procedure code is executed much more quickly than the code that must be recompiled each time. Another major difference is that a series of SQL queries can (usually) be run within a stored procedure, with all of the processing occurring on the server. This is obviously good for performance, and especially security.

Stored procedures allow developers to put logic that requires a large amount of data processing for a small result to be placed as close to the database as possible. By doing this we avoid passing the data between the database layer and the application layer – something that takes a considerable amount of time. We could bring all of the data down from the database into our web server and then manipulate it as a `Recordset` or array but this will often be quite inefficient. Often it is better to move that piece of programmatic logic out of our web development language and into the language of our database server.

What we are effectively doing is placing some of our business rules in the database. For example the first procedure that we looked at above enforced the business rule that only one organization of the same name could exist. Of course this is a very simple example; we can create very complex rules inside stored procedures to meet the specific requirements of our business problems. Another advantage of keeping business rules in the database itself is that it makes it more difficult for users of the database to accidentally forget to implement a certain rule in one of their applications. You could even go so far as to deny write access or even all access to the underlying tables in the database and permit access only through stored procedures. This sort of approach may take more time initially, but gives you, the DBA, a greater degree of control over how the database is used, especially where you are dealing with a large team of developers.

Some examples of where I have used stored procedures in recent projects are:

- **Inserting orders into an e-commerce system**
  Instead of executing an `INSERT` statement for every `OrderItem` for each order, I called a stored procedure passing the whole order in XML as a parameter. The database engine then disassembled the XML and inserted the line items. In this way I was able to insert the order with just a single call to the database. This is not only much faster but it is also much more reliable – we don't run the risk of our ASP/PHP/JSP code failing halfway through and the order being inserted without its order lines. We will take a look at this need for reliability a bit later in this chapter when we look at transactions. I strongly recommend using stored procedures when you need to insert data into more than one table at once such as in the order and order lines situation.

- **Checking for an alarm state in time series data**
  We recently built a web application that displays time series river flow data to users. Part of this process is to determine if a particular river has reached alarm threshold levels. This is a fairly complex calculation, which involves checking for a level being over a certain value for a certain period of time within another time period. It is not something that can easily be calculated in simple SQL but doing it programmatically on the web server would have brought the whole system to a crawl: for each site, the time series was about 30,000 rows. Instead we used a stored procedure to calculate whether a site had gone into alarm. We could simply pass the `SiteID` to the procedure and it would return a True/False. By using a stored procedure here we saved a whole lot of load on our web server.

- **Performing actions on remote databases**
  We had a project recently that required us to communicate with a number of database servers remotely over the Internet. In this situation the amount of data we sent back and forth would have had a major impact on the speed of the application. Instead of executing SQL against the remote databases and returning result sets, we used stored procedures to pass only the information that we actually needed – we probably cut the amount of network traffic by 80% using this approach.

Triggers

Triggers are a special type of stored procedure that is called when some specified action is performed on a table. Basically a trigger automates the execution of some code when a `SELECT`, `UPDATE`, `INSERT`, or `DELETE` statement is executed against a table. For example you could write a trigger that checked the stock levels of a product every time an **OrderItem** row was inserted for that product. The trigger could write a low stock alert to a log file or even e-mail a system administrator to let them know that there could be a problem.

In some RDBMSs, such as Oracle and SQL Server, it is also possible to define what is known as an `INSTEAD OF` trigger. In this case the trigger is executed whenever one of the aforementioned actions occurs, but rather than being executed **after** that action the trigger is executed **in place** of that action. This is useful in situations where you may want to leave the syntax of the three statements untouched, but handle them quite differently from the way the database might usually treat them. For example you may have a business rule that says products must not be deleted from the database: rather they should have their `Deleted` column set to True.

You could write an `INSTEAD OF DELETE` trigger that would update the `Deleted` column instead of actually deleting the data. In this way application developers could still use the syntax that they are comfortable with and you, as the database developer, can ensure that the business rules are properly enforced. This approach would also allow us to modify the business rules behind our application without modifying the application itself.

One of the keys to the power of triggers is that while they are executing we have access to virtual tables, which give us access to the inserted and deleted records for the statement that has fired the trigger. We can then use these virtual tables just like we would any other table while we are inside the procedural code of our trigger. Let's take a look at an example of the DDL for a trigger that makes use of these tables. This example is based around the first real-world situation I describe in the bulleted list below, dynamic table creation:

Note that again this is Transact SQL from SQL Server, and that I have commented the code, the lines beginning with '--'.

```
CREATE TRIGGER CreateObjects
ON SiteMeasurements
FOR INSERT
AS
DECLARE @ItemNumber as Varchar(20)

--Here we are getting the inserted site from the
virtual 'inserted' table.
SELECT @ItemNumber = inserted.SiteMeasurementID FROM inserted

--This is the function that actually creates a new table
EXEC('CREATE TABLE dbo.DataTable' + @ItemNumber + '(
  [DateStamp] [datetime] NOT NULL ,
  [DataValue] [float] NOT NULL ,
  CONSTRAINT [DataTablePrototype'  + @ItemNumber + '_PK] PRIMARY KEY  CLUSTERED
  (
    [DateStamp]
  )
) ')
```

Note that this is a very simplified version of the ideal solution. Most notably it only deals with the insertion of a single row. But we could easily modify it to work for multiple inserted rows by selecting all of the rows from the 'inserted' table and then iterating through those rows creating a corresponding new table for each one. But this example does show the usefulness of the inserted table. Note also that the syntax is fairly similar to what we saw above under stored procedures. In most RDBMSs the syntax of the body of triggers and stored procedures is the same.

One thing to be aware of with triggers is that you may get into a situation whereby you start cascading triggers across multiple tables, which can in turn create a circular reference. The trigger on table **A** inserts into table **B**, and the trigger on table **B** trigger inserts into table **A**. Most RDBMSs prevent cascading triggers. If you do need the support for cascading-type trigger actions you may need to use stored procedures to implement them.

Some examples where I have used triggers in recent projects are as follows:

- **Dynamic table creation**
  We did a project recently in which we had to generate graphs from large volumes of time series information for real-time display on the Web. The time series related to water flows at various sites and as such we had 30,000 rows for each site. For performance reasons (which I will discuss under indexing below)

I decided to store the data in a denormalized fashion, one table for each site. Obviously I didn't want to have to go back into the database every time the client needed to set up a new site so I wrote a trigger that automatically created a table to store the time series data whenever a row was inserted into the **Sites** table. I also wrote a trigger that automatically dropped the appropriate table when a row was deleted from the **Sites** table.

- **Content aggregation**
  In another situation we wanted to build a mall that displayed products from a number of e-commerce stores that we hosted. We used triggers on each of the local e-commerce store **Products** tables to insert, update, and delete information from the **Products** table of the mall database. When the **Products** table is updated in a particular store's database we perform the same action on the global mall database.

- **Maintenance of audit data**
  One of our web applications is a document-management portal targeted at legal and accounting firms. The ability to trace the lineage of a document is very important to these organizations. We use an `ON DELETE` and `ON UPDATE` trigger to shift any deleted or modified file information into an audit table along with a date stamp to allow for a full reconstruction of the document history.

- **Inserting into denormalized columns**
  In our e-commerce product we store summary data about each product, such as the number of times it has been purchased, in a denormalized form; that is we store the purchase count as a column in the **Products** table rather than calculating it on the fly when we query that column. The reason that we do this is to improve the speed of queries of this summary data. We use triggers to insert this summary data. When an **OrderItems** row is inserted we use an `ON INSERT` trigger on the **OrderItems** table to update the corresponding product rows summary data.

Physical Database Tuning

Most of what we have discussed in earlier chapters has been generic enough to apply for any of the major database platforms. The reason for this is that we have been looking at databases by way of an abstract model. We tell the database about the model that we want created and it will deal with all of the detailed things like deciding where to store rows on the disk and so forth.

This abstraction is brilliant for us database designers. It means that we can think about the problem without having to get bogged down in the specifics of the solution. There are, however, downsides to this logical/physical abstraction. When we give the DBMS our logical model it will make a number of assumptions on our behalf as to how the data is to be stored. These presumptions are not always valid and as a result the physical model that the database uses may not be the optimal solution.

In most situations the performance improvements that we might be able to achieve by 'fiddling' with the internals of our databases are outweighed by the time we would have to spend 'fiddling'. In some situations, though, particularly for very large databases or databases that serve a large number of users, we can see some marked improvements by optimizing the physical layer of the database ourselves.

> *In some situations we can see improvements by optimizing the physical layer of the database ourselves*

In this section we will discuss ways of helping the database make the right decisions when it comes to the physical storage of our data. We will start by looking at the way databases break down our queries in order to fetch the requested data. Then we will look at ways of indexing our data to best optimize different types of queries. Finally we'll delve into denormalization, which involves purposely bending and breaking some of the database rules we saw in earlier chapters in order to improve the efficiency of our databases.

Query Plans, or, What's That RDBMS Thinking?

You will recall from the last few chapters that when it comes to database querying there are often several ways to 'skin a cat'. Once the query reaches the database, there are generally even more options available for the RDBMS to choose from. When we execute queries against our database the RDBMS will break our query down into a language that it can understand. It will then examine the various ways that it could execute the query and pick the approach that it thinks would have the lowest cost in terms of system resource usage.

As I noted above, there are times when we might want to pre-empt the RDBMSs guesses with some ideas of our own. Most RDBMSs allow us to do this by making use of a couple of special commands:

- Explanation commands
- Hint commands

The purpose of these two commands is pretty simple. The first is used to ask the RDBMS to explain how it would approach a given query. The second is used to give the RDMBS some hints as to how we would like it to execute the query.

Make sure that you don't start trying to optimize queries until there is a realistic amount of data in the database. Most query processors are pretty clever and if there are only a few rows in a table they will bypass all of the indexes and go straight to disk to save time. Optimization really shouldn't be performed until there are operational volumes of data in the database, or even after the database is operational.

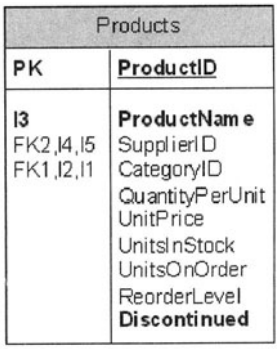
Explaining Queries

Before we can begin to give the RDBMS any direction on how it should deal with the physical retrieval of data, we really need to get a handle on how it is currently approaching a given problem. Most RDBMSs have some sort of command or utility that can be used to explain the approach that it is taking to a query.

The explanation will usually include reference to the indexes it is using to look up rows, as well as the order in which it is performing those lookups. While this book can't cover the ins and outs of each database platform's `EXPLAIN` functionality, the following table shows the keywords that you should look up in your database online Help in order to find out about its explain features:

Database	Explain Commands/Features	Comments
MS Access	Performance Analyzer	Go to the menu: *Tools > Analyze > Performance*
MySQL	`EXPLAIN`	
Oracle	`EXPLAIN PLAN`	
SQL Server	`SET SHOWPLAN` `SET STATISITCS`	SQL Server also ships with a visual tool called *Query Analyzer*, which displays execution plans in a graphical layout.
Postgres	`EXPLAIN`	

Let's take a look at the explanation of a query in the SQL Server query analyzer to get an idea of the sort of information that these explain commands return. I am going to use the Northwind sample database that ships with SQL Server and MS Access. The reason that I am not using the sample database that we have been looking at so far is that it doesn't really have enough data in it to give valid results from an `EXPLAIN` command. We'll look at a query against the Northwind **Products** table. The following diagram illustrates the basics of the table; the data types aren't so important:



The query we will look at is just a modification of one of the queries we looked at when discussing sub-queries previously.

```
SELECT Products. *
FROM Products
WHERE UnitPrice > (SELECT Avg(UnitPrice) FROM Products);
```

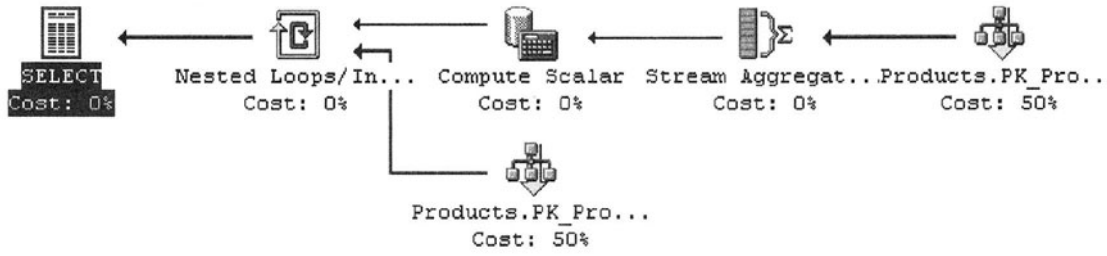
First let's look at the sort of output we get from the non-graphical explanation commands in SQL Server. We can enable `SHOWPLAN` executing the following command in SQL Server *Query Analyzer*:

```
SET SHOWPLAN_ALL ON
```

Once we have done this, any queries that we run will return a query plan instead of a result set. Let's look at the plan generated for the `SELECT` statement above:

```
SELECT Products.* FROM Products WHERE UnitPrice > (SELECT Avg(UnitPrice) FROM
Products);
  |--Nested Loops(Inner Join, WHERE:([Products].[UnitPrice]>[Expr1002]))
       |--Compute Scalar(DEFINE:([Expr1002]=If ([Expr1012]=0) then null else
([Expr1013]/Convert([Expr1012]))))
       |    |--Stream
Aggregate(DEFINE:([Expr1012]=COUNT_BIG([Products].[UnitPrice]),
[Expr1013]=SUM([Products].[UnitPrice])))
       |         |--Clustered Index
Scan(OBJECT:([Northwind].[dbo].[Products].[PK_Products]))
       |--Clustered Index
Scan(OBJECT:([Northwind].[dbo].[Products].[PK_Products]))
```

SQL Server actually returns a lot of other information but there is far too much of it to include here. Fortunately SQL Server also supports a graphical view of the query plan. Most major DBMSs offer graphical tools to view query plans. Take a look at the following screenshot from *Query Analyzer* showing the execution plan:



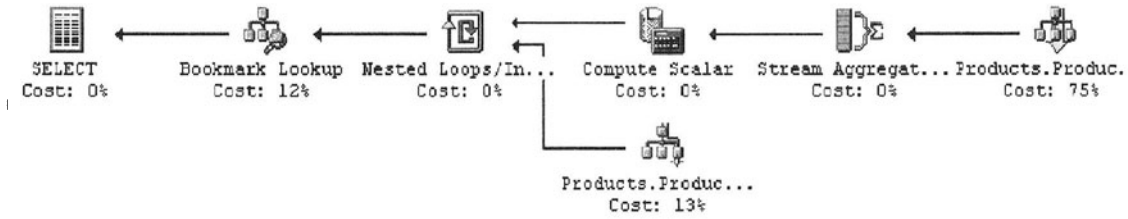
Each of the little icons represents a different physical database operation. The numbers expressed as percentages are the cost of that particular operation relative to the whole query. If we mouse over each of the icons we can view more details for that particular operation. In this case we have hovered the mouse over the far right-hand side icon, which is dealing with scanning through the table in the sub-query to build the average:

Clustered Index Scan
Scanning a clustered index, entirely or only a range.

Physical operation:	Clustered Index Scan
Logical operation:	Clustered Index Scan
Row count:	77
Estimated row size:	117
I/O cost:	0.0375
CPU cost:	0.000163
Number of executes:	1
Cost:	0.037742(50%)
Subtree cost:	0.0377
Estimated row count:	77

Argument:
OBJECT:([Northwind].[dbo].[Products].[PK_Products])

The goal with physical optimization is to try and get as much of the cost occurring in operations that cannot be optimized any further. In this case most of the cost is being borne by two 'clustered index scans'. Ideally we want to avoid 'scans' of tables or indexes if at all possible. When it performs a scan the RDBMS will iterate through all of the rows in the table or items in the index, which is not particularly efficient. The reason that the RDBMS has to scan in this case is that the query is doing a lookup based on `UnitPrice` in the `WHERE` clause, but there is no index specified on `UnitPrice`. If we add a new index to the `UnitPrice` column we can see the cost shift away from that part of the query:



You can see that the bottom branch of the tree now only accounts for 13% of the cost and that its icon has changed to a 'clustered index seek. The unavoidable index scan which is required for generating the average product price now accounts for a much larger proportion of the cost. When analyzing queries using this cost-based assessment it is important to note that the cost will always add up to 100%, all that we are doing by adding indexes or giving hints is shifting the proportion of time that is spent on each task. As we do this, the total time and resource that the query consumes will be lowered.

Hints

As well as identifying situations where an index might be useful, explain statements can sometimes identify situations where indexes are not being used even though they exist. In situations like this some RDBMSs allow us to insert 'hints' into our queries in order to force the query processor to follow a different query plan to the one it might otherwise determine as the optimum. The following table lists the hint capability of some of the more popular database platforms.

Database	Hint Commands/Features	Comments
MS Access	None	
MySQL	None	
Oracle	`/*+ hint */` syntax	Search for *Query Hints* in the documentation.
SQL Server	Query hints.	Allows hints to be given in a very wide range of situations.
Postgres	None	Possible to coerce query planner with certain join syntaxes.

Hints are a complex beast and, as developers and database designers we need to be very careful about how we use them. You should always make sure that you profile any situation where you might need to use hints thoroughly. It is also a good idea to go back and re-profile hinted queries at later stages in the life cycle of the database to see if things have changed as different volumes of data become available in the various tables. The following SQL shows the syntax for hinting SQL Server. In this case we are telling the query processor to optimize the query plan for returning five rows by using the `FAST` query hint:

```
SELECT Products. *
FROM Products
WHERE UnitPrice > (SELECT Avg(UnitPrice) FROM Products)OPTION (FAST 5);
```

Indexes

We looked at the concept of indexes briefly in *Chapter 2*. In this section we'll look at indexes from a more practical viewpoint. Indexes basically allow us to locate our data faster. Think of them like you would a library card index; I hope you're not too young to remember those because it's analogy time!

Most libraries used to have an index for Subject, and index for Author, and an index for Title. The actual books were usually arranged on the shelves using a fairly arbitrary numeric cataloging system. The card indexes would then reference this numbering system to allow us to find the book easily on the shelf. The process is similar in an RDBMS. The database stores the data on the disk using its own storage rules or numbering system. When the time comes to find a row, say when we do a `SELECT` statement, the database looks up the row we are searching for in an index and that index points to where the database has physically stored the row. In the same way that libraries can have indexes to allow us to search for books in multiple ways, so can databases have indexes on multiple columns to allow us to find the rows we want in different ways. If we need to find a book in a library by some criterion that is not indexed, say the color of the book's cover, the only way we can do this is to go and scan the shelves looking for the color we are after. In a database, if we do not have an index on a column and we need to select a row based on that column, then the RDBMS has to scan through all the rows stored on disk for that table in order to make that selection. In both the library and the database a 'scan' is a pretty onerous task.

The easiest solution might be to simply say, "let's just index everything," (that is, all of the columns on all of our tables). This would obviously make it much quicker to access data, but it is not without its pitfalls. Think back to the library; if we indexed every possible characteristic of a book then every time a new book came into the library and had to be shelved, the poor librarian would have to write out and file several dozen index cards. The same problem applies to our database; every time we insert a row into a table the database must go and update all of the relevant indexes to point to that new row. This is very time-consuming and resource-intensive. In addition to this, database indexes, just like the card file drawers at the library, are of limited size. Every so often the database server will need to completely rebuild the index, a bit like the librarian taking all of the index cards out of their drawers and reorganizing them into new drawers with more space.

The purpose of that rather long story is to try and explain to you how indexing is a trade-off. We trade off speed of retrieval against speed of insertion. Getting a good grasp of this trade-off is crucial to us as web developers, because more than almost any other database user we need to be able to optimize highly for one or the other. The crucial determinate is always the ratio of `INSERT` traffic to `SELECT` traffic.

So what are some good rules for indexes on web databases?

1. Primary key columns and foreign key columns should always be indexed. These columns will be used extensively when retrieving data from tables and joining tables. Many databases will index these columns by default for you.

2. Any column which is used in the `WHERE` clause of `SELECT` or `UPDATE` statements in the publicly accessible area of your web site should be indexed. Most web sites will have data that is frequently read by end users but only ever edited or updated by administrators. These sorts of tables should be heavily indexed because there will seldom be enough server load from administrators to make the performance costs of index creation notable. The exception to this rule is the next point:

3. Tables that are primarily written to more than they are read in the public area of the site, should have as few indexes on them as possible. For example, say we have a table that logs visitors to your site including such things as IP addresses and so forth. This sort of table commonly produces huge volumes of data.

My preferred approach is to simply dump the log data into a table completely denormalized and un-indexed and then as a batch job reconcile this into a more suitable separate data structure for analysis only. In this way my logging code is faster, which is very important as it is called on each page, but I still have the ability to access the data in a useful and efficiently indexed form.

Types of Index

Several of the more advanced database platforms offer a variety of different index types that are particularly well suited to certain operations. Oracle for example supports an extensive range of index types, some of which are specifically designed for aggregate queries and so forth. It is beyond the scope of this title to delve into these advanced index types, but the documentation for these platforms usually provides sufficient detail to enable you to decide when and where they are well suited. There are two main types of index, non-clustered and clustered, or physical.

> *Be careful not to include too many indexes on a table*

Non-Clustered Indexes

A **non-clustered** index has a structure that is completely separate from the table. This type of index contains a key that you define, based on columns in your table, and each key in the index has a pointer to the row of data in the table. You can define as many non-clustered indexes on a table as necessary or as many as your RDBMS permits. Be careful not to include too many indexes on a table, though, as each one takes up a little storage space, and each one adds to the time taken to perform `INSERT` operations.

When you define an index on a table, you typically specify whether or not the index key is to be unique. A unique index key specifies that no other index key can have the same value. Non-unique index keys allow two or more index rows to have the same key value. This could be useful when you want to index a table on a phone number and you have two or more people with the same phone number: a family, or flatmates perhaps.

Clustered or Physical Indexes

A **clustered** or **physical** index is part of the table itself and sorts and stores the rows of data in a table based on the key of the index. Therefore, since the index key sorts the data in the table, there can only be one clustered index on a table. Because of the way clustered indexes work, they can impose a fairly heavy performance hit when adding data to the table, as well as taking a long time to add. This results from the fact that every time you add data, the index will have to reorganize the table so that everything is in the right order – the larger the table, the longer this will take. Clustered indexes are best suited for tables where change is unlikely, or limited.

The term physical index can be taken to mean other things, but in this case we are using it to describe a situation where the database designer specifies the order in which rows are stored on disk. Usually this is determined by the RDBMS, and, like most of the cases we have seen so far, the RDBMS generally knows best. In some situations, however, it can be very useful to tell the database server how you want the data ordered on disk.

Generally the queries that you will be optimizing by this approach are what are called range select queries, or range retrievals. If you think back to one of my examples earlier in the chapter I discussed storing time series data in the database. The data consists of samples taken at 15-minute intervals and we store six months' worth of data. Say I want a month's data from three months ago, I would execute a query like this:

```
SELECT DateStamp, DataValue
FROM DataTable131
WHERE DateStamp BETWEEN GetDate()-90 AND GetDate() - 60
```

If we're using a normal index on the `DateStamp` column the database would seek through the index to find the start of my range, then scan through the index until it found the end. Because a normal index simply points to rows, and those rows could be anywhere on the disk, every time the database server fetches a row it may be going to a completely different part of the disk. What I really need is either for the rows to be stored in the same order as they are in the index, that is all together, or for the rows and the index to be stored as one item, that is the data stored with the index. If that is the case the database can simply seek the start of the range in the index and then read the rows of the disk sequentially until it reaches the end of my range. The performance difference gained by physical indexing is quite astronomical, especially on large tables.

It is important to note though that you can only have one physical index on a table. This should be fairly obvious – you can only want to store the rows on disk once and as such you can only store them in index order once.

Managing Concurrency

Concurrency is all about the number of users you can have executing SQL against a database at any one time. In web-based situations concurrency management is extremely important because we may ultimately want to have hundreds, if not thousands, of users working against our database at any one point in time.

Most RDBMSs manage concurrency by using what is known as locking. Some databases also allow the grouping of statements into a single execution unit that can be executed in isolation from other execution units. These are known as transactions.

Locking

Locking involves placing a flag on a certain object in a database to indicate that it is in use. These locks restrict what other users can do with that object while the lock is in place. So, for example a user might want to lock a table from being used by others while they perform updates to the data in that table.

> *Locks vary in terms of the object that they affect*

Locks vary in terms of the object that they affect, and the influence that they have on other users of that object. For example some locks prevent all access to an object whereas others simply prevent editing or writing to that object. These properties of locks are referred to as granularity and type.

While most databases allow locking of the physical objects stored in the database, the common levels of lock granularity at the logical level are, from smallest to largest:

- Field
- Row
- Table
- Database

The support for these lock granularities varies between RDBMS platforms. Many only support database and table locking. The different type of locks supported by the various RDBMSs also varies widely. Generally each vendor uses a different name for their lock types, but they can be broadly grouped into two categories:

- Shared
- Exclusive

A shared lock is used when an application wants to read data from the database. A shared lock can be obtained on an object provided that the object does not have any exclusive locks taken out on it. An exclusive lock is used when an application wants to write data to an object. Each object can only ever have one exclusive lock at any one point in time and the DBMS must wait until all shared locks are released before obtaining an exclusive lock.

While most RDBMSs will use locks any time we execute commands against them, locks don't really become an issue until we start looking at transactions. Generally we as developers and administrators do not need to set locks explicitly ourselves. Instead the RDBMS will set the locks for us based on the isolation level of the transaction we are executing.

Transactions

> *Transactions tell the database that all of the commands we execute within that transaction must succeed or fail as a group*

When we build our applications there are often times when a logical unit of work involves several inserts, updates, or deletes in the database. For example if we look at what happens when we place an order in our e-commerce store. The very least that will happen is that we will insert a row in the **Orders** table, and at least one **OrderItems** row. In terms of our understanding of the application this is a single logical unit of work. But to our database we have really executed at least two units of work – one to insert the order, and one or more to insert the item(s). In most situations this will not be a problem, but consider the situation where our web server crashed after we had inserted the order but before we had inserted the items. In our application we would be left with inconsistent data, that is, an order that is meaningless. What we really need is for the logical unit of work in our database to be the same as the logical unit of work in our application.

Fortunately databases allow us to group commands that we execute against them into logical units. These units are called transactions. By grouping all of the `INSERT` commands associated with inserting our order we can secure ourselves against ending up with invalid or inconsistent data in our database. Transactions tell the database that all of the commands we execute within that transaction must succeed or fail as a group. Thus, if our web server crashes, or we experience an error part way through inserting the details of an order, we will not be left with inconsistent data.

As developers, we are able to tell the database server when a transaction begins and ends. When we tell the database server to end the transaction we will be telling it to either save our changes and **COMPLETE** the transaction, or undo all of our changes and **ROLLBACK** the transaction. Most databases support the SQL commands **BEGIN TRANSACTION** (to start a transaction), **COMMIT TRANSACTION** (to finish the transaction and save the results), and **ROLLBACK TRANSACTION** (to finish the transaction and undo any changes it has made). Some middleware also supports transactions explicitly, for example the ADO `Connection` object has a `BeginTrans` method.

Putting the ACID on Transactions

Database transactions have a set of properties that is often abbreviated by the term ACID. These properties basically define what a transaction is, or needs to be. The ACID acronym stands for:

- Atomic
- Consistent
- Isolated
- Durable

In order to be really robust any RDBMS that supports transactions should implement them in such a way that they have these four properties. Let's look at each of these properties in turn.

Atomic

For a transaction to be atomic it must either all succeed or all fail. Atomicity allows transactions to be used to represent parts of business processes that must either all succeed or all fail. For example, when placing an order into our e-commerce database we need to enter both an order in the **Orders** table, and one or more order items in the **OrderItems** table. Placing the order is a single atomic unit of work.

Consistent

Transactions must always leave the database in a consistent state – both internally and externally. That is, the data in the database must be valid both against the integrity rules within our database and the business rules for our application that operate from outside our database. While it may be the case that the database is in an inconsistent state during the transaction, for example having an order row with no matching order items, once the transaction has completed the database must always be in a valid state.

Isolated

While a transaction is running it should appear as though it is running all by itself. That is, the running transaction should not be able to see the results of any other uncommitted transactions and neither should any other transaction be able to see the results of a running transaction until it has been committed. This property is easy enough to achieve, but from a practical point of view it seriously limits the number of concurrent users our database can support. Because of this most RDBMSs allow this isolation requirement to be modified to fit within a certain level of isolation. We'll take a look at isolation levels shortly.

Durable

Once committed, the results of a transaction must be permanently stored in the database. If there is a database crash or some other failure after a transaction has been committed, the RDBMS must guarantee that the committed data will not be lost. Conversely, data must not be made permanent until the transaction has been committed. If there is a database failure while a transaction is in progress the database must guarantee that the uncommitted data will not be saved permanently and that the whole transaction will be rolled back.

Isolation Levels

As mentioned above, most RDBMSs allow the isolated property of transactions to be bent a little, largely because of the time involved in transactions on large data sets. If they didn't do this then in many situations a database could only ever handle one request at a time and the rest would end by piling up in a queue somewhere waiting to obtain a shared lock, which is not an ideal situation, especially in a web-based environment. In order to facilitate a more flexible isolation regime the ANSI SQL-92 standard specifies a set of four different Isolation Levels, which offer varying degrees of isolation between transactions. They are in order of least isolated to most isolated:

- Read uncommitted
- Read committed
- Repeatable read
- Serializable

Let's look at each of these isolation levels in the context of a couple of typical transactions and the problems that each one can potentially present.

Read Uncommitted

This is the lowest level of isolation available. Basically it allows any statement in a transaction to read data from other transactions that have not yet been committed. The big advantage of Read Uncommitted transactions is that they will not block any other transactions from going about their business by locking rows or tables. Read Uncommitted transactions are great when the absolute accuracy of the data is not essential. For example they would be an excellent choice for a transaction that retrieved and manipulated hit counts for a certain category in an e-commerce store. By using a Read Uncommitted transaction to work with the hit count, other transactions in the system can continue to update the hit count column without having to wait for locks to be released.

> *By using a Read Uncommitted transaction to work with the hit count, other transactions can update the column*

Read Uncommitted transactions also bring with them a whole host of problems. The first is called the **Dirty Read** problem. Imagine for a moment that a system administrator was uploading a large batch of products into our e-commerce store. They were doing so within the context of a transaction. At the same time a customer was browsing the store and placing orders – again they were working in a transaction, but the isolation level of that transaction is set to Read Uncommitted. Because our customer can see the uncommitted data that the system administrator is inserting they could potentially view and place orders for products that are not yet committed to the database. If the system administrator then needs to roll back their transaction we could be left in a conundrum – there could be orders in the database that point to products that are about to be (or have been) rolled back. Different databases will handle this sort of situation in different ways, but the outcome is unlikely to be ideal for either or both of our parties.

Read Committed

As its name implies a Read Committed transaction can only access data from other transactions once it has been committed to the database. While Read Committed saves us from the potential chaos identified above it is still not a perfect solution. If we use a Read Committed transaction there is still the potential for another database to change existing data in the middle of our transaction.

For example, say we were writing an online banking application. When a user wants to transfer money between accounts we have to perform three steps. First we read from the account they are transferring from to check they have sufficient funds; then, if they have enough money to complete the transaction we remove the funds from the source account; finally, we add them to the destination account. What happens if between the first step and the second step some other action occurs – say an automatic payment is made from the account? The automatic payment could potentially bring the account balance below the required amount to complete our transfer but we can't tell this until it is too late because we have already checked the balance and it seemed fine. This problem is called a **non-repeatable read** – it occurs when a transaction makes identical requests from a database and receives different data each time. What we really need is some way of blocking other transactions from editing the data that we are interested in until our transaction has finished.

Repeatable Read

The Repeatable Read isolation level solves the problem of data being changed by one transaction while in use by another transaction. The way this is implemented is by using locking. Basically, any time that our Repeatable Read transaction reads data it will lock the rows that it read. It will then hold that lock until the transaction is completed – either committed or rolled back. In this way, any other transaction that wants to edit the data will be forced to wait until our Repeatable Read transaction is finished. Repeatable Read transactions are fairly robust; they do however place fairly major concurrency limitations on the database because they hold their locks for the duration of the transaction.

Repeatable Read transactions are still not completely isolated from other transactions; there is still one small issue that can arise in some situations. This is called the **Phantom Insert** problem. This is similar to the non-repeatable read issue that we looked at above. The difference this time is that rather than getting inconsistent values in existing rows for the same query, the transaction can see new rows that have been inserted since the original query. These phantom rows will probably not have a major impact – the most common situation where they do cause problems is when a transaction needs to get consistent reads of some aggregate value such as an `AVG` or `COUNT`. Fortunately there is one last isolation level that we can turn to.

Serializable

Serializable transactions are completely isolated. The term Serializable is used to signify that while in most cases transactions can still run side by side, the result must always be the same as if those transactions had been run in series, one after the other. Serializable transactions solve the Phantom Insert problem, but in doing so they severely limit the concurrency of the database. This is because they are generally implemented by using either database-wide or table-wide locks which are held for the duration of the transaction. You should think very carefully before using Serializable transactions in web-based databases.

Please note that not all RDBMSs support all of the above isolation levels. Some don't even support transactions. The outline that we have given applies to most databases in general but each will have its own quirks and differences. You should take a look at the RDBMS documentation for details on exactly how your database handles transactions and isolation levels.

Isolation Levels in Action

The syntax for setting different isolation levels will vary across database platforms. In SQL Server it looks like this:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ
```

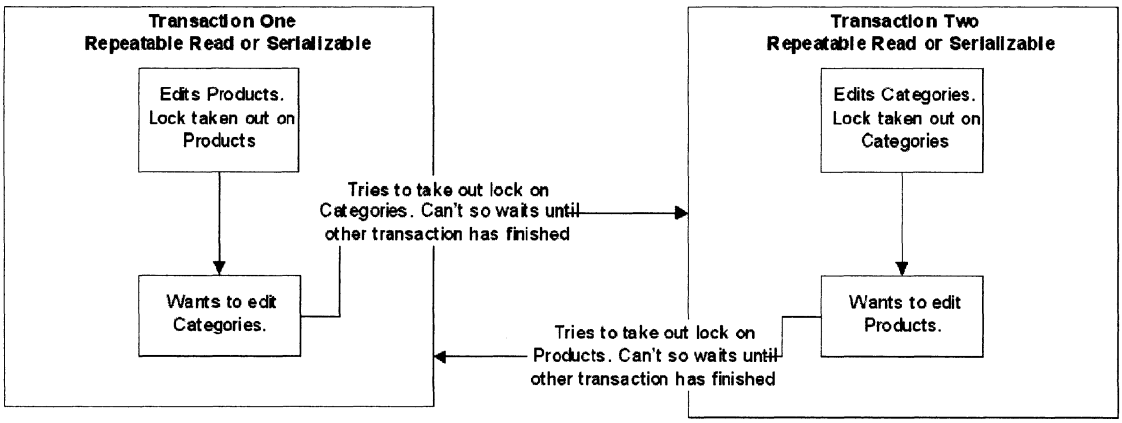
In cases where you are setting the isolation level from code (on the web server) isolation levels may be explicitly supported by your middleware such as ADO. So for example the ADO `Connection` has an `IsolationLevel` property that developers can set, like this:

```
objConnection.IsolationLevel = adXactRepeastableRead
```

You should always try to use the lowest isolation level possible. When working out which isolation level I need I usually grab a piece of paper and draw out a couple of transactions and where they are liable to conflict in terms of wanting access to the same data. I then work back from a `SERIALIZABLE` isolation level deciding if I can accept the disadvantages and risks presented by each lower isolation level. As a general rule you should be all right with Read Committed in most cases.

Deadlocks

A deadlock occurs when two transactions are holding locks and are both trying to take out locks on the data that the other is holding. It is probably best described by the following diagram:



Each transaction is left waiting for the other transaction, but because the transactions are waiting on each other neither transaction will ever finish – they will remain waiting forever. This sort of situation would wreak havoc with our application if they were left unchecked. Fortunately most RDBMSs will look after us and resolve this deadlock situation. They do this by picking a winner and a loser. The loser is aborted and rolled back, the winner can then get the locks that it needs and complete. The application code that was executing the losing transaction would then have to reattempt the transaction. It is up to the RDBMS how it chooses a winner and loser but generally the losing transaction will return an error code, which you can catch in your application to allow you to rerun the transaction.

In systems where we are balancing the constraints of data integrity with concurrency we will inevitably see some deadlocks. They can be very hard to diagnose and eliminate because they generally only occur when the system is under load. There are however a few simple rules which can reduce the number of deadlocks in operational systems:

> *"It is up to the RDBMS how it chooses a winner and loser"*

- Use the lowest isolation level possible for all transactions.
- Keep your transactions as short as possible, both in duration and in the number of commands that are executed. Remember the Atomic principle of transactions that we discussed above.
- Access data in the same order in all of your transactions, wherever possible. For example if we had accessed Products first in both of the above transactions a deadlock would not have occurred.

Summary

I know this chapter was fairly heavy going and that your mind is probably in a bit of a spin now, but it is important to at least get a grasp of some of these advanced concepts before you kick off on designing large web-based databases. While you may be able to make do without taking advantage of advanced features such as transactions and advanced indexing, it is important to at least understand what they might offer to your system as it grows further down the track.

We have covered a lot of ground in this chapter. First we looked at sub-queries and how they can be used to provide more complex `WHERE` clause conditions in our queries. Then we looked at views, a mechanism for building virtual tables from other `SELECT` statements. We then moved into some of the features generally reserved for the more advanced database platforms. This included the ability to store procedural logic in the database by way of stored procedures and triggers as well as the ability to group SQL commands into all succeed or all fail transactions.

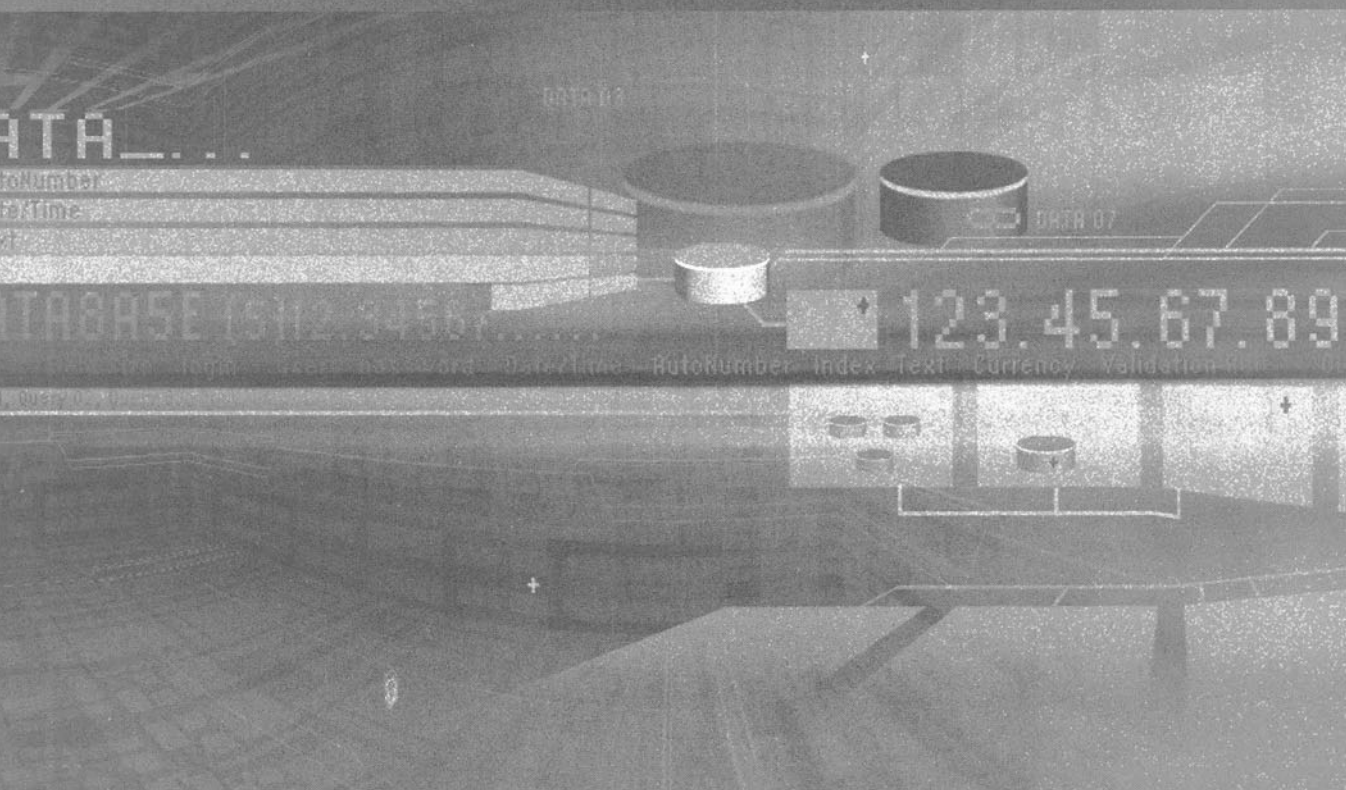
We also spent some time in this chapter considering the importance of indexing our databases in order to maintain performance and as part of that we looked at how the database will use those indexes and how it decides the best approach to take when processing a query.

The actual implementation for many of these advanced features differs significantly across the various database platforms. The goal of this chapter was to provide you with the theory you need to decide when you might need to take advantage of these advanced features. Having made that decision you will need to take a look at the specific documentation for your database platform to see if and how it supports that feature.

9

- Backing up your data
- Modifying tables
- Re-organizing data and indexes for efficiency

Author: Thearon Willis



Database Maintenance

In a production environment, your DBA is usually responsible for any and all database maintenance. This includes such tasks as backing up your database and ensuring the data and indexes are reorganized at frequent intervals to maintain optimal performance of your database. They also perform other tasks such as adding new indexes when appropriate to further optimize performance, and archiving old data. Finally, your DBA is usually also responsible for replicating data and schema changes from your development/test environments to the production environment.

In a development environment these tasks sometimes fall onto the developer to perform. Depending on your environment and size of your team, you may or may not have to work with your DBA on these tasks in the development environment. Ensuring that you have proper backups of your development database is critical to the development cycle as some changes may cause poor performance and need to be rolled back. A quick restore of your database is then all that is needed to recover from these types of changes.

This chapter takes a look at some of the common database maintenance tasks that you as a developer are likely to face in the development cycle. We will discuss some of the common challenges that you are likely to encounter and provide techniques to quickly solve those challenges.

In this chapter, we will cover:

- Database backup and restore
- Modifying tables
- Reorganizing data and indexes
- Building scripts for production implementation

Please note that we have used some SQL Server-specific code to demonstrate some of these techniques in places where there isn't a SQL standard to do such things. In these cases, if you are using a different DBMS, you should look up the specifics that apply to your chosen DBMS then modify the code appropriately.

Why a Developer Should Know This

In many companies a developer is responsible for the databases that they set up and use to build applications. A DBA or Data Modeler may get involved in the development cycle to help guide you on database design principles, but a lot of the time this task is left to the development staff.

Along with this comes the responsibility to maintain the development database during the development cycle. This not only includes backing up and restoring your development database but also other maintenance tasks such as altering tables to add or drop columns and adding or dropping indexes as the project progresses.

With all of this maintenance being performed against a development database and with data being inserted, updated, and deleted, the data in the tables and the indexes will become fragmented. When we refer to fragmentation, we are referring to the sequence of data in your indexes and tables. Typically, your data is organized sequentially based on a key. When it becomes fragmented, the sequence is broken and the data is stored in other parts of the table or index, thereby reducing its efficiency. This is when you will want to reorganize your table and re-index the data to improve performance.

These are all common tasks that every developer should be familiar with to help improve the performance of their development database. In some smaller companies, key developers are often called upon to perform two roles, developer and DBA. What you learn in this chapter will go a long way in helping you perform some of those DBA tasks that you may need to do if you are performing two roles in your company.

This chapter focuses on the tasks that a developer may need to perform in the development environment. What you learn here can also be applied to the test or QA environments as well as the production environment.

Backup and Restore

Having a backup of your database is key not only for recovering from a catastrophic hardware or software failure, but also for recovering from mistakes made during the development cycle. You may be thinking "what mistakes could you make that would cause you to have to restore a database?" The answer is accidents, or human error. For example:

- How many times have you executed a query to delete data from a table, only to forget the `WHERE` clause and end up deleting all data?
- How many times have you clicked on a database object and dropped it, only to realize that as you clicked OK in the confirmation dialog that it was the wrong object?
- How many times have you been testing your code only to find out that you forgot a key line of code and totally corrupted the data in your database?

These are all common mistakes and a good backup plan with multiple backups of your database will help ease the pain of these mistakes by allowing you to restore your database to a stable point. As a developer myself, I make backups of my development database at key points during the development cycle and especially before altering tables or dropping objects. I too am prone to errors, and have had to use a backup to restore my database to recover from mistakes.

Backing Up Your Database

Most RDBMSs contain two physical files associated with a database:

- A **data file** that contains the actual database objects such as tables, constraints, views, and stored procedures
- A **log file** (or journal) that contains transactions that have been performed against the database such as inserts, updates, and deletes

When you set out to back up your database, you must decide on how you want to recover your data should the need arise. That is, do you want to recover your data to the point where you made your last backup? Or would you like finer control over the recovery and be able to recover to a specific time?

> *make a full backup of your database every evening*

Let's take a look at the first option first, recovering your database to the point of your last backup. This is known as a **simple recovery model** as you simply restore your database using your last backup. Let's assume that you make a backup of your database every evening before you go home. The next afternoon, you make some changes that wipe out some important data that would take too long to recreate. You decide to restore your database with the backup you made last evening. This is fine but you have to keep in mind that you will lose all work that was made during the current day. Depending on how much work you have done with the database, this may or may not be OK.

Now let's look at how you can take this recovering one step further to recover your database to a specific point in time. This is known as a **full recovery model** as it allows you to recover more data, and to recover to any given time of your last transaction log backup. Using this model, you make a full backup of your database every evening as in the previous example. Using this model, however, backups are made of your transaction log every hour (or whatever time interval you specify). Using this model, you would restore your database using the last backup and then apply the transaction log backups for the day to the point of your last transaction log backup, or to the point where you want to recover the last changes made in your database.

Using a simple recovery model allows you to quickly recover your database. You are more likely to lose more development work, depending on how much is being performed and the number of developers working with the database. Using a full recovery model provides you with more control over how much data is to be recovered, as you have the option of restoring all of the transaction log backups or just a few of them, up to the point where you want to recover your database. Let's look more closely at how to back up a database.

Complete Backup

A lot of RDBMSs allow **differential backups** that allow you to only back up the data that has changed. A differential backup can only be performed after a complete backup has been performed. When restoring your database, however, you must first restore the database using your complete backup and then restore using the differential backups. To keep our simple recovery model really simple, we will just be focusing on using complete backups of our database, as development databases are typically smaller than a normal production database.

The BACKUP SQL command is part of the SQL-92 Data Definition Language (DDL), which is used to manage all objects in your database. Each RDBMS may implement variations to the BACKUP command so check the documentation for your RDBMS that you are using for the exact syntax it uses.

The syntax for the BACKUP command is quite lengthy so here we will only cover the basic syntax here, which will allow us to perform a complete backup of our database:

```
BACKUP DATABASE database_name
TO DISK = path_and_filename
WITH INIT
```

The interesting parts of this syntax are as follows:

- The DATABASE keyword specifies that we want to back up our database
- The *database_name* parameter represents the name of the database to be backed up
- The *path_and_filename* parameter represents a full path and filename of the backup to be created – this should be on a different drive to the database we're backing up, preferably removable media such as DAT or CD
- The WITH INIT keywords specify that, if the backup file already exists, the backup being performed will overwrite the existing backup file. If these keywords are not specified, then each subsequent backup will be appended to the physical backup file. This would obviously cause the backup file to keep growing as each backup was performed

The example BACKUP command shown below will back up the database named WebDB to a file named WebDB_Backup.bak located in the BACKUPS directory on our removable media drive E (perhaps a CD rewriter). Each time we run this command, the backup file will be overwritten if it already exists.

```
BACKUP DATABASE WebDB
TO DISK = 'E:\BACKUPS\WebDB_Backup.bak'
WITH INIT
```

Once this backup command has been run, it will return messages similar to the ones shown below. This message informs us how much data was backed up for our data and log files and also that the backup completely successfully. Because we are backing up the entire database, the log file (which is also part of the database) is automatically backed up.

```
Processed 88 pages for database 'WebDB', file 'WebDB_Data' on file 1.
Processed 1 pages for database 'WebDB', file 'WebDB_Log' on file 1.
BACKUP DATABASE successfully processed 89 pages in 0.266 seconds (2.713 MB/sec).
```

> *A page is the fundamental unit of data storage in your database*

Here we see that the database has specified how many pages were processed. A page is the fundamental unit of data storage in your database. For a table, this is the number of rows that will fit onto a page. For indexes, this is the number of index entries that will fit onto a page. A typical page size is 8KB, but it will vary from RDBMS to RDBMS.

Now that we understand the basic syntax of the BACKUP command, let's take this one step further and see how we can apply it in the real world to perform daily backups. Obviously, one backup file will not do us much good, should we decide we need to restore our database to a stable condition that it was in a couple of days ago. For this we would need multiple backups of our database.

The query that follows builds a backup path string in a local variable using some date functions (in Microsoft SQL Server). This allows us to append a date to our backup filename, and so maintain multiple backups, and quickly identify when each was made.

In the following code, we use the GETDATE() function to return the current date and then use the DAY, MONTH, and YEAR functions to extract the day, month, and year from the date returned by the GETDATE() function. At the same time, since we are concatenating these numeric dates to a string, we have to change their data types from a numeric to a character data type. We do this using the CAST function, which casts an expression from one data type to another. Consult the documentation for the RDBMS that you are using for similar functions.

Once our local variable is built, it is then used in the BACKUP command to specify our backup path and name. Depending on which country you live in, the date format that you use will vary and itcan be adjusted to your liking. Also, the extension that you use for your backups is irrelevant as the database is looking at the contents of your backup when restoring the database and not the file extension.

```
DECLARE @BackupPath VARCHAR(100)
SET @BackupPath = 'E:\BACKUPS\WebDB_Backup_' + CAST(DAY(GETDATE()) AS VARCHAR(2))
+ CAST(MONTH(GETDATE()) AS VARCHAR(2)) + CAST(YEAR(GETDATE()) AS CHAR(4)) +
'.bak'

BACKUP DATABASE WebDB
TO DISK = @BackupPath
WITH INIT
```

After this query is executed, our backup file will be located in the path E:\BACKUPS with a filename of WebDB_Backup_2892002.bak. As you can see, the filename contains the date that the backup was run, in this case 28 September 2002.

Transaction Log Backups

We need to implement the full recovery model for those developers who need more control over the restore process and want to restore their database to a specific point in time. As we just discussed this involves making a full backup, and making frequent transaction log backups. Remember that for full recovery, you will have to restore your database using your latest backup, and in addition to that you will have to restore each transaction log backup up to the point where you want to recover.

As we mentioned earlier, the log contains transactions that have been performed against your database such as inserts, updates, and deletes. It also contains any modifications you made to your database structure, such as adding or dropping columns from a table, and modifications made to stored procedures and views.

When we examined the basic syntax of the BACKUP command earlier, we did so from the standpoint of backing up your database. Now let's examine it from the perspective of backing up your transaction log:

```
BACKUP LOG database_name
TO DISK = path_and_filename
WITH INIT
```

Again, the interesting parts of this syntax are as follows:

- The `LOG` keyword specifies that we want to backup our database log file
- The *database_name* parameter represents the name of the database whose log file we are backing up
- The *path_and_filename* parameter represents a full path and filename of the backup to be created
- The `WITH INIT` keywords specify that if the log file already exists, the backup being performed will overwrite the existing backup. If these keywords are not specified, then each subsequent backup will be appended to the physical log file backup

Here's a query that will perform a backup of your log file. Notice that we have used the `LOG` keyword instead of the `DATABASE` keyword in our query. We have also changed the filename to indicate that this is a log backup instead of a database backup.

```
BACKUP LOG WebDB
TO DISK = 'E:\BACKUPS\WebDB_Log.bak'
WITH INIT
```

Once this query has been executed, you will see messages indicating the number of pages of the transaction log that were backed up and also that the log file was backed up successfully as shown below:

```
Processed 1 pages for database 'WebDB', file 'WebDB_Log' on file 1.
BACKUP LOG successfully processed 1 pages in 0.147 seconds (0.003 MB/sec).
```

We also want to include the time so we know when the backup was performed

Using the same types of function as we did earlier, we can modify this query to include the date in the backup name. We will also want to include the time so we know when the backup was performed since we will probably be backing up the transaction log every hour or so.

The query below has added the hour and minute to our backup name. Using the `DATEPART` function, we are able to retrieve the hour and minute from the current date and time returned by the `GETDATE()` function. Then using the `STR` function, we are able to convert the numeric data returned into character data. We tell the `STR` function that the length of data to be returned is two digits. We always want the hour and minute fully represented by two digits each so we use the `REPLACE` function to replace a space if one exists with the number `0`. This allows us to represent a single digit with a prefix of `0` (so that, for example, `8` becomes `08`):

```
DECLARE @BackupPath VARCHAR(100)
SET @BackupPath = 'E:\BACKUPS\WebDB_Log_' + CAST(DAY(GETDATE()) AS VARCHAR(2)) +
CAST(MONTH(GETDATE()) AS VARCHAR(2)) + CAST(YEAR(GETDATE()) AS CHAR(4)) + '-' +
REPLACE(STR(DATEPART(hh,GETDATE()),2),' ','0') +
REPLACE(STR(DATEPART(mi,GETDATE()),2),' ','0') + '.bak'
```

```
BACKUP LOG WebDB
TO DISK = @BackupPath
WITH INIT
```

Now we are able to run this query whenever we want and we will get a backup of the transaction log with a name similar to this: `WebDB_Log_2892002-0850.bak`. We can look at the name of our backup and quickly determine the exact date and time the backup was performed.

When backing up your web database, there are a couple of things that you need to keep in mind. First is the timing of your backups. In a development environment it really doesn't matter when you perform a backup, as you know who is accessing the database and when.

In a production environment however, you'll need to schedule your backups to occur when you have the least amount of traffic on your web site. Performing a backup of your database may briefly lock out transactions against tables that are being backed up as well as cause performance to be degraded while the backup is being performed.

In a production environment, you want the maximum recoverability available, so it is a good idea to maintain a complete backup of your database and incremental backups of your transaction logs. This will ensure that you can recover to any point in time, back to the last transaction log backup.

Now that we have covered backing up our database and transaction log, it's time to move on and learn how to restore our backups using both a simple recovery model and a full recovery model.

Restoring Your Database

In this section we will examine how to restore your database from a backup using both a simple and full recovery model. It is important to note that both models start out by restoring the database using the latest available full backup.

It is important to note that you should test your recovery model by restoring your backups to a test database to verify that your backups are good. This is extremely important when using new media such as CD or TAPE.

> *"it is a good idea to maintain a complete backup of your database and incremental backups of your transaction logs"*

In the case of a simple recovery model, no other actions are required beyond restoring your database using the latest full backup. After the restore, your database will be recovered to the point that it was when the backup was performed.

In the case of a full recovery model, you will need to restore each transaction log, **in sequence**, up to the point that you want to recover your database. This can be to the point of your last transaction log backup or any point before that.

Like the BACKUP command, the RESTORE command has a long, complex syntax that provides all kinds of options, such as restoring specific files within the backup for backups that contain multiple files. To keep this section simple, we will only examine the basic syntax of the RESTORE command. Consult the documentation for your DBMS for the full syntax available and an explanation of all of the restore options. The basic syntax to restore your database is shown here:

```
RESTORE DATABASE | LOG database_name
FROM DISK = path_and_filename
WITH NORECOVERY | RECOVERY
```

- The DATABASE keyword specifies that we want to restore our database using a backup of our database
- The LOG keyword specifies that we want to restore our database transaction log using a backup of our transaction log. Choose one of these two options depending on the type of restore you want to perform
- The *database_name* parameter represents the name of the database to be restored
- The FROM DISK keywords specify that we are restoring the database or transaction log from a backup on disk
- The *path_and_filename* parameter represents a full path and filename of the backup that is to be used to restore with
- The NORECOVERY keyword specifies that uncommitted transactions should not be rolled back. This option is used when you are using the full recovery model and allows you to restore one or more transaction log backups to recover the transactions in those backups. It should be noted that when the NORECOVERY keyword is specified, the database remains unusable until the final transaction log has been restored
- The RECOVERY keyword specifies that any uncommitted transactions should be rolled back. You should use this keyword when performing a restore of the database using a simple recovery model where you are restoring a full backup. You should also use this keyword when you are restoring the last transaction log so that all uncommitted transactions on your transaction log backup are rolled back

If neither NORECOVERY nor RECOVERY is specified, SQL Server uses RECOVERY as the default keyword. Check your DBMS to see if it provides a default if neither keyword is specified.

Let's look at an example query that will use our latest backup to restore the database. This query makes use of the simple recovery model as indicated by the RECOVERY keyword.

```
RESTORE DATABASE WebDB
FROM DISK = 'E:\BACKUPS\WebDB_Backup_2892002.bak'
WITH RECOVERY
```

After this query has been run, you will see messages indicating how many pages were restored and that the restore was successful. After the restore is complete, the database is ready for use.

```
Processed 88 pages for database 'WebDB', file 'WebDB_Data' on file 1.
Processed 1 pages for database 'WebDB', file 'WebDB_Log' on file 1.
RESTORE DATABASE successfully processed 89 pages in 0.219 seconds (3.296 MB/sec).
```

Using the simple recovery model is quick and restores our database to the point of the last backup. Because this was a complete backup of the database, it contained both the data file and the log file. Any uncommitted transactions in the log file were rolled back so the database was recovered to a stable point.

> *a complete backup of the database contained both the data file and the log file*

Let's move on now and take a look at the queries that should be used to restore our database using a full recovery model. Remember that the full recovery model allows us to restore the database using a full backup; we then continue to restore all transaction log backups to a point in time where we want to be.

It should be noted that the transaction logs must be restored in the order in which they were taken. For example, three transactions logs backed up at 8:00, 9:00, and 10:00 AM must be restored in the same order starting with the log backed up at 8:00 AM. This ensures that the transactions are applied in the correct order.

The query to restore our database looks (at first glance) like the one we saw previously for the simple recovery model. There is one minor difference however: the use of the `NORECOVERY` keyword. Remember that we must specify this keyword using the full recovery model to let the RDBMS know that we will be restoring transaction log backups to recover transactions. Also note that the database remains unusable until we restore the last transaction log backup and specify the `RECOVERY` keyword.

```
RESTORE DATABASE WebDB
FROM DISK = 'E:\BACKUPS\WebDB_Backup_2892002.bak'
WITH NORECOVERY
```

We would see a message like the one below after this query has been run:

```
Processed 88 pages for database 'WebDB', file 'WebDB_Data' on file 1.
Processed 1 pages for database 'WebDB', file 'WebDB_Log' on file 1.
RESTORE DATABASE successfully processed 89 pages in 0.202 seconds (3.573 MB/sec).
```

To restore our transaction log backups, our query would specify the `LOG` keyword and the transaction log backup path and filename. **Remember that you must restore each transaction log in the order in which it was taken and you cannot skip a transaction log backup.**

> *you cannot skip a transaction log backup*

Again, you would specify the `NORECOVERY` keyword for each transaction log backup that you are restoring until you get to the final transaction log backup. At this point you must specify the `RECOVERY` keyword so that once the restore of that backup is complete, the database will be in a usable state.

The query below assumes this is the last transaction log backup being restored.

```
RESTORE LOG WebDB
FROM DISK = 'E:\BACKUPS\WebDB_Log_2892002-0850.bak'
WITH RECOVERY
```

Once the restore has been completed, you will see messages similar to the ones shown below:

```
Processed 1 pages for database 'WebDB', file 'WebDB_Log' on file 1.
RESTORE LOG successfully processed 1 pages in 0.006 seconds (0.170 MB/sec).
```

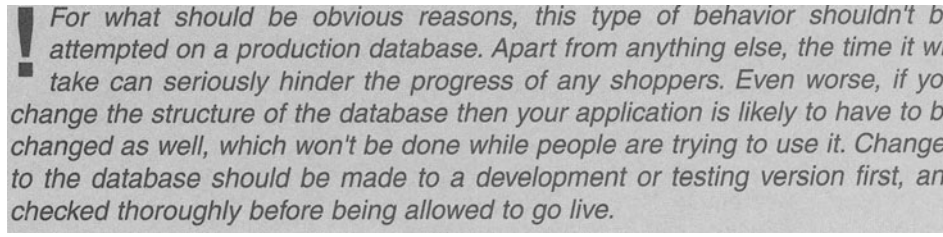
At this point we have shown you how to use simple and full recovery models to back up and restore your database. The model you choose depends on how often your data changes.

If a lot of development work (for example creating stored procedures, modifying schemas or data) is being performed against the database, it would be a good idea to use the full recovery model, making a full backup every day and incremental transaction log backups throughout the day.

Modifying Tables

As part of every development project, you will more likely than not need to modify the database design at some point. Designs and requirements change and it is our job to be able to adapt and meet those changes rapidly. These changes may be in the form of adding or dropping tables from your database, or they may be a little more complex, perhaps altering tables to change, add, or drop columns.

This section of the chapter will build on the discussion of `ALTER TABLE` in *Chapter 3*, and take a look at modifying tables as we go through what is required to change, add, and drop columns from a table as well as adding additional indexes to improve the performance of our queries. But why do we have to do this with SQL, you may well ask: why can't we use a GUI? The answer is that by using commands it is possible to keep a record of the changes made, making it easy to apply them to other instances of the database.



Before we begin you will need to create a test table so that you can follow along. The script shown below will create a table named `TestTable` in your chosen database, as we saw in *Chapter 3*.

```
CREATE TABLE TestTable
  (
  TableKey    INTEGER          NOT NULL,
  SerialNum   VARCHAR(30)      NOT NULL,
  Assigned    CHAR(1)          NOT NULL,
  Notes       VARCHAR(1000)    NULL,
  PRIMARY KEY (TableKey)
)
```

Once you have created your table, you will need to add some data to the table for testing purposes. You can use the script shown next, adding your own data as required. If you do add your own data, for the purposes of the discussions that follow please make sure you insert a numeric value of `0` or `1` in the `Assigned` column.

```
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1000,'12345','0',NULL)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1001,'89654','1','Assigned on 28/9/2002')
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1002,'741263985741','1','Nothing new to report')
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1003,'9632100145','0',NULL)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1004,'852369741202587','0','Unassigned on 28/9/2002')
```

Changing Columns

Sometimes a table is designed with what seems to be the appropriate data type and a sufficient length to hold the data expected. During the development cycle, however, it comes to light that the length of the data type is not sufficient or the data type chosen is not optimal for the queries being run against it. Or, it might be that the requirements change and it is decided that the column in question will be too small, as additional information may need to be added.

Regardless of the exact circumstances, you may need to alter the size or data type of a column. In this section we will examine what you need to do to alter the size and data type of a column in your table.

Let's first examine some constraints on altering columns in a table. The specific constraints will vary from RDBMS to RDBMS, so consult the documentation for your chosen software. The constraints listed below are basic constraints on changing a column in a table:

- Columns that are used in primary or foreign key constraints cannot be changed
- Columns used in other constraints such as one that sets a default value cannot be changed

There may be other constraints, specific to your RDBMS, so please check the documentation if you run into problems trying to change a column in a table.

Now let's examine the syntax for altering tables. The SQL server command `ALTER TABLE` has a large and complex syntax and we are only going to examine the basics of changing a column in this section. The basic syntax to alter a column in a table is.

> *"check the documentation if you run into problems"*

```
ALTER TABLE table_name
ALTER COLUMN column_name
      new_data_type NULL | NOT NULL
```

In the syntax above:

- The *table_name* parameter specifies the name of the table in which we are altering a column
- The `ALTER COLUMN` keywords specify that we want to alter a column in a table
- The *column_name* parameter specifies the column name that we are altering
- The *new_data_type* parameter specifies the new data type for the column being altered
- Finally, the `NULL` and `NOT NULL` keywords specify whether or not the column being altered will accept null values

Changing a Column to Allow or Disallow Null Values

The most common change to a column is to modify whether it accepts nulls or not. Changing a column that is set to not allow null values so that it allows null values is very simple and less prone to errors than changing a column that currently allows null values to not allow null values.

Let's take a look at a query that will change the `Assigned` column in our test table to allow null values. Even though we are not changing the data type of the column, we must still specify it in our query. The only change to the column is changing the `NOT NULL` attribute of our column to `NULL` as shown below:

```
ALTER TABLE TestTable
ALTER COLUMN Assigned CHAR(1) NULL
```

The query shown above will work on any column in your table, with the exceptions outlined at the start of this section.

You may want to change a column that currently accepts null values to disallow null values. The following query can be run to change a column to disallow null values.

```
ALTER TABLE TestTable
ALTER COLUMN Assigned CHAR(1) NOT NULL
```

Again, we have to specify the data type even though we are not changing it. This query will change the column attribute from `NULL` to `NOT NULL`.

> *You will be warned by your RDBMS if you try to change a column with null in to one that doesn't accept null*

There is a caveat here that you should be aware of. This query will not work on a table that currently has null values in the column you want to change. For example, the `Notes` column in our table contains null values and this query will not work unless you populate the `Notes` column with some data in every row of the table, even if it is a single blank space. You will be warned by your RDBMS if you try to change a column with null in to one that doesn't accept null.

Changing the Length of a Column

Sometimes it is desirable to increase the length of the data type in a column. For example, suppose it was determined that the default length of 1,000 characters will not be sufficient for the `Notes` column in our table and it has been decided that we should increase it to 2,000 characters.

The query below will alter this column increasing the length of the `Notes` column from 1,000 characters to 2,000 characters:

```
ALTER TABLE TestTable
ALTER COLUMN Notes VARCHAR(2000) NULL
```

Notice that in this query we have changed the length of the `VARCHAR` data type to 2,000 characters. Also notice that we have also specified the `NULL` keyword even though it is not needed for this operation.

One thing that you should be aware of here is that, if you do not specify the `NULL` or `NOT NULL` keyword, and the column currently has an attribute of `NOT NULL`, it will be changed to `NULL`. The reason behind this is that `NULL` is the default value in SQL Server when neither keyword is specified. Therefore, it is good coding practice to always specify this keyword. Check the documentation for your RDBMS to determine if it implements a default value when none is specified.

Changing the Data Type of a Column

A requirement may arise to completely change the data type of a column (it may become necessary for performance reasons). There are two different scenarios that we want to explore here:

- Changing a data type to something similar
- Changing a data type to something rather different

Similar Substance

Let's look at the first type of change, changing the data type to something similar. Let's look at the `Notes` column again. Suppose it was decided that 2,000 characters would not be enough space after all and that the prudent thing to do would be to change the column to a data type of `TEXT`. Both the `TEXT` data type and the `VARCHAR` data type allow for character data.

This type of change is a similar change and the query below will change the data type from a `VARCHAR(2000)` data type to a `TEXT` data type.

```
ALTER TABLE TestTable
ALTER COLUMN Notes TEXT NULL
```

Different Substance

Now let's look at changing the data type of a column to something different. A good example of this would be changing a data type that allows character data to a data type that only allows numeric data.

Take, for example, our `Assigned` column. It is currently defined with a data type of `CHAR(1)` and contains only numeric data anyway. Let's assume that this column only contains a value of `0` or `1`. It makes more sense for this column to be of a numeric data type than a character data type. Let's also assume that we want to change this column to a data type of `BIT`, which only allows a value of `0` or `1`.

The following query will alter this column to have a new data type of `BIT` and still not allow null values.

```
ALTER TABLE TestTable
ALTER COLUMN Assigned BIT NOT NULL
```

As we can see, there is quite a lot that can be done to alter the existing columns in our tables. These changes can be as simple as increasing the size of a data type or a little more complex, for example changing the data type altogether.

Another common method of altering a column in a table is to add the new column, copying the existing data from the old column to the new column and then drop the old column. This method requires a lot more SQL statements and accomplishes the same goal as our SQL statements above. Using the method in our example above, the RDBMS takes care of moving the data for us, and requires a lot less code.

Adding Columns

During the development process, the database design may change either due to issues discovered during development or the requirements may change. Either way, sometimes you will find that you may need to add additional columns to your tables.

> *Columns will typically be added to the end of the table*

When you add a new column to a table, you have the option of also adding constraints for that column such as a default value, index, or primary or foreign keys. We don't have the space to cover all of these topics here so consult the documentation for your RDBMS for the specific constraints that can be added. Columns will typically be added to the end of the table.

What we do want to cover here is adding additional columns to our table with a default value constraint. This will be the most typical scenario that you will come across in your development cycle. That said, let's look at the basic syntax for altering a table to add a new column.

```
ALTER TABLE table_name
ADD column_name data_type NULL | NOT NULL
    DEFAULT constant_expression WITH VALUES
```

Note that not all databases support altering tables and columns in this manner. Check your RDBMS documentation for details.

In the syntax above:

- The *table_name* parameter specifies the name of the table in which we are altering
- The `ADD` keyword specifies that we want to add a new column to our table

- The *column_name* parameter specifies the column name that we are adding. We can add a series of new columns, each one separated by a comma
- The *data_type* parameter specifies the data type for the column being added, and the `NULL` and `NOT NULL` keywords specify whether or not the column being added will accept null values
- The `DEFAULT` keyword specifies that we will be adding a constraint that provides a default value for the column
- The *constant_expression* parameter specifies the value that we want added to the column as the default
- Finally, the `WITH VALUES` keywords specify that the default value being added to the new column will also be added to existing rows in the table

The query below will alter our table named `TestTable` by adding two columns, `Model` and `Price`. Notice that the `Model` column will allow null values while the `Price` column will not. We have also specified a default value of `0.00` that is to be placed in the `Price` column. Also, note the use of the `WITH VALUES` keywords. This will place the default value of `0.00` that is added to existing rows of data in this new column:

```
ALTER TABLE TestTable
ADD Model  VARCHAR(30) NULL,
    Price  SMALLMONEY  NOT NULL
    DEFAULT 0.00 WITH VALUES
```

> *you cannot add a new column to an existing table that has the NOT NULL keywords specified **without** providing a default constraint*

There is one thing to note here: you cannot add a new column to an existing table that has the `NOT NULL` keywords specified **without** providing a default constraint. It does not matter whether the table contains any data or not.

Suppose you want to add a new column and you want to provide a default value for that column but do not want that default value applied to existing rows? You can accomplish this by omitting the `WITH VALUES` keywords but you must specify that the new column allow null values by using the `NULL` keyword. Take at look at the query below:

```
ALTER TABLE TestTable
ADD DateAquired DATETIME NULL
    DEFAULT GETDATE()
```

This query adds a new column to our test table and specifies a default value for new rows that are added to the table if no value is specified for this column during the insert. It does not, however, affect any existing rows of data in the table, as we have not used the `WITH VALUES` key words.

Dropping Columns

Just as your database design may change and require you to add new columns, it will surely change and require you to drop existing columns from a table. You'll probably want to make a backup of your database before you start dropping columns. The requirements may change again, and you may need to retrieve a column, along with its data.

Let's take a look at the basic syntax for dropping a column from a table:

```
ALTER TABLE table_name
DROP CONSTRAINT constraint_name
     COLUMN column_name
```

In the syntax above:

- The *table_name* parameter specifies the name of the table that we are altering
- The `DROP` keyword specifies that we want to drop a constraint or column
- The `CONSTRAINT` keyword specifies that a column constraint is to be dropped
- The *constraint_name* parameter specifies the name of the constraint that we want to drop
- The *column_name* parameter specifies the column that we are dropping

If no constraints exist on the column that you want to drop, you simply omit the `CONSTRAINT` keyword and the *constraint_name* parameter. If the column to be dropped contains any constraints, you must specify those associated with the column or it will not be dropped.

> *"Any column that has a default value, an index, or is a primary or foreign key has a constraint on it"*

You may be wondering how you know if a column has a constraint defined for it. Any column that has a default value, an index, or is a primary or foreign key has a constraint on it. In SQL Server you can run a system stored procedure called `sp_help` to list all constraints defined on a table. Consult the documentation for your RDBMS to determine the equivalent system stored procedure.

Let's look at the syntax for this system stored procedure. It expects one parameter: an object name. The values returned will depend on the object you run it against:

```
sp_help TestTable
```

In our case, since we are requesting help on a table, `sp_help` will return a wealth of information, including column names and data types, primary keys, indexes, and constraints.

Once you have determined whether or not a column has a constraint, you will know if you need to include the `CONSTRAINT` keyword in your query. If you do not specify the constraints to be dropped, you will receive an error and the command will fail.

Let's look at two examples, one query for a column that has a constraint and one query for a column that does not.

The first query that we want to look at is a query to drop the `DateAquired` column. Remember that this column has a constraint for adding a default value of the current date. You will need to execute the system stored procedure `sp_help` or equivalent on the `TestTable` to determine the constraint name that has been defined in your database. The results of executing `sp_help` are listed here:

```
Name
Owner
Type                                Created_datetime
------------------------------------------------------------------
------------------------------------------------------------ ----------
------------------------------------------------------------------
---------------------------------------- -------------------------
- ------------------------------------------------------
TestTable
dbo
user table                          2002-10-30 07:43:17.237


Column_name
Type
Computed                                Length       Prec  Scale Nullable
TrimTrailingBlanks                      FixedLenNullInSource
Collation
------------------------------------------------------------------
------------------------------------------------------------ ----------
------------------------------------------------------------------
---------------------------------------- -------------------------
----- ----------- ----- ----- ---------------------------------- -
---------------------------- ---------------------------------- -
------------------------------------------------------------------
--------------------------------------------------
TableKey
int
no                                      4            10    0      no
(n/a)                                   (n/a)
NULL
SerialNum
varchar
no                                      30                        no
no                                      no
SQL_Latin1_General_CP1_CI_AS
Assigned
bit
no                                      1                         no
(n/a)                                   (n/a)
NULL
Notes
text
no                                      16                        yes
(n/a)                                   (n/a)
SQL_Latin1_General_CP1_CI_AS
Model
varchar
no                                      30                        yes
no                                      no
SQL_Latin1_General_CP1_CI_AS
Price
smallmoney
no                                      4            10    4      no
(n/a)                                   (n/a)
NULL
```

```
DateAquired
datetime
no                                      8                       yes
(n/a)                                   (n/a)
NULL


Identity
Seed                                            Increment
Not For Replication
------------------------------------------------------------------
------------------------------------------------------------ ----------
------------------------- ----------------------------------------
--------------
No identity column defined.
NULL                                            NULL
NULL


RowGuidCol
------------------------------------------------------------------
------------------------------------------------------------
No rowguidcol column defined.


Data_located_on_filegroup
------------------------------------------------------------------
------------------------------------------------------------
PRIMARY


index_name
index_description
index_keys
------------------------------------------------------------------
------------------------------------------------------------ ----------
------------------------------------------------------------------
------------------------------------------------------------------
------------------------------------------------------------ ---------------
------------------------------------------------------------------
------------------------------------------------------------------
------------------------------------------------------------------
--------------------
PK__TestTable__7A9C383C
clustered, unique, primary key located on PRIMARY
TableKey


constraint_type
constraint_name
delete_action update_action status_enabled status_for_replication constraint_keys
------------------------------------------------------------------
------------------------------------------------------------------
-- ---------------------------------------------------------------
------------------------------------------------------------ -------
- ------------- -------------- ---------------------- -------------
```

```
------------------------------------------------------------------------
------------------------------------------------------------------------
------------------------------------------------------------------------
----------------------
DEFAULT on column DateAquired
DF__TestTable__DateA__7C8480AE
(n/a)          (n/a)          (n/a)          (n/a)
(getdate())
DEFAULT on column Price
DF__TestTable__Price__7B905C75
(n/a)          (n/a)          (n/a)          (n/a)                    (0.00)
PRIMARY KEY (clustered)
PK__TestTable__7A9C383C
(n/a)          (n/a)          (n/a)          (n/a)
TableKey


No foreign keys reference this table.
No views with schema binding reference this table.
```

Once you have the constraint name, you can add it to your query so that it looks similar to the one shown below. After executing this query, the `DateAquired` column will be dropped from your table.

```
ALTER TABLE TestTable
DROP CONSTRAINT DF__TestTable__DateA__08EA5793,
  COLUMN DateAquired
```

Now let's take a look at a query that drops a column that does not contain a constraint. The column that we added in the last section (`Model`) does not have a constraint on it so we can execute a query like the one shown below to drop this column:

```
ALTER TABLE TestTable
DROP COLUMN Model
```

After executing this query, the `Model` column is dropped from the table.

We have examined altering existing columns, adding new columns, and dropping columns from our tables. The `ALTER TABLE` command is a very powerful command that allows you to alter existing tables in your database to perform a wide range of tasks.

While we have only explored the basic syntax and tasks here, there is a lot more functionality in the `ALTER TABLE` command. Consult your documentation for the RDBMS that you are using to see the complete syntax and to discover how this powerful command can assist you in altering your tables to meet the design needs of your project.

Adding Indexes

Every RDBMS has a unique way in which it manages indexes behind the scenes. In an effort to try and keep this section as generic as possible, we will mention how indexes are used and not detail how they are managed. Also remember that indexes are not, strictly speaking, part of the relational model.

Indexes are intended to increase the performance of your queries as they point to the exact location of the data that you want to retrieve as long as the WHERE clause of your query contains an indexed column. Unfortunately, like the rest of the database, the index has to be designed well in the first place, and kept up to date with the changes in the tables.

> *indexes are not, strictly speaking, part of the relational model*

Let's look at an example of using an index in a query. The query below executes very efficiently because the RDBMS knows the exact location of the row of data we want. The reason is that, when the table was created, we specified that the column TableKey was a primary key for the table and we specified the CLUSTERED keyword indicating that a clustered index be created for this key.

```
SELECT SerialNum, Assigned
FROM TestTable
WHERE TableKey = 1003
```

You determine the indexes that you need on a table based on the WHERE clauses of your queries. For example, if we determined that we had a lot of queries selecting data based on the SerialNum column then we would want to index that column to help optimize our queries. This becomes increasingly important as the table grows larger and larger.

The basic syntax to create an index is shown below:

```
CREATE UNIQUE CLUSTERED | NONCLUSTERED INDEX index_name
      ON table_name (column_name)
```

In the syntax above:

- The UNIQUE keyword specifies that the index being created will not allow duplicate keys in the index.
- The CLUSTERED and NONCLUSTERED keywords specify the type of index to be created; choose one of these options.
- INDEX is a keyword specifying that an index is to be created
- The *index_name* parameter specifies a name for the index
- The *table_name* parameter specifies the table on which we want the index created
- The *column_name* parameter specifies the column in the table to be indexed. You can create an index that contains multiple columns as the index key by specifying all columns that should make up the index key separated by commas

Let's take a look at how to create an index on our TestTable. The query below creates a unique non-clustered index on the column SerialNum. Since we have specified the UNIQUE keyword, every serial number that we enter into the SerialNum field must be unique. If we try to insert a duplicate serial number, we will get an error.

```
CREATE UNIQUE NONCLUSTERED INDEX IX_SerialNum
  ON TestTable (SerialNum)
```

We're calling the index `IX_SerialNum` so it's easy to remember – it has an `IX_` prefix, so it's an index, and it's called `SerialNum`, so it's on the `SerialNum` column.

If you suspect there could be duplicate data in a column that you want to index, do not specify the `UNIQUE` keyword. You will still gain the performance benefits by indexing a column with non-unique values.

> *You will gain performance benefits by indexing a column*

Now that we have indexed the `SerialNum` column in our table, we can execute a query such as the one below and the DBMS will use the index on the `SerialNum` column to select the data that we want. This query will execute more efficiently because we have added an index on it:

```
SELECT  TableKey, SerialNum
FROM  TestTable
WHERE  SerialNum = '12345'
```

Let's look at some statistics on this theory. Before the index was added, I executed this query in SQL Server's *Query Analyzer* and requested that the execution plan be shown for the query. The execution plan that was used is shown opposite:

Clustered Index Scan
Scanning a clustered index, entirely or only a range.

Physical operation:	Clustered Index Scan
Logical operation:	Clustered Index Scan
Row count:	1
Estimated row size:	36
I/O cost:	0.0375
CPU cost:	0.000084
Number of executes:	1
Cost:	0.037662(100%)
Subtree cost:	0.0376
Estimated row count:	1

Argument:
OBJECT:([WebDB].[dbo].[TestTable].[IX__PrimaryKey_TableKey]), WHERE:([TestTable].[SerialNum]=[@1])

Notice that SQL Server was scanning the clustered index when it executed this query. Remember that our clustered index on the table organizes data in our table based on the values in the `TableKey` column.

Next, the query we run after the index was created on the `SerialNum` column and the following execution plan was shown:

Notice that in this execution plan SQL Server is doing an index seek, which is much more efficient than scanning an entire index or table for the data that we need. You can also look at the *I/O Cost* and *Cost* rows to see some significant changes in the numbers, in this case a drop to 17%.

Index Seek
Scanning a particular range of rows from a non-clustered index.

Physical operation:	Index Seek
Logical operation:	Index Seek
Row count:	1
Estimated row size:	36
I/O cost:	0.00632
CPU cost:	0.000080
Number of executes:	1
Cost:	0.006408(100%)
Subtree cost:	0.00640
Estimated row count:	1

Argument:
OBJECT:([WebDB].[dbo].[TestTable].[IX_SerialNum]), SEEK:([TestTable].[SerialNum]=[@1]) ORDERED FORWARD

The bottom line here is that simply adding an index on the column helped improve the performance of our query. Don't get carried away with this thought, however, and start indexing lots and lots of columns in your tables.

> *for small tables there is probably little to be gained from indexing*

As we've seen in earlier chapters, there is an associated cost with having an index on a table and that is the maintenance of that index by the RDBMS. Whenever you insert or update a row of data in a table, the data in the column or columns that are indexed must be inserted or updated in the index. Likewise when you delete a row of data in your table, the associated index row must also be deleted. This is extra processing that takes place by the RDBMS to keep your indexes in sync with the data in the tables. In fact, for small tables there is probably little to be gained from indexing and their addition could even make things worse.

Indexing a few key columns that are used in the `WHERE` clauses of your queries, however, offsets the cost of having an index as it will improve the overall performance of your queries. This becomes increasingly important as your tables become larger and larger.

Reorganizing Data and Indexes

As your tables grow larger and larger, your indexes will become fragmented. This is because the data in the indexes are moved around to make room for new data that gets inserted and points to the data in your tables. As your indexes become fragmented it slows down their performance and ultimately your queries.

How fragmented indexes become over time depends on how your RDBMS manages their contents. Typically, the data in indexes will be stored in the order of the columns that are part of the index key. For example, suppose you have the following values in the `SerialNum` column in the `TestTable`. These values are stored in the `TestTable` in the following order (the order in which we inserted them):

```
12345
89654
741263985741
9632100145
852369741202587
```

However, the index stores these values in the index in this order because the column type is a `VARCHAR` data type:

```
12345
741263985741
852369741202587
89654
9632100145
```

As you can see, when you insert a new row of data in your table, the DBMS may have to shift the data around in your index to insert the new value in the appropriate place in the index. This also happens when you update an indexed column in your table.

All this moving data around in your index causes fragmentation, so reorganizing your index will help improve its performance. Suppose for example that you insert another row of data into your table and the `SerialNum` column contains a value of `88123`. This row of data is inserted at the end of your table but the index will have to insert an index row for this new value before `89654`.

Depending on the RDBMS that you are using, there may or may not be an index defined on the `TestTable` that we created earlier. Microsoft SQL Server created a primary key constraint on our table that functions like an index but is actually a constraint. Therefore we cannot drop an index on this table to reorganize it until we have a true index on this table to begin with. If you are using SQL Server, follow the next few steps to drop the constraint and to add a true index on the table. This will allow you to follow along in the rest of this section.

Execute the `sp_help` stored procedure to determine the constraint name on the `TestTable` using the following query:

```
sp_help TestTable
```

Once you have the constraint name, use it in the query below substituting your constraint name for `PK__TestTable__00551192`. This query will drop the constraint that was created by SQL Server.

```
ALTER TABLE TestTable
DROP CONSTRAINT PK__TestTable__00551192
```

Next, we want to add a unique clustered index on our table. Execute the query shown below to add the index to the `TestTable` table.

```
CREATE UNIQUE CLUSTERED INDEX IX_PrimaryKey_TableKey
ON TestTable (TableKey)
```

Dropping an Index

Your RDBMS may have specific built-in functions to help solve this problem and help you reorganize your indexes to maximize performance. You can do this by dropping them and recreating them, which is what we will discuss in this section.

The syntax to drop an index on a table is very simple and is shown below.

```
DROP INDEX table.index
```

In the syntax above, you specify the keywords `DROP INDEX` and the name of the table where the index resides followed by a period and the index name.

For example, to drop the unique clustered index on the `TestTable` you would use a query similar to the one shown below substituting the appropriate index name:

```
DROP INDEX TestTable.IX_PrimaryKey_TableKey
```

Once the index is dropped, you recreate the index using the `CREATE INDEX` SQL statement, as shown below.

```
CREATE UNIQUE CLUSTERED INDEX IX_PrimaryKey_TableKey
ON TestTable (TableKey)
```

Dropping and recreating the index causes the index to be rebuilt and therefore the index will be reorganized with no fragmentation.

> *a clustered index stores the data in the table in the order of the index key*

Dropping and recreating an index can be costly in terms of processing and time. When an index is dropped and recreated, the RDBMS puts on a lock and no other processing can occur against that table until the index has been rebuilt.

Remember that a clustered index actually stores the data in the table in the order of the index key defined, so dropping and recreating a clustered index will take longer to perform than dropping and recreating a non-clustered index.

In order to load lots of data into a table, it is sometimes more efficient to first drop the index, insert the data and then recreate the index on the table. The reason behind this is that every time you insert a row of data, the appropriate index key is inserted into the index. Dropping the index first allows for faster loading of the table; creating the index after the load completes then ensures that your index is not fragmented.

Drop Existing

You can increase the performance of reorganizing your indexes, both clustered and non-clustered, by not using the `DROP INDEX` statement. You may be wondering how this is possible. The answer to that question lies in the `CREATE INDEX` statement. The `CREATE INDEX` syntax contains a special keyword called `DROP_EXISTING`. This keyword causes an existing index to be dropped and rebuilt all in one statement.

There are a few restrictions here that you should be aware of. First, the index name that you are using in the `CREATE INDEX` statement must be the same as that of an existing index. Secondly the index must not be corrupted. If it is, you must use the `DROP INDEX` statement to drop the index and then use the `CREATE INDEX` statement without the `DROP_EXISTING` keyword to recreate it.

You should also reorganize your non-clustered indexes after reorganizing a clustered index, so that the non-clustered indexes can pick up the new clustering keys created by the clustered index.

The query shown below uses the same syntax for creating a new index but specifies the name of an existing index along with the `DROP_EXISTING` keyword.

```
CREATE UNIQUE CLUSTERED INDEX IX_PrimaryKey_TableKey
ON TestTable (TableKey)
WITH DROP_EXISTING
```

This query causes our clustered index to be dropped and recreated all in step. As you can see, this is not only more efficient from a system standpoint, but more efficient for us as we only have to execute one query.

Which method you use to reorganize your indexes depends on whether your DBMS supports the `DROP_EXISTING` keyword. If it does, then the `CREATE INDEX` statement is the way to go as it involves less code and is more efficient to process. Otherwise, you must use the `DROP INDEX` statement followed by the `CREATE INDEX`.

Building Scripts for Production Implementation

When you are ready to deploy your development database to the test, QA, or production environment, how you proceed will be determined by your company's policy for implementing databases in these environments. For example, one company may allow you to simply copy the database to the new environment and the DBA may simply delete all existing data in the tables that need to be cleared. Another company may insist on SQL scripts to recreate your database in these environments. It is the latter policy, being the more sensible and professional approach that we want to talk about and explore in this section.

When we talk about scripts, we are talking about the SQL statements used to create your tables, constraints, indexes, stored procedures, views, and all the other objects in your database. How you script your database for implementation in the test, QA, or production environment will depend on the RDBMS that you are using and what tools it provides.

For example, Microsoft SQL Server provides several visual tools that allow you to create scripts to build your database. You could use *Enterprise Manager*, the primary administration tool for SQL Server, which allows you to script the various objects in your database either as individual scripts (one script per object) or as one large script file that contains all the individual scripts to recreate your entire database. SQL Server also has *Query Analyzer*, a graphical tool that allows you to create and run SQL commands and scripts against your database, and has built-in menu items that allow you to create scripts of your database.

Once you have generated the scripts to recreate your database, you can pass them on to the DBA to implement in the test or production environment.

Scripts to Populate Tables

As we mentioned above, the scripts that you generate will be used to create your tables, constraints, indexes, stored procedures, views, and all the other objects in your database. There may, however, be some tables in your database that contain common data that is required by your application. This data does not change or get updated.

Let's assume that your company processes payrolls for other companies. You may have a table in your database that contains payroll types and codes that are used by the processes in your application. These values are required and don't get updated by your application. This table would need to have its data populated before your application can run.

Along with the scripts to build your database, you will also need a script to populate this type of table. That being said, let's take a look at how you can generate these scripts using normal SQL statements.

When we started talking about modifying tables, we provided a script to insert data into the `TestTable`: this script is shown below to refresh your memory. We don't want to code a script like this by hand, especially for tables that contain lots of columns or rows of data.

```
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1000,'12345','0',NULL)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
```

```
  VALUES(1001,'89654','1','Assigned on 28/9/2002')
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1002,'741263985741','1','Nothing new to report')
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1003,'9632100145','0',NULL)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes)
  VALUES(1004,'8523697412025 87','0','Unassigned on 28/9/2002')
```

What we want to do here is to create our script by selecting the data from our existing tables that need to be populated in the new environments.

> *We will use some built-in functions in SQL Server to massage the data returned*

The SQL statement that we will use to accomplish this is a `SELECT` statement that uses a combination of column names, string constants, and concatenation characters. We will also use some built-in functions in SQL Server to massage the data returned. You will need to consult the documentation for your RDBMS to determine what characters are valid for including string constants in your SQL statement, and what character should be used for concatenating data. You will also need to determine if there are the equivalent functions in your RDBMS to massage the data.

Our SQL statement to extract the data into a script that can be executed in another database looks like this:

```
SET NOCOUNT ON
SELECT    'INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)' +
          CHAR(13) + CHAR(9) + 'VALUES(' +
          CAST(TableKey AS VARCHAR(10)) +
          ',' + QUOTENAME(SerialNum,'''') + ',' +
          CAST(Assigned AS CHAR(1)) + ',' +
          COALESCE(QUOTENAME(CAST(Notes AS VARCHAR(1000)),''''),'NULL') +
          ',' + CAST(Price AS VARCHAR(11)) + ')'
FROM TestTable
```

The results that are returned look like this.

```
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1000,'12345',0,NULL,0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1001,'89654',1,'Assigned on 28/9/2002',0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1002,'741263985741',1,'Nothing new to report',0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1003,'9632100145',0,NULL,0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1004,'8523697412025 87',0,'Unassigned on 28/9/2002',0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1005,'88123',0,'Unassigned on 28/9/2002',0.00)
```

Let's examine our SQL statement line by line so you know what's going on. The first line of our SQL statement turns off the "rows affected" message that you normally see after executing a `SELECT` statement. This statement really isn't necessary as you could just select all data returned and then delete the "rows affected" message from the data selected:

```
SET NOCOUNT ON
```

Our `SELECT` statement starts with a string constant. This string constant, which is self-explanatory, will become the first line of data in our output returned from this query:

```
SELECT    'INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)' +
```

Notice the plus (+) sign at the end of this line: this is the concatenation character in SQL Server.

Next we specify the `CHAR` string function, which converts an ASCII character code into a character. The ASCII value of `13` represents a carriage return and will cause our output to be on two separate lines. The ASCII value of `9` represents a tab and will tab the second line of our output one tab space. The last part of this line of code is another string constant and this string will be placed on the second line of our output.

Using a carriage return and tab character just makes our output easier to read as we will have the `INSERT` statement followed by the columns that we want to insert data into on the first line of our output and the values to be inserted on the second line of our output.

```
    CHAR(13) + CHAR(9) + 'VALUES(' +
```

The plus sign (+) in SQL Server is not only used as a concatenation character but is also used in arithmetic operations to add two numeric values together. When SQL Server encounters a column of data that has a numeric data type it tries to add it to the value in the SQL statement, which causes an error because you cannot add a numeric value to a string value.

To solve this issue, we use the `CAST` function, which will cast a numeric data type to a character data type. In this next statement we are casting the value in the `TableKey` column to a `VARCHAR(10)` value. Since an `INTEGER` data type can contain up to 10 digits, we want to ensure that the data type that we are casting to is large enough to handle the maximum value in this column.

```
    CAST(TableKey AS VARCHAR(10)) +
```

The `SerialNum` column in our table is character data. In SQL Server, we use a single quote delimiter around our character data when inserting it into a table or when used in a query. The `QUOTENAME` function will add a delimiter that we choose around our character data in our output.

> *we have chosen to use a single quote for our delimiter*

The `QUOTENAME` function, specific to SQL Server, has two parameters: the character string to be quoted, and the quote character to use. The column name is used as the character string to be quoted, and we have chosen to use a single quote for our delimiter. Since a single quote is the standard delimiter in SQL Server we must specify two consecutive single quotes in order to have it recognize that a single quote is what we want to use. If we had chosen to use a double quote as our delimiter we could simply have specified a single quote followed by a double quote followed by a single quote as demonstrated in this line of code:

```
QUOTENAME(SerialNum,'"').
```

We extract the data from the SerialNum column, have it quoted for us and then specify a string constant that will place a comma in our output. We then specify another string constant that will place a comma in our output after the data from the SerialNum column:

```
',' + QUOTENAME(SerialNum,'''') + ',' +
```

In this next statement we are casting the value in the Assigned column to a CHAR(1) value. Remember that this column has a data type of BIT, which is represented by the numbers 0 or 1.

When using the CAST function you need to take into account the maximum number of digits that can be returned from the numeric data type and make the character data type that you are casting to large enough to hold the maximum value. We'll see more on this in a minute.

We do not use the QUOTENAME function here because this is a numeric data type and we don't want the output generated here to contain quotes. This ensures that our output is built correctly with all string values quoted and all numeric values not quoted.

The last part of this line of code adds another comma to our output as we have specified a comma in a string constant:

```
CAST(Assigned AS CHAR(1)) + ',' +
```

The next line of code presents a whole unique set of challenges and you will want to pay close attention here. First, the Notes column in our table is a TEXT data type, which means that it can hold up to 2,147,483,647 characters. This is more data than would normally fit into a regular VARCHAR data type and SQL Server requires us to convert this data type to a VARCHAR data type in our SELECT statement because we are going to quote the name. Normally you would not have a data type like this in a table that contains data needed by your application. You would typically have smaller data types that provide codes and keys, which are relatively small.

Before we quote the output from the Notes column using the QUOTENAME function, we want to cast the data type of TEXT to a data type of VARCHAR(1000). Keep in mind that if you use a TEXT data type in your table and you do use this type of conversion that it is possible that you may lose some data.

Next, because this column in our table does contain null values, we need to handle them correctly in our output. The COALESCE function will return the first non-null value from the expressions supplied. So in our code below, if the Notes column contains some data, the COALESCE function will return the quoted data.

If the Notes column does contain a null value, then the COALESCE function will return the string NULL which will not be quoted in our output. This is the correct behavior that we want, as NULL is a valid expression that can be used in an INSERT statement to insert a null value into a column:

```
COALESCE(QUOTENAME(CAST(Notes AS VARCHAR(1000)),''''),'NULL') +
```

The last line of code in our SELECT statement converts the data in the Price column to a data type of VARCHAR(9). Remember that the Price column in our table contains a data type of SMALLMONEY, which is considered numeric data. We need to convert this data type to a character data type so SQL Server does not try to add it to the string, which would cause an error.

The SMALLMONEY data type can return a maximum value of 214,748.3647. When we calculate the number of characters needed in our VARCHAR data type we need to take into account all numeric digits to the left of the decimal, the decimal itself, and all digits to the right of the decimal. We do not need to account for the commas as they are not stored or returned in a SELECT statement.

Since we want the returned value to be represented as numeric data, we do not quote the value returned.

The final string constant in this SELECT statement provides the ending parentheses around our data to be inserted:

```
    ',' + CAST(Price AS VARCHAR(11)) + ')'
```

The last line of code is the table that we want to select data from:

```
FROM  TestTable
```

Having just gone through all the code in detail, let's take another look at the results that get returned (shown again below).

Notice the values that are going to be inserted into the table. The character data is properly quoted and our numeric data does not contain quotes. Where the Notes column contained a null value we have specified the NULL expression, which will cause a null value to be inserted into the new table where this script is run.

```
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1000,'12345',0,NULL,0.00)


INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1001,'89654',1,'Assigned on 28/9/2002',0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1002,'741263985741',1,'Nothing new to report',0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1003,'9632100145',0,NULL,0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)
  VALUES(1004,'852369741202587',0,'Unassigned on 28/9/2002',0.00)
INSERT INTO TestTable (TableKey,SerialNum,Assigned,Notes,Price)


  VALUES(1005,'88123',0,'Unassigned on 28/9/2002',0.00)
```

As you can see, this type of script can be a time saver and a means of populating the same table in another database. This type of script works particularly well when you have a database application that you sell to customers and the customer is responsible for running your scripts to build and populate their database. The script is run and the tables that need to be populated are populated with the data that your application needs.

When to Perform Database Maintenance

When performing database maintenance tasks, one very important thing that must be given a lot of thought to is **when** to perform them. If you are running a fairly high-traffic database-driven online shopping site for example, you really don't want to even think about changing indexes and updating data types at peak usage times. It is better to choose a time when the least possible number of people are using the site. The best way to do this is as follows:

> *choose a time when the least possible number of people are using the site*

1. First, perform some simple usage analysis on your site to work out what time of day your site gets the least amount of traffic coming to it. If you are not sure how to do this, check out a suitable resource, such as *Practical Web Traffic Analysis* (Peter Fletcher, glasshaus, ISBN: 1904151183.) This will ensure that your site suffers the least amount of downtime possible. Downtime means lost business in the case of business sites but, whatever the case, it's annoying for the site users, so we want to reduce it to a minimum.
2. Next we need to prepare a "work in progress – please try again later" (or something similar) message to replace your homepage while the work is going on. Try to provide as much information as possible on this page, such as when the site will be back up.
3. Perform the maintenance work. When you have finished, make sure you test it to see if it works before uploading the updated code to the live site, otherwise you could run into untold problems.
4. After you have checked your work and it comes up OK, put your site back online. A nice touch is to provide a feedback address (if you haven't done so already), for your users to report any problems they find with the updated site, or to suggest improvements.

Summary

In this chapter we have taken a look at quite a few database maintenance tasks that you as a developer may encounter and may be required to perform.

We started out by discussing how to perform a complete backup of your database and transaction logs followed by how to perform a simple and full recovery of your database. You should appreciate the importance of the proper backups and now know how to properly perform a simple recovery to restore your database to the point of your last backup. You should also know how to recover your database using a full recovery model to restore it to a given point in time using your database and transaction log backups.

You may be required to modify the table structure of your tables from time to time and you should now be aware of how to alter existing columns not only to change their data types but also how to change a column from not allowing null values to allowing null values.

Along with this you should now also know how easy it is to add new columns and drop existing columns. Just as a reminder, be cautious about dropping existing columns. Ensure that another table does not use the columns that you want to drop; if they are and have constraints defined on them, you will need to drop the constraints before you drop the column.

We moved on to talk about indexes and how to add indexes to your tables to increase the performance of your queries and the overall performance of your database.

Along with increasing the performance of your database, we examined how to reorganize the data in your indexes to increase their performance. You should realize that reorganizing the data in your indexes from time to time is required to maintain optimal performance of your database.

Finally, we talked about generating scripts for your database to be used by a DBA or customer to recreate your database. Specifically, we examined in detail how to select existing data from a table to create a script that can be used to populate a table with data. This becomes very important when you need to populate a new database table with values that are required by your application.

In summary, you should now know how to:

- Perform a complete backup of your database
- Perform a backup of your transaction log
- Restore your database using a simple and full recovery model
- Alter existing columns, add new columns, and drop existing columns from a table
- Add a new index and reorganize existing indexes
- Create a script that can be run to insert values into a new table

Appendix

- MySQL 3.23
- Oracle 9*i*
- SQL Server 2000

Appendix
Available Database Platforms

Throughout the book, we've made reference to the array of database platforms available, and how some of them differ from the SQL-92 standard. In this appendix, we take a look at some of the key features of three of the more popular relational database management systems available today:

- MySQL
- Oracle
- SQL Server

For each RDBMS, we'll look at its availability, the operating systems it supports, and programming languages that offer support. We'll examine some of the tools available, and perhaps most importantly, we'll see the data types supported by each product, and how they relate to SQL-92 data types.

Let's start with the popular free database platform, MySQL.

MySQL

MySQL is, according to its founders, "The World's Most Popular Open Source Database". The MySQL web site (*http://www.mysql.com/*) serves over 27,000 downloads of the MySQL server per day. Some of the users of MySQL include NASA, Yahoo, *slashdot.org*, *freshmeat.net*, and Compaq. Novell Netware version 6 and above now ships with MySQL included.

MySQL is free, fast, and reliable, and is easy to install, configure, and manage. Add to that the fact that it runs on a wide variety of platforms and has a wealth of APIs for different client languages and you begin to see why it is so popular. Unfortunately, MySQL doesn't support ACID transactions at present, so could be considered lightweight. Look for PostgreSQL if you want open source with transactions.

In this section we will take a look at some of the things that make MySQL so popular – we'll look at what platforms it runs on, what APIs are available for it and some of its key features. To be fair we will also look at some of the drawbacks of using the MySQL database.

Supported Operating Systems

MySQL supports the following operating systems as platforms, either natively or emulated:

AIX	OpenBSD
Amiga	OS/2 Warp
BSDI	SGI Irix
DEC Unix	Solaris
FreeBSD	SunOS
HP-UX	Caldera (SCO) OpenServer
Linux	Caldera (SCO) UnixWare
Mac OS X Server	Tru64 Unix
NetBSD	Windows 9x, Me
Novell Netware	NT, 2000, and XP

For brevity's sake we have not listed the specifics for each operating system, but these are clearly outlined in the MySQL manual, along with notes for each operating system.

Supported APIs

An API, or Application Programming Interface, enables a programmer to connect to and interact with a MySQL database. The functions and methods in the API are used for manipulating the database and retrieving information from it. If you are using MySQL as the back-end database for a web site, you will need to use one of the APIs in order to get and send data between your web pages and the database.

The C API is used as the basis for most of the other language APIs. The API is distributed with MySQL as the `libmysqlclient` library and was developed by the MySQL developers. C++ programmers can use the MySQL++ API, available at *http://www.mysql.com/downloads/api-mysql++.html*.

If you are using a language that does not provide native MySQL support, or does not have an API or wrapper that you can use to interface with MySQL, you can still connect to it as long as your language has ODBC or JDBC support. You can get both the ODBC (MyODBC) and JDBC (Connector/J) drivers from the MySQL web site. MyODBC is available at *http://www.mysql.com/downloads/api-myodbc.html*, and Connector/J can be found at *http://www.mysql.com/downloads/api-jdbc.html*.

PHP includes functions for accessing MySQL. The MySQL support is included in the Windows binary installation of PHP, and most Linux users will find it built-in as well, but Unix users will need to compile the support in. Most, if not all, ISPs that allow you to host PHP pages will have MySQL included in their PHP installation.

A variety of other languages have user-contributed APIs or wrappers that you can use to connect to MySQL:

- **Perl**. The Perl API is simply a database driver (DBD) for use with the Perl database interface (DBI), and is available from *http://www.mysql.com/downloads/api-dbi.html*
- **Eiffell**
- **.NET**. The .NET APIs are available from *http://www.mysql.com/downloads/api-dotnet.html*
- **Python**. The Python APIs are available from *http://www.mysql.com/downloads/api-python.html*
- **Ruby**. The Ruby APIs are available from *http://www.mysql.com/downloads/api-ruby.html*
- **Tcl**

Other API's and wrappers can be found under the user-contributed downloads section of the MySQL web site, *http://www.mysql.com/doc/en/SEC668.html*.

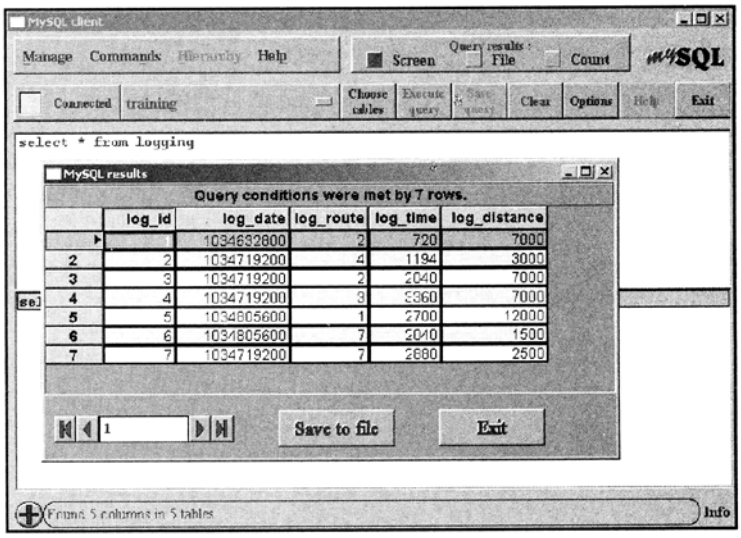
Available Tools

There are a number of tools available for administering your MySQL databases. All MySQL installations include the command-line `mysql` monitor program, which lives in the `mysql/bin` folder of your installation. For those of you who would prefer a graphical interface, some of the available options are listed below.

The first two are developed by MySQL AB and are available from the *http://www.mysql.com* web site.

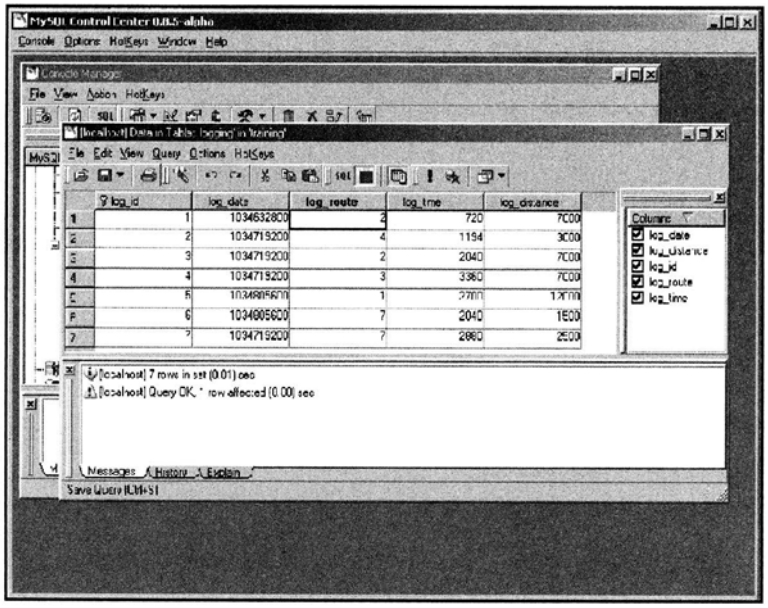
MySQL GUI

MySQL GUI is a fairly basic interface. Once connected you can select the database you want to work on and then enter your queries in the text area below. Query results are displayed in a pop-up window:



The simple MySQL GUI

MySQL Control Center

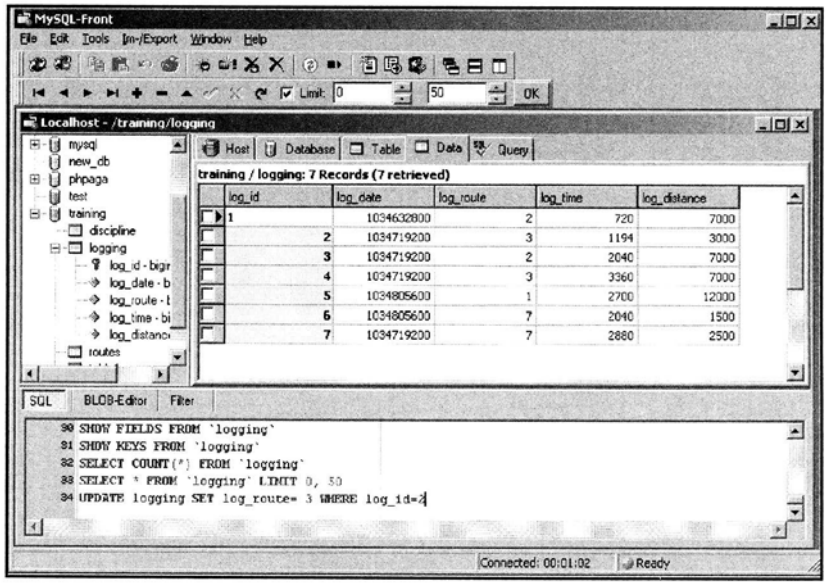


MySQL Control Center

MySQL CC is the new GUI client from MySQL AB. It is platform-independent, but currently is only available for Windows and Linux. Other platforms including Mac OS X are planned. The client has a much cleaner look than MySQL GUI and has many more features.

The next two options are third-party products, but are available for free.

MySQL Front

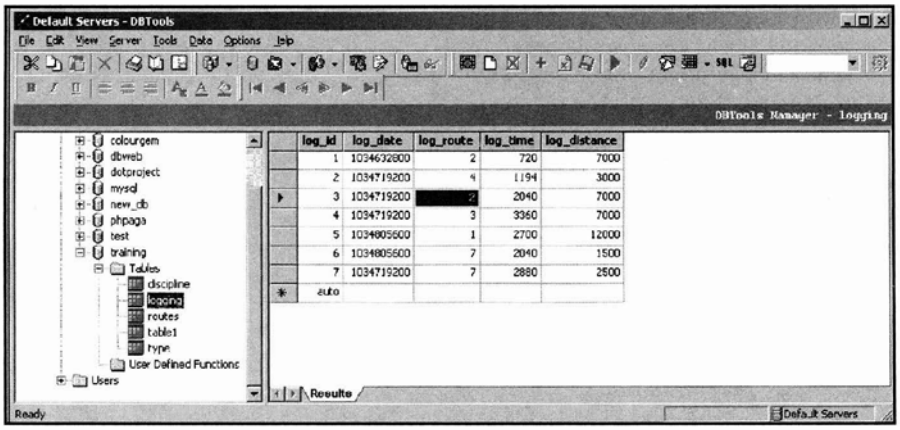


MySQL-Front

MySQL Front is a popular client for MySQL, but its development has been discontinued. The latest version is still available for download from *http://mysqlfront.venturemedia.de/*.

DBTools Manager

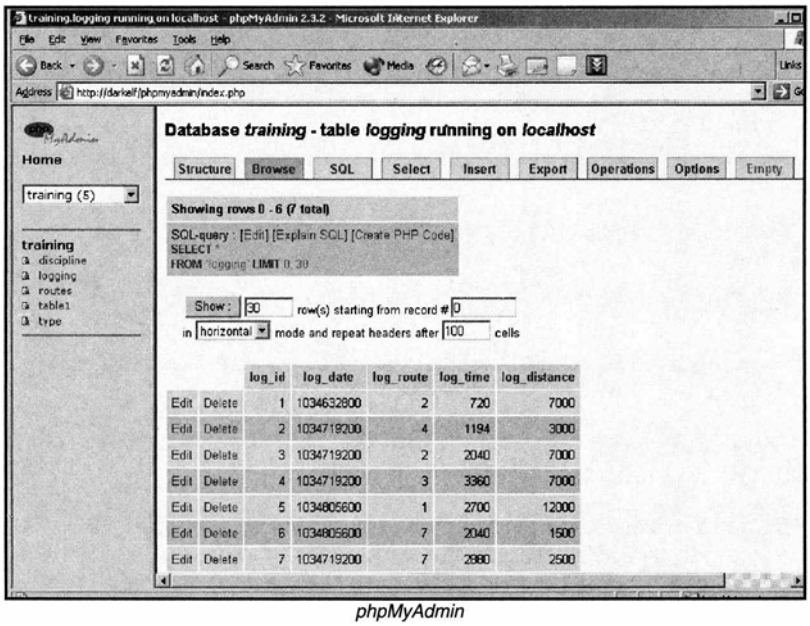
The latest version of DBTools Manager, available from *http://www.dbtools.com.br/EN/*, now also supports the PostgreSQL database.



DBTools Manager

phpMyAdmin

Perhaps the most popular of all MySQL front ends is the phpMyAdmin package, available from *http://www.phpwizard.net/projects/phpMyAdmin/*. It is entirely web-based and uses the PHP scripting language, so you can use it on any machine that can run a PHP-enabled web server. It allows you to edit, drop, or create tables, add and modify data, manage keys, and import and export data.

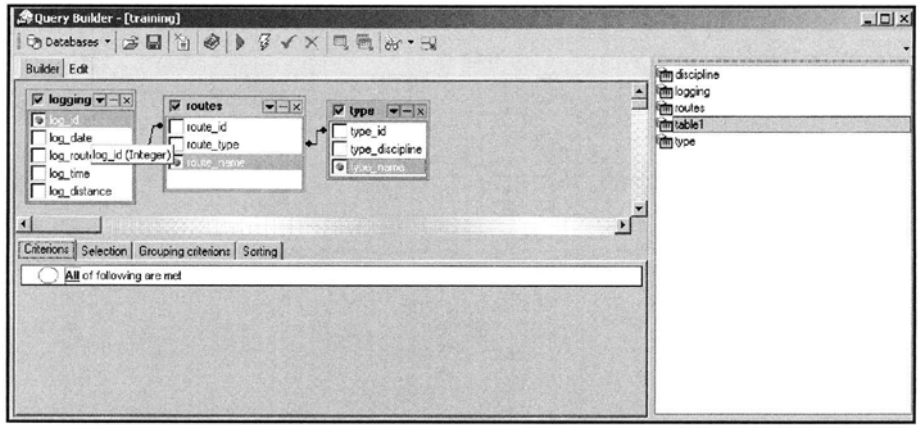


phpMyAdmin

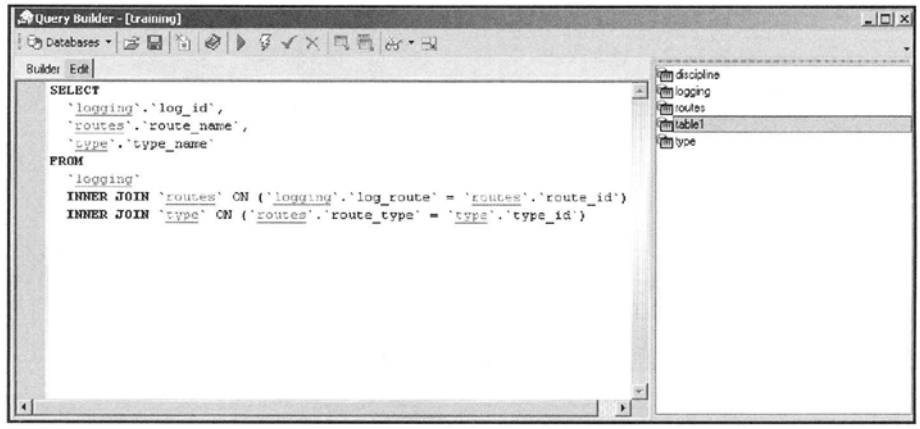
The last option that we will look at is EMS MySQL Manager.

MySQL Manager

MySQL Manager is a commercial product, but you can get a 30-day evaluation from the web site at *http://ems-hitech.com/mymanager/*. Besides the usual features that you will find in the other programs mentioned here, MySQL Manager has a nifty tool called the Query Builder. In the Query Builder you can visually create relationships between your tables and the Query Builder will then generate the SQL query for you:



Query Builder creating queries visually



The resulting SQL

Support for SQL-92

MySQL 3.23.52 was the latest stable release at the time of writing, although MySQL 4.0 had recently entered public beta testing. MySQL offers the following support for the SQL-92 standard, including full support for `GROUP BY` and `ORDER BY`, and full operator and function support in `SELECT` and `WHERE` clauses.

The following tables discuss the available data types in MySQL, and how they compare to the SQL-92 standard. Let's start with character data:

SQL-92	MySQL	Comments
CHAR(n)	CHAR	Both CHAR and VARCHAR support a range of 1 to 255 characters
VARCHAR(n)	VARCHAR	
	TINYTEXT	Maximum length of 255 characters
	TEXT	Maximum length of 65535 characters
	MEDIUMTEXT	Maximum length of 16777215 characters
	LONGTEXT	Maximum length of 4294967295 characters

Next we have numeric data:

SQL-92	MySQL	Comments
	TINYINT	Signed range is -128 to 127. Unsigned is 0 to 255
SMALLINT	SMALLINT	Signed range is -32768 to 32767, unsigned range 0 to 65535
	MEDIUMINT	Signed range -8388608 to 8388607. Unsigned 0 to 16777215
INTEGER	INT or INTEGER	Signed ranges from -2147483648 to 2147483647. Unsigned ranges from 0 to 4294967295
	BIGINT	Signed range is -9223372036854775808 to 9223372036854775807. Unsigned is from 0 to 18446744073709551615
FLOAT(p)	FLOAT	Single-precision floating-point number, is always signed. Allowable values are -3.402823466E+38 to -1.175494351E-38, 0, and 1.175494351E-38 to 3.402823466E+38
DOUBLE PRECISION REAL	DOUBLE	Double-precision floating-point number, also is always signed. Allowable values are -1.7976931348623157E+308 to -2.2250738585072014E-308, 0, and 2.2250738585072014E-308 to 1.7976931348623157E+308
DECIMAL	DECIMAL	Unpacked floating-point number. Is always signed. The number is stored as a string, using one character for each digit

The next table looks at date and time data:

SQL-92	MySQL	Comments
DATE	DATE	Dates are stored in 'YYYY-MM-DD' format. Supported range is '1000-01-01' to '9999-12-31'
TIME	TIME	Times are stored in 'HH:MM:SS' format. Supported range is '-838:59:59' to '838:59:59'
TIMESTAMP	DATETIME	A date/time combination. Format is 'YYYY-MM-DD HH:MM:SS'. Supported range is '1000-01-01 00:00:00' to '9999-12-31 23:59:59'
	TIMESTAMP	Supported range is '1970-01-01 00:00:00' to sometime in 2037. When creating the field an optional width for the field can be given – 6, 8, 12 or 14. Depending on the width the field will display as YYMMDD, YYYYMMDD, YYMMDDHHMMSS, or YYYYMMDDHHMMSS respectively
	YEAR	Can be in either 4- or 2-digit format. Allowable values are 1901 to 2155 for 4-digit format, and 1970 to 2069 in 2-digit format (70 – 69)

Finally, we look at data types for storing raw binary data:

SQL-92	MySQL	Comments
BIT(n)	TINYBLOB	Maximum length of 255 characters
BIT VARYING(n)	BLOB	Maximum length of 65535 characters
	MEDIUMBLOB	Maximum length of 16777215 characters
	LONGBLOB	Maximum length of 4294967295 characters

MySQL also supports a number of data types not covered by the SQL-92 standard:

Miscellaneous	Comments
ENUM	A string object that can only have one of a list of possible values. The list is specified and can have a maximum of 65535 distinct values.
SET	A string object that can have 0 or more values, each value chosen from a list of a maximum of 64 possible values.

Version 4.0 of the database server, in public beta testing at the time of writing, will have the following additional features:

- Increased speed
- Transactions
- Row-level locking
- Foreign key constraints
- New character sets

Current Drawbacks

Version 4.0 of the MySQL database server is not yet ready for a production environment, and as such, you would not have transactions and row-level locking available as features in your production environment. Without these features, MySQL fails the ACID test for relational database management systems.

Other features missing from the current production version of MySQL are:

- Sub-queries
- Stored procedures
- Triggers

Oracle

Oracle was the first commercial RDBMS product to be released, in 1979. The first copy of Oracle was sold to Wright-Patterson Air Force Base; the CIA was another early customer. It scales all the way from individual PCs up to hundred-machine distributed clusters.

Oracle has almost always been the number one RDBMS product since it was first released. Depending on how you measure it, Oracle may or may not still be top of the heap, but IBM's DB2 and Microsoft's SQL Server are definitely snapping at its heels. Other RDBMS products have either barely appeared on the radar (like Sybase) or have been acquired (like Informix, which was acquired by IBM).

The current version at the time of writing is Oracle9*i* Release 2 (the "*i*" stands for "Internet"), which was released mid-2002. Oracle9*i* is a large and extremely complex RDBMS product that can be used for everything from web database systems, to data warehouses, to video-on-demand systems. It can take years to fully learn, but is also the database of choice for large and highly reliable systems, especially on UNIX and Linux. This is partly due to its ability to scale into hundreds of megabytes per table. The RDBMS is only the tip of the iceberg though; Oracle also sells business intelligence tools, query and analysis tools, data-mining products, application development tools, and an application server that integrates closely with the RDBMS. The product is continually growing, with version 10*i* due out some time in 2003.

Let's take a quick whistle-stop tour of Oracle9*i*. We'll briefly look at some of the tools available, how Oracle's SQL dialect differs from SQL-92, the physical side of Oracle, and Oracle's support for web development.

Supported Operating Systems

Oracle runs on a multitude of different platforms, everything from Windows XP to mainframes to Linux (they even released in September 2002 a beta version for Apple's Mac OS X). Oracle9*i* is currently available for the following operating systems:

AIX	OS/390 (IBM mainframes)
HP-UX (HP 9000 systems)	Solaris (32- & 64-bit)
Linux for Intel (effectively Red Hat)	Tru64 UNIX (Compaq)
Linux for IA64	Windows NT/2000/XP
Linux for IBM mainframes	64-bit Windows (in development)
Mac OS X v10.2 Jaguar	

Oracle also comes in three editions, which cater for different needs:

- Personal Edition, which mainly is intended for use by single users on individual workstations for development and deployment purposes. It can, for example, be installed and used on a PC running Windows or Red Hat Linux (although the system requirements are somewhat hefty).

- Standard Edition, which is intended for medium-scale deployments, such as workgroup or department-level databases, or smaller, less demanding web applications. It includes the core DBMS, management tools, analytic functions, and a handful of specialized add-ons that support things like full-text searching.
- Enterprise Edition, which is intended for high-end applications such as high-volume transaction processing systems, data warehouses, and large-scale web applications. It comes with "the works": everything from Standard Edition, plus support for hardware clusters, table partitioning, advanced security features, additional management tools, OLAP and data-mining tools, and support for spatial data.

Supported APIs

Oracle supports a wide range of APIs and languages for application development. First, Oracle provides a built-in proprietary programming language called PL/SQL, which is used to create triggers and stored procedures. PL/SQL is derived from Ada and is very similar to the procedural language defined in the SQL:1999 standard.

Oracle also supports embedding SQL directly into C, C++, COBOL, and Java source code. The first three languages are supported by the Pro* series of embedded SQL compilers (Pro*C/C++, Pro*COBOL), while the SQLJ tool provides similar support for Java. (SQLJ is also an ANSI standard specification for embedding SQL into Java source code.)

Oracle9*i* in particular has very strong support for Java, with built-in Java virtual machine and standard libraries, plus the ability to store Java code in the database. You can also access Oracle databases using both JDBC and ODBC, and drivers for these APIs can be downloaded from Oracle's web site.

Oracle provides two native APIs called the Oracle Call Interface (OCI) and the Oracle C++ Interface (OCCI). OCI provides a dynamic runtime library that can be used with most programming languages. OCCI is an extended version of OCI that takes advantage of various C++ features.

Finally, Oracle provides Oracle Objects for OLE (OO4O), which provides support for any programming or scripting language that supports Microsoft COM Automation and ActiveX technologies. This includes languages like Visual Basic, Visual C++, VBScript, JavaScript, and so on.

Tools for Oracle

As you would expect, Oracle comes with many basic tools as standard, including graphical database administration and manipulation tools, and command-line SQL tools. If you want to go beyond these, however, you will either need to buy add-on products from Oracle, or look for third-party software.

Oracle provides several add-on tools and products for Oracle9*i*. The list below includes most of the major products, but be warned that Oracle has somewhat of a tendency to change the names of their products at the drop of a hat. See *http://www.oracle.com/* for the most up-to-date information on these products, but beware: it can often be surprisingly difficult to find useful information on their web site!

- **Oracle9*i* Designer** is an entity-relationship modeling tool. You can design and build databases for many different RDBMS products. It integrates well with both Developer and JDeveloper (see below) for building complete application systems.

- **Oracle9*i* Developer** provides tools for forms development, reporting on data, and querying, and enables applications to be deployed across multiple platforms, including the Web.
- **Oracle9*i* JDeveloper** is an integrated Java 2 Enterprise Edition (J2EE) development environment. It provides UML modeling support and can be used to build three-tier web-based applications.
- **Oracle9*i* Repository** acts as a single source of metadata for both Oracle Designer and Oracle Developer. The kinds of things that can be managed by Repository include entity-relationship and UML models, database objects, forms, Java classes, XML files, and so on.
- **Oracle9*i* Warehouse Builder** provides support for building any kind of data warehouses.
- **Oracle9*i* Application Server** is a J2EE application server that closely integrates with the Oracle9i RDBMS. It's intended for deploying Java-based Internet applications, including J2EE applications and web services.
- **XML Developer's Kits** for building XML-based applications using Java, JavaBeans, C, C++, or PL/SQL.

An entire third-party industry has also grown up around Oracle. Quest Software (see *http://www.quest.com/*) in particular produces two excellent Oracle tools:

- **TOAD (Tool for Oracle Application Developers)**, which is a Windows-based integrated development environment for Oracle SQL.
- **Spotlight**, which is a database performance-monitoring tool for Oracle, DB2, and SQL Server. It uses highly effective visualizations to quickly highlight problems in the database.

Support for SQL-92

Oracle9*i* claims to fully or partially support all core SQL:1999 features. Oracle has been compliant with SQL-92 Entry SQL since at least version 8. Oracle does however extend the base standard considerably in many areas, some of which we'll look at now.

Data Types

Oracle supports all of the standard SQL data types, but also introduces many data types of its own. The data types supported by Oracle9*i* are shown in the tables below. Again, we'll start with character data:

SQL-92	Oracle9*i*	Comments
`CHAR(n)`	`CHAR(n) [BYTE | CHAR]` `NCHAR(n)`	Fixed-length character string, exactly *n* bytes (`BYTE`) or characters (`CHAR`) long; $1 \le n \le 2000$ (default 1); `NCHAR` uses Unicode
`VARCHAR(n)`	`VARCHAR2(n) [BYTE | CHAR]` `NVARCHAR2(n)`	Variable-length character string, up to *n* bytes or characters; $1 \le n \le 4000$; `NVARCHAR2` uses Unicode
	`LONG`	Variable-length character data up to 2GB
	`CLOB` `NCLOB`	Character large object up to 4GB (single-byte characters for `CLOB`, Unicode characters for `NCLOB`)

Next we have numeric data:

SQL-92	Oracle9*i*	Comments
INTEGER SMALLINT	INTEGER SMALLINT	Automatically mapped to NUMBER(38) (see below)
NUMERIC(p,s) DECIMAL(p,s)	NUMERIC(p,s) DECIMAL(p,s)	Automatically mapped to NUMBER(p,s) (see below)
FLOAT(p) DOUBLE PRECISION REAL	FLOAT(p)	Automatically mapped to NUMBER (see below)
	NUMBER(p,s)	Number with precision p and scale s; $1 \le p \le 38$, $-84 \le s \le 127$

The next table looks at date and time data:

SQL-92	Oracle9*i*	Comments
DATE	DATE	Dates ranging from January 1, 4712 BC to December 31, 9999 AD
TIME TIMESTAMP	TIMESTAMP	Date (year/month/day) plus time (hours/minutes/seconds); can include time zone
INTERVAL	INTERVAL	Time interval

Finally we look at data types for storing raw binary data:

SQL-92	Oracle9*i*	Comments
BIT(n) BIT VARYING(n)	RAW(n)	Variable-length raw binary data, maximum n bytes; $1 \le n \le 2000$
	LONG RAW	Variable-length raw binary data up to 2GB
	BLOB	Binary large object up to 4GB
	BFILE	"Pointer" to a large binary file stored outside the database (up to 4GB)

Oracle9*i* also supports a number of data types not covered by the SQL-92 standard:

Miscellaneous	Comments
ROWID	Physical record address
User-defined types	Object types, VARRAY (variable length arrays), REF (object references), nested tables
XMLType	Specialized CLOB with member functions for storing and manipulating XML data
URI types	For storing Uniform Resource Identifiers (including URLs)
"Any" types	"Anonymous" data types that can be used for procedure parameters when the actual data type is unknown
Spatial data types	Provided by the *Oracle Spatial* add-on
Multimedia data types	Provided by *Oracle interMedia* add-on

Notice that several of the standard SQL-92 data types are actually re-mapped directly onto Oracle's native data types (it also does the same for some DB2 data types). The two most obvious instances of this are with variable-length character data and numeric data. The SQL standard specifies `VARCHAR` for storing variable-length character strings. Oracle uses its own variant of this called `VARCHAR2`, which is basically identical apart from the name. If you declare something as `VARCHAR`, Oracle will automatically re-map it to `VARCHAR2`. (Oracle actually recommends not using `VARCHAR` because its semantics may change in the future.)

For numeric data, Oracle has only one built-in data type: `NUMBER`. This is used to represent integers, decimal numbers, and floating-point numbers, but `NUMBER` is probably closest toSQL-92's `NUMERIC` data type in style. The precision `p` of a `NUMBER` represents the maximum number of digits that it can hold (ignoring the decimal point), so a `NUMBER(24)` can have at most 24 digits. The scale `s` represents the number of digits after the decimal point, so a `NUMBER(10,2)` can have up to eight digits before the decimal point, and up to two digits after. If `s` is negative, any data entered is rounded `s` decimal places to the left of the decimal point, so a `NUMBER(10,-2)` can have up to ten digits before the decimal point, but is always rounded to the nearest hundred (10^2).

As noted in the table, Oracle also supports user-defined data types, including variable-length arrays, nested tables, and **objects**. Oracle has supported objects since version 8. It provides all the usual features you would expect for an object implementation: (single) inheritance, object methods, and so on. Oracle also provides features that allow you to blend object structures with relational structures, including object tables and object views. Oracle9*i* can therefore be said to be an **object-relational DBMS**, rather than just a purely relational one.

Data Manipulation

Oracle supports all the standard features of `SELECT`, including all the SQL-92 join types: natural joins, inner joins, and the three varieties of outer join. Oracle also adds some particularly powerful data manipulation operators to the `SELECT` command:

- Extensions to `GROUP BY` for OLAP-style queries (`CUBE`, `ROLLUP`, and `GROUPING SETS`).
- Random sampling of rows rather than reading the entire table (`SAMPLE`).
- Hierarchical queries (`CONNECT BY`).
- Factored (reusable) sub-queries (`WITH`).
- On-the-fly table construction within a query using collections.

Security

Oracle supports the usual `GRANT` and `REVOKE` commands, and implements a full role-based security model. Security privileges are split into three categories: system-level privileges like `CREATE USER`; and roles; and object-level privileges like `ALTER TABLE`.

Stored Procedures and Triggers

As noted earlier, Oracle stored procedures are written using a built-in proprietary programming language called PL/SQL, but as of version 9*i* you can also call out to functions written in Java or C. Java procedures in particular can also be stored in the database.

Oracle provides very powerful triggers. Triggers in Oracle are effectively a special kind of stored procedure and are able to do most things that a stored procedure can do, with a few minor limitations. As you will recall, a trigger is typically fired when some specified data-manipulation event occurs on the table the trigger is attached to. Oracle generalizes this to triggers that can be fired by a broader range of events, such as creating a table or a particular user logging in.

Oracle also provides something called an `INSTEAD OF` trigger. This is a trigger that's attached to a view rather than to a table. The purpose of an `INSTEAD OF` trigger is to enable a view to be updated that could otherwise not be. It does this by re-mapping `INSERT`, `DELETE`, and `UPDATE` operations on the view into appropriate operations on the underlying base tables. For example, if we were to create a view that combined the **Order** and **OrderItem** tables, we could create an `INSTEAD OF` trigger that re-mapped `INSERT` operations on the view into `INSERT` operations on **Order** and **OrderItem** respectively.

System Architecture and Physical Storage

Oracle has a slightly idiosyncratic architecture compared to most other RDBMS products. In many RDBMS products, a "database" is something that you create when you need it for a specific project or application. Oracle has a different philosophy: each Oracle installation has only *one* "database" that is created at install time, and everyone shares this instance. What most people would normally consider a "database", Oracle calls a **schema**. Schemas in Oracle are effectively just user accounts, so when you want to create a new "database" for a specific project, in Oracle you create a new user account. In practice this distinction makes very little difference, but the variation in terminology can be confusing for people switching either to or from Oracle.

Oracle Enterprise Edition has very powerful physical tuning capabilities, including advanced forms of partitioning and clustering, which, providing it's well configured, allow it to handle very large data sets indeed with very little performance degradation. Oracle's table-partitioning features in particular allow you to physically split a table over multiple disks to enable true simultaneous access to different parts of the same table. This is particularly useful for balancing the I/O load in databases that are accessed by large numbers of simultaneous transactions (such as large e-commerce web databases).

Oracle also supports the usual B-tree indexes and hashing, but also provides the following advanced index types:

- Index-organized tables, where the table data are stored inside a B-tree index rather than separately.
- Bitmap indexes, which are good for queries that involve complex WHERE clauses on columns with a fixed set of possible values.
- Function-based indexes, which are B-tree indexes whose index key is based on the result of a user-defined function rather than a value stored in the database (not to be confused with a hash function, which is quite a different beast).
- Join indexes, which are useful for data warehouses.
- Domain indexes, which are custom indexes designed for a particular type of data, such as spatial data or video. Oracle provides powerful text-indexing features through this mechanism.

Web Support

Oracle first began to focus on the Internet and the Web with the release of version 8*i*, which provided the first steps towards greater integration with the Internet. The current release is now well down this road. Oracle9*i* provides several key technologies that are ideal for building web applications and databases. These include:

- A built-in Java virtual machine and Java libraries, and support for calling out to Java from triggers and stored procedures.
- The Apache open source web server is installed by default, and pre-configured with support for Java servlets. This means that you can start developing web applications straight out of the box.
- Built-in support for XML with the XMLType data type and numerous functions for accessing and manipulating XML data (and support for XML query languages like XQuery is in the works).
- The ability to provide hints to the query optimizer on how best to execute a query.
- **Materialized views**. A materialized view is similar to a normal view in that its content is derived from a SELECT query. Unlike a normal view, however, a materialized view takes the results of the query and stores them as an actual physical table (hence "materialized"; normal views are "virtual" and dynamically regenerated every time they are accessed). While this provides obvious performance advantages (especially for pre-computing joins), data in a materialized view are not automatically updated when the original data changes. The solution to this problem is to define an automatic refresh interval; for example, refresh the materialized view every four hours.

For More Information

For the most up-to-date information about Oracle, check both Oracle's main web site (*http://www.oracle.com/*) and their developer technical site, the Oracle Technology Network (OTN; see *http://otn.oracle.com/*). In particular, the entire documentation for the RDBMS is available online through OTN (you'll need to register first, but this is free). You can also download beta and early-release versions of the RDBMS for development purposes, but be warned that these are very large downloads: in the order of 1.5GB for Enterprise Edition.

Microsoft SQL Server

Microsoft SQL Server 2000, the latest release of SQL Server, is distributed in two major categories, **SQL Server 2000 Server Editions** and **SQL Server 2000 for Special Uses**. The Server editions of SQL Server include the following:

- SQL Server 2000 Enterprise edition
- SQL Server 2000 Standard edition

The Special Uses editions of SQL Server include the following:

- SQL Server 2000 Personal edition
- SQL Server 2000 Developer edition
- SQL Server 2000 Evaluation edition
- SQL Server 2000 Desktop Engine (MSDE 2000)
- SQL Server Windows CE edition

You can read more on these editions to determine which edition is best suited for your needs at *http://www.microsoft.com/sql/techinfo/planning/SQLResKChooseEd.asp*.

Supported Operating Systems

Since Microsoft SQL Server was designed and developed by Microsoft, it currently only runs on Microsoft operating systems. The various editions of SQL Server 2000 run on the low-end Windows operating systems up to the high-end Windows operating systems. The following table details the editions of SQL Server and the operating systems that they run on:

	Enterprise Edition	Standard Edition	Personal Edition	Developer Edition	Evaluation Edition	Desktop Engine	CE Edition
Windows 2000 Datacenter Server	✓	✓	✓	✓	✓	✓	
Windows 2000 Advanced Server	✓	✓	✓	✓	✓	✓	
Windows 2000 Server	✓	✓	✓	✓	✓	✓	
Windows NT Server 4.0 Enterprise Edition	✓	✓	✓	✓	✓	✓	
Windows NT Server 4.0	✓	✓	✓	✓	✓	✓	
Windows XP Professional			✓	✓	✓	✓	
Windows XP Home Edition			✓	✓	✓	✓	
Windows 2000 Professional			✓	✓	✓	✓	
Windows NT Workstation 4.0			✓	✓	✓	✓	
Windows ME			✓			✓	
Windows 98			✓			✓	
Windows CE 4.1							✓

Support for SQL-92

As with the other databases considered SQL Server supports the SQL-92 standard to the following degree. First we look at character data:

SQL-92	SQL Server	Comments
CHAR(n)	CHAR NCHAR	CHAR and VARCHAR support a maximum of 8,000 characters. NCHAR and NVARCHAR provide support for Unicode data and thus only support a maximum of 4,000 characters
VARCHAR(n)	VARCHAR NVARCHAR	
	TEXT NTEXT	TEXT supports a maximum of 2,147,483,647 characters. NTEXT provides support for Unicode data and supports a maximum of 1,073,741,823 characters

Next we have numeric data:

SQL-92	SQL Server	Comments
SMALLINT	TINYINT SMALLINT,	TINYINT takes values 0 through 255 SMALLINT takes values -32,768 through 32,767
INTEGER	INTEGER BIGINT	INTEGER takes values -2,147,483,648 through 2,147,483,647 BIGINT takes values -9,223,372,036,854,775,808 through 9,223,372,036,854,775,807
FLOAT(p)	FLOAT	FLOAT takes values -1.79E + 308 through -2.23E - 308, 0, and 2.23E + 308 through 1.79E + 308
REAL DOUBLE PRECISION	REAL	REAL takes values -3.40E + 38 through -1.18E - 38, 0, and 1.18E - 38 through 3.40E + 38
DECIMAL NUMERIC(p,s)	DECIMAL(p,s)	DECIMAL(p,s) takes values: -10^38 +1 through 10^38 –1
	MONEY SMALLMONEY	MONEY takes values: -2^63 through 2^63 - 1 SMALLMONEY takes values: -214,748.3648 through +214,748.3647

The next table looks at date and time data:

SQL-92	SQL Server	Comments
DATE TIME	DATETIME SMALLDATETIME.	DATETIME provides date and time data from January 1, 1753, through December 31, 9999, with an accuracy of three-hundredths of a second, or 3.33 milliseconds SMALLDATETIME provides date and time data from January 1, 1900, through June 6, 2079, with an accuracy of one minute
TIMESTAMP	TIMESTAMP	TIMESTAMP provides a database-wide unique number that gets updated every time a row gets updated

Finally, we look at data types for storing raw binary data:

SQL-92	SQL Server	Comments
BIT(n) BIT VARYING(n)	BINARY VARBINARY IMAGE	BINARY provides fixed-length Unicode data with a maximum length of 4,000 characters VARBINARY provides variable-length Unicode data with a maximum length of 4,000 characters IMAGE provides variable-length Unicode data with a maximum length of 2^30 - 1 (1,073,741,823) characters

SQL Server also supports a number of data types not covered by the SQL-92 standard:

Miscellaneous	Comments
UNIQUEIDENTIFIER	UNIQUEIDENTIFIER provides a globally unique identifier (GUID).
TABLE	TABLE provides a special data type that can be used to store a result set for later processing. Its primary use is for temporary storage of a set of rows, which are to be returned as the result set of a table-valued function.
SQL_VARIANT	SQL_VARIANT provides a data type that stores values of various SQL Server-supported data types, except TEXT, NTEXT, IMAGE, TIMESTAMP, and SQL_VARIANT.

Available Tools

Two major tools come with SQL Server 2000:

- Enterprise Manager
- Query Analyzer

Enterprise Manager

The *Enterprise Manager* is the primary administrative tool that allows you to fully manage all databases in your network. Using this tool, you can create and delete databases, manage SQL Server configuration options on other SQL Servers, and perform administrative tasks such as backing up and restoring your databases. Not only does *Enterprise Manager* provide you with a rich set of tools for performing maintenance on your databases and SQL Server installations, but it also provides a full set of wizards to help the newcomer perform these tasks and will step you through these tasks step by step.

Query Analyzer

The *Query Analyzer* is a tool primarily used by developers to create and test stored procedures and views as well as run ad-hoc queries. *Query Analyzer* provides an *Object Browser* that allows you to quickly see all objects in a database such as tables, columns in a table, constraints, stored procedures, and views. You can also edit the data in tables directly if you have the appropriate permissions. *Query Analyzer* also provides many templates to help you perform common tasks such as creating new databases using SQL commands as well as tables, stored procedures, and views. You can even define and implement your own custom templates to help simplify common tasks and queries.

Server Editions

In this section we want to examine the typical uses of SQL Server 2000 Enterprise and SQL Server 2000 Standard editions. We will also take a look at the system requirements for these two editions, which includes not only the hardware that these editions require but the operating systems that they require as well.

Both server editions of SQL Server 2000 are well suited for e-commerce, data warehousing, data analysis, online transaction processing, and line-of-business applications. The Enterprise edition supports the full range of data warehousing and analysis features, while the Standard edition supports a limited set of these features.

The Enterprise edition is well suited for organizations that demand the full set of features from a database along with high scalability and performance for mission-critical applications. This edition of SQL Server is particularly well suited for data warehousing and online analytical processing (OLAP) and allows you to define, create, and link OLAP cubes.

The Standard edition is well suited for small to medium organizations that do not require the advanced online analytical processing features supported in the Enterprise edition. This edition does provide support for data warehousing and the core OLAP features for minor OLAP functionality.

The table below provides a quick glance at some of the major features supported by both editions of SQL Server 2000:

Feature	Enterprise Edition	Standard Edition
Advanced Analysis Features for OLAP	✓	
Analysis Services	✓	✓
Comprehensive Analysis Services	✓	
Data Mining	✓	✓
Distributed Partitioned Views	✓	
Data Transformation Services	✓	✓
XML Support	✓	✓
Full Text Search	✓	✓
Replication	✓	✓
English Query	✓	✓
Scalability	✓	✓
High Availability	✓	

For a more detailed look at the features supported, check out *http://www.microsoft.com/sql/evaluation/features/choosing.asp*.

Special Uses Editions

The Special Uses editions of SQL Server have been designed to meet a variety of customer needs and to run on a variety of Windows platforms all the way down to hand-held CE devices. Let's take a quick look at each of the Special Uses editions to see what they were designed for.

Personal Edition

The **Personal** edition of SQL Server was designed for mobile users who are disconnected from the network most of the time but who require SQL Server to run their applications. This edition provides the GUI tools, *Enterprise Manager* and *Query Analyzer*, which allow a mobile user to administer the database, create new databases, and to create and run ad-hoc queries.

Replication is the process by which data in one database is synchronized with that in another, creating a replica database. Replication is broken into two main categories, publisher and subscriber. A publisher publishes data for replication and a subscriber consumes the published data.

This edition of SQL Server is functionally equivalent to the Standard edition of SQL Server with a few exceptions. First, since this edition was designed for the mobile user who is disconnected from the network most of the time, this edition cannot be used as a replication publisher. It can however be a replication subscriber. Next, this edition only supports five concurrent queries and has a built-in workload governor that will degrade performance when more than five concurrent queries are being run.

Developer Edition

The **Developer** edition of SQL Server provides all of the functionality of the Enterprise edition but has been designed to be run not only on the Windows Server family of operating systems, but also on the Windows professional desktop operating systems (Windows NT, Windows 2000 Professional, and Windows XP Professional).

This edition was designed for developers and independent software vendors for developing applications that will run on server editions of SQL Server.

Evaluation Edition

The **Evaluation** edition of SQL Server is just that, an evaluation edition and has a 120-day evaluation period after installation. This edition is functionally equivalent to the Enterprise edition of SQL Server and can be installed on the same operating systems as the Developer edition. This provides a company with the opportunity not only to evaluate the server features available in SQL Server Enterprise edition, but it also allows developers and database administrators to evaluate the features that they would use. After the 120-day evaluation period has expired, the software will cease to function and must be uninstalled. There is no cost for this edition and it can be downloaded for free at: *http://www.microsoft.com/sql/evaluation/trial/default.asp*.

Desktop Engine Edition

The **Desktop Engine** edition is also commonly known as **MSDE 2000** or **MSDE**. This edition of SQL Server was designed primarily as a means to distribute the SQL Server relational database engine with client-side applications. This version of SQL Server does not contain any GUI tools to administer a database or to run queries against it. All of the administration and query processing must be performed by your application, unless you have a copy of *Enterprise Manager* from another installation. You can also use Access 2000 as a front end for MSDE.

MSDE has a couple of restrictions, the first of which is that, like the Personal edition, the concurrent workload is limited to five transactions. Also, the maximum database size that MSDE will support is 2 GB, which should be enough for any desktop application. MSDE also cannot be used as a publisher in a replication plan but it can be used as a subscriber.

This edition of SQL Server will run on all Microsoft Server editions of Windows as well as all professional and consumer Windows operating systems, with the exception of Windows 95.

CE Edition

SQL Server CE edition is a compact SQL Server database designed specifically for mobile devices running the Microsoft CE operating system. This edition of SQL Server brings enterprise data management capabilities to mobile devices. With a small footprint of 1 MB, this edition provides you with powerful database capabilities such as acting as a subscriber in a replication plan and providing optimizing query processor for processing queries.

Like MSDE, SQL Server CE supports databases up to 2 GB, which is a lot of data on a mobile device. In addition to providing replication support, SQL Server CE provides remote data access with other editions of SQL Server allowing you to receive data, process it offline, and then synchronize that data back to other SQL Server databases when the CE device is connected to the network.

Features and Requirements

Now that we have had a look at the Special Uses editions of SQL Server, let's take a comparative look at the features and requirements of these different editions of SQL Server. The following table summarizes the major features of SQL Server and which editions support them:

Feature	Personal Edition	Developer Edition	Evaluation Edition	Desktop Engine	CE Edition
Multiple Instance Support	✓	✓	✓	✓	
Analysis Services	✓	✓	✓		
Data Mining	✓	✓	✓		
Data Transformation Services	✓	✓	✓	✓	
XML Support	✓	✓	✓	✓	✓
Full Text Search	✓	✓	✓		
Replication	✓	✓	✓	Subscriber Only	Subscriber Only
English Query	✓	✓	✓		
GUI Tools	✓	✓	✓		

Summary

In this appendix, we've looked at three of the more popular RDBMS products available today: MySQL, Oracle, and SQL Server. We have made no effort to compare the three, as this would have led us into pages of rhetoric about whether it's better to have transactions or to be freely available to modify. Each of the products we've looked at here has its advantages, and each its disadvantages.

For example, MySQL is free, but doesn't support stored procedures. Oracle scales incredibly well, but is complex and expensive. SQL Server ships with wonderful tools, but only runs on Windows.

Appendix

- MySQL 3.23
- Oracle 9*i*
- SQL Server 2000

Index

A Guide to the Index

The index is arranged in word-by-word order (so that, for example, New York would appear before Newark). Acronyms have been preferred to their expansions as main entries and unmodified headings usually indicate the principal treatment of a topic. Comments specifically about the index would be welcome at *billj@glasshaus.com*.

1NF (First Normal Form), 39
2NF (Second Normal Form), 40
3NF (Third Normal Form), 40
4NF (Fourth Normal Form), 41

A

access permissions
  column-level permissions, 183
  restricting access with stored procedures, 206
  setting permissions, 185
  stored procedure permissions, 184
  table-level permissions, 183
  view permissions, 184
ACID transaction properties, 218
ADD form, ALTER TABLE command, 64, 238
administration *see* DBA.
administrative privileges
  protecting administrator accounts, 193
  warning about, 187
administrative requirements and the planning process, 142
administrative tools
  MySQL, 259
  Oracle9*i*, 266
  SQL Server, 275
ADO (ActiveX Data Objects)
  Command object, 189
  Connection object transaction support, 218, 221
  Error object, 191
advanced SQL queries, 197-203
  sub-queries, 198
aggregate functions, 87
  many-to-many recursive relationships, 113
  pre-computed aggregate functions, 89
  renaming columns, 92
  use of GROUP BY clauses, 88
  using triggers, 209
  using views, 204
aliasing
  table and column names, 90
  tables in correlated sub-queries, 201
ALL operator, 203
ALTER COLUMN keywords, 236, 237
  changing data type, 237
ALTER form, ALTER TABLE command, 64
ALTER TABLE command, 64, 234-243
  adding columns, 238
  dropping columns, 240, 243
  limitations, 65
  syntax, 235
alternate keys
  introduced, 35
ANSI (American National Standards Institute)
  SQL standards and, 55
ANY operator, 203
architectural peculiarities of Oracle, 270
artificial primary keys *see* surrogate keys.
AS keyword, 91
associative tables
  see also many-to-many relationships.
  creating during implementation, 159
  e-commerce store example, 108
  implementing many-to-many relationships, 108
  naming, 109

associative tables (Cont'd)
web store e-commerce application
entity-relationship diagram, 160
ProductCategory table, 168
asterisks
significance in SELECT commands, 73
used as multiplication operators, 87
atomicity of transactions, 218
attributes
attribute integrity, 121
attribute settings, 122
database columns as, 30
ER modeling elements, 103, 104
identifying in the planning process, 146
audit data
maintaining using triggers, 209
authentication methods, choosing, 179
auto_increment columns, MySQL
numeric sequence keys, 118
surrogate keys defined as, 163
automatic columns, 163
see also surrogate keys.
autonumber columns, Access
Globally Unique Identifiers, 119
numeric sequence keys, 118
surrogate keys defined as, 163

B

B2B (Business to Business) sites,
security requirements contrasted with e-commerce, 178, 179
BACKUP command, 228
backups, 226-234
complete and differential backups, 227
recovery models, 227
scheduling, 231
transaction log backups, 229
BEGIN TRANSACTION command, 218
BETWEEN operator
introduced, 63
specifying date ranges in queries, 75
BIT data type, 238
BLOBs (Binary Large Objects), 31
book publishing example
database models, 10-17
bookstore, online
ER diagram example, 103
Boolean data types, 31
introduced only in SQL:1999, 56
Boolean logic
problems with, caused by nulls, 61
Boolean operators
WHERE clauses, 70
B-tree indexes
indexed access methods, 47
business rules
domain integrity and, 121
incorporating into stored procedures, 206
modifying with triggers, 207-208

C

caching hierarchical tree data, 112
cardinality of relationships
common structures, 105
introduced, 105
web store e-commerce application, 147
carriage return coding in SQL Server, 251
CASCADE referential action, 130
cascading triggers, 208
CASE (Computer Aided Software Engineering) tools
forward engineering, 134
reverse engineering, 132
use for data modeling, 132
case sensitivity
SQL not case-sensitive, 58
CAST function, 229
scripting use, 252, 252
catalog, metadata storage, 44
category trees
using recursive relationships, 109-110
web store e-commerce application, 167
CHAR data type
choice between VARCHAR and, 59
character data types, 123
rules for setting lengths, 124
web store e-commerce application, 161
CHECK constraints
domain integrity and, 127
example, 63
introduced, 60
sub-type/super-type relationships, 115

checkboxes
  using the IN operator with, 202
Chen, Peter, Entity-Relationship models and, 9
CLOBs (Character Large Objects), 31
clustered indexes, 215
  CLUSTERED keyword, 244
  scans and seeks, 212, 245
CMS (Content Management Systems), example of web database use, 20, 139
COALESCE function, 252
Codd, Edgar F, relational database pioneer, 9
  null values and, 33
  relational algebra and relational calculus, 73
code downloads, 3
collaboration systems
  example of web applications using databases, 19
column constraints
  distinguished from table constraints, 58, 59
column naming
  importance of descriptive names, 77
  INSERT commands, 67
  qualifying names recommended, 78
  renaming, 92
    introduced, 90
  web store e-commerce application, 157
column-level permissions, 183
columns
  adding, 238
  adding and dropping, alternative to, 238
  dropping, 239
  modifying, 235
    allowing null values, 236
    changing data type and field length, 237
  order immaterial, 29
  representing attributes, 104
  shared columns and natural joins, 77
command groups, SQL, 56
comments in SQL, 90
  trigger example, 208
COMMIT TRANSACTION command, 218
common structures, ER modelling, 105-116
compilation of stored procedures, 206
complete backups, 227
composite primary keys, 34
Computer Associates Jasmine ODBMS, 14
concatenated primary keys, 117
  web store e-commerce application, 168, 170
concatenating numeric dates, 229
concatenation character, SQL Server
  scripting use, 251
conceptual data model, web store e-commerce application, 151
concurrency management, 216
  limitations of transaction isolation levels, 220
conditional joins, 78
  affecting several tables, 79
consistency of transactions, 218
constants
  string constants, 251
  UNION operator and, 94
constraints
  adding columns and, 238
  check, not null and unique constraints, 60
  column and table constraints, 59
    introduced, 58
  CONSTRAINT keyword, 61, 240
  domains and, 32
  DROP CONSTRAINT keywords, 240
  preventing column alterations, 235
  relational integrity constraints, 165
correlated sub-queries, 201
  EXISTS operator and, 203
country codes, 169
CREATE INDEX command, 66, 244, 247
  adding an index to a primary key, 247
  reorganizing indexes with, 248
CREATE TABLE command, 57
  setting default values, 127
  web store e-commerce application, 172
creation of databases
  implementation specific, 57
crow's foot notation, 103
  web store e-commerce application, 148
CRUD (Create Read Update Delete) operations, 67
CryptoAPI, 183
CSV (Comma Separated Value) files, 16
currency
  money type variation between RDBMSs, 124
  unsuitability of FLOAT data types, 62
customer information
  privacy legislation and, 144
customer order information
  web store e-commerce application, 143

D

data access
identifying needs, security modeling and, 180
restricting using views, 191
data access methods, 45
direct access methods, 48
indexed access methods, 47
sequential access methods, 46
data dictionary, metadata storage, 44
data integrity, 42
built-in rules, 43
user-defined rules, 43
data manipulation
see also DML.
best done at database level, 198
data modeling
see also entity-relationship modeling.
importance, 102
process diagram, 102
testing models with sample data, 137
use of CASE tools, 132
data models
alternative definitions, 25
entity-relationship diagrams and, 25
theoreticians' definition, 25
data redundancy, 37
see also denormalized tables.
ER diagrams and, 104
removed by normalization, 39
data storage
potential of modern RDBMSs, 10, 46
shortcomings of spreadsheets, 6-7
shortcomings of Word documents, 5
data type and size and domain integrity, 122
data types
changing column types, 235
changing for a column, 237
changing lengths, 237
changing with the CAST function, 229
choice of, stock portfolios example, 124
choosing, in implementation stage, 161
domains and, 31
RDBMS support for, 122
representing in ER diagrams, 104
SQL:1999 support for structural and user-defined types, 56
SQL-92 compared to MySQL, 263
SQL-92 compared to Oracle, 267
SQL-92 compared to SQL Server, 273
SQL-92 standard, 59
data validation *see* validation.
database creation as implementation specific, 57
Database Model Diagram template, MS Visio 2002, 133
database models, 10
distributed databases, 15
flat file storage, 15
hierarchical database model, 10
historical development, 8
multimedia databases, 15
network database model, 11
object relational database model, 14
object-oriented database model, 14
relational database model, 12
XML databases, 17
database platforms, Appendix comparing, 257
database tables *see* tables.
database-level security, 183
databases
advantages of using, 8
definition, 6
reasons not to use, 20
web application use, 18
date data types
web store e-commerce application, 162
date formats
differ between RDBMSs, 68
geographical variations, 229
specifying for different RDBMSs, 76
date ranges
specifying in SQL queries, 75
DATEPART function, 230
DATETIME data type, 124
current date and time recording, 126
DBA (Database Administrator) role
combined with development, 226
DBMS tools for, 18
development databases and, 225
security role, 177, 192

DBMS
see also RDBMS.
diagram of some key components, 49
role of, 17
DBTools Manager, MySQL, 261
DCL (Data Control Language) SQL syntax group, 57
DDL (Data Definition Language), 57-66
BACKUP command, 228
output form MS Visio, 134
setting default values, 127
specifying CHECK constraints, 127
SQL syntax group, 57
web store e-commerce application, 172
deadlocks, 221
DECIMAL data type, 123
declarative nature of SQL, 54
DECLARE command, 229
default value constraints, 238, 239
default values
alternatives to nulls, 34
default categories
as relationship data, 148
web store e-commerce application, 168
domain integrity and, 126
setting with ALTER TABLE, 65
delete anomalies
RI problem, 129
un-normalized data, 38
DELETE command, 70
TRUNCATE command and, 69
deleting records, applying referential actions, 130
denormalized tables
inserting, using triggers, 209
normalized databases and, 83
performance and, 39, 131
deploying a development database, scripts for, 249
design stage, 101
security issues, 178
developer role
database maintenance and, 225, 226
data access needs, 181
diagramming tool, Microsoft Visio as, 133
dialects of SQL, 55
differential backups, 227
direct access methods, 48
dirty reads, 220
discussion forums, example of web database use, 19
disk access times, 45
disk storage, character data types, 123
DISTINCT keyword
aggregation operations, 87
removing duplicate values, 86
sub-query example, 200
distributed databases, 15
DML (Data Manipulation Language), 67-98
four possible operations on data, 67
Oracle additional operators, 269
SQL syntax group, 57
domain integrity, 121
aspect of built-in data integrity, 44
domains
data types and, 31
RDBMS support for, 32
RDBMS support for, 122
relational database columns, 29
DRI (Declarative Referential Integrity), 184
DROP COLUMN keywords, 243
DROP CONSTRAINT keywords, 239, 243
adding an index to a primary key, 247
DROP_EXISTING keywords, 248
DROP form, ALTER TABLE command, 65
DROP INDEX command, 66, 247
DROP TABLE command, 63
SQL injection example, 187
duplicate values
removing with the DISTINCT keyword, 86
durability of transactions, 219
dynamic content
reasons to use databases, 20
dynamic table creation with triggers, 208

E

e-commerce sites
content aggregation using triggers, 209
denormalization and, 39, 131
example of pre-computed joins, 83
example of web applications using databases, 19
denormalization and, 131
read-only and write data access needs, 180
security requirements contrasted with B2B, 178, 179

e-commerce store example

e-commerce store example
  see also web store e-commerce application.
  associative tables, 108
  common structures illustration, 105
  database diagram, 116
  inserting data using a sproc, 207
  many-to-many recursive relationships, 112
encrypting passwords, 182
Enterprise Manager, 275
  SQL Server scripting aid, 249
entities
  ER modeling elements, 103, 104
  identifying in the planning process, 146
  representation by tables, 30, 104, 157
entity integrity, 117
  aspect of built-in data integrity, 43
  primary keys constraints specify, 60
entity relationships
  see also ER diagrams; relationships between tables.
  identifying in the planning process, 147
entity-relationship modelling, 103
  see also data modelling.
  common structures, 105-116
  history of database development and, 9
ER (Entity-Relationship) diagrams
  conceptual data model, 151
  example of one data model type, 25
  generating by reverse engineering, 135
  one-to-many recursive relationships, 150
  relationships between entities, 105
  web store e-commerce application, 148-151,159-165
    including associative tables, 160
error handling
  generic error handling, 186-187, 190
  meaningful error messages, 191
Eureka, Ca, sample query featuring, 171
Exabyte data storage units, 10
Excel spreadsheet closeness to a database, 6
EXCEPT operator, 93
exclusive locks, 217
execution plans *see* query plans.
existing database content, web application use, 19
EXISTS operator, 203
  stored procedure example, 205
EXPLAIN functionality tabulated for some RDBMSs, 210

file system example, illustrating one-to-one relationships, 114-115
files, physical, making up a database, 227
filtering using WHERE clauses, 71
First Normal Form (1NF), 39
flat file storage, 16
FLOAT data types
  related types and, 124
  unsuitable for monetary values, 62
foreign keys
  defining with CREATE TABLE commands, 62
  e-commerce store example, 106
  existence preventing deletes, 71
  indexing, 171
  introduced, 36
  natural joins and, 78
  recursive relationships and, 111
  referencing non-primary key columns, 173
  referential integrity and, 164
  representing in ER diagrams, 104
  should always be indexed, 214
  sub-type/super-type relationships, 115
forms, downloading data integrity rules onto, 43
forward engineering
  CASE tools and, 134
  generic data types and, 122
  portable data types, 125
Fourth Normal Form (4NF), 41
fragmentation
  introduced, 226
  need to reorganize indexes and, 246
FROM clauses
  building a SELECT query, 95
  effects of listing multiple tables, 76
full outer joins, 81
full recovery model
  introduced, 227
  restore operations, 231
functional dependencies
  fourth normal form and, 41
  second normal form and, 40
  third normal form and, 41

G

generic error handling, 186
ADO Error object and, 191
vulnerability to hackers, 187, 190
GETDATE() function
backups and, 229
transaction log backups, 230
use as a default value, 239
GRANT command, setting access permissions, 185
granularity of locks, 217
graphical tools for query plan display, 212
GROUP BY clauses, 88
building a SELECT query, 95
nesting sub-queries in, 199
RDBMS support, 89
GUID (Globally Unique Identifiers)
suitability for web applications, 119
surrogate key choice, 118-119

H

hackers *see* security issues.
hash functions, direct access methods, 48
HAVING clauses, 89
nesting sub-queries in, 199
header-lines, 62
heterogeneous distributed databases, 15
hidden form fields, surrogate keys in, 164
Hierarchical database model, 10
historical significance, 9
survival as XML, 50
hierarchical tree structures, 111
HINT functionality tabulated for some RDBMSs, 213
history of database development, 8
history of query language development, 55
homogeneous distributed databases, 15

I

identity columns, SQL Server
numeric sequence keys, 118
surrogate keys defined as, 163
images storing as URLs, 167
implementation process
choice of target database system, 155
choosing data types and nullability, 161
creating associative entities, 159
creating indexes, 171
foreign keys and relational integrity, 164
naming conventions, 156
natural and surrogate primary keys, 163
table and column definitions, 165
IN operator, 202
introduced, 75
sub-query example, 200
indexed access methods, 47
update frequency considerations, 48
indexes, 213
adding, 243
cost associated with, 246
CREATE INDEX and DROP INDEX commands, 66
drawbacks of, 214
dropping, 247
for faster table loading, 248
EXISTS operator and, 203
index types, 215
Oracle, 270
introduced, 45
not part of relational model, 243
reorganizing, 246
by dropping and rebuilding, 248
using DROP EXISTING, 248
rules for, on web databases, 214
uniqueness, 47, 215, 244
web store e-commerce application, 171
information gathering stage, planning process, 140
Ingress database system, 9
inline SQL, 186
preventing abuse of, 187
insert anomalies
RI problem, 129
un-normalized data, 38
INSERT command, 67
incorporating a query, 84
web store e-commerce application, 175
inserting data
dropping indexes for efficiency, 248
inserting into multiple tables with a sproc, 207
inserting summary data using triggers, 209
performance and non-clustered indexes, 215

INSTEAD of triggers

INSTEAD of triggers, 207
INTEGER data type, 123
integrity aspect
    see also data integrity.
    provided by data models, 26
integrity constraints, 116
    attribute or domain integrity, 121
    entity integrity, 117
integrity rules
    defined using constraints, 59
    testing with sample data, 137
intelligent keys, 118
    surrogate keys compared to, 118, 120
intermediary tables *see* associative tables.
internationalization
    date formats, 229
    designing tables to support, 169
INTERSECT operator, 93
ISO (International Organization for Standardization)
    date format, 68
    SQL standards and, 55
isolation of transactions, 219
iterative nature of modeling, 125

J

Jasmine ODBMS, 14
Java, Oracle support for, 271
JavaScript, data integrity in web forms, 43
JOIN keyword, SQL-92 standard, 79
joining tables *see* associative tables.
joins, 77-84
    condition based joins, 78
    foreign keys and, 37
    joins in a web environment, 82
    natural joins, 77
    outer joins, 80
    self joins, 91

K

keywords, SQL, numbers in different dialects, 56

L

leaf nodes, indexes, 47
left outer joins, 81
    example, 96
legacy databases, data modeling usefulness, 103
LIKE operator, 174
linking tables *see* relationships.
Lists, using the IN operator with, 202
locking
    caused by backup operations, 231
    concurrency management and, 216
    deadlocks, 221
    limitations of transaction isolation levels, 220
    lock granularity and type, 217
log files
    backup example, 229
    transaction logs and full recovery backups, 227
LOG keyword, BACKUP command, 230
logging tables, indexing and performance, 214
logical and physical storage distinguished, 26, 30
logical data models, 26
logins
    reserved logins for data insertion, 192
    two needed for read-only and write access, 180

M

maintenance, 225
    scheduling, 254
management hierarchies and recursive relationships, 111
many-to-many recursive relationships, 112
many-to-many relationships, 107
    see also associative tables.
    creating associative tables, 159
    introduced, 13
    web store e-commerce application, 149
        resolving, 159
materialized views, 84, 271
mathematical basis of relational model, 27
menus and recursive relationships, 111
metadata, 44

Microsoft Corporation
see also Excel; MSDE; SQL Server; Visio modeling tool; Word.
free stress testing tool, 137
mobile devices, Microsoft CE OS and SQL Server CE Edition, 277
modeling *see* data modeling.
modeling tools and optionality choice, 126
MONEY data type, variation between RDBMSs, 124
motor racing, 14
MSDE (Microsoft Data Engine), 277
multidimensional data storage, 6
multimedia databases, 15
multi-valued dependencies, 41
MySQL Control Center tool, 260
MySQL databases
administrative tools, 259
APIs supported, 258
Appendix comparing three RDBMSs, 257-264
availability, 54, 257
drawbacks of, 264
operating systems supported, 258
RI support, 128
SQL-92 support, 267
TEXT data type, 166
MySQL Front tool, 260
MySQL GUI tool, 259
MySQLManager tool, 262

N

naming conventions and implementation process, 156
natural joins, 77
nested categories, 111
nesting of SELECT commands using sub-queries, 198
Network database model, 11
historical significance, 9
survival as objects, 50
network security, 192
network traffic reduction using stored procedures, 207
news sites, reasons to use databases, 20
non-clustered indexes, 215
implication of reorganizing clustered indexes, 248
NONCLUSTERED keyword, 244
non-repeatable reads, 220
NORECOVERY keyword, 232
restoring full recovery backups, 233
normalization, 37
see also denormalization; *individual normal forms*.
higher normal forms, 42
need for table joins and, 77
not equal to operator, RDBMS support, 72
NOT NULL constraints
ignored by temporary columns, 81
introduced, 60
null values
see also optionality of relationships.
aggregation operations, 88
care required with, 34
cause of problems with Boolean logic, 61
changing column lengths may affect, 237
changing columns to allow or disallow, 236
CHECK constraints and, 127
display of, in tables, 33
in result of a join, 97
introduced, 32
NULL keyword, 68
populating a table, 252
prohibited in primary keys, 117
recursive relationships and, 111
restrictions on manipulating, 33
temporary columns ignore NOT NULL constraints, 81
UNION operator and, 93
Nullability, defining in implementation stage, 162
numeric data types, web store e-commerce application, 161
numeric sequence keys, surrogate key choice, 118

O

object databases, 14
OQL similarity to SQL, 55
object relational database model, 14
Oracle9*i* as, 269
SQL:1999 support for object storage, 56
ODMG (Object Data Management Group), 50, 54
ON keyword and join conditions, 79

one-to-many recursive relationships, 109
  sub-query example, 200
  web store e-commerce application, 150
one-to-many relationships, 106
  introduced, 13
  rule about location of primary key, 164
  web store e-commerce application, 151
one-to-one relationships, 113
  introduced, 13
open-source products
  see also MySQL.
  emergence of open-source RDBMSs, 10
  open-source XML databases, 17
  postgreSQL support for SQL-92 standard, 54
operating systems
  supported by MySQL, 258
  supported by Oracle, 265
  supported by SQL Server, 272
operational aspect, provided by data models, 26
optimization *see* performance tuning.
optionality of relationships
  domain integrity and, 126
  introduced, 105
OQL (Object Query Language), 54-55
Oracle databases
  APIs supported, 266
  Appendix comparing three RDBMSs, 265-271
  availability, 265
  NUMBER data type preferred to those of SQL-92, 59
  operating systems supported, 265
  Oracle 8*i* outer join syntax, 82
  SQL-92 support, 267
  SYS_OP_GUID() function, 119
Oracle Magazine, building a SELECT query, 95
ORDER BY clauses, 85
  building a SELECT query, 95
order histories, example of a pre-computed join, 83
OS authentication, 179
  see also Windows Authentication.
outer joins, 80
  RDBMS support, 82

P

pages, data storage units, 228
parameters and stored procedures, 205
parent tables and child tables, foreign keys and, 37
passwords
  encryption, 182
  using strongly-typed passwords, 182, 192
payment processing
  third-party payment processors, 145
  web store e-commerce application, 145
performance improvements
  denormalization and, 131
  range select queries, 216
  using stored procedures, over SQL strings, 206
  views and, 204
performance penalty
  associated with normalization, 39
  INSERTS with clustered indexes, 215
  joins in a web environment, 82
performance tuning, 209-213
  needs realistic data volumes, 210
  using sample data, 211
permissions *see* access permissions.
personal names, B-tree index example, 47
Petabyte data storage unit, 10
phantom inserts, 220
phpMyAdmin package, 261
physical data independence
  logical data models and, 26
  physical storage and, 45
physical database tuning *see* performance tuning.
physical files for a database, 227
physical indexes *see* clustered indexes.
physical storage
  see also data access methods.
  peculiarities of Oracle, 270
PL/SQL, SQL:1999 general purpose language resembles, 56
planning process, 139
  identifying entities and attributes, 146
  identifying entity relationships, 147
  information gathering stage, 140
  preparing the conceptual data model, 151

portable data types, 125
postgreSQL support for SQL-92 standard, 54
pre-computed aggregate functions, 89
pre-computed joins, 83
prefixes, naming conventions based on
  column names, 158, 160
  table names, 157
pre-SQL-92 specification of joins, 38
  example queries compared with SQL-92, 80
primary key constraints function as indexes, 247
primary keys
  adding indexes to, 48
  always indexed by the RDBMS, 171
  choice for Product table, 166
  declaring composite keys, 172
  definition of, as a table constraint, 58
  natural and surrogate keys contrasted, 163
  representing in ER diagrams, 104
  requirements of, 34
  risks of declaring indexes on, 173
  second normal form and, 40
  should always be indexed, 214
  surrogate keys alternative to concatenating, 63
  validation criteria, 117
privacy legislation, 144
  see also security issues.
product categories, web store e-commerce application, 142
product information display, web store e-commerce application, 141
product prices and averages example
  stored procedure example, 206
  using sub-queries, 198, 201, 203
production databases, warning about changing directly, 234
profiling and use of query hints, 213
programming languages
  supported by MySQL, 258
  supported by Oracle, 266

Q

QUEL query language, 9, 55
queries, SQL, 72
  see also SELECT command.
  advanced SQL queries, 197-203
  benefits of efficient queries, 74
  incorporating into INSERT commands, 84
  SQL versions of sample queries, 74
  web store e-commerce application, 173
Query Analyzer, SQL Server, 275
  execution plan display, 211-212
  graphical scripting tool, 249
  index performance example, 245
query hints, 213
query languages
  see also SQL.
  physical data independence and, 26
query plans
  generated once for stored procedures, 206
  graphical views of, 211-212
  indexing sample query, 245
  methods of pre-empting, 210
  views and, 204
QueryBuilder tool, MySQL Manager, 262
Quest Software, 267
QUOTENAME function, 251, 252
quoting character data, 67
  see also single quotes.
  delimiters within scripts, 251
quoting date values, 68
  single quotes, escaping, 180, 190

R

range select queries
  B-tree indexes, 48
  clustered indexes and, 216
  hash functions unsuitable, 49
RDBMS (Relational Database Management Systems)
  Appendix comparing three, 257
  choice of, 156
  emergence of open-source, 10
  list of well-known products, 27
  support for null values, 34
  support for SQL-92 standard, 54
Read Committed isolation level, 220
Read Uncommitted isolation level, 219
reading data *see* querying.

read-only access
  avoiding SQL injection, 187
  use in conjunction with views, 192
records and record types, Network database model, 11
RECOVERY keyword, 232
recovery models, database backups, 227
recursive relationships
  many- to-many recursive relationships, 112
  one-to-many recursive relationships, 109
  programming for, 111
  sub-query example, 199
references *see* foreign keys.
referential actions, 130
referential constraints, 128
referential integrity *see* RI.
reflexive relationships *see* recursive relationships.
relational algebra and relational calculus, 73
relational database model, 12
  characteristics, 12
  implementing many-to-many relationships, 108
  indexes not part of, 243
  role of Edgar F Codd, 9, 27
relational integrity, web store e-commerce application, 164
relations *see* tables.
relationship data, default categories as, 148
relationships
  distinguishing from attributes, 146
  ER modeling elements, 103
relationships between entities
  ER diagrams and, 105
relationships between tables
  foreign key role, 36
  illustration, 8
  many-to-many recursive relationships, 112
  many-to-many relationships, 107
  one-to-many recursive relationships, 109
  one-to-many relationships, 106
  one-to-one relationships, 113
  sub-type/super-type relationships, 114
  types of relationship, 12
  web store e-commerce application, 147
Repeatable Read isolation level, 220
repeating groups forbidden by 1NF, 39
REPLACE function, 230
replication, 277
reporting features, SELECT commands, 84
reporting requirements and the planning process, 143
reserved words and table and column naming, 157
response times, perceived, 1
RESTORE command, 232
restore operations, 231
  see also backups.
RESTRICT referential action, 130
reverse engineering and CASE tools, 134
Reverse Engineering Wizard, Microsoft Visio, 135
REVOKE command, access permissions, 185
RI (Referential Integrity), 128
  aspect of built-in data integrity, 44
  DRI (Declarative Referential Integrity), 184
  foreign keys constraints specify, 60
  practical implications, 129
  RDBMS support for, 128
  recursive relationships and, 111
  web store e-commerce example, 164
right outer joins, 81
roles, security, and table-level permissions, 183
ROLLBACK TRANSACTION command, 218
round trip engineering, 134
rows
  identification with primary keys, 34
  order immaterial, 29

sample data
  importance for database model validation, 136
  Northwind database, 211
salability and good design, 24
schemas viewed as data models, 25
schemas, Oracle, 270
scripting languages, checkbox choices returned as CSV lists, 202
scripts, SQL, 249
  populating tables using, 249
search mechanisms and the planning process, 142
Second Normal Form (2NF), 40
security features of DBMSs, 18
security issues, 177
  see also access permissions; passwords.
  advantages of views, 204
  database-level security, 183

designing a security model, 178
network security, 192
security model diagram, 178
SELECT * command, 74
surrogate keys, 164
SELECT * syntax dangers, 174
SELECT clause
building a SELECT query, 95
SELECT commands
expressions and constants in, 90
extracting data into a SQL scripts , 250
nesting in sub-queries, 198
reporting features, 84
SQL querying based on, 73
techniques for building, 94-98
example, 95
using with views, 204
web store e-commerce application
searching for products, 173
self joins, 91
sub-query alternative to, 200
semicolons
indicator of SQL injection, 188
SQL command terminator, 58
SEQUEL, precursor of SQL, 55
sequence objects, Oracle
numeric sequence keys, 118
surrogate keys defined as, 163
sequential access methods, 46
serial data type, PostgreSQL
numeric sequence keys, 118
surrogate keys defined as, 163
Serializable isolation level, 221
server-side includes, 20
SET NOCOUNT ON command, 250
SET NULL referential action, 130
set operations
joins viewed as, 82
SQL set operators, 92
set operators, sub-queries, 199
SET SHOWPLAN command, SQL Server, 211
set types, Network database model, 11
sets and the relational database model, 28
shared locks, 217
simple recovery model
introduced, 227
restore operations, 231, 233
single quotes
escaping in input, 188
escaping in names, 190
SiXDML (Simple XML Data Manipulation Language), 55
smallint data type, web store e-commerce application, 167
SMALLMONEY data type, scripting example, 253
sorting
see also clustered indexes.
ORDER BY clauses, 85
sorting by multiple columns, 86
sorting into categories, one-to-many recursive relationships, 109
sp_help stored procedure
displaying table characteristics, 240, 247
sample results, 241-243
Spotlight performance monitoring tool for Oracle, 267
spreadsheets
shortcomings for data storage, 6-7
superficial resemblance of relations to, 29
sprocs *see* stored procedures.
SQL (Structured Query Language), 53
see also DCL; DDL; DML; queries, SQL; scripts, SQL.
data types and, 32
dialects, 55
history of database development and, 9
inline SQL security issues, 186
RDBMS departures from standard SQL, 2
SQL injection, 186-190
ADO Command object and, 189
stored procedures and, 188
SQL Server
see also Enterprise Manager; Query Analyzer.
Appendix comparing three RDBMSs, 271-278
availability, 271
NULL as default value, 237
operating systems supported, 272
security examples use of, 177
SET SHOWPLAN command, 211
Special Uses Editions, 276
features and requirements, 277
SQL-92 support, 273
Standard and Enterprise Editions compared, 275
stored procedure example, 205

SQL Server Authentication, 179
SQL syntax, command groups, 56
SQL:1999, 56
SQL-92 standard
  BACKUP command, 228
  example queries compared with earlier syntax, 80
  most widely supported version, 54
  MySQL support for, 262
  Oracle support for, 267
  specifying joins before, 78
  SQL Server support, 273
  syntax for conditional joins, 79
  transaction isolation levels, 219
SSL (Secure Sockets Layer), 183
stakeholder requirements, web store e-commerce application, 139
standardization
  RDBMS departures from standard SQL, 2
  SQL-92 standard, 54
static data and denormalization, 83
stock portfolios example, domain integrity, 121
stored procedure permissions, 184
stored procedures, 206
  see also sp_help.
  displaying table characteristics, 240
  examples of use, 207
  executing using ADO Command objects, 190
  inline SQL security issues and, 186
  introduced, 205
  Oracle support for, 269
  security advantages, 188
  syntax resembles that of triggers, 208
  usefulness in a web environment, 98
STR function, 230
stress testing, importance and free MS tool for, 137
string constants, 251
strings, zero-length, contrasted with nulls, 32
strongly-typed parameters, 205
strongly-typed passwords, 182
  Windows Authentication and, 192
structural aspect provided by data models, 26, 28-42
sub-queries, 198-203
  available operators, 200
  combining with the IN operator, 200
  correlated sub-queries, 201
  RDBMS support for, 199
  set operators, 199
sub-type/super-type relationships, 114
support and feedback for this book, 3
surnames, B-tree index example, 47
surrogate keys, 35, 118-121
  alternative to concatenating primary keys, 63
  guaranteeing uniqueness, 118
  intelligent keys and, 118, 120
  web store e-commerce application, 169
  when not to use, 120
SYS_OP_GUID() function, Oracle, 119
system architecture, Oracle, 270
System R, 9, 27
  history of query language development, 55

tab character coding in SQL Server, 251
table constraints
  distinguished from column constraints, 58, 59
  primary keys as, 58
table joins *see* joins.
table renaming, 90
table rows *see* rows.
table scans, 46
  indexing sample query, 245
  library analogy, 214
  performance tuning and, 212
table-level permissions, 183
table-like structures and the relational model, 28
tables
  aliasing in correlated sub-queries, 201
  characteristics in relational database model, 29
  dynamic table creation, 208
  modifying, 234
  naming associative tables, 109
  naming conventions, 157
  predominantly written to, indexing, 214
  representation of, and examples, 30
  representing entities as, 30, 104, 157
  SQL scripts to populate, 249
target audience, identifying, security modeling and, 178
telephone number storage, 124
testing
  see also performance tuning.
  building a SELECT query, 95
  stress testing, 137

TEXT data type, MySQL databases, 166
theory, need for, 20
Third Normal Form (3NF), 40
time series data
  checking for alarm states, 207
  clustered indexes and, 216
  using triggers, 208
TOAD (Tool for Oracle Application Developers), 267
trade-off between retrieval and modification speeds, 131, 214
traffic analysis and scheduling maintenance, 254
transaction isolation levels, 219
  choosing and setting, 221
transaction logs *see* log files.
transactions, 217-222
  introduced, 216
Transact-SQL
  stored procedure example, 205
  trigger example, 208
transitive functional dependencies, 41
triggers, 207
  examples of use, 208-209
  introduced, 205
  Oracle support for, 269
  recursive relationships and, 111
  syntax resembles that of sprocs, 208
true/false *see* Boolean data types.
TRUNCATE command, contrasted with DELETE, 69
T-SQL *see* Transact SQL.
tuples, 30
  see also rows.

U

UNION operator, 93
UNIQUE constraints, 60
UNIQUE keyword, CREATE INDEX command, 66
unique values, selecting or generating, 118
uniqueidentifier column, GUIs, 119
uniqueness of indexes, 47, 215
  UNIQUE keyword, 244
update anomalies
  RI problem, 129
  un-normalized data, 38
UPDATE command, 69
update frequency and indexed access methods, 48
updating records, applying referential actions, 130
URI (Uniform Resource Identifiers) and physical data independence, 26
usable form design, internationalization, 169
user preferences, database storage, 19
user-defined integrity rules, 43
users
  defining users and groups, 181
  identifying target audience, security modelling, 178

V

'valid' columns, simulating nulls, 34
validation of data
  avoiding SQL injection, 187
  combining with data integrity rules, 44
VARCHAR data type, 123
  size limits of RDBMSs, 166
  web store e-commerce application, 161
view permissions, 184
views, 204
  chaining SELECT statements, 204
  use to restrict data access, 191
virtual tables accessible using triggers, 208
Visio modeling tool,
  CASE demonstration using Visio 2002, 132
  Examples using, 103
  forward engineering, 134
  reverse engineering, 135

W

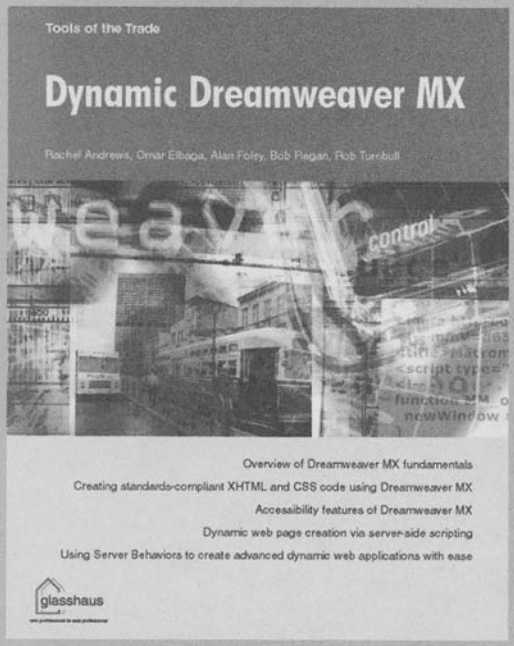
web access technologies and history of database development, 10
web applications
  database back-ends, 19
  denormalizing for performance, 131
  example database uses, 139
  Oracle support for, 271
  reasons to use databases, 18
web store e-commerce application
  creating associative tables, 159
    entity-relationship diagram, 160
  creating indexes, 171
  database diagram, 170

web store e-commerce application (Cont'd)
ER diagrams, 148-151, 159-165
including associative tables, 160
foreign keys and relational integrity, 164
identifying entities and attributes, 146
identifying entity relationships, 147
information gathering stage, 140
introduced, 139
preparing the conceptual data model, 151
sample DDL, 172
sample SQL queries
adding products to a cart, 174
searching for products, 173
table and column definitions, 165
table layouts
Category table, 167
Customer table, 168
OrderItem table, 170
Orders table, 169
Product table, 166
ProductCategory table, 168
WHERE clauses, 71
aggregation operations and, 89
building a SELECT query, 95
columns in, as candidates for indexing, 214
conditional joins, 78
incorporating sub-queries, 198
indexing and performance, 244
may refer to columns not in query output, 74
specifying multiple conditions, 70, 98
UPDATE command, 69
wildcard characters, use with LIKE operator, 174
Windows Authentication, 179
group logins and, 181
network security and, 192
Windows Explorer resembles a hierarchical database, 11
WITH INIT keywords, BACKUP command, 228, 230
WITH VALUES keywords, adding columns to tables, 239
Word documents, shortcomings for data storage, 5
Workflows, detemining data access needs using, 180

XML (Extensible Markup Language)
Oracle support for, 271
retrieving hierarchical tree data, 112
use as a database format, 16
XML databases, 17
XML:DB initiative and SiXDML, 55
XPath, querying XML databases, 17
XQuery, querying XML databases, 17
XSLT (Extensible Stylesheet Language Transformations), XML display, 16

Also from glasshaus:

glasshaus
web professional to web professional



Dynamic Dreamweaver MX

Rob Turnbull, Bob Regan, Omar Elbaga, Paul Boon, Rachel Andrew

1-904151-10-8

US: $29.99
C : $21.99
UK: £46.99

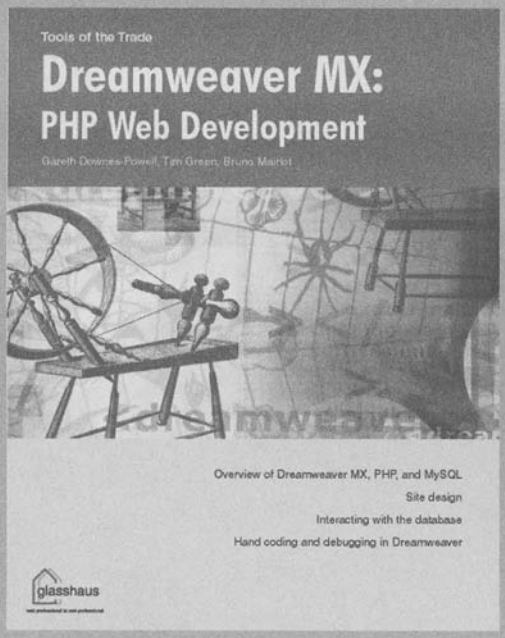
July 2002

This book gets you up to speed on using Macromedia Dreamweaver MX, the new version of Macromedia's premier visual web site design tool, to produce dynamic, creative, visually stunning sites that comply with web standards and accessibility guidelines. It gets straight to the heart of the matter so you spend less time reading, and more time building your site.

- **Rachel Andrew** is a member of the Web Standards Project's Dreamweaver Task Force, responsible for improving Dreamweaver's standards compliance and accessibility
- **Omar Elbaga** started out as a fine artist and moved to computer graphic arts. He is also a member of Team Macromedia
- **Alan Foley** is an Assistant Professor of Instructional Technology who teaches and consults on web accessibility and usability issues
- **Bob Regan** is the Senior Product Manager for Accessibility at Macromedia
- **Rob Turnbull** is also a member of Team Macromedia

Also from glasshaus:

glasshaus
web professional to web professional



Dreamweaver MX:
PHP Web Development

Bruno Mairlot, Gareth Downes-Powell
Tim Green

1-904151-11-6

US: $39.99
C : $61.99
UK: £28.99

July 2002

This book is all about making dynamic PHP web sites with Dreamweaver MX . It covers PHP, enough to get the reader up to speed with the technology, and how to use Dreamweaver MX to produce PHP code quickly and efficiently. It also covers site design and databases and SQL. It uses an example project, a hotel reservation system, that is built up through the chapters to demonstrate the concepts explained

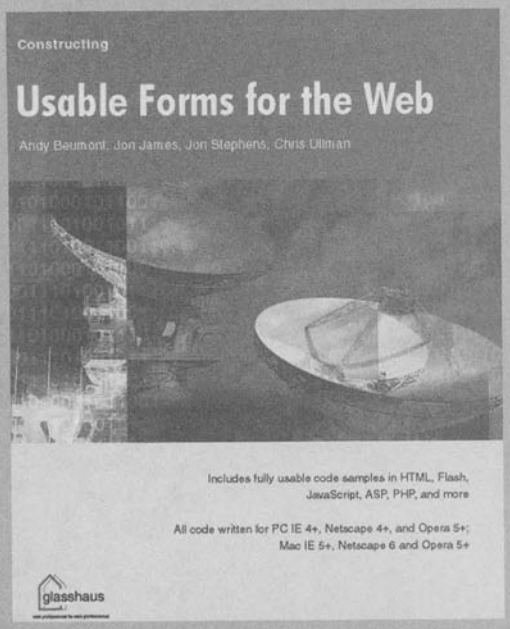
Web Professionals have been calling for years for Dreamweaver/ UltraDev to support PHP, as it's the premier free open-source server-side scripting language. With Macromedia's landmark new release of Dreamweaver, PHP is fully supported in the familiar Dreamweaver visual environment.

It's is a no-fluff 400 pages, so you can learn enough PHP to make real dynamic web pages, spending less time reading and more time on the job.

Aimed at web professionals who want to use Dreamweaver MX to produce PHP web sites. It doesn't assume any knowledge of PHP, and it doesn't hold your hand when talking about Dreamweaver, so experience of Dreamweaver would be useful. It assumes knowledge of HTML and Web design.

Also from glasshaus:

glasshaus
web professional to web professional



Usable Forms for the Web

Jon James,
Andy Beaumont,
Jon Stephens,
Chris Ullman

US: $24.99
C: $38.99
UK: £18.99

June 2002

Forms are an integral part of many web sites, whether they are registration forms, feedback forms, or order forms. However, forms are time consuming for the site user to fill out, and need to be implemented to be as usable as possible, otherwise they can be frustrating and annoying, spoiling the user experience.

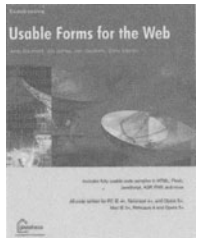
This book takes all the hassle out of implementing forms in whatever way you wish, dealing with client-side forms in HTML and Flash, client- and server-side form validation, and server-side data processing. It provides code samples fully adaptable to your own needs, along with walkthrough tutorials on how they work, and an HTML form element reference.

This book is for intermediate to advanced web professionals who need to implement a form on a web site as quickly as possible, with the minimum of hassle.

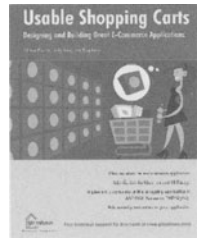
Also from glasshaus:

glasshaus
labor-saving devices for web professionals

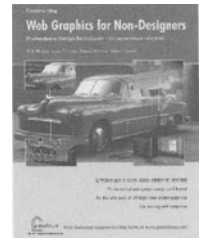
1-904151-02-7
Usable Web Menus



1-904151-09-4
Usable Forms
for the Web



1-904151-14-0
Usable Shopping Carts

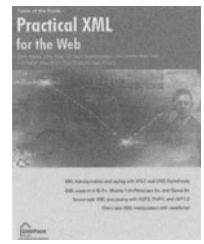


1-904151-15-9
Web Graphics for
Non-Designers

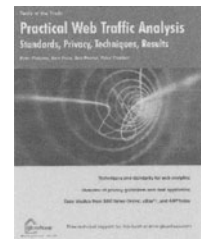
1-904151-03-5
Usability:
The Site Speaks for Itself

1-904151-10-8
Dynamic
Dreamweaver MX

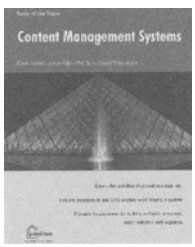
1-904151-11-6
Dreamweaver MX:
PHP Web Development



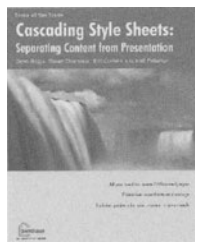
1-904151-08-6
Practical XML
for the Web



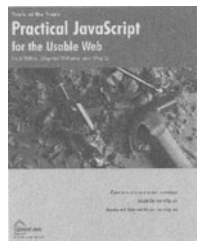
1-904151-18-3
Practical Web Traffic
Analysis



1-904151-06-X
Content Management
Systems

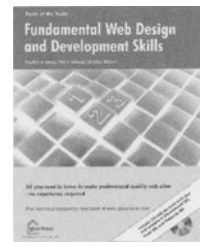


1-904151-04-3
Cascading Style Sheets



1-904151-05-1
Practical JavaScript
for the Usable Web

1-904151-00-0
Accessible Web Sites



1-904151-17-5
Fundamantal Web Design
and Development Skills

glasshaus books: labor-saving devices for web professionals